

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

Работа допущена к защите
Руководитель ОП
_____ А.В. Щукин
« _____ » _____ 2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
РАБОТА БАКАЛАВРА
ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ IOS-ПРИЛОЖЕНИЯ КАК
КОМПЛЕКСНОГО ПОМОЩНИКА ДЛЯ МУСУЛЬМАН

по направлению подготовки 09.03.03 Прикладная информатика
Направленность (профиль) 09.03.03_03 Интеллектуальные инфокоммуникацион-
ные технологии

Выполнил
студент гр. 5130903/20301

М.Р. Сабирзянов

Руководитель
к.т.н., доцент ВШПИ

А.В. Сергеев

Консультант
по нормоконтролю

Е.Е. Андрианова

Санкт-Петербург
2026

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

УТВЕРЖДАЮ

Руководитель ОП

_____ А.В. Щукин

« _____ » _____ 2026 г.

ЗАДАНИЕ
на выполнение выпускной квалификационной работы

студенту Сабирзянову Мирсаиту Раисовичу, гр. 5130903/20301

1. Тема работы: «Проектирование и реализация iOS-приложения как комплексного помощника для мусульман».
2. Срок сдачи студентом законченной работы: 18.05.2026.
3. Исходные данные по работе:
 - 3.1. Коран: переводы смыслов на русский язык [Электронный ресурс] // Quran.com: [официальный сайт]. — URL: <https://quran.com/ru> (дата обращения: 01.02.2026). — Режим доступа: свободный.
 - 3.2. Apple Developer Documentation. Swift Programming Language [Электронный ресурс] // Apple Developer: [официальный сайт]. — URL: <https://developer.apple.com/documentation/swift>
 - 3.3. Apple Developer Documentation. SwiftUI Framework [Электронный ресурс] // Apple Developer: [официальный сайт]. — URL: <https://developer.apple.com/documentation/swiftui>
 - 3.4. PyTorch Documentation [Электронный ресурс] // PyTorch: [официальный сайт]. — URL: <https://pytorch.org/docs/stable/index.html>
 - 3.5. Hugging Face Transformers Documentation [Электронный ресурс] // Hugging Face: [официальный сайт]. — URL: <https://huggingface.co/docs/transformers>
 - 3.6. FastAPI Documentation [Электронный ресурс] // FastAPI: [официальный сайт]. — URL: <https://fastapi.tiangolo.com/>

- 3.7. Spring Framework Documentation [Электронный ресурс] // Spring: [официальный сайт]. — URL: <https://docs.spring.io/spring-framework/docs/current/reference/html/>
4. Содержание работы (перечень подлежащих разработке вопросов):
- 4.1. Анализ предметной области и обзор существующих решений для мусульманской аудитории.
 - 4.2. Формулирование функциональных и нефункциональных требований к приложению.
 - 4.3. Проектирование архитектуры системы (iOS-клиент, backend, RAG-пайплайн, БД).
 - 4.4. Разработка и интеграция модулей приложения.
 - 4.5. Тестирование работоспособности системы.
5. Перечень графического материала (с указанием обязательных чертежей): Нет.
6. Перечень используемых информационных технологий, в том числе программное обеспечение, облачные сервисы, базы данных и прочие сквозные цифровые технологии (при наличии):
- 6.1. СУБД PostgreSQL.
 - 6.2. Облачный API-сервис OpenRouter (LLM-генерация).
 - 6.3. Фреймворк FastAPI (Python).
 - 6.4. Фреймворк Spring Boot (Java).
7. Консультанты по работе:
- 7.1. Старший преподаватель ВШПИ, Е.Е. Андрианова (нормоконтроль).
8. Дата выдачи задания: 22.12.2025.

Руководитель ВКР _____ А.В. Сергеев

Задание принял к исполнению 22.12.2025

Студент _____ М.Р. Сабирзянов

РЕФЕРАТ

На 116 с., 68 рис., 13 табл., 3 прил.

Ключевые слова: iOS-приложение, RAG, LLM, искусственный интеллект, халяль

Тема выпускной квалификационной работы: «Проектирование и реализация iOS-приложения как комплексного помощника для мусульман».

В выпускной квалификационной работе рассмотрены проектирование, разработка и тестирование многофункционального iOS-приложения для мусульман, объединяющего инструменты анализа халяльности продуктов, чат с искусственным интеллектом, модуль чтения Корана и систему расчета времени намаза.

В работе проведен анализ существующих исламских цифровых сервисов и сформулированы функциональные и нефункциональные требования к системе. Разработана архитектура программного комплекса, включающая iOS-клиент на SwiftUI, backend-сервис на Spring Boot и отдельный LLM-сервис на FastAPI. Для формирования ответов AI-ассистента реализован retrieval-augmented generation (RAG) подход с использованием векторного поиска по текстам Корана.

Реализован модуль анализа состава продуктов с использованием OCR и классификации ингредиентов по категориям «халяль», «халал» и «сомнительный». Выполнено проектирование REST API, базы данных PostgreSQL и механизма взаимодействия между сервисами.

Проведено тестирование клиентской и серверной частей системы, а также оценка качества работы RAG-подсистемы и проверки состава продуктов. Результаты показали корректность работы основных модулей приложения и возможность применения разработанной системы в качестве комплексного цифрового помощника для мусульман.

В разработке использованы: Xcode, Swift, Java 17, Python 3.9, SwiftUI, Spring Boot, FastAPI, PyTorch, Sentence Transformers, LangChain, PostgreSQL, Docker, Docker Compose, OpenRouter, XCTest, XCUITest, Swift Testing, JUnit 5, Mockito, MockMvc, pytest, JaCoCo.

ABSTRACT

On 116 pages, 68 figures, 13 tables, 3 appendices

Keywords: iOS application, RAG, LLM, artificial intelligence, halal

The subject of the graduate qualification work is «Design and implementation of an iOS application as a comprehensive assistant for Muslims».

This thesis presents the design, development and testing of a multifunctional iOS application for Muslims that combines halal product analysis, an AI-based assistant, Quran reading functionality and prayer time calculations.

The paper includes an analysis of existing Islamic digital services and defines functional and non-functional requirements for the system. A software architecture consisting of an iOS client developed with SwiftUI, a Spring Boot backend and a separate FastAPI-based LLM service was designed. A retrieval-augmented generation (RAG) approach with vector search over Quran texts was implemented for the AI assistant.

A product ingredient analysis module based on OCR and ingredient classification into halal, haram and doubtful categories was implemented. REST API architecture, PostgreSQL database structure and service interaction mechanisms were also designed.

Testing of the client, backend and LLM subsystems was carried out, including evaluation of the RAG pipeline and ingredient recognition quality. The results demonstrated the correctness of the implemented modules and the applicability of the developed system as a comprehensive digital assistant for Muslims.

The following tools were used in development: Xcode, Swift, Java 17, Python 3.9, SwiftUI, Spring Boot, FastAPI, PyTorch, Sentence Transformers, LangChain, PostgreSQL, Docker, Docker Compose, OpenRouter, XCTest, XCUITest, Swift Testing, JUnit 5, Mockito, MockMvc, pytest, JaCoCo.

СОДЕРЖАНИЕ

Введение	8
1 Теоретическая часть	11
1.1 Обзор исследований	11
1.2 Обзор существующих программных решений	13
1.3 Требования к разрабатываемому программному обеспечению	15
1.3.1 Функциональные требования	15
1.3.2 Нефункциональные требования	17
1.4 Выбор архитектуры системы	17
1.5 Выбор фреймворка для мобильного приложения	18
1.6 Выбор языка программирования для мобильного клиента	19
1.7 Выбор backend-технологий	20
1.8 Выбор системы управления базами данных	21
1.9 Выбор векторного хранилища	22
1.10 Выбор моделей и поставщиков LLM	23
1.11 Выбор инструмента для расчета времени намаза	23
1.12 Формулирование гипотез	24
1.13 Проектирование общей схемы взаимодействия компонентов	25
1.14 Use Case диаграммы	26
1.15 Схема базы данных	30
1.16 Алгоритмы обработки данных	32
1.16.1 Алгоритм обработки чат-запроса с использованием RAG	33
1.16.2 Алгоритм анализа состава продукта	34
1.16.3 Алгоритм расчета времени намаза	36
1.17 Проектирование REST API	38
1.18 Архитектура iOS-приложения	40
1.18.1 Общее описание архитектурного подхода	40
1.18.2 Управление навигацией через паттерн Coordinator	41
1.18.3 Внедрение зависимостей	42
1.18.4 Сервисы	42
1.18.5 Экраны и их состояние	42
1.18.6 Взаимодействие с backend	43
1.19 Архитектура Backend	44
1.19.1 Трехслойная архитектура	44
1.19.2 Аутентификация через JWT	44

1.20 Архитектура LLM-service.....	45
1.20.1 REST API слой	45
1.20.2 RAG слой.....	46
1.20.3 Слой интеграции с LLM	46
1.21 Макеты интерфейсов	46
2 Практическая реализация	53
2.1 Реализация Backend-сервера	53
2.2 Реализация LLM-сервиса	55
2.2.1 Основные компоненты сервиса.....	55
2.2.2 Экспериментальный подбор параметров и моделей	58
2.3 Реализация iOS-приложения	62
2.3.1 Структура проекта	63
2.3.2 Реализация навигации.....	64
2.3.3 Реализация функциональных модулей.	66
2.3.4 Реализация сетевого взаимодействия.....	77
3 Тестирование.....	79
3.1 Тестирование iOS-клиента.....	79
3.1.1 Unit-тестирование	79
3.1.2 UI-тестирование	84
3.2 Тестирование backend-сервиса	86
3.3 Тестирование LLM-сервиса	87
3.4 Проверка соответствия требованиям	90
3.4.1 Проверка качества распознавания состава продукта	90
3.4.2 Проверка корректности классификации ингредиентов	94
3.4.3 Проверка реализации других модулей приложения	95
3.4.4 Проверка производительности системы.....	101
3.4.5 Проверка надежности системы.....	104
3.4.6 Проверка безопасности.....	108
Заключение	113
Список использованных источников.....	115
Приложение 1 Код LLM-Service	117
Приложение 2 Код Backend	132
Приложение 3 Код iOS-приложения	160

ВВЕДЕНИЕ

В современную эпоху цифровизации мобильные приложения становятся неотъемлемой частью повседневной жизни. Смартфоны используют около 70% населения планеты, а на их долю приходится более 85% всех используемых мобильных устройств - что создает благоприятную среду для мобильных сервисов и супераппов в целом [1].

Одновременно мусульманское население мира стремительно растет и составляет значительную часть глобальной аудитории: по оценкам, к 2030 году число мусульман может превысить 2,2 млрд человек - примерно 30% населения Земли [2].

В рамках повседневного потребления для данной аудитории принципиальное значение имеет соблюдение требований халяль. Под халяльными продуктами понимаются продукты питания и товары, соответствующие нормам исламского права (шариата), включая запрет на использование определенных ингредиентов (например, свинины, алкоголя и их производных), а также требования к процессам производства, хранения и логистики. При этом определение халяльности продукта в современных условиях не всегда является однозначной задачей. Состав многих товаров включает сложные пищевые добавки, ферменты и ароматизаторы, происхождение которых затруднительно установить без специализированной информации. Дополнительную сложность создает отсутствие единых международных стандартов халяль и различия в требованиях сертифицирующих организаций, что осложняет осознанный выбор для конечного потребителя. Демографический сдвиг приводит к увеличению потребительских расходов в халяль-сегменте экономики. Согласно одной из оценок, общий объем глобального рынка халяльных продуктов питания и услуг к середине 2020-х годов исчисляется триллионами долларов - например, объем рынка халяльной еды превысил 2,6 трлн USD в 2024 году, и ожидается, что он будет расти более чем на 10% в год [3].

На уровне повседневного потребления цифровые технологии выступают для мусульманских пользователей основным каналом доступа к информации и сервисам. Отраслевые аналитические обзоры указывают на высокий уровень мобильной активности данной аудитории: в ряде ключевых регионов проникновение смартфонов превышает 80%, а мобильные устройства активно используются для получения контента, совершения покупок и взаимодействия с цифровыми услугами [4].

На цифровом рынке мобильных приложений в исламском пространстве уже есть яркие примеры больших и активно используемых продуктов. Так, популярное приложение Muslim Pro было скачано более 30 млн раз, а во время рамадана им пользовались от 3 до 4 млн активных пользователей в день [5]. Кроме него существуют специализированные сервисы, такие как Muzz - мусульманское приложение для знакомств, которое насчитывает свыше 8 млн пользователей и сыграло роль в более чем 600 000 браков [6].

Вместе с тем существуют серьезные риски и ограничения, которые отражают религиозные и юридические рамки:

- А. отсутствие единых международных стандартов халяль, поскольку каждая страна и сертифицирующая организация формируют собственные правила - это усложняет проверку и соответствие продуктов и сервисов на глобальном рынке [7].
- В. правовые различия в отдельных юрисдикциях могут ограничивать распространение и функциональность приложений, если они затрагивают религиозные аспекты, например, интерпретацию исламских норм (фетвы), что приводит к необходимости локализации и соблюдения нормативов в каждой стране. (общая проблема стандартизации характерна для них как цифровых, так и физических халяль-продуктов) [7].
- С. религиозно-культурные требования требуют от цифровых продуктов высокой точности, толерантности к различным толкованиям и возможности кастомизации под локальные конфессии и подходы, что создает вызовы для UX/UI и алгоритмов рекомендаций.

Эти факторы усиливают необходимость интеграции ИИ-механизмов, которые могут автоматически учитывать сложные правила, языковые особенности и культурный контекст в анализе и выдаче рекомендаций. Экосистема совмещающая мобильное приложение, серверную инфраструктуру и модели искусственного интеллекта может обеспечить комплексное, адаптивное и соответствующее религиозным ожиданиям решение для глобального рынка.

Целью работы является разработка, реализация и тестирование многофункционального iOS приложения для мусульман со сканером продуктов, чатом с искусственным интеллектом на основе Корана, напоминаниями о намазах и с текстами сур и аятов для повседневных религиозных практик пользователей.

Для достижения указанной цели были поставлены следующие задачи:

- A. Провести анализ предметной области, рассмотреть существующие решения для мусульман и проанализировать конкурентов.
- B. Сформулировать функциональные и нефункциональные требования к разрабатываемому приложению на основе анализа существующих решений.
- C. Разработать общую архитектуру, включая схему взаимодействия iOS-клиента, серверной части, базы данных и RAG-модуля.
- D. Спроектировать структуру базы данных для хранения и обработки информации о халальности продуктов, пользовательских настройках, а также текстов Корана.
- E. Разработать API для обеспечения взаимодействия между клиентской и серверной частями приложения..
- F. Реализовать LLM сервис с RAG-пайплайном для формирования ответов на пользовательские вопросы в чате на основе текстов Корана.
- G. Разработать iOS-приложение, учитывающее функциональные и нефункциональные требования к продукту.
- H. Осуществить интеграцию клиентской и серверной частей, а также провести тестирование работоспособности всех модулей приложения и оценку качества ответов от RAG-системы.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Обзор исследований

Современные исследования показывают устойчивый рост интереса к применению цифровых технологий и искусственного интеллекта в мусульманском сообществе, особенно в контексте халяль-индустрии и религиозно ориентированных цифровых сервисов. В обзорной работе, опубликованной в *Indonesian Journal of Halal Industry*, отмечается, что искусственный интеллект перестал быть исключительно концептуальной темой и постепенно внедряется в ключевые процессы халяль-индустрии, включая контроль качества, сертификацию, маркетинг и управление цепочками поставок. Авторы подчеркивают, что AI-системы способны автоматически выявлять нехаляльные ингредиенты и обеспечивать соответствие продукции стандартам халяль на протяжении всего производственного цикла, что делает данные технологии практически значимыми для реальных отраслевых задач [8].

Важным направлением исследований является анализ мобильных приложений, ориентированных на мусульманскую аудиторию. В ряде работ подчеркивается, что исламские мобильные приложения демонстрируют более высокий уровень загрузок по сравнению с приложениями других религиозных направлений. Согласно исследованию, посвященному анализу исламских мобильных приложений, наиболее востребованными функциями являются Коран с переводом, напоминания о времени молитв, направление киблы и азан. Пользователи характеризуют такие приложения как «all-in-one» решения, объединяющие религиозные, информационные и сервисные функции, что указывает на высокий запрос на комплексные цифровые продукты для повседневной религиозной практики [9].

Цифровизация религии, в частности ислама, также рассматривается в отечественных исследованиях. В работах, опубликованных на платформе CyberLeninka, подчеркивается, что исламские религиозные практики все чаще реализуются в онлайн-формате, включая виртуальные формы джума-намаза, хаджа и умры. При этом ключевой характеристикой цифровых религиозных сервисов остается их соответствие нормам шариата и требованиям халяль. Авторы указывают, что цифровые технологии становятся языком диалога религии с современным обществом, однако любое IT-решение в исламском контексте должно учитывать культурные и богословские ограничения [10, 11].

Ряд зарубежных исследований фокусируется на применении искусственного интеллекта в религиозных информационных системах. В частности, показано, что чат-боты и голосовые ассистенты основанные на AI могут выступать интерактивными платформами для получения информации о Коране и исламском учении, особенно среди молодежи. Такие цифровые инструменты способствуют повышению вовлеченности пользователей и положительно влияют на их духовное благополучие, однако подчеркивают, что внедрение таких технологий требует модерации со стороны человека и учета религиозных норм и ответственного использования технологий [12].

Более прикладные исследования рассматривают использование искусственного интеллекта для распознавания халяль-продуктов. В работе HALALCheck предлагается многофакторный подход к интеллектуальному анализу упакованных продуктов питания, демонстрирующий высокую точность классификации халяль и харам. Авторы отмечают, что для 66% мусульман халяль-питание является ключевым фактором, особенно в условиях путешествий, однако подчеркивают наличие методологических проблем, связанных с несбалансированностью данных и различиями в интерпретации норм среди исламских ученых [13].

С точки зрения архитектуры интеллектуальных информационных систем, в последние годы значительное внимание уделяется retrieval-augmented generation (RAG). В фундаментальной работе «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks» показано, что сочетание параметрической памяти языковой модели с непараметрической памятью в виде векторного индекса позволяет генерировать более точные и фактически обоснованные ответы по сравнению с моделями, не использующими механизм RAG. Авторы подчеркивают, что RAG-подход особенно эффективен для knowledge-intensive задач, где требуется опора на внешние источники данных [14].

Дополняют данный подход исследования в области плотного векторного поиска. В работе В. Карпучина «Dense Passage Retrieval for Open-Domain Question Answering» показано, что retrieval в системах открытого вопросно-ответного поиска может эффективно реализовываться с использованием dense-представлений и dual-encoder архитектур, что обеспечивает масштабируемость и высокую семантическую точность поиска [15]. Эти идеи получили дальнейшее развитие в работе RocketQA, где dense passage retrieval рассматривается как новая парадигма семантического поиска для интеллектуальных систем [16]. Для религиозных приложений особенно важно, чтобы они могли гарантировать полноту и корректность

сообщаемой информации пользователю. Именно поэтому в моей работе было принято решение использовать RAG-модель совместно с векторным поиском, что позволяет опираться на первоисточник при формировании ответа, тем самым максимально избегать дезинформации и являться достоверным источником для пользователей.

Подводя итог, анализ существующих исследований показывает, что, несмотря на активное развитие исламских мобильных приложений и рост интереса к применению AI в халяль-индустрии, большинство работ либо носит обзорно-социокультурный характер, либо фокусируется на отдельных прикладных задачах, таких как распознавание продуктов. В своей работе я хочу сфокусироваться на объединение современных подходов, которые показали свою успешность по отдельности, для создания единого продукта, закрывающего все потребности пользователя. Создать приложение и адаптировать современные генеративные технологий для работы с чувствительными данными и религиозным контекстом, для осознанного и ответственного внедрения AI-инструментов в религиозный контекст.

1.2 Обзор существующих программных решений

Рассмотрим более конкретно несколько существующих программных решений на рынке, а именно: HalalCheck, HalalGuide, Muslim Pro и Noor AI (таблица 1.1).

Таблица 1.1

Сравнение существующих решений

Критерий	HalalCheck	HalalGuide	Muslim Pro	Noor AI
Основное назначение	Определение халяльности продуктов питания	Многофункциональное исламское приложение с разделом проверки продуктов	Многофункциональное исламское приложение	Исламский AI-ассистент с консультационными функциями
Функция определения халяльности продуктов	Реализована через справочную базу ингредиентов	Реализована через сканирование штрихкода и анализ состава	Прямой анализ состава отсутствует	Консультационная оценка по текстовому запросу пользователя

Продолжение табл. 1.1

Критерий	HalalCheck	HalalGuide	Muslim Pro	Noor AI
Метод проверки	Справочник добавок, ингредиентов и продуктов	Распознавание текста состава, база данных ингредиентов	Поиск халяльных заведений и ресторанов	Ответы на основе текстового запроса без официальной базы сертификации
Наличие AI-ассистента	Отсутствует	Отсутствует	Отсутствует	Реализован в виде диалогового помощника
Дополнительные функции	Сохранение результатов проверки, формирование отчетов	Время намаза, кибла, исламский календарь, карта халяльных заведений	Время намаза, Коран, кибла, исламский календарь, поиск халяльных мест	Ответы на религиозные вопросы, консультационная поддержка
Модель распространения	Бесплатная версия	Бесплатная версия с дополнительными платными функциями	Бесплатная версия с дополнительными платными функциями	Бесплатный доступ с ограничениями либо подписка
Преимущества	Узкая специализация на анализе продуктов, автоматизация проверки	Комплексность и широкий функционал	Высокая популярность и широкий спектр исламских сервисов	Наличие AI-ассистента, возможность задавать вопросы в свободной форме
Недостатки	Узкая специализация, ограничения в рамках одной базы данных, поддержка только английского и немецкого языков, только web-версия, отсутствие интеллектуального помощника	Часть функций доступна только по подписке, распознавание состава ограничено в бесплатной версии, перегруженный интерфейс, отсутствие полноценного AI-ассистента	Отсутствие функции сканирования и детального анализа ингредиентов, слабая локализация для российского рынка, отсутствие интеллектуального анализа состава	Отсутствие специализированной базы ингредиентов, нет сканирования составов, ответы носят консультационный характер и не всегда подтверждаются официальной сертификацией

Анализ решений показал, что ни одно из рассмотренных решений не сочетает одновременно:

- А. автоматизированный анализ состава продуктов
- В. наличие полнофункционального AI-ассистента
- С. полноценную локализацию под российский рынок

HalalCheck ориентирован на базовую проверку ингредиентов, однако не обладает интеллектуальным диалоговым модулем и имеет ограниченную языковую поддержку.

HalalGuide предоставляет более широкий функционал, но специализированный анализ состава ограничен подпиской, а интеллектуальный ассистент отсутствует.

Muslim Pro является популярным исламским приложением, однако не ориентирован на анализ состава продуктов и не содержит инструментов глубокого определения халяльности конкретных товаров.

Noor AI реализует формат AI-ассистента, однако не имеет специализированной базы продуктов и механизма верификации состава. Его ответы носят консультативный характер и зависят от формулировки запроса пользователя.

Таким образом, существует потребность в создании приложения, которое объединит:

- А. автоматическое распознавание состава и выявления харам ингредиентов в нем.
- В. расширенную базу ингредиентов с регулярным обновлением и модерацией.
- С. встроенного AI-ассистента для консультаций.
- Д. адаптацию под региональные стандарты и локализацию.
- Е. удобный и не перегруженный интерфейс.
- Ф. отсутствие критических ограничений базового функционала.

Разработка такого решения позволит объединить сильные стороны существующих решений на рынке и устранить их выявленные недостатки.

1.3 Требования к разрабатываемому программному обеспечению

1.3.1 Функциональные требования

- а) Определение халяльности продуктов

Приложение должно обеспечивать автоматизированное определение халяльности пищевых продуктов на основании анализа их состава.

б) Сканирования состава продукта

Приложение должно включать модуль анализа состава, обеспечивающий:

- А. загрузку фотографии состава продукта
- В. предварительную обработку изображения
- С. передачу распознанного текста в систему анализа

в) Распознавание текста состава и анализ его

Приложение должно реализовывать механизм распознавания текста для извлечения состава продукта с упаковки, для дальнейшего сопоставления ингредиентов продуктов с базой данных пищевых добавок и компонентов, классифицированных по категориям: халяль, харам, сомнительный.

г) Обоснование результатов

Система должна предоставлять аргументированное объяснение результата проверки, включая:

- А. перечень распознанных ингредиентов, которые есть в системе
- В. классификацию ингредиента
- С. итоговый статус продукта

д) Чат с искусственным интеллектом

Приложение должно включать чат с AI-ассистентом, который формирует ответы на основе встроеной базы знаний. Для пользователя есть возможность задавать вопросы в свободной форме, связанные с исламом и халяль-продукцией.е)

Отображения текстов Корана

Система должна обеспечивать доступ к тексту Корана с возможностью:

- А. выбора суры и аята (главы и стихи в нем)
- В. отображения оригинального текста
- С. отображения перевода на выбранный язык
- Д. удобной навигации по структуре текста

ж) Напоминаний о времени молитв

Приложение должно обеспечивать:

- А. определение времени молитв на основе геолокации пользователя
- В. персонализированные уведомления
- С. возможность настройки времени и способа уведомлений

з) Локализация

Система должна поддерживать многоязычный интерфейс, включая обязательную адаптацию под русский язык.

1.3.2 Нефункциональные требования

а) Надежность

Программное обеспечение должно обеспечивать корректность и стабильность работы при обработке пользовательских запросов.

б) Актуальность данных

База данных ингредиентов должна регулярно обновляться с учетом изменений стандартов и нормативов.

в) Производительность

Время отклика приложения на действия пользователя не должно превышать 2 секунды, время выполнения серверных запросов - не более 3 секунд.

г) Удобство использования

Интерфейс должен быть интуитивно понятным, не перегруженным второстепенными функциями и ориентированным на основные функции.

д) Доступность Базовый функционал системы должен быть доступен без обязательной платной подписки.

е) Безопасность

Система должна обеспечивать защиту персональных данных пользователей в соответствии с действующим законодательством.

ж) Масштабируемость

Архитектура программного обеспечения должна предусматривать возможность расширения функционала и увеличения объема базы данных без снижения производительности.

1.4 Выбор архитектуры системы

Для разрабатываемого продукта выбирается гибридная архитектура, сочетающая локальную обработку на устройстве и клиент-серверное взаимодействие. В качестве клиентской части будет реализовано мобильное приложение для iOS.

А. Клиентская часть

В. Выполняет локальный анализ состава продукта.

- С. Обработывает фотографии упаковки, распознает текст через OCR и классифицирует ингредиенты как халяль, харам или сомнительные.
- D. Формирует локальный отчет для пользователя без необходимости подключения к серверу.
- E. Серверная часть
- F. Хранит централизованную базу ингредиентов и религиозных источников.
- G. Обработывает запросы AI-ассистента, предоставляющего консультации по вопросам халяльности и религиозным нормам.
- H. Управляет дополнительными сервисами: тексты Корана, персонализированные уведомления о намазе, история проверок и статистика.

Также для формирования более точных и аргументированных ответов от AI-ассистента предполагается использование Retrieval-Augmented Generation (RAG), для полноценной его работы мы будем использовать векторную базу данных, способную хранить семантическое представление документов и ингредиентов.

1.5 Выбор фреймворка для мобильного приложения

Выделим два подхода: нативные фреймворки iOS (SwiftUI и UIKit) и кроссплатформенные решения (Flutter и Kotlin Multiplatform). Проведем сравнение их возможностей, преимуществ и ограничений для конкретных задач нашего приложения (таблица 1.2).

Таблица 1.2

Сравнение фреймворков для мобильной разработки

Критерий	SwiftUI	UIKit	Flutter	KMP
Производительность UI	++	++	+-	+
Доступ к нативным API	++	++	-	+-
Качество кастомного UI	++	++	+-	+
Интеграция on-device ML	++	++	-	-
Скорость разработки (MVP)	+	-	++	+-
Тестируемость / Unit & UI тесты	++	++	+	+

Продолжение табл. 1.2

Критерий	SwiftUI	UIKit	Flutter	KMP
Степень повторного использования бизнес-логики (между платформами)	-	-	+	++
Экосистема, пакеты, поддержка (iOS)	++	++	+	+-
Поддержка accessibility, локализация	++	++	+	+-

Сравнение показало, что SwiftUI и UIKit обеспечивают наилучший уровень производительности UI, доступ к нативным API, интеграцию on-device ML и работу с камерой, что критично для функционала приложения. SwiftUI выигрывает благодаря декларативному синтаксису и быстрому созданию кастомного интерфейса, тогда как UIKit обеспечивает полный контроль для сложных нативных решений. Flutter подходит для быстрой кроссплатформенной разработки, но уступает в нативных интеграциях и доступе к ML. KMP оптимален для повторного использования бизнес-логики между платформами, однако UI и интеграцию с нативными API требует реализации на iOS/Android отдельно. В целом, для iOS-клиента предпочтителен SwiftUI с возможной интеграцией UIKit для производительных и сложных экранов.

1.6 Выбор языка программирования для мобильного клиента

Поскольку для реализации мобильного клиента было принято решение использовать SwiftUI с возможной интеграцией UIKit, выбор языка программирования определяется выбранным фреймворком. Нативные фреймворки iOS поддерживаются только языком Swift, который обеспечивает полную интеграцию с системными API, высокую производительность интерфейса и прямой доступ к библиотекам CoreML, VisionKit и другим нативным сервисам. Таким образом, использование Swift является естественным и необходимым следствием выбора фреймворка.

1.7 Выбор backend-технологий

Для разработки серверной части были рассмотрены несколько технологий (таблица 1.3).

Таблица 1.3

Сравнение backend-технологий

Критерий	Spring Boot	Node.js (Express)	Ktor (Kotlin)	Go	Python (FastAPI)
Производительность (латентность, throughput)	+	+	+	++	+
Масштабируемость / поддержка микросервисов	++	+	+	++	+
Знакомство с фреймворком	++	-	-	-	+

Для разработки распределенной backend-части приложения мы оценили несколько технологий: Spring Boot, Node.js (Express), Ktor, Go (Gin/Fiber) и Python (FastAPI). Основными критериями были производительность, масштабируемость и знакомство с фреймворком.

Spring Boot показывает высокую масштабируемость и богатую экосистему, что удобно для сложной бизнес-логики, однако его производительность уступает Go. Node.js хорошо подходит для быстрых прототипов и интеграции с LLM, благодаря простой архитектуре и богатой экосистеме npm, но не так эффективен для высоконагруженных систем. Ktor обеспечивает тесную интеграцию с Kotlin и позволяет писать лаконичный код, но пока имеет меньшую экосистему и сообщество. Go обеспечивает лучшую производительность и низкую латентность, что важно для масштабируемых сервисов, но требует более глубоких знаний языка. Python (FastAPI) отлично подходит для интеграции с LLM сервисом и быстрого прототипирования, обладает хорошей экосистемой библиотек для ML, однако уступает по производительности Go и Spring Boot.

С учетом архитектурного паттерна low coupling / high cohesion оптимальным выбором для нашего backend является сочетание Python FastAPI и Java Spring Boot. High cohesion (высокая внутренняя связность) означает, что каждый сервис выполняет строго определенную задачу и содержит все необходимые компоненты для ее реализации: FastAPI отвечает за интеграцию LLM, обработку RAG-запросов и ML–

функционал, а Spring Boot за управление пользователями, JWT-аутентификацию и бизнес-логику. Low coupling (низкая связанность) обеспечивает минимальные зависимости между сервисами, что позволяет масштабировать ресурсоемкие компоненты, например AI-обработку запросов, без влияния на основную бизнес-логику. Такой подход повысит читаемость кода, упростит тестирование, обеспечит стабильность, гибкость и легкое расширение архитектуры (рисунок 1.1).

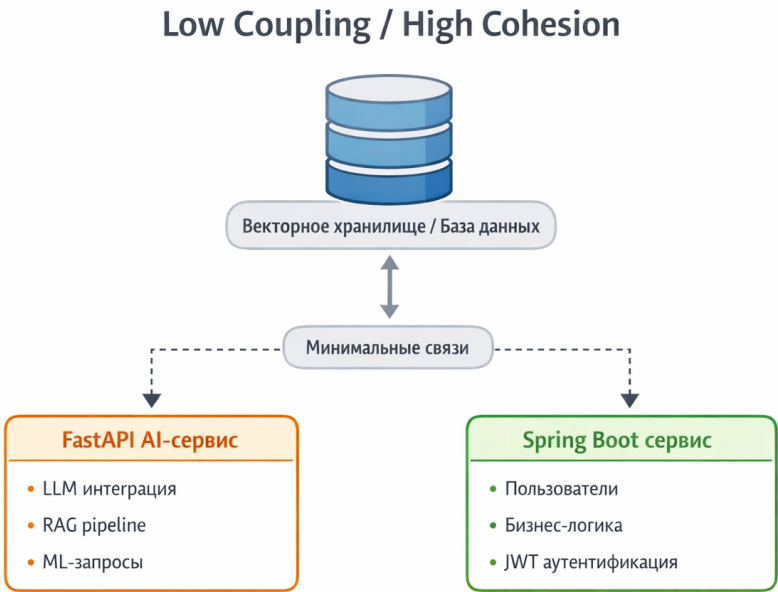


Рис.1.1. Схема low coupling / high cohesion

1.8 Выбор системы управления базами данных

Для хранения пользовательских данных и текстов Корана были рассмотрены несколько СУБД (таблица 1.4).

Таблица 1.4

Сравнение СУБД				
СУБД	Тип	Лицензия	ACID	Комментарий
PostgreSQL	SQL	PostgreSQL License	++	Оптимальный выбор
MySQL	SQL	GPL	+	Ограничения лицензии
MongoDB	NoSQL	SSPL	-	Юридические риски
Cassandra	NoSQL	Apache 2.0	-	Избыточна

- PostgreSQL выбран в качестве основной СУБД по следующим причинам:
- А. открытая лицензия без ограничений коммерческого использования
 - В. строгая транзакционность (ACID)
 - С. поддержка JSONB
 - Д. зрелая интеграция с Spring Data JPA
 - Е. высокая надежность и масштабируемость

Применение NoSQL-СУБД нецелесообразно, так как проект ориентирован на строгую согласованность пользовательских данных, а также с учетом возможных лицензионных ограничений некоторых NoSQL-платформ.

1.9 Выбор векторного хранилища

Для хранения векторных эмбеддингов были рассмотрены несколько решений (таблица 1.5).

Таблица 1.5

Сравнение векторных хранилищ

Решение	Тип	Стоимость	Контроль данных	Сложность
Файловое (PyTorch)	Local	0	++	+
FAISS	Local	0	++	+
Milvus	Server	Средняя	+	++
Pinecone	Cloud	Высокая	—	+

В рамках проекта планируется использование локального файлового векторного хранилища на основе PyTorch-тензоров и вычисления косинусного сходства для реализации семантического поиска в RAG-pipeline.

- Преимущества выбранного решения:
- А. отсутствие дополнительных финансовых затрат
 - В. простота развертывания и сопровождения
 - С. достаточная производительность для реализации MVP и прототипа системы

1.10 Выбор моделей и поставщиков LLM

Для сценариев работы AI-ассистента предусмотрим интеграцию с внешними языковыми моделями через OpenRouter API. OpenRouter предоставляет единый интерфейс для работы с различными облачными LLM, позволяя использовать любые доступные модели без необходимости интеграции с каждой платформой отдельно. Сервис обеспечивает масштабирование, обновление моделей и управление квотами.

Преимущества:

- А. Возможность выбрать оптимальную модель для конкретного сценария.
- В. Расширение функционала локальной модели для сложных задач или длинных диалогов.
- С. Быстрое тестирование разных моделей на одном и том же сценарии без сложной настройки.

Недостатки:

- А. Оплата за использование внешних моделей.
- В. Передача данных третьим сторонам и необходимость соблюдения законодательства о приватности.

1.11 Выбор инструмента для расчета времени намаза

Для реализации функционала расчета времени намаза можно рассмотреть 3 подхода: локальный расчет по формулам, использование сторонней библиотеки или обращение к внешнему API.

1. Локальный расчет по формулам

В данном подходе предполагается самостоятельно рассчитывать времени намаза на основе астрономических формул (положение солнца, географические координаты, временные зоны).

2. Использование внешнего API

В этом подходе предполагается использование сторонних сервисов для получения времени намаза.

3. Использование open source библиотеки

Третий подход заключается в использовании библиотеки Adhan Swift, реализующей расчеты времени намаза непосредственно на устройстве. Сравнение всех трёх подходов приведено в таблице 1.6.

Таблица 1.6

Сравнение методов расчета времени намаза

Критерий	Локальный расчет (формулы)	Библиотека (Adhan Swift)	Внешний API
Точность	Зависит от реализации	Высокая, проверенные алгоритмы	Высокая
Сложность реализации	Высокая	Низкая	Низкая
Поддержка методик расчета	Требуется реализовывать самостоятельно	Встроенная поддержка	Зависит от API
Зависимость от интернета	Нет	Нет	Да
Поддержка и обновления	Требуется самостоятельно	Обеспечивается библиотекой	Обеспечивается сервисом

Использование библиотеки Adhan Swift является наиболее рациональным решением, так как он сочетает точность вычислений, простоту интеграции и оффлайн доступ.

1.12 Формулирование гипотез

Гипотеза 1

Применение подхода retrieval-augmented generation, включающего векторный поиск релевантных документов и последующую передачу найденных фрагментов в качестве контекста языковой модели, приводит к повышению точности ответов и снижению частоты галлюцинаций по сравнению с использованием языковой модели без механизма retrieval.

Обоснование: Использование внешнего контекста позволяет уменьшить зависимость ответа от внутренних представлений языковой модели и обеспечивает опору на проверенные источники, что способствует повышению достоверности и обоснованности генерируемых ответов.

Метод проверки: Проверка гипотезы осуществляется путем сравнения двух режимов функционирования системы: без использования retrieval-механизма и с использованием RAG-подсистемы.

Критерии подтверждения: Гипотеза считается подтвержденной при выполнении следующих условий:

- А. увеличение количества корректных ответов
- В. снижение галлюцинаций

Гипотеза 2

Дообучение модели эмбедингов на специализированном корпусе (тексты Корана) приводит к повышению качества семантического поиска релевантных документов по сравнению с использованием исходной (базовой) версии модели.

Обоснование: Базовые модели эмбедингов обучаются на универсальных текстовых корпусах и могут недостаточно точно отражать семантические особенности узкоспециализированной предметной области. Дообучение на специализированном корпусе позволяет адаптировать модель к специфической терминологии и структуре текста.

Метод проверки: Для каждой рассматриваемой embedding модели проводится сравнение двух состояний: базового (без дообучения) и дообученного на специализированном корпусе. Сравнение осуществляется при фиксированных параметрах генерации и идентичном наборе тестовых запросов. Оценке подлежат релевантность найденных документов и качество итоговых ответов.

Критерии подтверждения: Гипотеза считается подтвержденной при наблюдении:

- А. увеличение количества корректных ответов
- В. повышения обоснованности ответов
- С. снижения доли нерелевантных источников

1.13 Проектирование общей схемы взаимодействия компонентов

Спроектируем общую схему взаимодействия компонентов системы (рисунок 1.2). Система будет состоять из трех основных частей: iOS-приложение, которое является точкой входа для пользователя, Spring Boot backend, отвечающий за аутентификацию, бизнес-логику и работу с базой данных и FastAPI LLM-сервис, изолированный в отдельный микросервис в соответствии с принципом low coupling, рассмотренным в разделе 2.4. Запросы к AI-ассистенту будут проходить через Spring Boot и проксироваться в LLM-сервис. Каждый из компонентов более подробно будет рассмотрен в пунктах ниже.

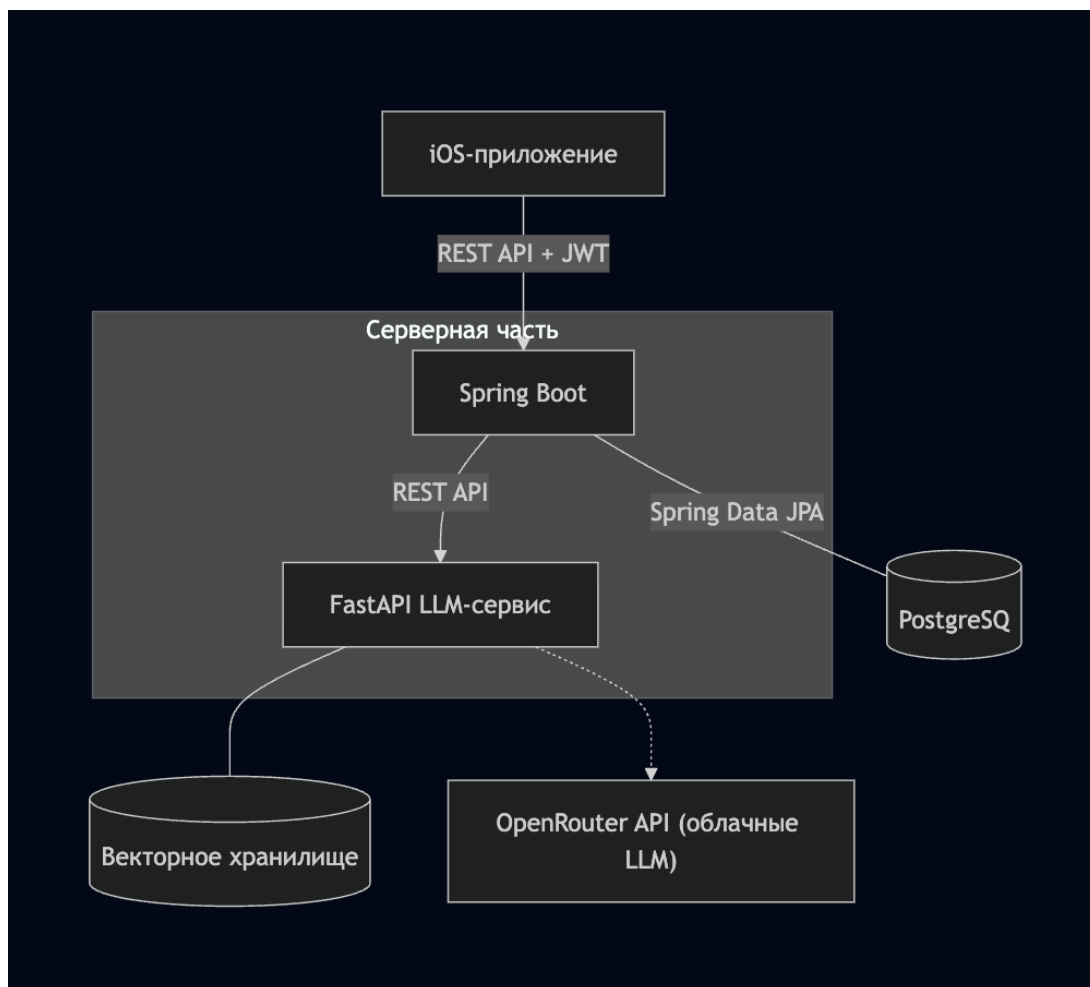


Рис.1.2. Общая схема взаимодействия компонентов системы

1.14 Use Case диаграммы

Диаграмма вариантов использования (Use Case Diagram) описывает функциональные возможности системы с точки зрения пользователя, определяя доступные в приложении действия без рассмотрения вопросов внутренней реализации. Она представляет собой первый этап проектирования, позволяющий зафиксировать все пользовательские сценарии до перехода к разработке диаграмм классов и алгоритмов.

В системе выделены три актора: гость, авторизованный пользователь и система. Гость - это пользователь, который запускает приложение без авторизации. Такой режим позволяет использовать базовые функции приложения без создания учетной записи, что снижает барьер входа для новых пользователей и позволяет ознакомиться с возможностями приложения. В режиме гостя доступен просмотр религиозного контента, а также информация о времени намаза.

Авторизованный пользователь обладает расширенными возможностями. Он наследует все варианты использования гостя и дополнительно получает доступ к функционалу сканера продуктов, AI-ассистента и расширенным настройкам приложения. Использование авторизации обусловлено необходимостью хранения персональных настроек пользователя, а также управлением доступом к расширенным функциям системы (рисунок 1.3).



Рис.1.3. Общая диаграмма вариантов использования системы

Все варианты использования объединены в шесть функциональных блоков:

А. Модуль аутентификации отвечает за идентификацию пользователя и предоставление доступа к персонализированным функциям приложения. Незарегистрированный пользователь может зарегистрироваться, войти в систему или продолжить работу в режиме гостя. После успешной аутентификации пользователь получает доступ к расширенным возможностям приложения (рисунок 1.4).

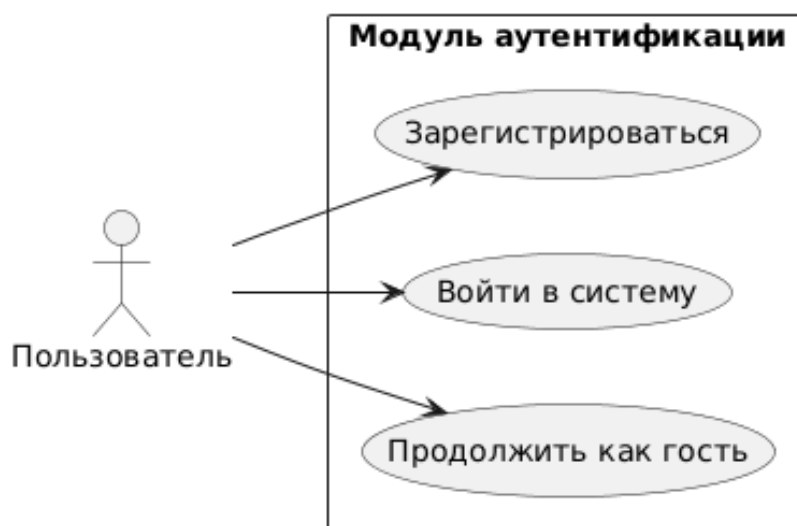


Рис.1.4. Use Case-диаграмма модуля аутентификации

В. Модуль сканера продуктов предназначен для анализа состава пищевых продуктов и определения их соответствия требованиям халяль. Основной пользовательский сценарий "Проверить состав продукта" доступен только авторизованным пользователям и может быть запущен через камеру устройства или из галереи изображений. Распознавание текста и классификация ингредиентов являются обязательными этапами обработки данных и включены в основной сценарий через отношение «include». Ручной ввод состава реализован как альтернативный путь выполнения и обозначен связью «extend» (рисунок 1.5).

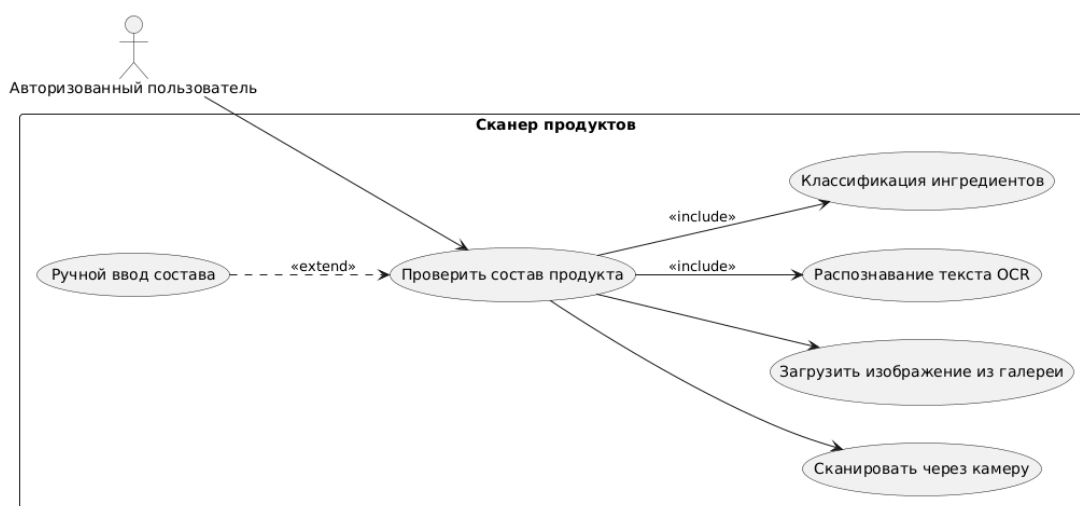


Рис.1.5. Use Case-диаграмма модуля "Сканер продуктов"

С. Модуль AI-ассистента предоставляет пользователю возможность задавать вопросы, связанные с халяльностью продуктов и ингредиентов. При каждом обращении к ассистенту автоматически формируется контекст

для генерации ответа на основе механизма retrieval-augmented generation (RAG), что обозначено отношением «include». Использование удаленной языковой модели через сервис OpenRouter реализовано как расширение основного сценария («extend») и активируется только в случае, если пользователь указал API-ключ в настройках приложения (рисунок 1.6).

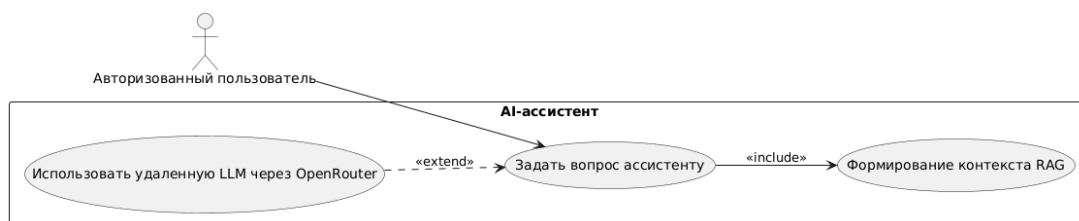


Рис.1.6. Use Case-диаграмма модуля AI-ассистента

- D. Модуль Коран предоставляет пользователю возможность просматривать список сур и читать выбранные суры и аяты. Данный функционал доступен как гостю, так и авторизованному пользователю.
- E. Модуль времени намаза отображает расписание пяти обязательных намазов на текущий день, рассчитанное на основе геолокации пользователя. Система автоматически отправляет уведомления в назначенное время. Пользователь может дополнительно настроить уведомления для каждого намаза, что реализовано как расширение основного сценария через связь «extend».
- F. Модуль настроек предназначен для управления параметрами приложения. В режиме гостя пользователю доступна возможность перейти к авторизации. После входа в систему открываются дополнительные функции: управление API-ключом для удаленных моделей, выбор используемой языковой модели (LLM), настройка параметра генерации max_tokens, а также выход из аккаунта (рисунок 1.7).

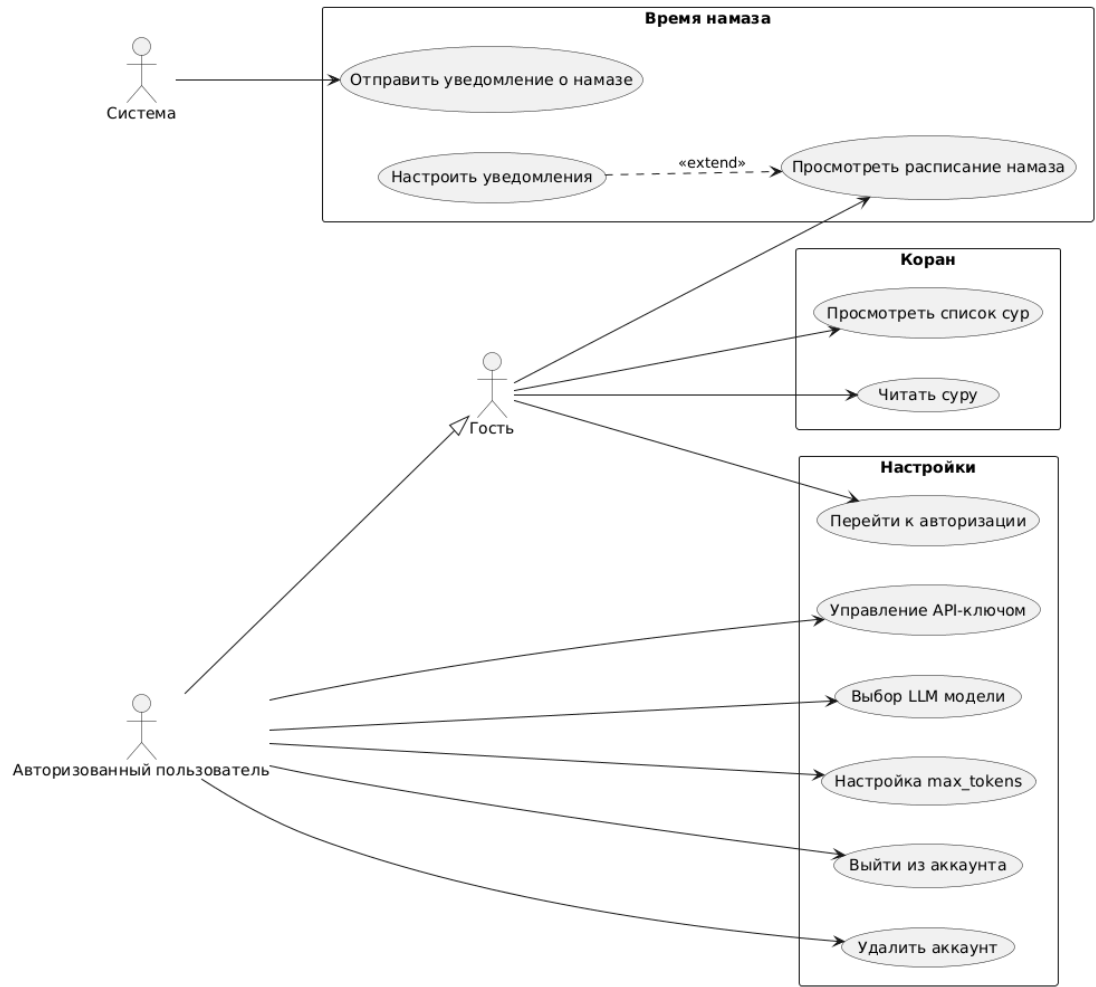


Рис.1.7. Use Case-диаграмма модулей "Коран "Время намаза"и "Настройки"

1.15 Схема базы данных

В разделе 1.6 была выбрала PostgreSQL качестве СУБД. Планируется создание двух таблиц: для хранения пользовательских аккаунтов (рисунок 1.8) и текстов Корана (рисунок 1.9). Общая схема базы данных показана на рисунке 1.2.

users			
bigserial	id	PK	
varchar_50	username		UNIQUE NOT NULL
varchar	email		UNIQUE NOT NULL
varchar	password		NOT NULL (bcrypt)
timestamp	created_at		NOT NULL
timestamp	updated_at		
boolean	enabled		DEFAULT true

Рис.1.8. Таблица users

verses			
bigserial	id	PK	
int	sura_index		NOT NULL
varchar_200	sura_title		NOT NULL
varchar_200	sura_subtitle		
int	verse_number		
text	text		NOT NULL

Рис.1.9. Таблица verses

Для LLM-сервиса планируется использовать файловое векторное хранилище - бинарный файл формата PyTorch (рисунок 1.10). Он будет содержать два объекта: список документов (фрагменты аятов с метаданными) и матрицу эмбедингов размерностью N на D , где N - это количество проиндексированных фрагментов, а D - размерность векторного представления выбранной модели эмбедингов. Значение D определяется моделью: например, для paraphrase-multilingual-MiniLM-L12-v2 оно составляет 384, для более тяжелых моделей 768 или 1024. Окончательный

выбор модели планируется произвести по результатам экспериментов в главе о разработке. Поиск векторному хранилищу будет выполняться через косинусное сходство, поэтому он намеренно выносится за пределы PostgreSQL - обновление индекса не должно затрагивать основную базу данных.

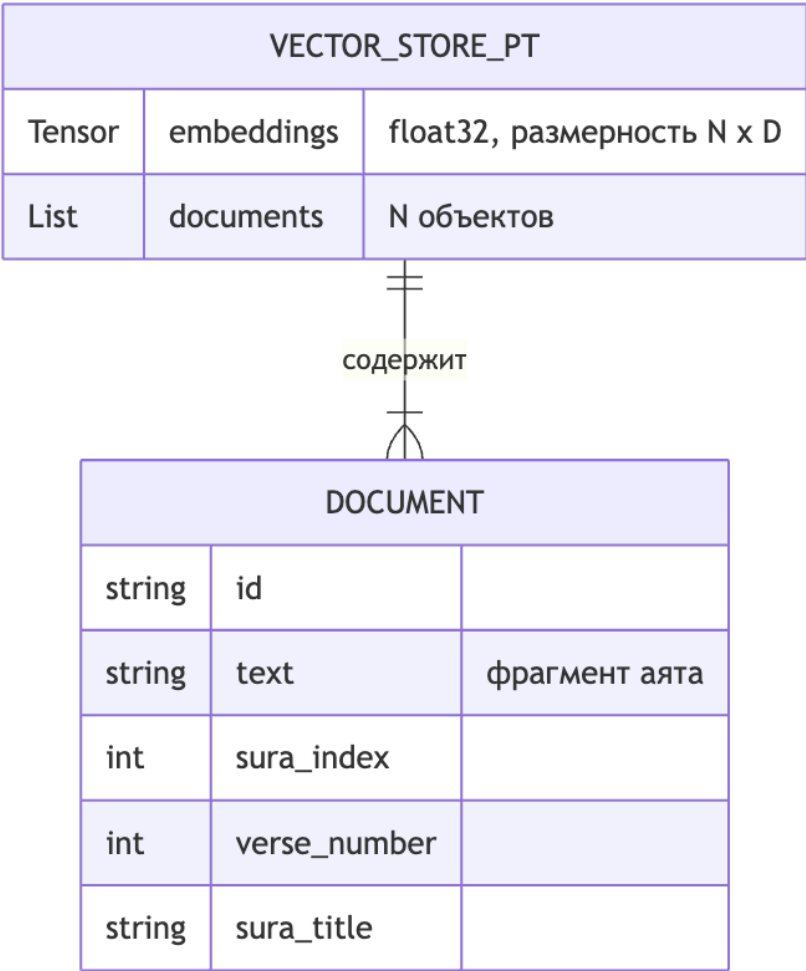


Рис.1.10. Структура векторного хранилища

1.16 Алгоритмы обработки данных

Рассмотрим 3 основных алгоритма, каждый из которых соответствует отдельному функциональному модулю приложения:

- А. Алгоритм обработки чат-запроса с использованием RAG. Он описывает процесс от получения пользовательского сообщения до формирования ответа.
- В. Алгоритм анализа состава продукта. Он описывает процесс от выбора источника изображения до классификации каждого ингредиента по ка-

тегориям "халяль" "харам" и "сомнительный" с формированием итогового вердикта о продукте.

- С. Алгоритм расчета времени намаза. Он рассчитывает время намаза на основе астрономических формул, координат пользователя и выбранного метода расчета.

1.16.1 Алгоритм обработки чат-запроса с использованием RAG

Алгоритм обработки чат-запроса выполняется на стороне LLM-сервиса, реализованного на языке Python с использованием веб-фреймворка FastAPI. Запросы поступают от backend-сервиса, который проксирует их в соответствии с принципом слабой связности (low coupling), рассмотренным в разделе 2.4. На вход алгоритм принимает следующие параметры: история диалога в формате последовательности сообщений с указанием ролей участников, максимальное количество токенов в генерируемом ответе, идентификатор языковой модели и API-ключ пользователя для доступа к OpenRouter.

Сам алгоритм состоит из нескольких этапов.

Первый этап - это предобработка и извлечение поискового запроса. Система последовательно проходит по всем сообщениям в обратном хронологическом порядке и выделяет первое сообщение, чей отправитель имеет роль пользователя. Выделенное сообщение представляет собой основной поисковый запрос. История диалога сохраняется в полном объеме для контекстуализации ответа от LLM.

На втором этапе семантический поиск в базе знаний с помощью RAG. Извлеченный поисковый запрос векторизируется. Данное преобразование выполняется таким образом, чтобы семантически связанные тексты получали близкие векторные представления. Полученный вектор запроса сравнивается со всеми векторными представлениями документов, хранящихся в базе знаний (аятов Корана), посредством вычисления функции косинусного сходства.

$$\cos(\theta) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \cdot |\vec{d}|} \quad (1.1)$$

На основе вычислений выбираются 3 наиболее релевантных результата. Для каждого выбранного документа сохраняются данные: номер суры, номер аята, текст аята, значение функции сходства.

Далее происходит формирование входных данных для языковой модели (см. рисунок 1.11). Выбранные источники объединяются в текстовый блок, где каждый

источник содержит: номер суры, номер аята, полный текст аята. К сформированному блоку добавляется системный промпт, содержащий инструкции для языковой модели:

- А. базировать ответы исключительно на предоставленных источниках.
- В. приводить в ответе конкретные ссылки на номера сур и аятов.
- С. воздерживаться от ответа при отсутствии релевантной информации в контексте.

Подготовленная последовательность сообщений отправляется в удаленную языковую модель через API OpenRouter. Когда языковая модель возвращает результат он отображается у пользователя в чате и добавляется в историю сообщений для обогащения контекста.

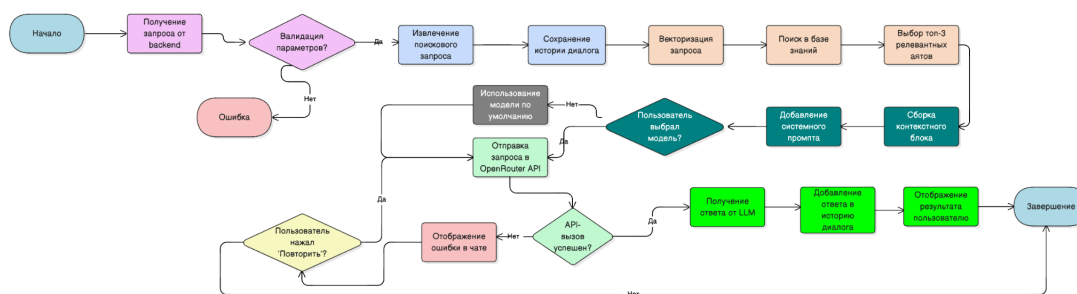


Рис.1.11. Блок-схема алгоритма анализа состава продукта

1.16.2 Алгоритм анализа состава продукта

Алгоритм анализа состава планируется реализовать полностью на стороне клиента. База ингредиентов будет храниться локально в приложениях в CSV-файлах и загружаться в память при первом запуске.

Алгоритм предполагает два возможных пути получения входных данных: фотографирование упаковки (или выбор из галереи) и ручной ввод текста состава. В первом случае перед анализом выполняется этап распознавания текста - изображение обрабатывается OCR-движком, результаты с нескольких фотографий объединяются в одну строку. При ручном вводе текст поступает на анализ напрямую.

Поскольку качество распознавания зависит от освещения, угла съемки и шрифта на упаковке, планируется предусмотреть информационный дисклеймер, который будет отображаться пользователю вместе с результатами анализа. Он будет содержать предупреждение о том, что результат основан на автоматически

распознанном тексте, и рекомендацию самостоятельно убедиться в корректности считанного состава перед принятием решения о покупке или употреблении.

Анализ текста планируется разбить на два независимых шага, которые выполняются последовательно.

Первым шагом выполняется поиск по Е-кодам. Из текста извлекаются все токены вида "Е"+ цифры (например, E471, E120). Каждый найденный код сверяется с базой ингредиентов по точному совпадению. Найденные ингредиенты исключаются из дальнейшего поиска во втором шаге.

Второй шаг - это поиск по названиям. Для каждого ингредиента из базы, не найденного на первом шаге, выполняется нечеткое сопоставление с текстом по русскому и английскому названиям. Каждому совпадению присваивается оценка (score) в диапазоне от 0 до 1. Точное совпадение названия ингредиента с текстом по границам слов дает $\text{score} = 1.0$. Для частичных совпадений вычисляется matchRatio - это доля значимых слов названия (длиной от 4 символов), найденных в тексте: например, если название содержит 3 значимых слова и 2 из них обнаружены в тексте, $\text{matchRatio} = 0.67$. Итоговый score для частичных совпадений вычисляется как $\text{matchRatio} * 0.85$. Коэффициент 0.85 намеренно снижает оценку относительно точного совпадения, чтобы при финальной фильтрации точные совпадения всегда имели приоритет над частичными. Кандидаты с оценкой ниже 0.7 отбрасываются. Затем из оставшихся кандидатов удаляются дублирующие подмножества. Например, если в тексте найдено и "молоко" и "сухое молоко" остается более специфичное совпадение.

После объединения результатов обоих шагов выставляется итоговый статус продукта по приоритетной схеме: наличие хотя бы одного харам-ингредиента делает продукт харамом вне зависимости от остальных, при отсутствии харам, но наличии сомнительных - статус "сомнительный если все найденные ингредиенты халяль" - статус "халяль если не удалось распознать, то статус "неизвестно"(рисунок 1.12).

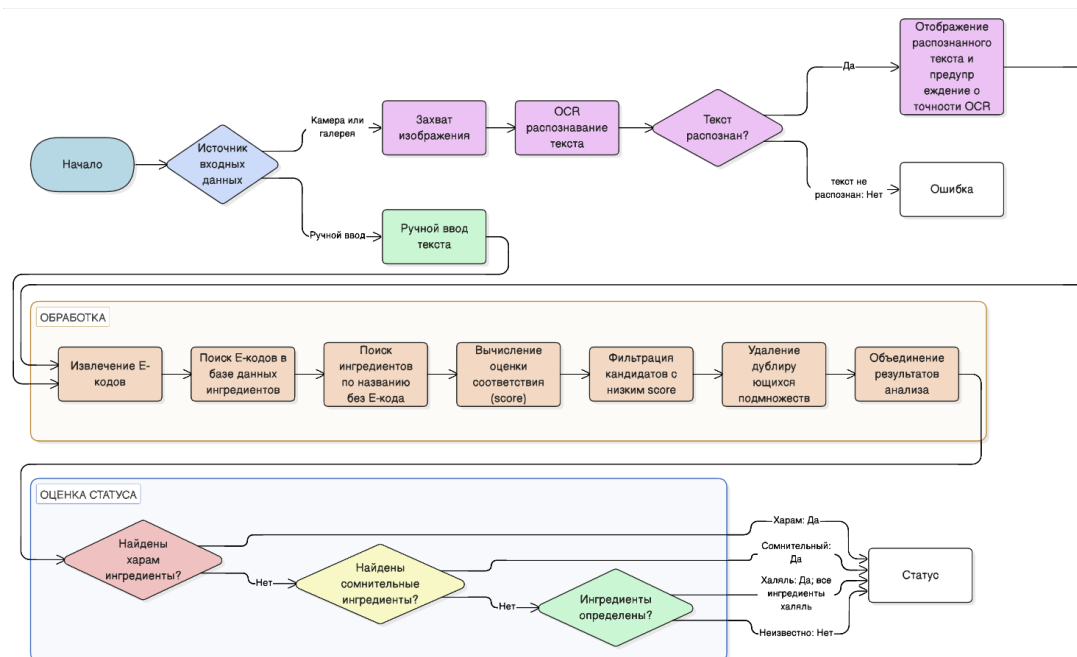


Рис.1.12. Блок-схема алгоритма анализа состава продукта

1.16.3 Алгоритм расчета времени намаза

Алгоритм расчета времени намаза реализуется полностью на стороне клиента (iOS) без обращения к серверу. В качестве вычислительного ядра используется библиотека Adhan Swift. Такой подход обеспечивает возможность работы в офлайн-режиме, снижает задержки при получении результатов и исключает зависимость от внешних сетевых сервисов.

В основе алгоритма лежат три группы входных данных: географические координаты пользователя (широта и долгота), дата (год, месяц, день), а также параметры расчета, задаваемые пользователем.

Параметры расчета включают несколько важных аспектов. Во-первых, это метод расчета времени намаза, представляющий собой набор правил и астрономических параметров, используемых в различных регионах мира. Разные исламские организации (например, в Саудовской Аравии, Турции, Европе) используют разные углы положения Солнца для определения начала некоторых молитв, что приводит к небольшим различиям во времени. Выбор метода позволяет адаптировать расчет под региональные стандарты.

Во-вторых, учитывается мазхаб - правовая школа в исламе, которая влияет на определение времени наступления молитвы Аср. В частности, различие заключается в длине тени объекта: в шафиитской традиции Аср начинается, когда длина

тени равна длине самого объекта, а в ханафитской - когда тень в два раза длиннее объекта. Это приводит к различию во времени начала данной молитвы.

Дополнительно могут задаваться пользовательские параметры для молитв Фаджр и Иша, которые определяются по углу положения Солнца относительно горизонта. Фаджр - это утренняя молитва, начинающаяся до восхода Солнца, а Иша - ночная молитва, наступающая после наступления темноты. Время этих молитв зависит от того, на сколько градусов Солнце опускается ниже горизонта (например, 18° , 15° и т.д.). Пользователь может задать эти значения вручную для более точной настройки расчета.

Поскольку точность вычислений напрямую зависит от корректности геолокации и системного времени устройства, в пользовательском интерфейсе предусмотрены состояния, информирующие о невозможности выполнения расчета. В частности, при отсутствии доступа к геолокации или невозможности определить координаты пользователю отображается соответствующее сообщение, а расчет не производится.

Алгоритм включает несколько последовательных этапов. На первом этапе выполняется получение и валидация местоположения пользователя. Приложение запрашивает разрешение на доступ к геолокации, после согласия пользователя мы получаем текущие координаты с точностью, достаточной для расчета времени намаза. При отсутствии координат дальнейшие вычисления не выполняются.

Далее формируются параметры расчета. В зависимости от выбранного пользователем метода задаются соответствующие параметры. Если пользователь задал собственные значения углов для Фаджра и Иши, они имеют приоритет и подменяют стандартные параметры метода.

На следующем этапе осуществляется непосредственный расчет времени молитв. На основе календарной даты формируются необходимые компоненты, после чего библиотеке передаются координаты, дата и параметры. В результате вычисляются временные метки для шести событий дня: Фаджр, Шурук, Зухр, Аср, Магриб и Иша. В случае невозможности выполнения вычислений результат считается недоступным.

Заключительный этап включает постобработку полученных данных и определение ближайшей молитвы. Результаты преобразуются во внутреннюю модель приложения, после чего определяется ближайшее предстоящее событие. Для этого выбирается первое время молитвы, превышающее текущий момент. Если все мо-

литвы текущего дня уже прошли, система фиксирует соответствующее состояние (рисунок 1.13).

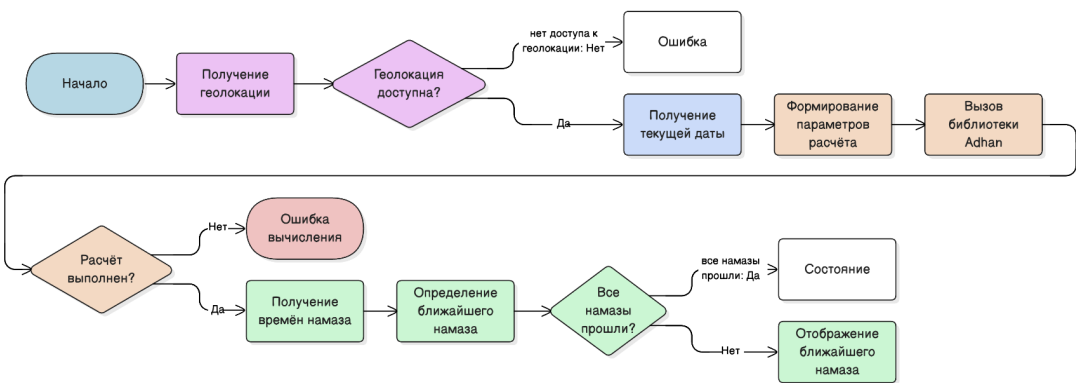


Рис.1.13. Блок-схема алгоритма расчета времени намаза

1.17 Проектирование REST API

Для обеспечения взаимодействия между компонентами системы используется REST API. Данный интерфейс определяет контракты обмена данными между iOS-приложением, backend-сервером и LLM-сервисом (таблица 1.7).

Таблица 1.7

REST API эндпоинты

Сервис	Метод	Endpoint	Request	Response
Auth	POST	/api/auth/register	username, email, password	JWT токен + данные пользователя
Auth	POST	/api/auth/login	username/email, password	JWT токен + данные пользователя
Auth	POST	/api/auth/refresh	refresh token	новый JWT токен
Quran	GET	/api/verse-of-the-day	-	аят (сура, текст, номер)
Chat	POST	/api/chat	messages[], model, api_key	ответ модели
Chat	GET	/api/models	-	список моделей
LLM Service	GET	/llm/health	-	status, готовность RAG/LLM
LLM Service	POST	/llm/chat	messages[], параметры	ответ модели

Рассмотрим каждый endpoint более подробно.

А. /api/auth/register (POST)

Endpoint предназначен для регистрации нового пользователя в системе. В теле запроса передаются имя пользователя, адрес электронной почты и пароль. В случае успешной регистрации сервер создает учетную запись, формирует JWT-токен доступа и возвращает его вместе с основными данными пользователя.

B. /api/auth/login (POST)

Endpoint используется для аутентификации ранее зарегистрированного пользователя. В запросе передаются учетные данные пользователя: имя пользователя или адрес электронной почты, а также пароль. В ответ сервер возвращает JWT-токен доступа и сведения о пользователе.

C. /api/auth/refresh (POST)

Endpoint предназначен для обновления access token без повторного ввода логина и пароля. В теле запроса передается refresh token, на основе которого сервер формирует новый JWT-токен доступа. Использование данного endpoint позволяет поддерживать пользовательскую сессию и повышает удобство использования приложения.

D. /api/verse-of-the-day (GET)

Endpoint используется для получения случайного аята дня. Запрос не требует передачи параметров. В ответ сервер возвращает информацию об аяте, включая номер суры, номер аята и текст.

E. /api/chat (POST)

Endpoint является основным входом для работы чат со стороны клиента. В теле запроса передается история сообщений, а также дополнительные параметры генерации, такие как выбранная модель и API-ключ пользователя для удаленной LLM. Backend-сервис принимает этот запрос, выполняет необходимую обработку и проксирует его в LLM-service. В ответ клиент получает сформированный текст ответа модели. Таким образом, данный endpoint выступает связующим звеном между iOS-приложением и LLM-service.

F. /api/models (GET)

Endpoint предназначен для получения списка доступных языковых моделей, которые могут быть использованы в системе. Запрос не требует передачи параметров. В ответ возвращается список моделей, доступных для выбора пользователем в настройках приложения.

G. /llm/health (GET)

Endpoint используется для проверки состояния LLM-service. В ответ возвращается информация о доступности сервиса, а также о готовности его ключевых компонентов, в частности RAG-подсистемы и модуля взаимодействия с языковой моделью.

Н. /llm/chat (POST)

Endpoint является основным для обработки чат-запросов в LLM-service. В теле запроса передается история сообщений пользователя, а также параметры генерации ответа, включая API-ключ, максимальное количество токенов и название удаленной модели. После получения запроса сервис выполняет извлечение поискового запроса, поиск релевантных фрагментов Корана с использованием RAG и обращение к внешней языковой модели через OpenRouter. В ответ возвращается готовый текст.

1.18 Архитектура iOS-приложения

1.18.1 Общее описание архитектурного подхода

iOS-приложение будет разработано с модульной многоуровневой архитектурой. Реализация будет выполнена на языке Swift с применением декларативного фреймворка SwiftUI.

Предварительно разделим код на следующие слои (см. рисунок 1.14):

- А. Слой представления (Presentation Layer) - отвечает за отображение пользовательского интерфейса и обработку действий пользователя.
- В. Слой координации навигации (Navigation Coordination Layer) - управляет переходами между экранами и навигационными сценариями приложения.
- С. Слой бизнес-логики (Business Logic Layer) - содержит сервисы, реализующие прикладную логику и координирующие обработку данных.
- Д. Слой данных (Data Layer) - отвечает за взаимодействие с backend-сервисами, локальными хранилищами и другими источниками данных.

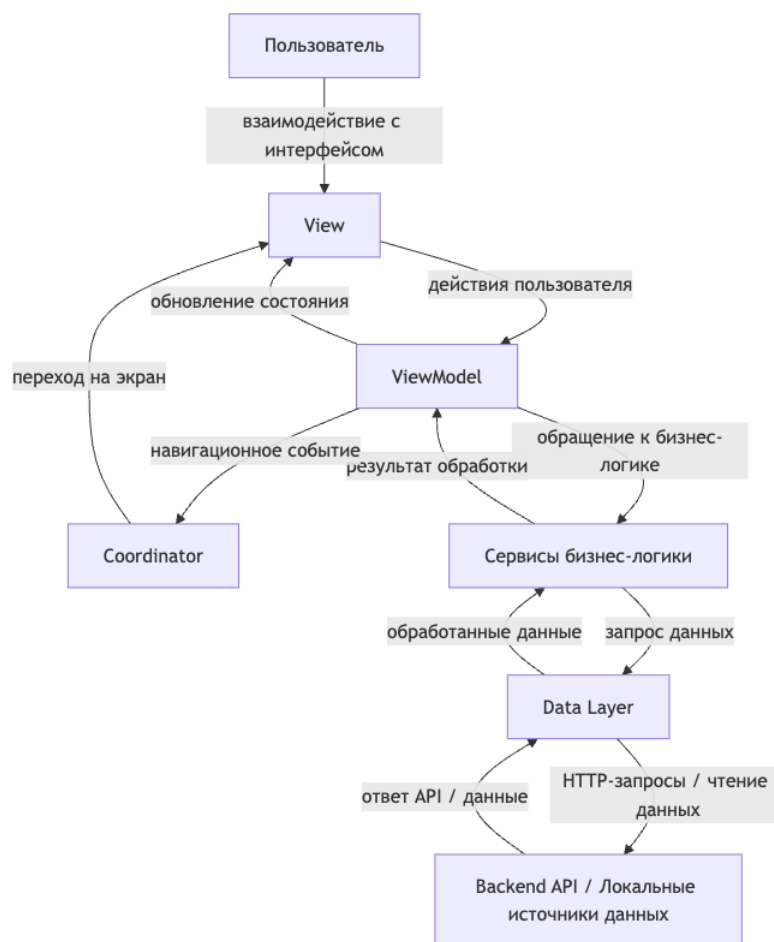


Рис.1.14. Архитектура iOS-приложения

1.18.2 Управление навигацией через паттерн Coordinator

Для управления навигацией будет использован паттерн Coordinator при котором логика переходов выносится из представлений и инкапсулируется в отдельном объекте - координаторе.

Глобальный координатор будет хранить стек активных экранов и управлять переходами между ними. Для каждой основной области приложения (чат, главный экран, настройки) будут созданы специализированные координаторы, которые управляют навигацией внутри этой области.

Данный подход позволяет легко добавлять новые сценарии навигации и делать контроллеры представления легковесными и сосредоточенными только на представлении данных и реагировании на ввод пользователя. Поскольку кон-

троллеры представлений не будут обрабатывать навигацию, становится проще их логику изолировать.

1.18.3 Внедрение зависимостей

Все сервисы и компоненты приложения создаются и управляются с использованием контейнера зависимостей (DependencyContainer). Контейнер зависимостей - это инструмент, позволяющий управлять, регистрировать и разрешать зависимости в приложении. Это не только упрощает процесс разработки и тестирования, но и повышает модульность и повторное использование кода.

Для создания экранов применяется фабричный подход (ScreenFactory). При необходимости отображения нового экрана приложение обращается к фабрике, которая формирует экземпляр экрана и внедряет в него все требуемые зависимости.

Основным преимуществом данного подхода является: централизованное управление зависимостями, возможность замены реальных сервисов на тестовые реализации(моки и стабы), снижение связности между компонентами системы.

1.18.4 Сервисы

Бизнес-логика будет реализована с помощью сервисов. Каждый сервис отвечает за конкретную область: управление чатом, аутентификация, работа с Кораном, геолокация, расписание молитв и т. д

Сервисы будут определены через протоколы (интерфейсы), что позволит легко менять их реализацию без изменения существующего кода. Реактивность будет достигнута через макрос @Observable, когда состояние сервиса меняется, все связанные экраны обновляются автоматически.

1.18.5 Экраны и их состояние

Для организации кода каждого экрана используется архитектурный паттерн MVVM (Model-View-ViewModel), предполагающий разделение на три компонента.

View (представление) - это пользовательский интерфейс, реализованный с использованием SwiftUI. View отвечает исключительно за отображение данных и передачу пользовательских действий во ViewModel. Представление не содержит бизнес-логики и не определяет источник данных.

ViewModel (модель представления) - компонент, связанный с View и отвечающий за управление состоянием экрана. ViewModel выполняет следующие

функции: управление состоянием, обработку пользовательских действий, а также преобразование данных, получаемых из сервисов, в формат, удобный для отображения.

Model (модель) - слой, представленный сервисами, содержащими бизнес-логику и данные приложения. Model не зависит от View и ViewModel и предоставляет методы для работы с состоянием (рисунок 1.15).

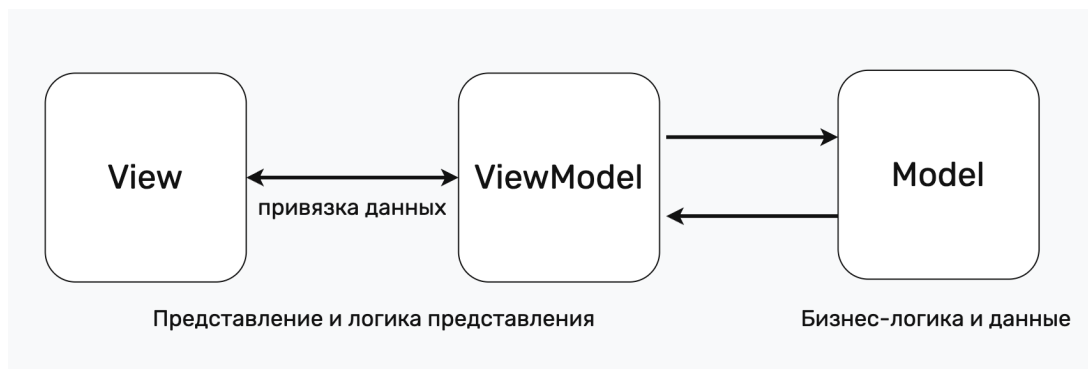


Рис.1.15. MVVM

1.18.6 Взаимодействие с backend

Взаимодействие приложения с backend-сервисом осуществляется посредством HTTP-запросов. Все сетевые операции выполняются асинхронно, что позволяет избежать блокировки пользовательского интерфейса и обеспечить корректную работу приложения.

Для управления сетевыми запросами выделяется NetworkLayer. Он представляет собой абстракцию над механизмами HTTP-коммуникации. Данный слой инкапсулирует следующие функции: формирование запросов (URL, заголовки, параметры), отправка запросов на backend-сервис, обработка ответов и ошибок, преобразование полученных данных во внутренний формат приложения. Это позволяет изолировать детали реализации сетевого взаимодействия от бизнес-логики. В результате сервисы не зависят от конкретного способа коммуникации с backend-сервисом. Это обеспечивает гибкость архитектуры: при необходимости изменения протокола взаимодействия (например, перехода на WebSocket) изменения затрагивают только NetworkLayer, не влияя на остальные компоненты системы.

1.19 Архитектура Backend

Backend-сервер будет разработан на Java с использованием фреймворка Spring Boot. Он отвечает за управление пользователями, хранение данных и предоставляет REST API, с которым взаимодействует iOS-приложение.

Архитектура backend-сервера построена по трехслойной модели: слой API, слой бизнес-логики и слой данных.

1.19.1 Трехслойная архитектура

Слой API определяет HTTP-эндпоинты, к которым обращается iOS-приложение. Каждый эндпоинт реализует конкретную функциональность, например: аутентификацию, получение профиля пользователя, работу с чатом или поиск ингредиентов. Данный слой не содержит бизнес-логики. Его основная задача - принять запрос, передать его в слой бизнес-логики и вернуть сформированный ответ клиенту.

В слой бизнес-логики реализуется основная логика приложения. Здесь выполняется валидация данных, проверка прав доступа и применение бизнес-правил. Этот слой не зависит от способа взаимодействия с клиентом и работает с абстрактными данными и моделями.

Слой данных отвечает за взаимодействие с базой данных PostgreSQL. Для работы с данными используется Spring Data JPA, которая позволяет оперировать объектами вместо написания SQL-запросов вручную. ORM автоматически преобразует операции над объектами в соответствующие SQL-запросы.

1.19.2 Аутентификация через JWT

Для аутентификации пользователей используется механизм JSON Web Token (JWT). Пользователь выполняет вход в систему, после чего backend-сервер генерирует JWT-токен. Полученный токен сохраняется на стороне клиента и передается в каждом последующем запросе. Backend проверяет валидность токена, включая его подпись и срок действия. JWT содержит информацию о пользователе и цифровую подпись, что позволяет выполнять проверку подлинности без обращения к базе данных (рисунок 1.16).

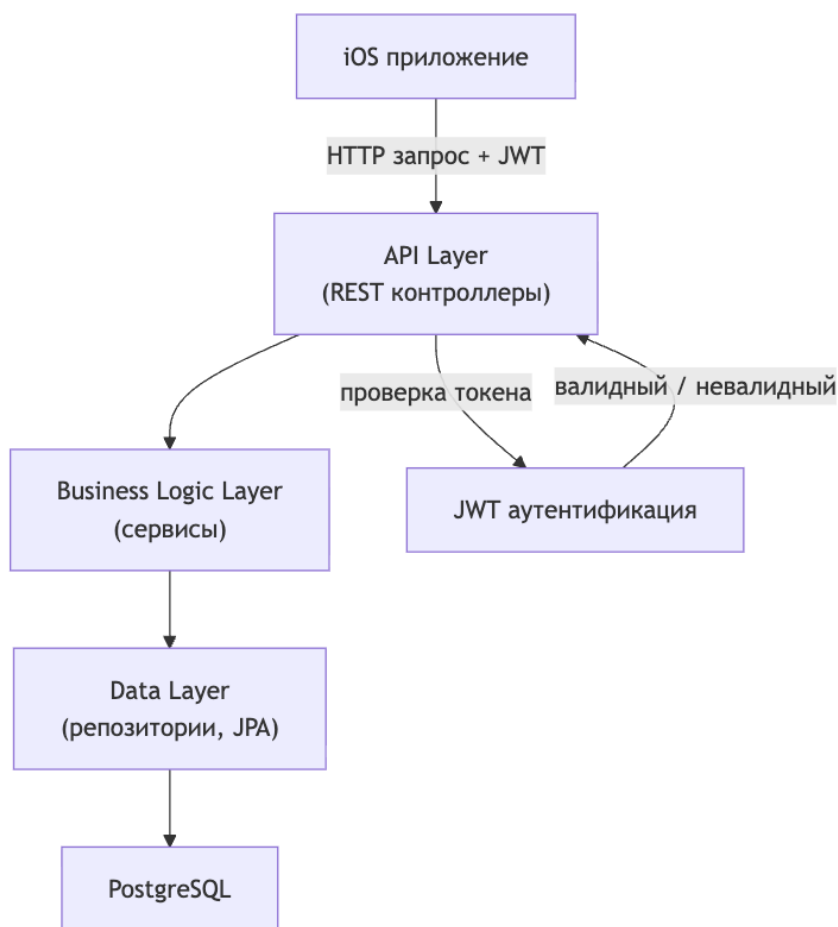


Рис.1.16. Архитектура backend

1.20 Архитектура LLM-service

LLM-service представляет собой отдельный микросервис, реализованный на Python, предназначенный для обработки пользовательских чат-запросов. Сервис принимает текстовый запрос, выполняет поиск релевантных фрагментов Корана и формирует ответ с использованием языковой модели.

1.20.1 REST API слой

Слой REST API реализован с использованием фреймворка FastAPI и определяет HTTP-эндпоинты, через которые основной backend-сервер взаимодействует с LLM-service. Каждый эндпоинт является точкой входа для обработки определен-

ного типа запросов и отвечает за входные данные и передачу их в последующие компоненты системы.

1.20.2 RAG слой

RAG (Retrieval-Augmented Generation) слой отвечает за поиск релевантного контекста в базе знаний. Данный слой включает следующие компоненты:

- A. Embeddings - выполняет преобразование текстового запроса в векторное представление.
- B. Vector Store - хранит векторные представления фрагментов Корана.
- C. Retriever - осуществляет поиск наиболее релевантных фрагментов на основе меры сходства.

1.20.3 Слой интеграции с LLM

Слой интеграции с языковой моделью обеспечивает взаимодействие с внешним сервисом OpenRouter. В рамках данного слоя формируется запрос, включающий пользовательский ввод и найденный контекст, после чего выполняется вызов языковой модели и получение сгенерированного ответа.

1.21 Макеты интерфейсов

На этапе проектирования были разработаны макеты основных экранов приложения. На рисунке 1.17 представлен макет главного экрана с виджетом времени намаза. Настройки уведомлений о времени намаза показаны на рисунке 1.18. Макет экрана сканирования состава продуктов приведен на рисунке 1.19. Экран чтения Корана представлен на рисунке 1.20. На рисунке 1.21 показан экран настроек приложения, на рисунке 1.22 - экран AI-чата, на рисунке 1.23 - экраны авторизации и регистрации.



Рис.1.17. Макет главного экрана

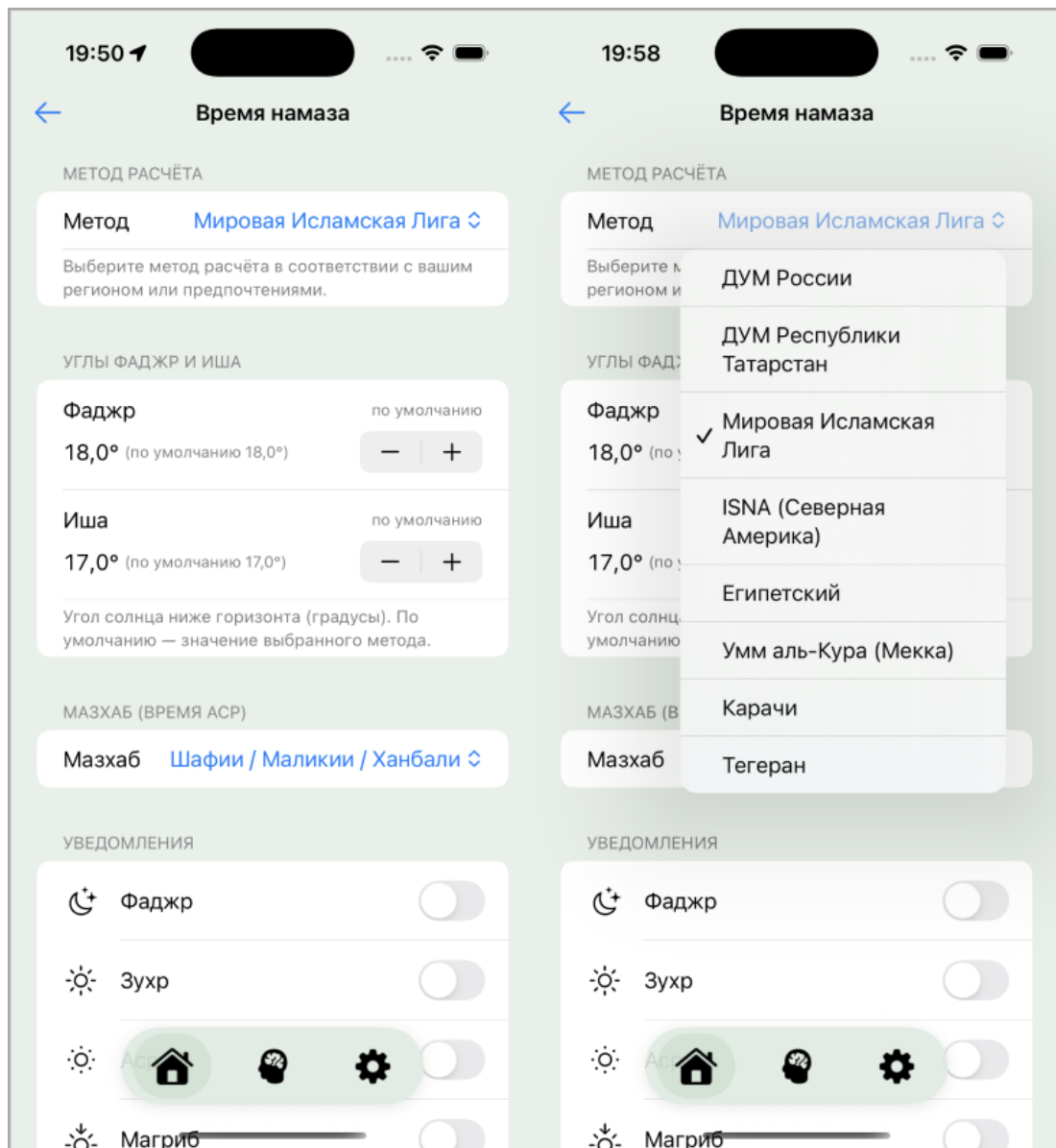


Рис.1.18. Макет экрана настроек времени намаза

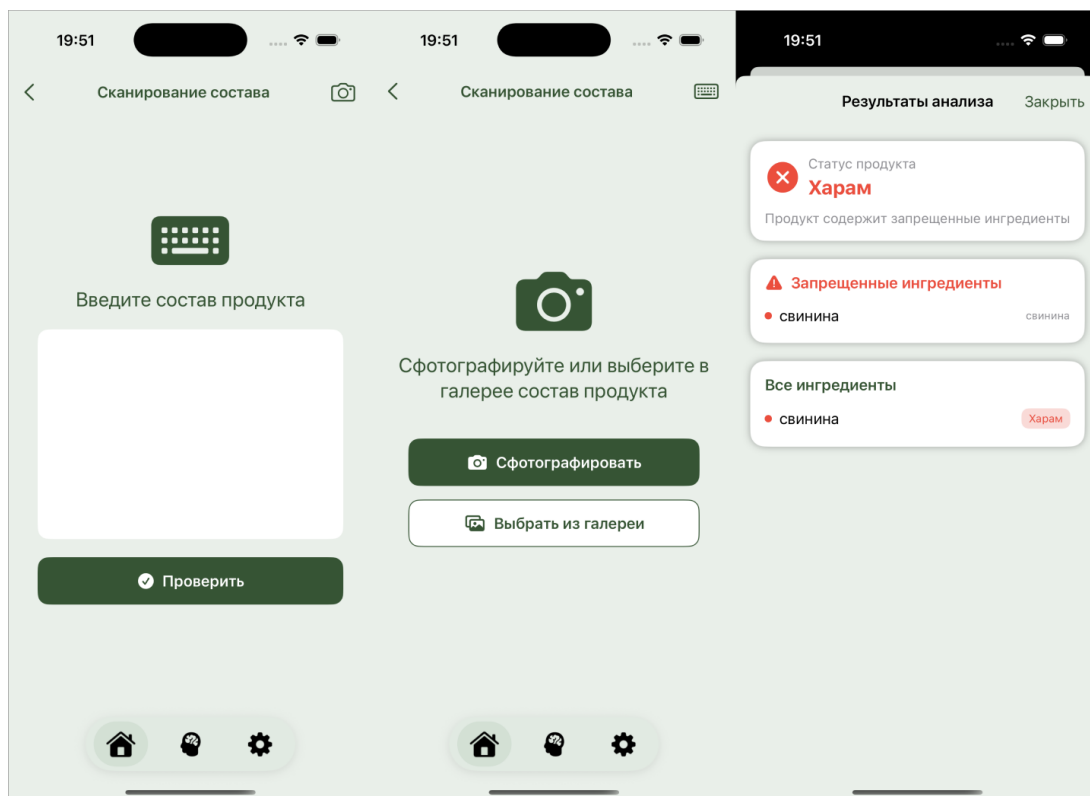


Рис.1.19. Макет экрана сканирования состава продуктов

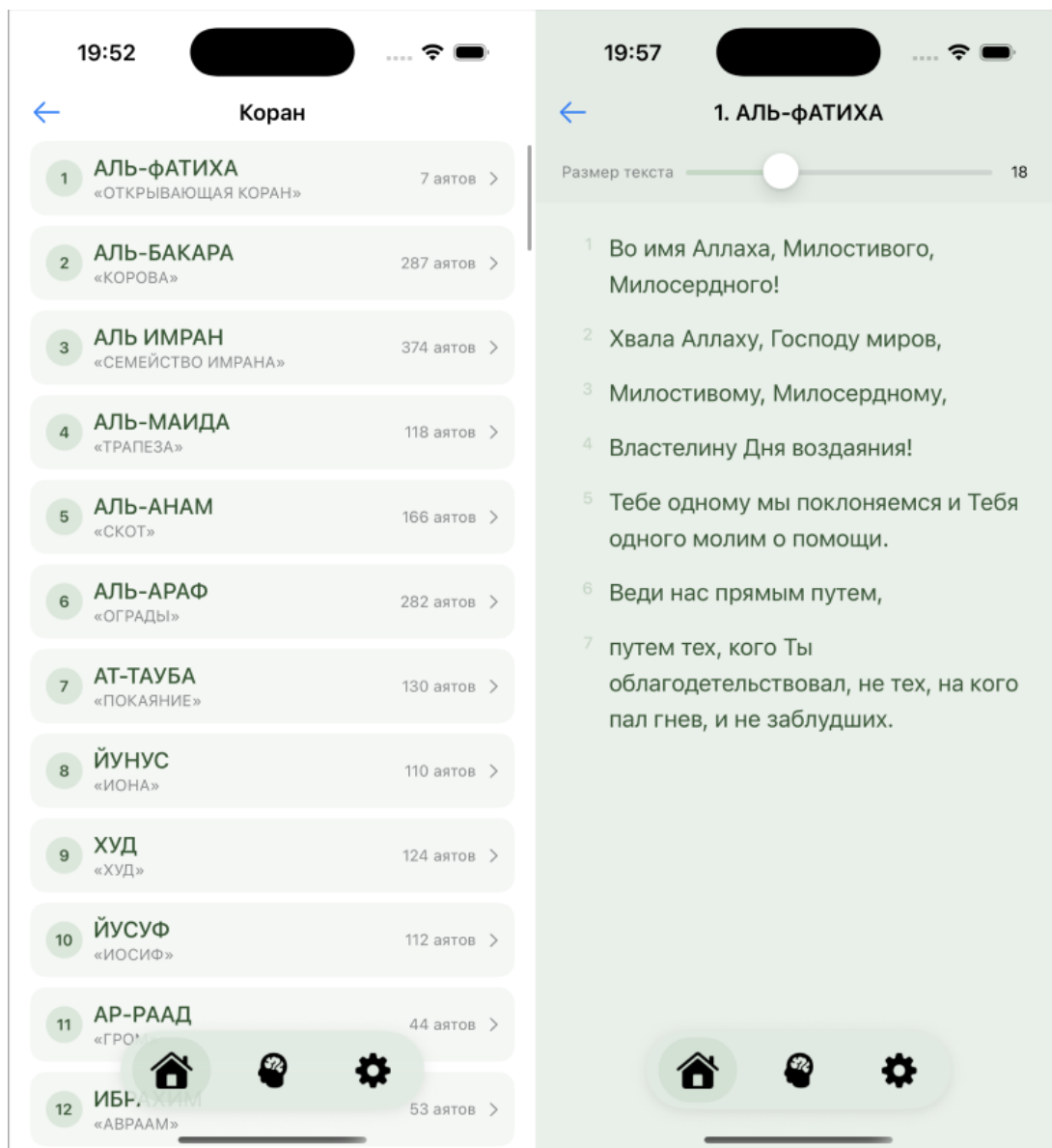


Рис.1.20. Макет экрана чтения Корана

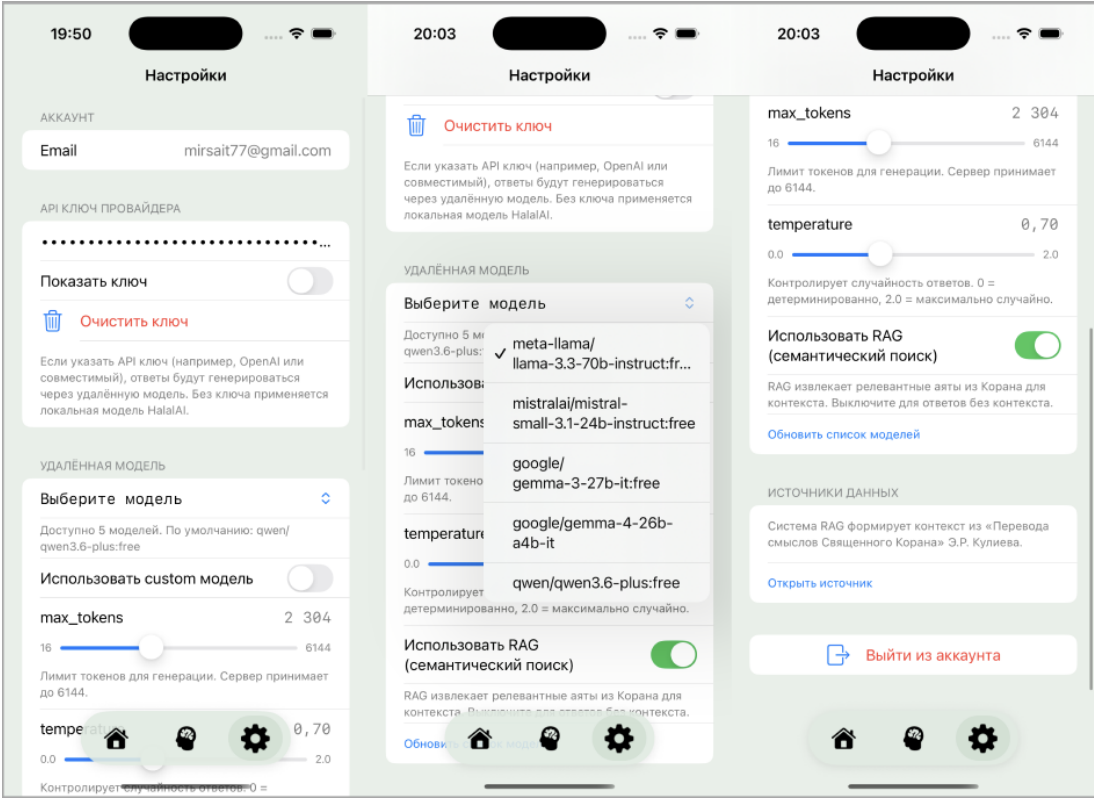


Рис.1.21. Макет экрана настроек

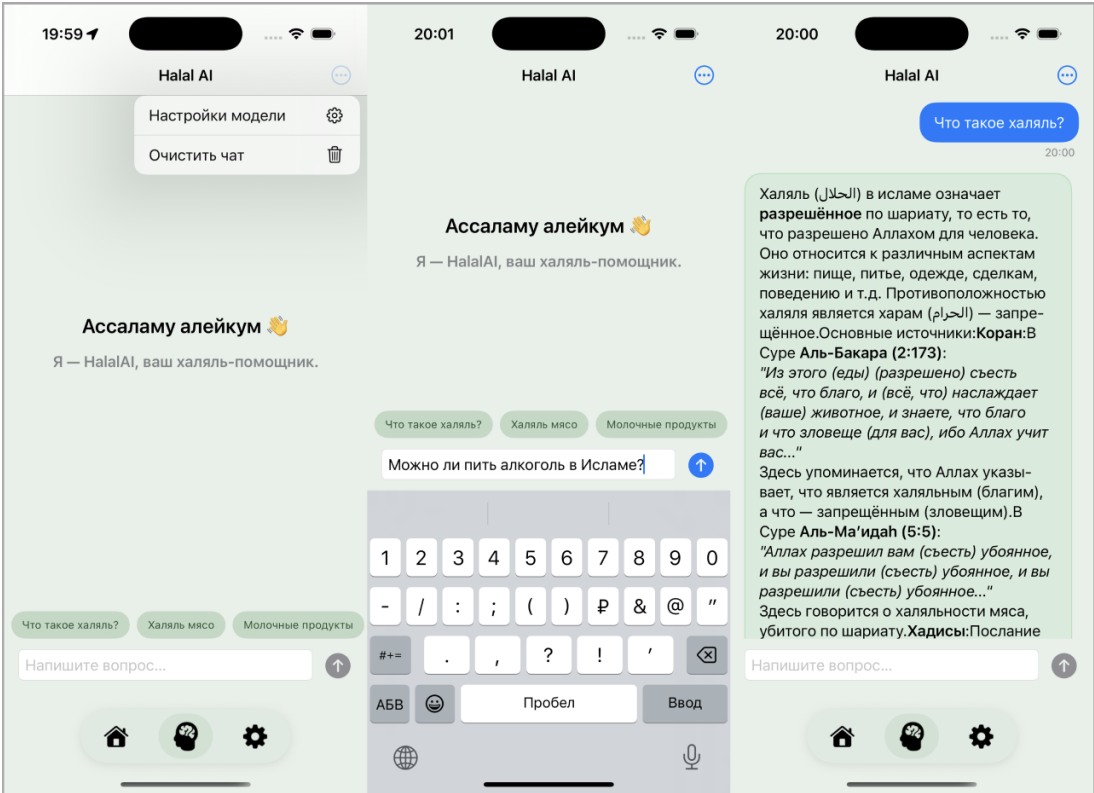


Рис.1.22. Макет экрана с AI-чатом

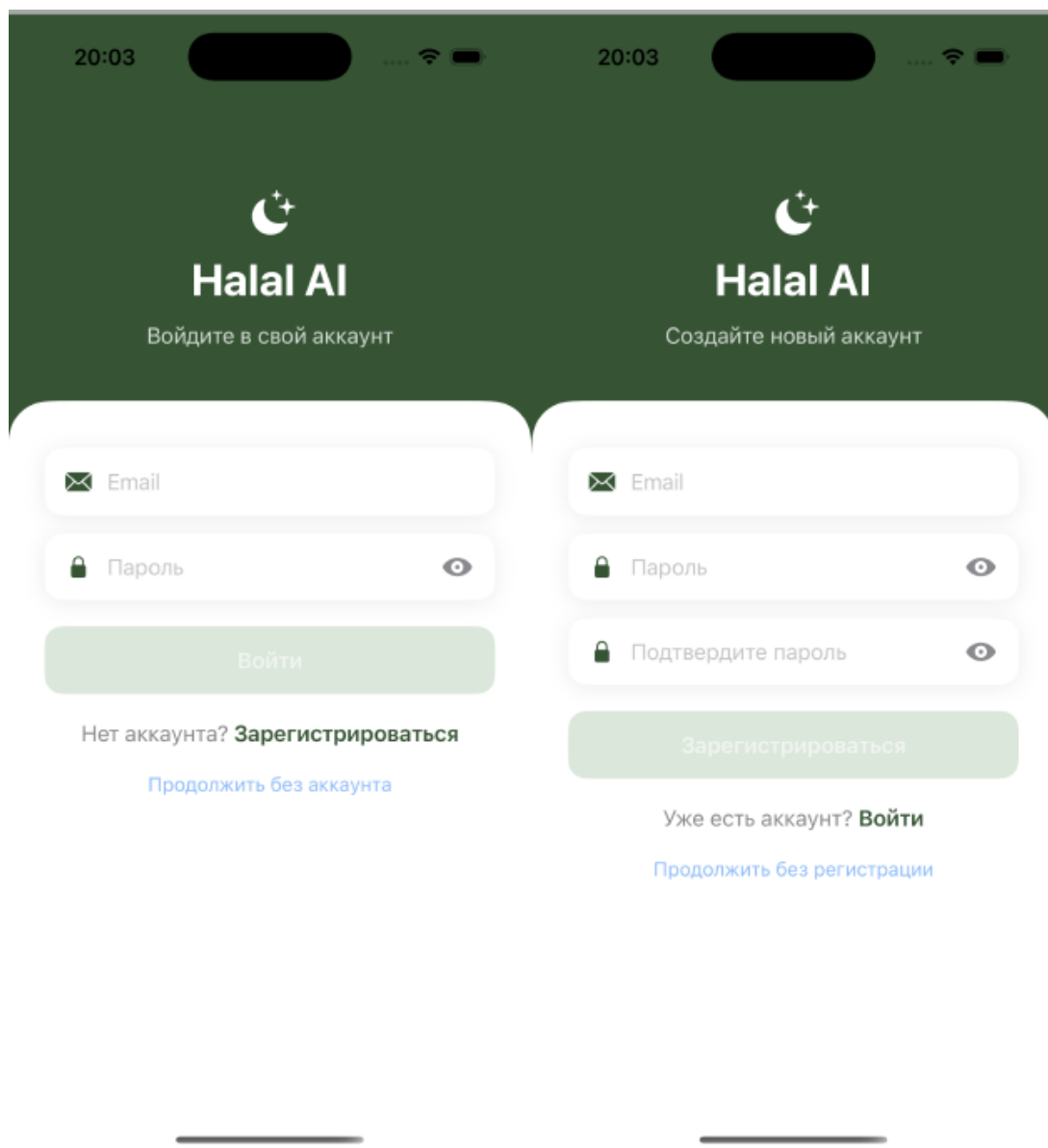


Рис.1.23. Макет экранов авторизации и регистрации

2 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ

В данной главе будет рассмотрен процесс реализации системы. На основе ранее спроектированной архитектуры и выбранных инструментов будет выполнена разработка компонентов системы: мобильное приложение, backend-сервер и LLM-сервис. Исходный код разработанных компонентов системы приведены в приложениях: LLM-сервис в приложении А, backend-сервер в приложении Б и iOS-приложение в приложении В.

2.1 Реализация Backend-сервера

Backend-сервер реализован на Java с использованием Spring Boot. Он выполняет роль центрального компонента системы и отвечает за обработку запросов от iOS-приложения, реализацию бизнес-логики, взаимодействие с базой данных PostgreSQL, а также за проксирование запросов к LLM-service.

Для описания внутренней структуры backend-сервера была построена uml диаграмма классов (см. рисунок 2.1), где показаны основные компоненты и связи между ними. На диаграмме представлены контроллеры, сервисы, интерфейсы, репозитории, модели предметной области, DTO-объекты и компоненты безопасности.

Диаграмма отражает не только состав основных классов, но и используемые архитектурные принципы. В частности, контроллеры взаимодействуют не с конкретными реализациями сервисов, а с их интерфейсами, что соответствует принципу инверсии зависимостей и упрощает тестирование и сопровождение системы.

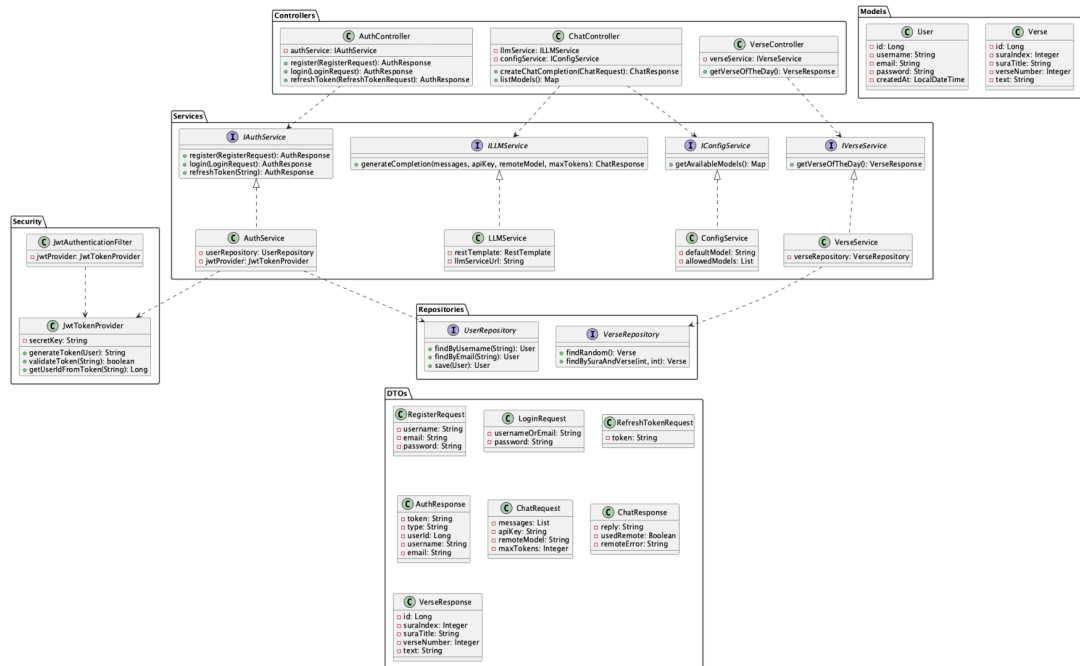


Рис.2.1. Диаграмма классов backend-сервера

На диаграммы видно, что backend-сервер разделен на несколько логических уровней.

На уровне контроллеров расположены классы AuthController, ChatController и VerseController. Они принимают HTTP-запросы от клиента, выполняют валидацию входных данных и передают обработку в сервисный слой. Каждый контроллер взаимодействует с системой через интерфейсы сервисов (IAuthService, ILLMService, IConfigService, IVerseService), что позволяет снизить связанность компонентов и реализовать принцип инверсии зависимостей (Dependency inversion principle).

Сервисный слой содержит основную бизнес-логику приложения. Класс AuthService реализует операции регистрации, аутентификации и обновления токена, используя репозиторий пользователей и компонент генерации JWT. Класс LLMService отвечает за взаимодействие с внешним LLM-service, включая отправку запросов на генерацию ответа. Дополнительно выделен ConfigService, который инкапсулирует конфигурацию системы, в частности список доступных моделей и значение модели по умолчанию. Это позволяет отделить конфигурационные данные от логики работы чат-сервиса. Класс VerseService реализует работу с данными Корана, включая получение случайного аята дня.

Слой доступа к данным представлен интерфейсами UserRepository и VerseRepository, которые используются сервисами для взаимодействия с базой данных PostgreSQL. Для хранения и передачи данных между слоями применяются DTO-объекты (RegisterRequest, LoginRequest, ChatRequest, AuthResponse,

ChatResponse, VerseResponse), что позволяет изолировать внутреннюю модель данных от внешнего API.

Отдельно на диаграмме представлены компоненты безопасности JwtTokenProvider и JwtAuthenticationFilter. Они обеспечивают генерацию, проверку и извлечение данных из JWT-токенов, а также выполняют аутентификацию входящих запросов. Это позволяет реализовать безопасный доступ к защищенным endpoint-ам без необходимости обращения к базе данных при каждом запросе.

Таким образом, представленная диаграмма классов демонстрирует, что backend-сервер реализован в соответствии с принципами модульности, слабой связанности и разделения ответственности, что обеспечивает масштабируемость и устойчивость системы к изменениям. Полный код backend-сервера приведен в приложении Б.

2.2 Реализация LLM-сервиса

2.2.1 Основные компоненты сервиса

LLM-сервис реализован в виде микросервиса, предназначенного для обработки запросов пользователей и генерации ответов на основе текстов Корана. Сервис построен с использованием архитектуры Retrieval-Augmented Generation (RAG), которая объединяет методы информационного поиска и генерации текста. Полный код LLM-сервиса приведен в приложении А.

Общая схема обработки запроса включает следующие этапы: пользовательский запрос поступает в API слой, затем передается в RAG-модуль для поиска релевантных фрагментов текста, после чего эти данные используются языковой моделью для генерации ответа.

Архитектура сервиса включает три основных слоя - API слой, RAG слой и LLM слой.

API слой реализован с использованием фреймворка FastAPI и обеспечивает обработку HTTP-запросов от backend-сервиса.

В рамках данного слоя реализованы следующие эндпоинты:

- А. /llm/health - проверка доступности сервиса.
- В. /llm/chat - основной интерфейс взаимодействия с системой.
- С. /llm/info - получение информации о сервисе.

Бизнес-логика обработки запроса, связанных с чатов реализована в классе ChatService. Данный класс выступает в роли координирующего компонента и реализует последовательность шагов, необходимых для формирования ответа пользователю. Выполняются следующие операции:

- А. извлечение последнего пользовательского сообщения из истории диалога.
- В. обращение к RAG-пайплайну для поиска релевантных фрагментов Копрана.
- С. преобразование найденных данных в текстовый контекст, используемый при генерации.
- Д. вызов LLM-клиента для получения ответа.
- Е. обработку возможных ошибок при взаимодействии с внешними сервисами.

Также для управления зависимостями используется DependencyContainer, который реализует подход dependency injection. В контейнере хранится: экземпляр RAG-пайплайна (интерфейс IRAGPipeline), клиент для взаимодействия с языковой моделью (интерфейс ILLMClient), а также экземпляр ChatService (интерфейс IChatService). Структура API-слоя показана на рисунке 2.2.

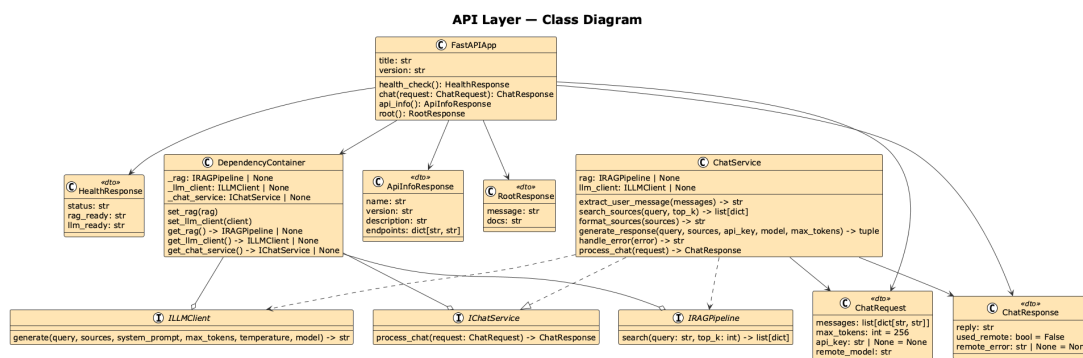


Рис.2.2. Диаграмма классов api-layer у llm-сервиса

RAG слой отвечает за поиск релевантной информации в базе знаний и включает следующие компоненты:

- А. EmbeddingModel - преобразует текст в векторное представление.
- В. VectorStore - хранит эмбединги документов и выполняет поиск.
- С. SimpleRAG - координирует процесс поиска.

Обработка запроса в рамках данного слоя включает:

- А. Преобразование пользовательского запроса в вектор (эмбединг).

В. Поиск наиболее похожих документов с использованием косинусного сходства.

С. Выбор top-k наиболее релевантных фрагментов текста.

Для генерации эмбеддингов используется библиотека SentenceTransformers, позволяющая учитывать семантическое сходство между текстами. Векторное хранилище реализовано в оперативной памяти, что обеспечивает высокую скорость поиска при большом объеме данных. Диаграмма классов RAG-слоя представлена на рисунке 2.3.

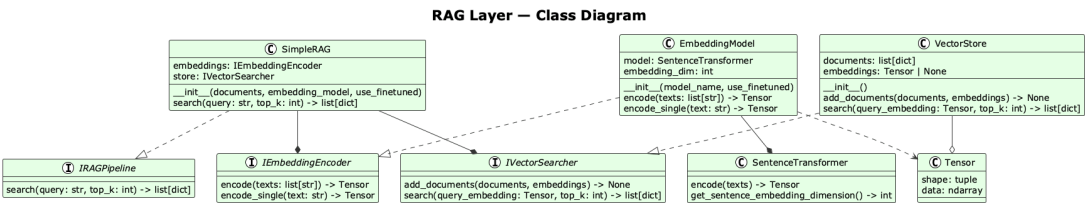


Рис.2.3. Диаграмма классов rag-layer у llm-сервиса

LLM слой отвечает за генерацию текстового ответа на основе пользовательского запроса и найденных фрагментов. Взаимодействие с языковой моделью осуществляется через сервис OpenRouter, который предоставляет унифицированный доступ к различным моделям.

Процесс генерации включает: формирование системного промпта, добавление найденных фрагментов Корана, пользовательского запроса и отправку запроса в языковую модель вместе с получением ответа от нее (рисунок 2.4).

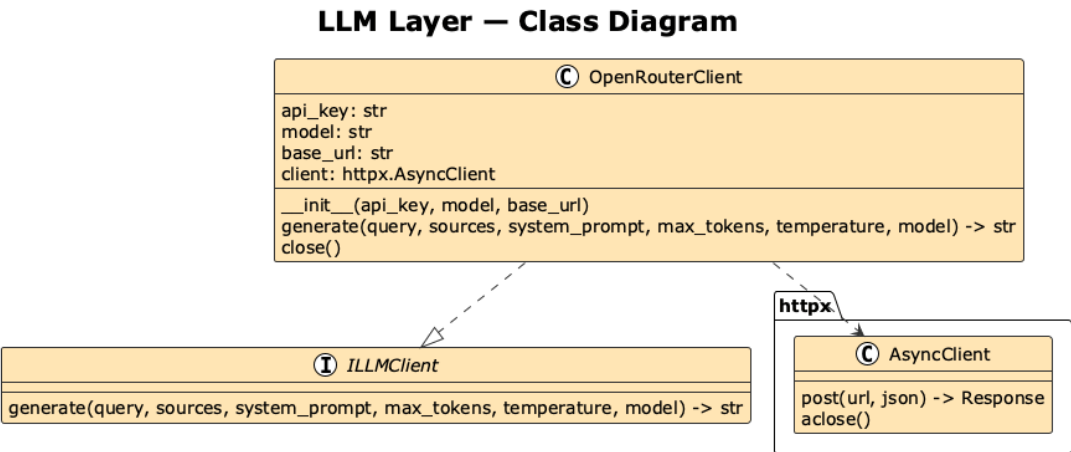


Рис.2.4. Диаграмма классов rag-layer у llm-сервиса

Для выполнения HTTP-запросов OpenRouterClient использует AsyncClient. Это позволяет не блокировать взаимодействие с внешним API и эффективно обрабатывать запросов в условиях одновременной работы нескольких пользователей.

2.2.2 Экспериментальный подбор параметров и моделей

После реализации функционала в LLM-сервиса была проведена серия экспериментов, направленных на повышение качества ответов системы. Основной задачей являлась проверка сформулированных ранее гипотез, а также выбор оптимальных параметров для максимально релевантного ответа для пользователя.

Эксперименты проводились на наборе тестовых запросов, включающих вопросы, связанные с исламской культурой и Кораном:

- А. Что Коран говорит о запрете свинины?
- В. Какие аяты в Коране говорят о запрете алкоголя?
- С. Почему в исламе запрещен алкоголь?
- Д. Что говорится в Коране о молитве и ее значении?
- Е. Как в Коране описывается поведение и скромность женщины?

Для проверки влияния ключевых параметров RAG-подсистемы был сформирован набор экспериментальных конфигураций, различающихся по двум факторам: использование retrieval-механизма и тип модели эмбедингов.

В качестве базового сценария рассматривалась генерация ответа без использования RAG. Для сценариев с RAG дополнительно варьировались тип embedding model (базовая и fine-tuned). Конфигурации тестирования приведены в таблице 2.1.

Таблица 2.1

Конфигурации RAG

No.	RAG	Embedding model	Состояние
C1	Нет	-	-
C2	Да	paraphrase-multilingual-mpnet-base-v2	Base
C3	Да	paraphrase-multilingual-mpnet-base-v2	Fine-tuned
C4	Да	ai-forever/sbert_large_nlu_ru	Base
C5	Да	ai-forever/sbert_large_nlu_ru	Fine-tuned

Для оценки итогового качества ответа языковой модели использовались следующие критерии: корректность (A), обоснованность (G), полнота (C) и галлюцинации (H) (таблица 2.2).

Таблица 2.2

Метрики оценки ответов

Критерий	Описание	Диапазон
Корректность (A)	Насколько ответ соответствует содержанию Корана	0 - 5
Обоснованность (G)	Насколько ответ опирается на релевантные источники	0 - 5
Полнота (C)	Насколько полно раскрыт вопрос	0 - 5
Галлюцинации (H)	Наличие недостоверной информации	0 - 5

По каждому критерию оценка проводится по шкале от 0 до 5. Для всех критериев, кроме галлюцинаций, оценка 0 - это полное несоответствие, а 5 - идеальное соответствие. Для критерия галлюцинаций 0 - это их полное отсутствие, а 5 - большое количество недостоверной информации. Итоговая оценка качества ответа языковой модели рассчитывается по формуле:

$$AnswerScore = 0,2 \cdot A + 0,3 \cdot G + 0,2 \cdot C - 0,5 \cdot H + 1 \quad (2.1)$$

Следует отметить, что указанные критерии применялись исключительно для оценки итогового ответа языковой модели и не использовались для непосредственной оценки качества retrieval-подсистемы. Это обусловлено тем, что значение косинусного сходства, получаемое на этапе retrieval, не всегда отражает фактическую релевантность найденных аятов. В ряде случаев документы с высоким значением сходства оказывались семантически близкими к вопросу, но не содержали прямого ответа на него. В связи с этим качество retrieval оценивалось отдельно, на основе ручного анализа найденных аятов.

Для оценки качества retrieval использовались следующие критерии: релевантность найденных источников (R) и точность top-3 выдачи (T) (таблица 2.3).

Таблица 2.3

Метрики оценки поиска

Критерий	Описание	Диапазон
Релевантность источников (R)	Насколько найденные аяты действительно соответствуют поставленному вопросу	0 - 5
Точность top-3 выдачи (T)	Доля релевантных аятов среди первых трех найденных документов по всем заданным вопросам	0 - 5

Оценка retrieval проводится вручную. Для каждого вопроса анализируются первые три аята, возвращённые retrieval-подсистемой. При оценке учитывается, содержат ли найденные аяты прямой ответ на поставленный вопрос, являются ли они частично релевантными либо не относятся к запрашиваемой теме вообще.

Показатель точности top-3 выдачи рассчитывается по формуле:

$$t_i = \frac{N_{rel}}{3} \times 5 \quad (2.2)$$

где N_{rel} - это количество релевантных аятов среди первых трех найденных документов.

Оценка всех вопросов считается по формуле:

$$T = \frac{1}{5} \times \sum_{i=1}^5 t_i \quad (2.3)$$

Итоговая оценка качества retrieval определяется по формуле:

$$RetrievalScore = 0.6R + 0.4T \quad (2.4)$$

Общая оценка экспериментальной конфигурации рассчитывалась с учетом как качества retrieval, так и качества итогового ответа языковой модели:

$$FinalScore = 0.4 \times RetrievalScore + 0.6 \times AnswerScore \quad (2.5)$$

Таким образом, оценка каждой экспериментальной конфигурации выполнялась в два этапа. На первом этапе анализировалось качество retrieval-подсистемы по найденным аятам, на втором - качество итогового ответа языковой модели.

Таблица 2.4

Результаты тестирования конфигураций RAG

Конфигурация	R	T	Retrieval Score	A	G	C	H	Answer Score	Final Score
C1	0	0	0	4	3	3	1	2,8	2,8
C2	3	3	3	4	3	5	1	3,2	3,12
C3	3	4	3,4	5	4	4	0	4	3,76
C4	1	1	1	4	3	4	1	3	2,2
C5	3	3	3	4	4	5	1	3,5	3,3

Результаты проведенных экспериментов, представленные в таблице 2.4, позволяют сделать выводы о влиянии рассматриваемых факторов на качество работы системы.

Прежде всего, сравнение базовой конфигурации C1 с конфигурациями, использующими retrieval-механизм (C2–C5), показывает, что применение RAG-подсистемы в целом положительно влияет на качество ответов. Несмотря на то, что языковая модель в конфигурации C1 демонстрирует приемлемые значения корректности и полноты, отсутствие опоры на конкретные источники приводит к снижению обоснованности ответов и увеличению риска появления галлюцинаций. В конфигурациях с использованием RAG наблюдается рост показателя обоснованности, что связано с использованием релевантных аятов при генерации ответа.

Анализ качества retrieval-подсистемы показывает, что эффективность поиска существенно зависит от используемой embedding модели. Конфигурации C2 и C4, использующие базовые версии моделей, демонстрируют более низкие значения показателей R и T по сравнению с соответствующими дообученными. Это говорит о том, что базовые модели не всегда способны корректно учитывать семантические особенности религиозных текстов, что приводит к включению нерелевантных аятов в выдачу. В то же время конфигурации C3 и C5, использующие дообученные модели эмбеддингов, показывают более высокие значения показателей релевантности и точности top-3 выдачи. Это подтверждает, что дообучение модели на специализированном корпусе позволяет существенно повысить качество семантического поиска и уменьшить количество нерелевантных документов.

Также сравнение архитектур embedding моделей показывает, что модель paraphrase-multilingual-mpnet-base-v2 демонстрирует более высокие результаты по сравнению с ai-forever/sbert_large_nlu_ru как в базовом, так и в дообученном

состоянии. Особенно заметно это проявляется в конфигурации C3, которая достигает наибольшего значения итоговой оценки среди всех рассмотренных вариантов.

Полученные результаты позволяют подтвердить сформулированные гипотезы. Вместе с тем проведенное экспериментальное исследование имеет ряд ограничений. Набор тестовых запросов ограничен и не охватывает все возможные сценарии использования системы. В дальнейшем планируется расширение корпуса тестовых данных, а также автоматизация оценки качества.

Гипотеза 1 о том, что применение RAG повышает качество ответов и снижает частоту галлюцинаций подтверждается. Конфигурации с использованием RAG демонстрируют более высокие значения показателя обоснованности за счет опоры на релевантные источники. Однако полностью устранить галлюцинации не удастся, модель продолжает генерировать частично некорректные формулировки, включая искажения религиозной терминологии.

Гипотеза 2 о влиянии дообучения embedding модели также подтверждается. Во всех случаях дообученные модели (C3 и C5) демонстрируют более высокое качество retrieval и, следовательно, более высокие итоговые оценки по сравнению с соответствующими базовыми моделями (C2 и C4).

На основании проведенного исследования для дальнейшей разработки была выбрана конфигурация, показавшая наилучшие результаты, а именно модель paraphrase-multilingual-mpnet-base-v2, дообученная на специализированном корпусе.

2.3 Реализация iOS-приложения

Клиентская часть системы реализована в виде iOS-приложения. В процессе разработки были реализованы функциональные модули аутентификации, домашнего экрана, чата с AI-ассистентом, сканирования ингредиентов, чтения Корана, расчета времени молитвы и настроек приложения. Структурно проект разделен на модули App, Navigation, Features, Core, Resources и Models, что позволяет разграничить пользовательский интерфейс, навигацию, бизнес-логику и инфраструктурные компоненты. Полный код iOS-приложения приведен в приложении В.

2.3.1 Структура проекта

Основная прикладная логика сосредоточена в Features, где каждая функциональная область оформлена как отдельный модуль. Так, модуль Auth отвечает за сценарии входа и регистрации, Home реализует главный экран приложения, Chat содержит экран взаимодействия с AI-ассистентом, Scanner предназначен для анализа состава продуктов, Quran обеспечивает просмотр сур и аятов, а Prayer реализует расчет времени молитвы и настройку уведомлений. Отдельно выделен модуль Settings, предназначенный для управления пользовательскими параметрами приложения (рисунок 2.5).

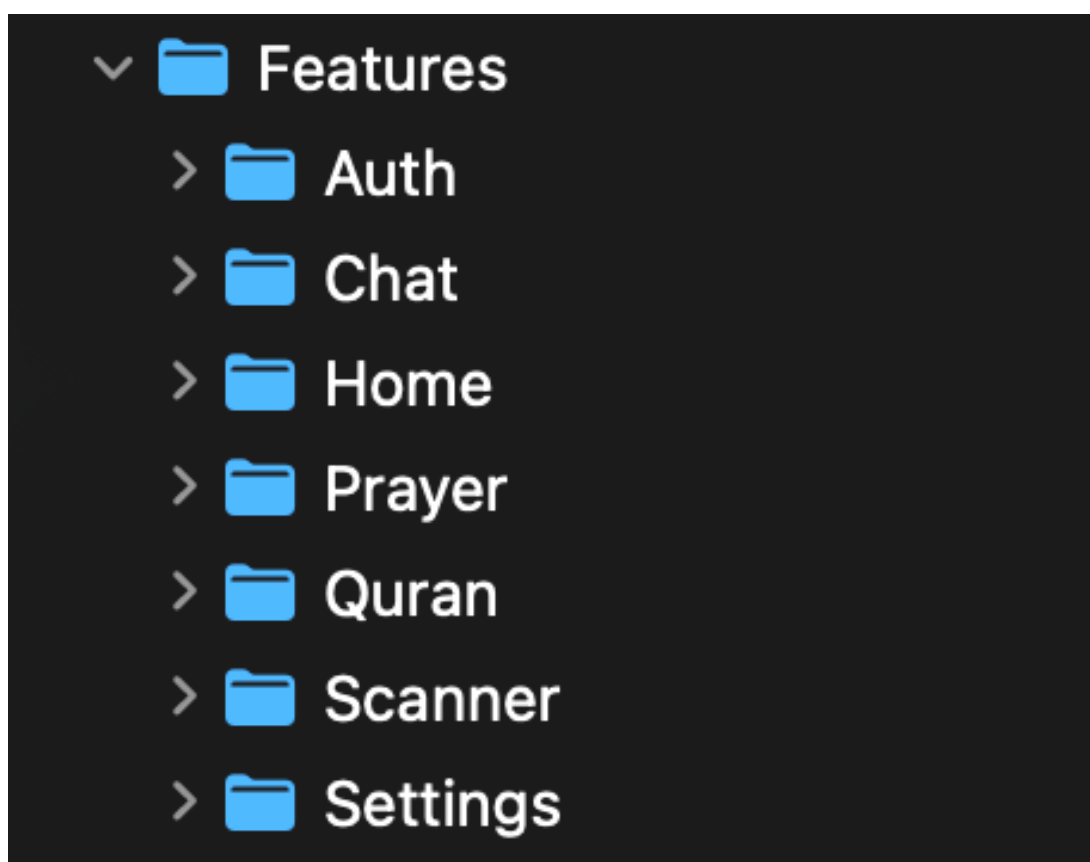


Рис.2.5. Модуль Features

Общая навигация приложения реализована в модуле Navigation, включающем главный координатор, корневой RouterView и координаторы для отдельных разделов приложения (рисунок 2.7). Общие компоненты вынесены в модуль Core, где реализованы сетевой слой, сервис геолокации, переиспользуемые UI-компоненты и вспомогательные расширения (рисунок 2.6).

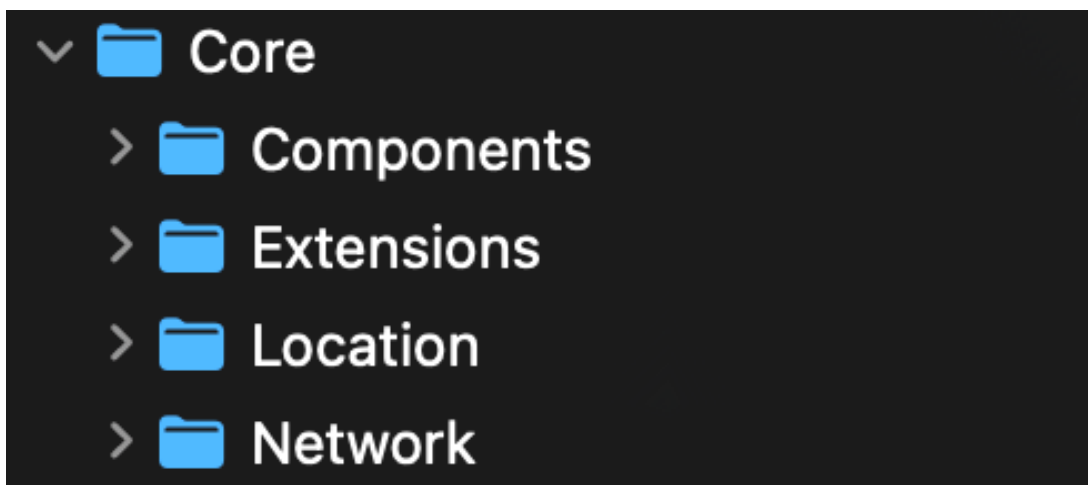


Рис.2.6. Модуль Core

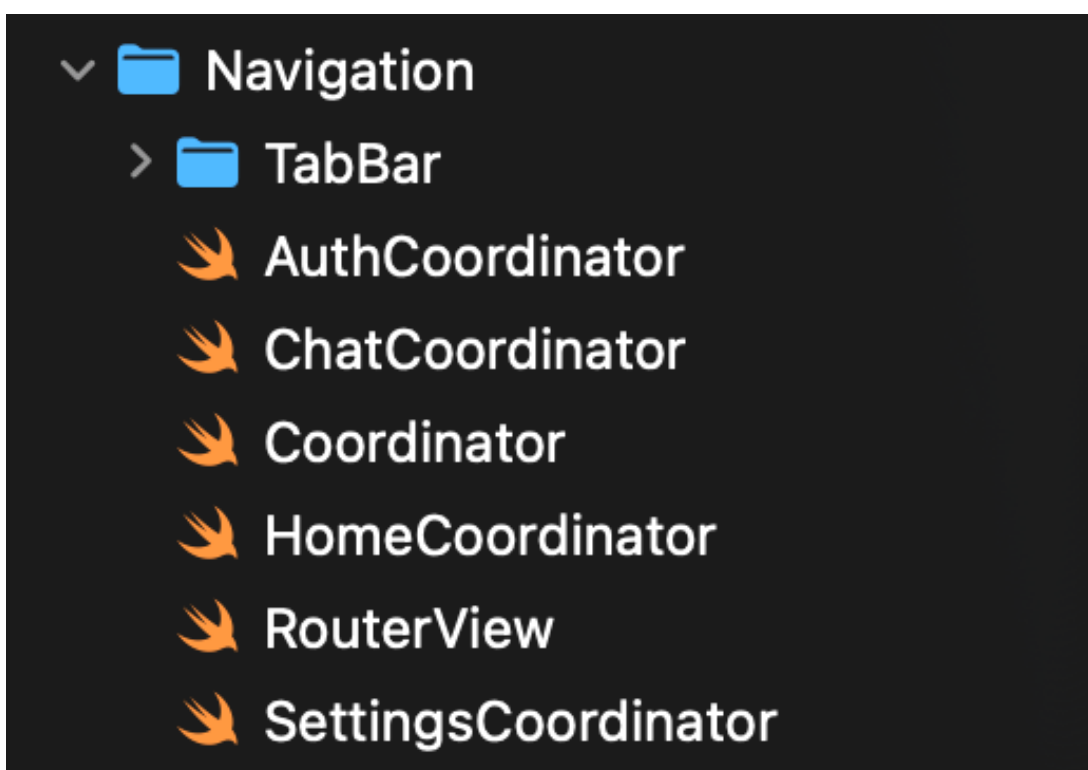


Рис.2.7. Модуль Navigation

2.3.2 Реализация навигации

Навигация в приложении реализована в модуле Navigation. В состав данного модуля входят главный класс Coordinator, корневое представление RouterView, а также координаторы HomeCoordinator, ChatCoordinator, SettingsCoordinator и AuthCoordinator. Дополнительно в навигационный слой включены компоненты TabBarItem и TabBarView, отвечающие за организацию переключения между основными разделами приложения.

Coordinator управляет стеком переходов и текущей выбранной вкладкой приложения. За счет этого навигация становится централизованной: экраны не создают и не открывают другие экраны напрямую, а только инициируют переход через методы координатора. Корневой контейнер навигации реализован в RouterView. Он связывает состояние координатора с интерфейсом приложения и обеспечивает отображение соответствующего стека экранов. Через RouterView пользователь может переключаться между основными вкладками приложения, а также переходить к вложенным экранам внутри отдельных функциональных модулей.

Для маршрутизации внутри отдельных разделов приложения используются специализированные координаторы. HomeCoordinator отвечает за переходы, связанные с главным экраном, сканером, списком сур, чтением суры и настройками молитвы. ChatCoordinator управляет экраном диалога с AI-ассистентом. SettingsCoordinator содержит маршруты, относящиеся к настройкам приложения. AuthCoordinator используется для сценариев авторизации, включая вход и регистрацию пользователя.

Отдельно реализован механизм переключения между основными вкладками приложения. Для этого используются TabBarItem, задающий набор доступных вкладок, и TabBarView, отвечающий за отображение панели навигации. В текущей реализации выделены три основные вкладки: Главный экран(Home), Чат(Chat) и Настройки(Settings) (рисунок 2.8).

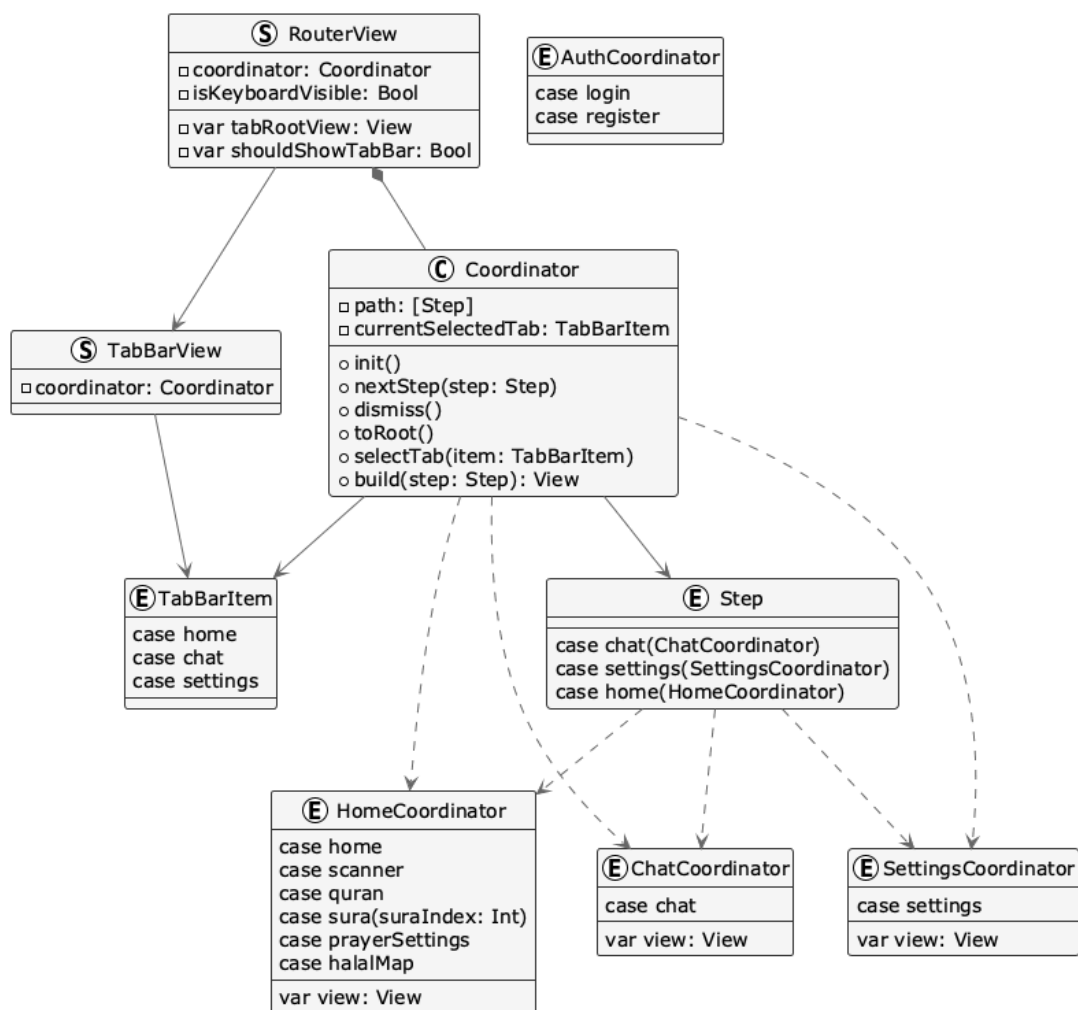


Рис.2.8. UML-диаграмма классов модуля навигации

2.3.3 Реализация функциональных модулей.

Каждый модуль оформлен как отдельная часть проекта и включает собственные представления, модели и сервисы. Это позволяет изолировать различные области функциональности друг от друга и упрощает дальнейшее масштабирование приложения. В текущей реализации выделены модули Auth, Home, Chat, Scanner, Quran, Prayer и Settings.

Создание экранов в приложении реализовано централизованно через ScreenFactoryImpl, который выступает фабрикой пользовательских представлений. Для каждого основного сценария в нем предусмотрен отдельный метод, возвращающий уже сконфигурированный экран, например makeLoginView(), makeChatView(), makeScannerView(), makeHomeView() и другие. Внутри фабрики используется DependencyContainer, который инициализирует основные сервисы приложения и предоставляет их экранным модулям. За счет этого зависимости передаются в представления через конструктор, а не создаются непосредственно внутри экранов.

2.3.3.1 Модуль аутентификации

Модуль аутентификации отвечает за вход и регистрацию пользователя. В нем содержатся модели данных, сервис взаимодействия с backend и пользовательские экраны авторизации. Данный модуль обеспечивает переход между гостевым режимом и полным режимом работы приложения, в котором пользователю становятся доступны персонализированные функции системы. Управление состоянием авторизации выполняется через AuthManager, а сетевое взаимодействие через AuthService (рисунки 2.9, 2.10).

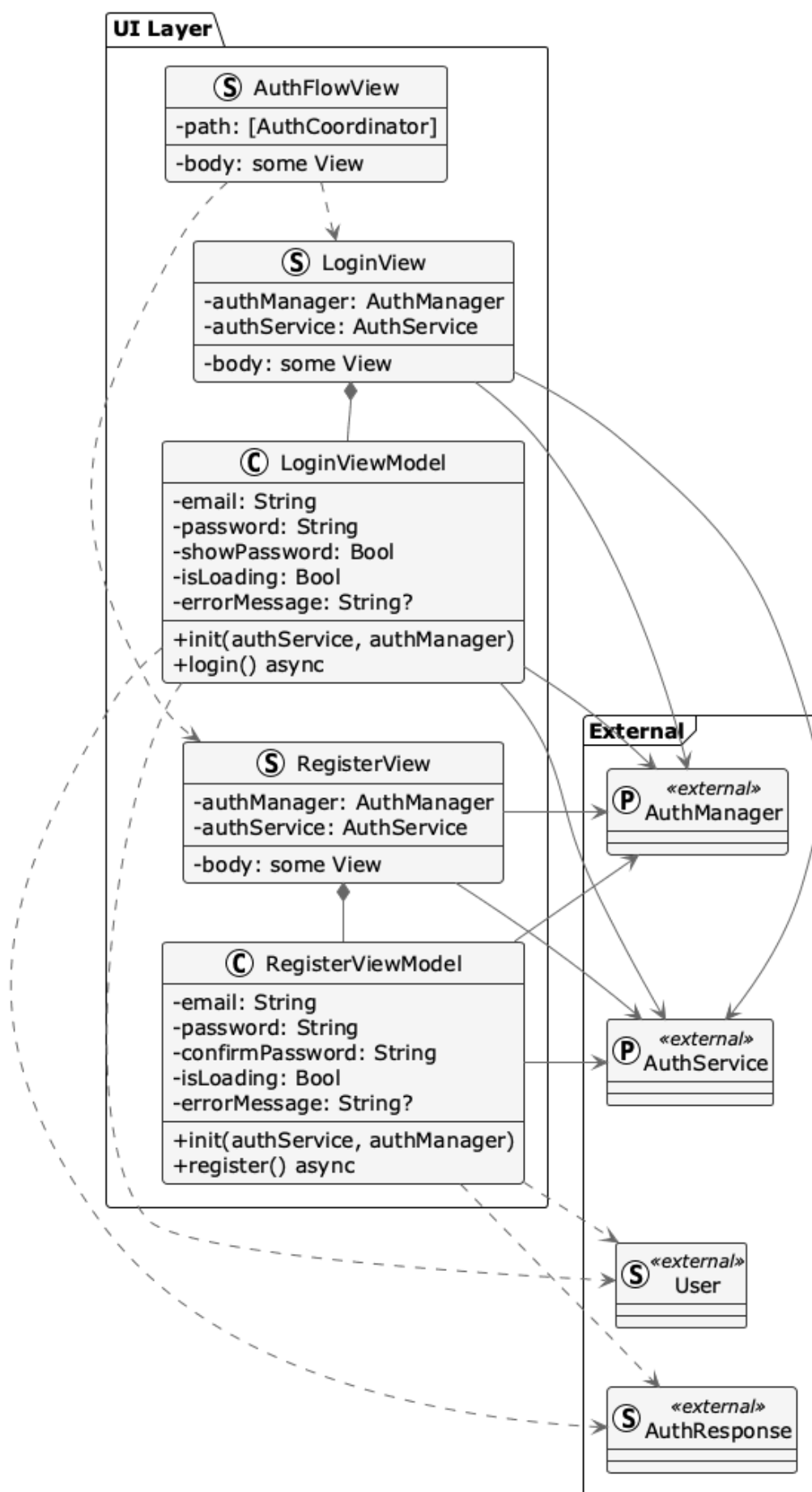


Рис.2.9. UML-диаграмма классов модуля аутентификации. UI

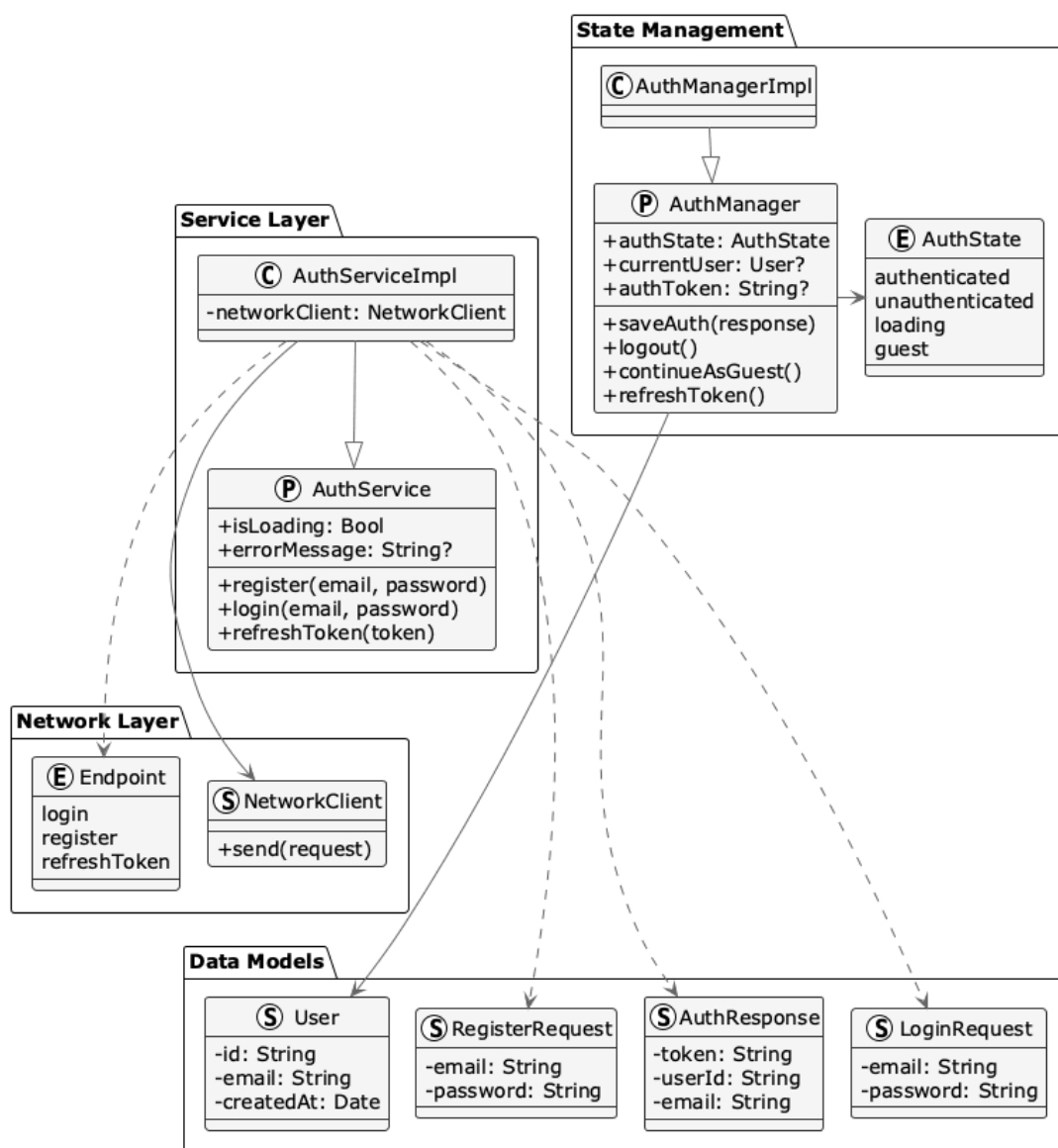


Рис.2.10. UML-диаграмма классов модуля аутентификации. Services

2.3.3.2 Модуль главного экрана

Модуль Home реализует главный экран приложения и служит основной точкой входа в остальные пользовательские сценарии. На главном экране отображаются ключевые элементы интерфейса: карточка со случайным аятом, карточка времени молитвы, баннер гостевого режима и кнопки быстрого перехода к основным функциям приложения.

2.3.3.3 Модуль чата

Модуль Chat является одной из ключевых частей приложения, он реализует взаимодействие пользователя с AI-ассистентом. Данный модуль включает экран диалога, модели сообщений, визуальные компоненты отображения переписки и

сервис отправки запросов на backend. Пользователь может вводить сообщения в свободной форме, после чего текст запроса передается на серверную часть.

На уровне взаимодействия с системой чат опирается на backend API. Клиент не обращается напрямую к LLM-service, а отправляет запросы на основной backend, который уже проксирует их в соответствующий сервис. Это позволяет скрыть от клиента внутреннюю организацию серверной части и сохранить единый интерфейс взаимодействия для мобильного приложения. Кроме того, в модуле предусмотрена поддержка выбора модели, что связано с использованием OpenRouter в серверной части системы (рисунок 2.11).

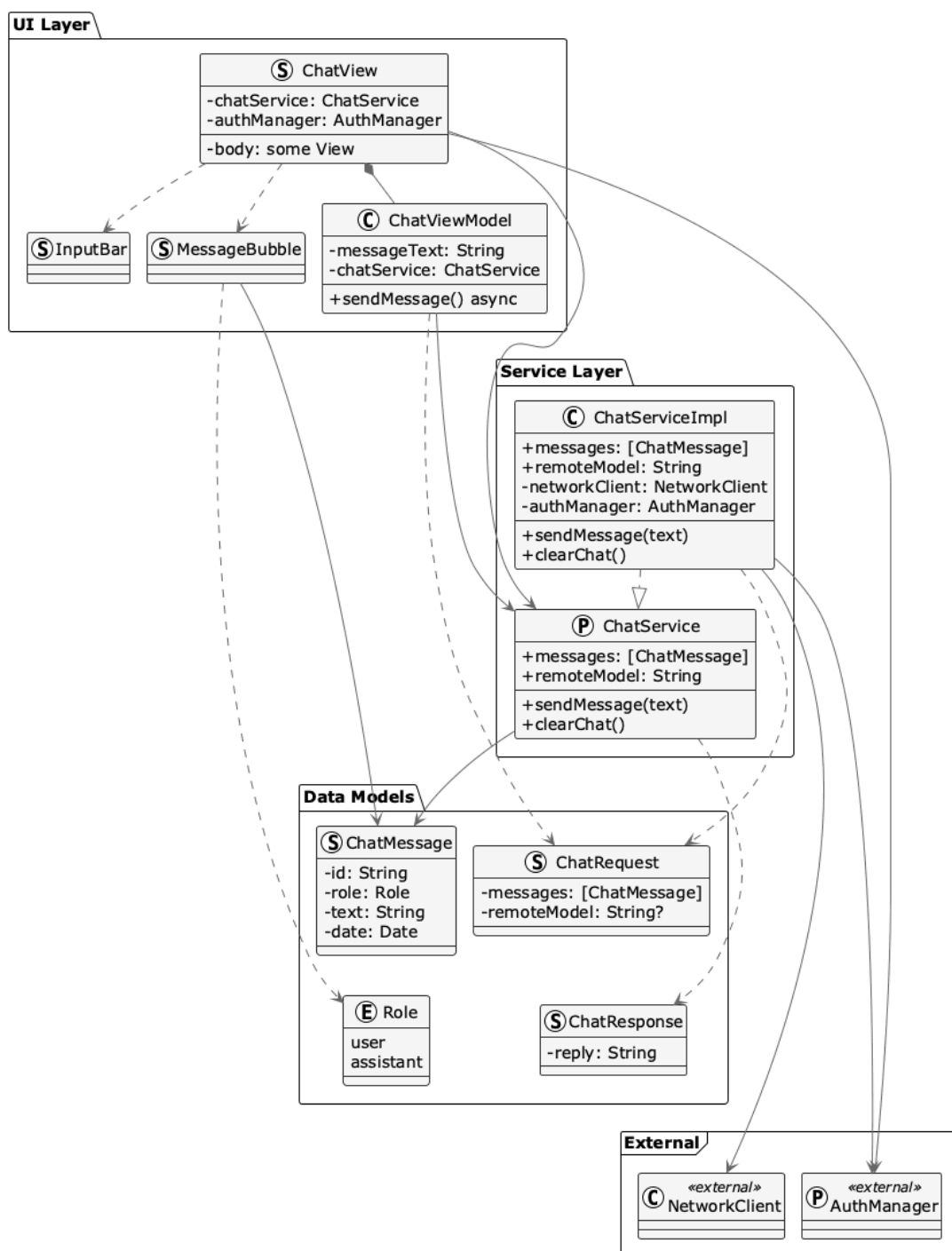


Рис.2.11. UML-диаграмма классов модуля чата

2.3.3.4 Модуль сканирования ингредиентов

Модуль Scanner предназначен для анализа состава продуктов и выявления в них запрещенных в исламе ингредиентов.

Структура модуль включает экран ScannerView, модель представления ScannerViewModel и сервис IngredientService. Экран отвечает за взаимодействие с пользователем и предоставляет несколько вариантов ввода данных: получение

изображения через камеру, выбор фотографии из галереи и ручной ввод текста состава. `ScannerViewModel` управляет состоянием экрана, координирует распознавание текста и инициирует анализ. `IngredientService` отвечает за загрузку базы ингредиентов и сопоставление распознанного текста с данными этой базы.

На первом этапе пользователь передает состав продукта одним из доступных способов. Если используется изображение, далее запускается OCR-распознавание текста средствами `Vision Framework`. Распознанный текст передается в сервис анализа, который извлекает названия ингредиентов и выполняет их сопоставление с внутренней базой ингредиентов. По результатам сопоставления формируется объект `ProductAnalysis`, содержащий перечень найденных ингредиентов, их статусы и общий итоговый результат для продукта.

При анализе учитываются как текстовые названия ингредиентов, так и пищевые добавки, обозначенные Е-кодами. Для каждого обнаруженного элемента определяется его статус, после чего вычисляется итоговый статус всего продукта. Если среди найденных компонентов присутствует хотя бы один харам-ингредиент, общий результат помечается как "харам". Если запрещенных компонентов нет, но присутствуют сомнительные, продукт получает статус "сомнительный". В случае, когда все распознанные ингредиенты относятся к разрешенным, результат помечается как "халяль". Такой подход соответствует логике, ранее заложенной в алгоритме анализа состава продукта (рисунок 2.12).

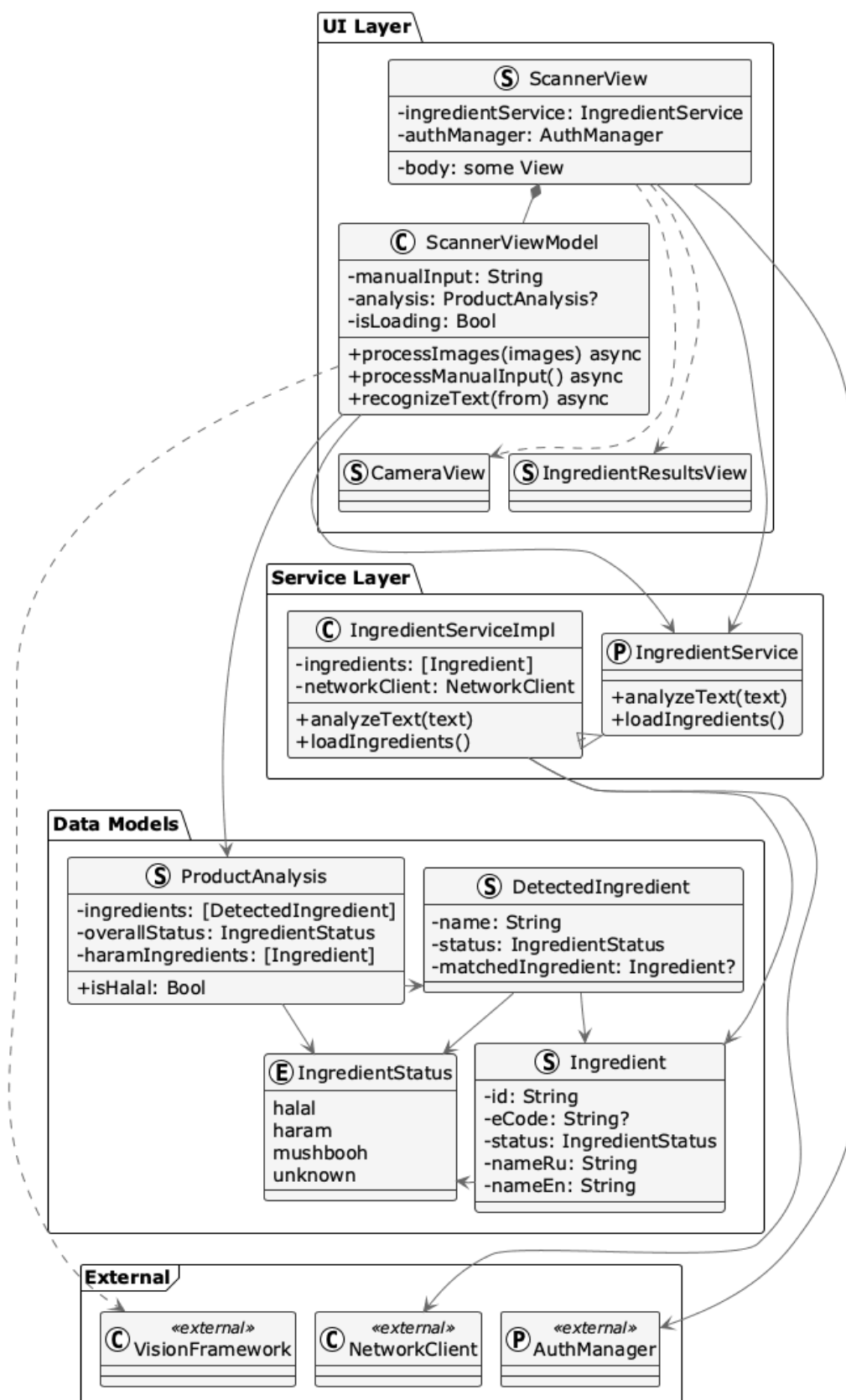


Рис.2.12. UML-диаграмма классов модуля сканирования ингредиентов

2.3.3.6 Модуль времени молитвы

Модуль Prayer отвечает за расчет времени молитвы, отображение расписания на текущий день, определение ближайшей молитвы и настройку уведомлений. В состав модуля входят сервис расчета времени молитвы PrayerTimeService, сервис уведомлений PrayerNotificationService, сервис геолокации LocationService, хранилище настроек PrayerSettingsStore, а также пользовательские экраны PrayerTimesCardView и PrayerNotificationSettingsView.

Центральным элементом модуля является PrayerTimeService, который отвечает за вычисление расписания молитв на основе текущих координат пользователя, даты и выбранного метода расчета. В реализации для этих вычислений используется библиотека Adhan, предоставляющая готовые алгоритмы расчета времени намаза на основе астрономических параметров.

Результатом работы сервиса является структура DailyPrayerTimes, содержащая время пяти обязательных молитв: fajr, dhuh, asr, maghrib и isha. Также на основе полученного расписания определяется ближайшая предстоящая молитва.

Для получения координат используется LocationService, представляющий собой обертку над системным фреймворком CoreLocation. После запроса разрешения у пользователя сервис получает текущее местоположение и передает его в модуль расчета времени молитвы.

Отдельная часть модуля связана с хранением пользовательских параметров. В PrayerSettingsStore сохраняются выбранный метод расчета, мазхаб (правовая школа в исламе) и настройки уведомлений.

Для напоминаний о времени молитвы используется PrayerNotificationService. Он запрашивает разрешение на отправку уведомлений, формирует локальные уведомления и позволяет пересоздавать их при изменении пользовательских настроек.

На уровне интерфейса основным элементом является PrayerTimesCardView, связанный с PrayerTimesCardViewModel. Эта модель представления получает данные из сервисов, управляет состоянием экрана и передает в интерфейс рассчитанное расписание, а также информацию о ближайшей молитве (рисунки 2.14, 2.15).

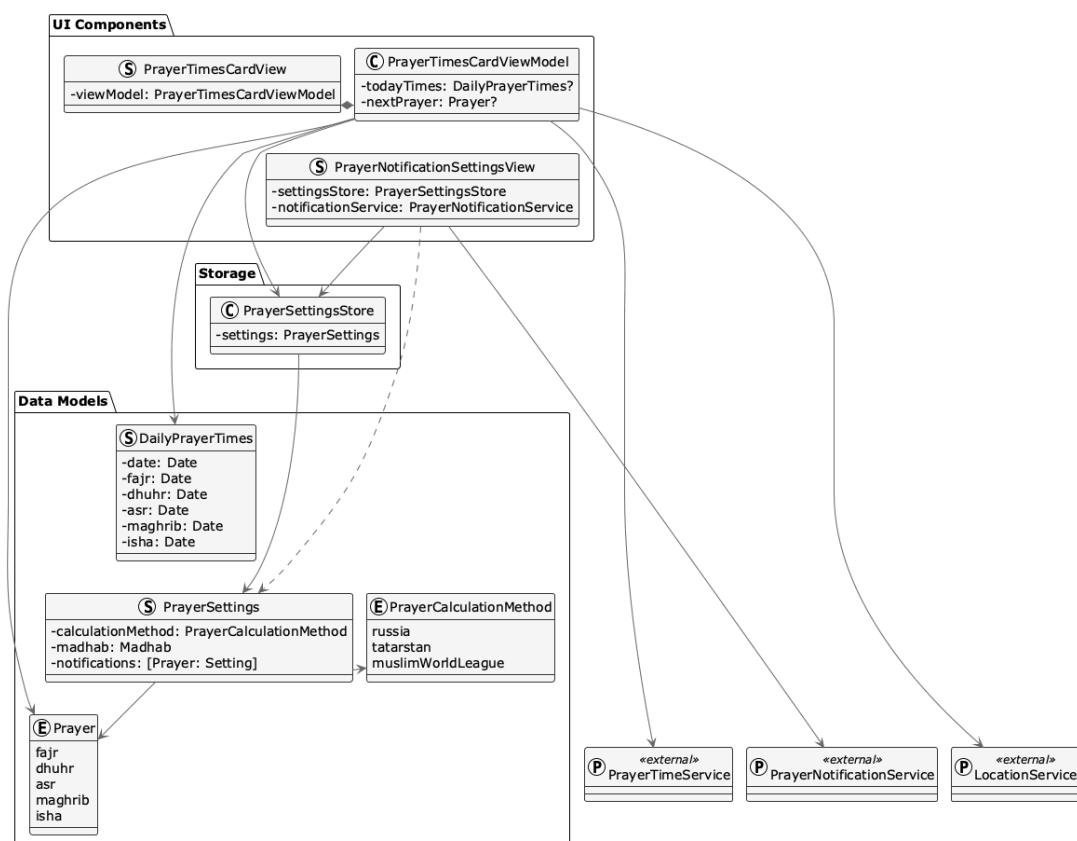


Рис.2.14. UML-диаграмма классов модуля времени молитвы. UI

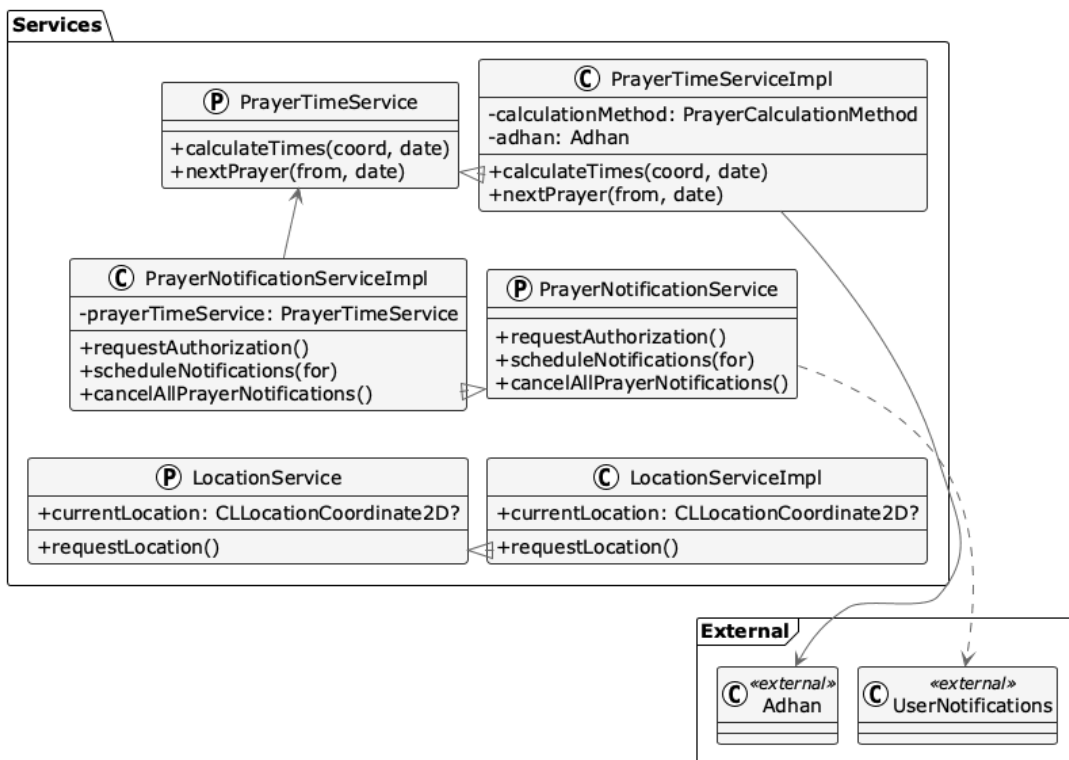


Рис.2.15. UML-диаграмма классов модуля времени молитвы. Services

2.3.3.7 Модуль настроек

Модуль Settings предназначен для управления параметрами приложения и завершает набор основных пользовательских сценариев клиентской части. Через данный раздел пользователь получает доступ к настройкам, связанным с профилем, параметрами работы чата и управлением авторизацией. В состав модуля входят экран SettingsView, модель представления SettingsViewModel, а также вспомогательный компонент RemoteModelSettingsSection, отвечающий за настройку параметров удаленной языковой модели.

С точки зрения архитектуры модуль настроек взаимодействует прежде всего с двумя сервисами: ChatService и AuthManager. AuthManager предоставляет сведения о текущем пользователе и его статусе, а также используется для выполнения выхода из аккаунта. ChatService, в свою очередь, отвечает за параметры работы AI-ассистента, включая выбор удаленной модели, ограничение на количество токенов, температурный параметр генерации и использование режима RAG.

На уровне интерфейса SettingsView объединяет несколько групп параметров. Во-первых, здесь отображается информация о текущем пользователе, если приложение работает в авторизованном режиме. Во-вторых, пользователю предоставляется возможность изменять параметры удаленной языковой модели. Для этого используется отдельный компонент RemoteModelSettingsSection, который инкапсулирует элементы управления, связанные с выбором модели и настройками генерации. Модель представления SettingsViewModel управляет состоянием экрана и связывает элементы интерфейса с соответствующими параметрами сервисов. Через нее поддерживается изменение значений, связанных с отображением API-ключа, ограничением числа токенов, температурой генерации и выбором пользовательской модели.

2.3.4 Реализация сетевого взаимодействия

Взаимодействие iOS-приложения с серверной частью реализовано через отдельный сетевой слой, вынесенный в модуль Core/Network. Это позволяет отделить логику пользовательских модулей от деталей HTTP-коммуникации и обеспечить единый механизм работы с backend API. В состав сетевого слоя входят компоненты NetworkClient, APIConfiguration, APIRequest, Endpoint и NetworkError.

NetworkClient выполняет отправку запросов и получение ответов от backend-сервера. Его задача заключается в том, чтобы инкапсулировать общую логику

формирования HTTP-запросов, обработки ответов, декодирования JSON и обработки сетевых ошибок. Благодаря этому прикладные сервисы, такие как AuthService и ChatService, не работают напрямую с URLSession, а используют унифицированный клиент.

Для описания запросов используется абстракция APIRequest, а перечисление Endpoint задает набор доступных маршрутов backend-сервера.

Все сетевые операции в приложении реализованы с использованием URLSession и механизма async/await. Это позволяет выполнять запросы асинхронно, не блокируя основной поток и не нарушая отзывчивость пользовательского интерфейса (рисунок 2.16).

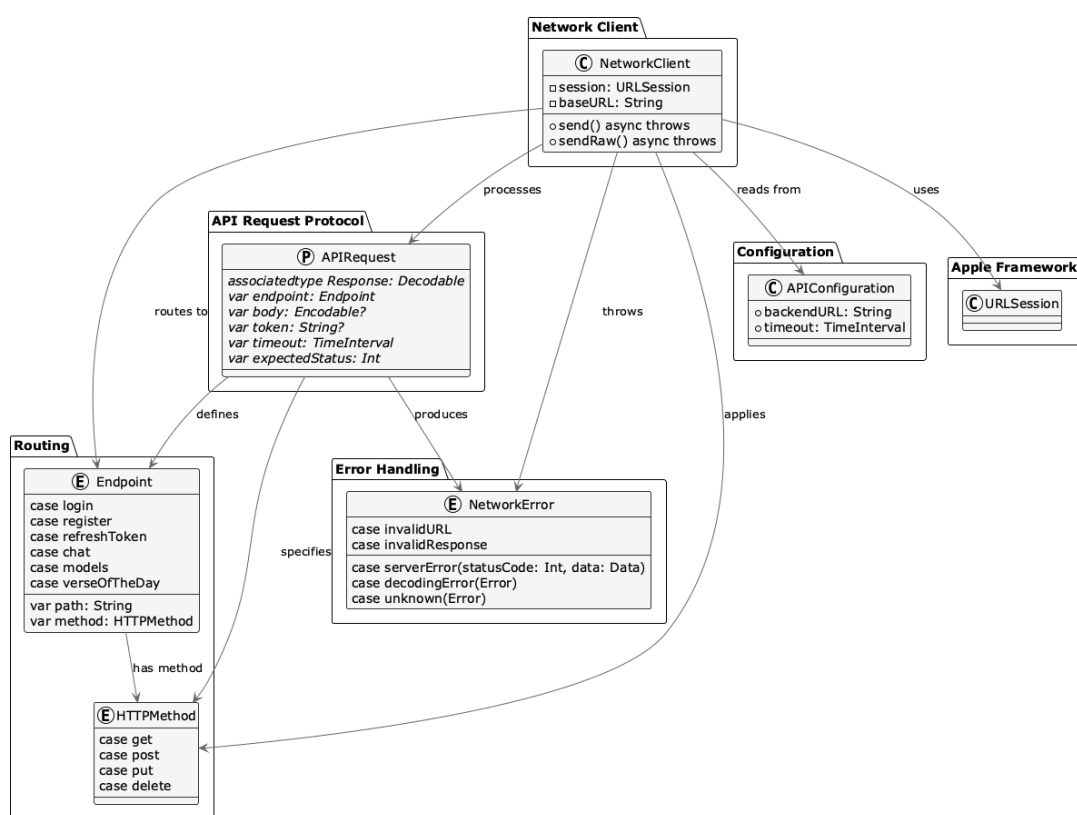


Рис.2.16. UML-диаграмма классов сетевого слоя

3 ТЕСТИРОВАНИЕ

3.1 Тестирование iOS-клиента

Проводилось тестирование iOS-клиента с целью проверки корректности работы пользовательского интерфейса, бизнес-логики приложения и взаимодействия клиентской части с backend. В процессе разработки были написаны модульные и UI-тесты, охватывающие ключевые компоненты приложения. Для написания тестов использовались встроенные инструменты: Swift Testing, XCTest и XCUITest.

Unit-тесты размещены в каталоге HalalAITests, а UI-тесты в HalalAIUITests. Всего было написано более 270 модульных тестов и 28 UI-тестов.

3.1.1 Unit-тестирование

Unit-тестирование использовалось для проверки отдельных компонентов приложения. Основное внимание уделялось тестированию модулей, реализующих ключевую бизнес-логику приложения.

В рамках тестирования системы авторизации были реализованы тесты компонентов AuthService, AuthManager, LoginViewModel и RegisterViewModel. Проверялись сценарии входа в систему, регистрации пользователя, обновления токена, обработки сетевых ошибок, сохранения пользовательской сессии и работы гостевого режима (рисунок 3.1).

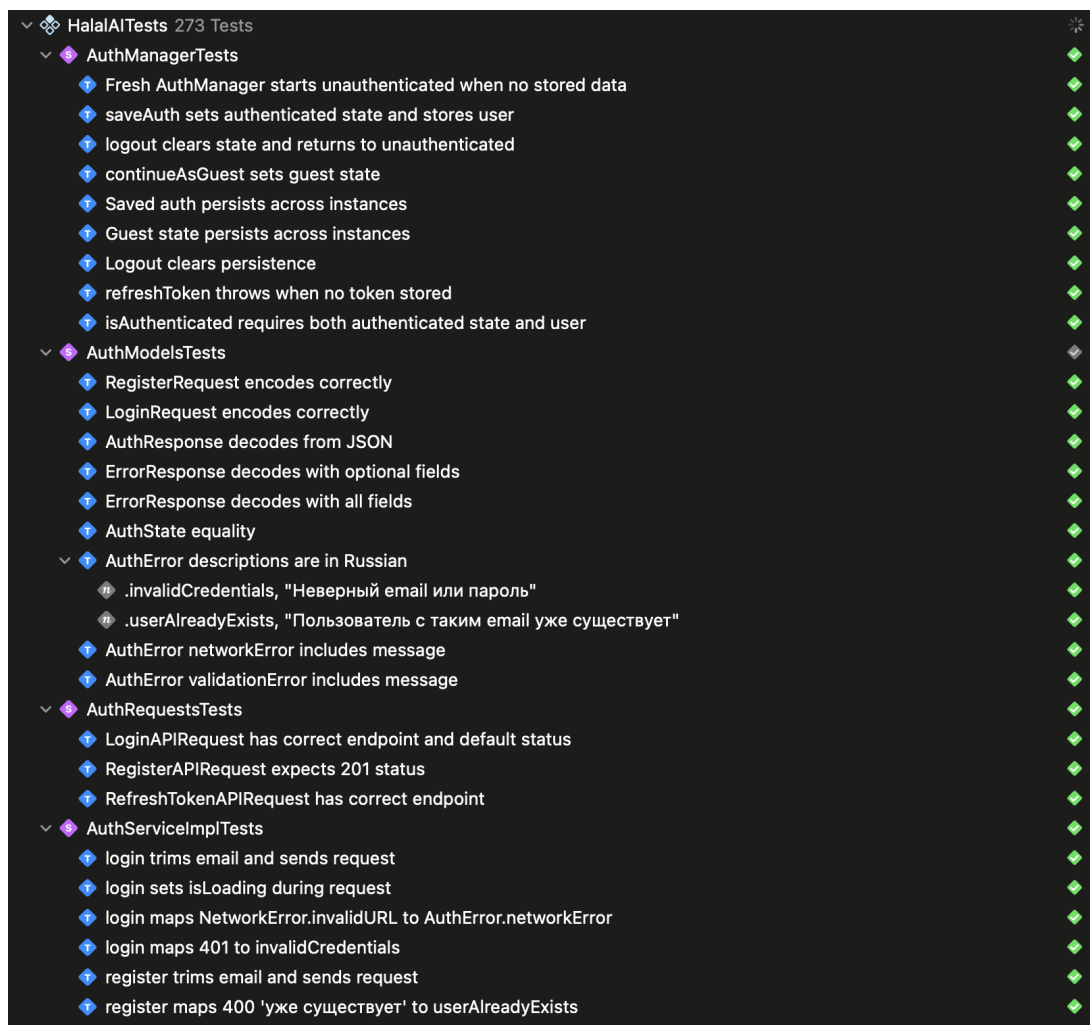


Рис.3.1. iOS Unit-тесты модуля авторизации

Был протестирован модуль анализа ингредиентов, так как данный компонент реализует основную функциональность приложения. Для модуля IngredientMatcher были реализованы тесты распознавания Е-кодов, определения статусов «халяль», «харам» и «сомнительно», а также обработки граничных случаев пользовательского ввода. Отдельно тестировались алгоритмы оценки совпадений ингредиентов, обработка специальных символов, а также регистронезависимый поиск (рисунок 3.2).



Рис.3.2. iOS Unit-тесты модуля анализа ингредиентов

Были написаны тесты для проверки корректности расчетов времени молитв, проверялось: корректность порядка времен молитв, поддержка различных методов расчета, различия между мазхабами, корректность определения следующей молитвы (рисунок 3.3).

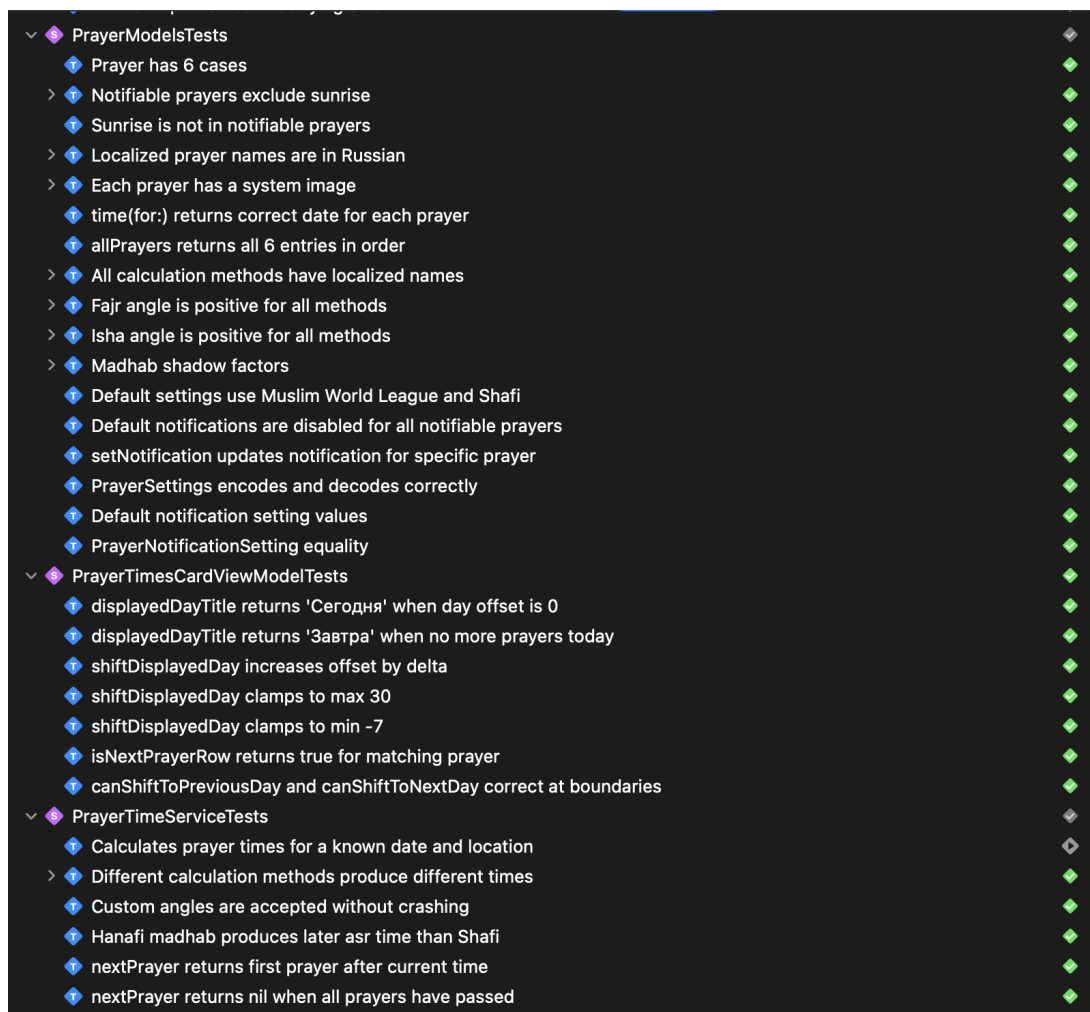


Рис.3.3. iOS Unit-тесты модуля расчета времени молитв

Также были реализованы тесты модулей AI-чата и сетевого взаимодействия. Проверялись сценарии отправки сообщений, обработка пустого пользовательского ввода, изменение состояния соединения, загрузка доступных моделей и обработка ошибок backend-сервиса (рисунок 3.4).

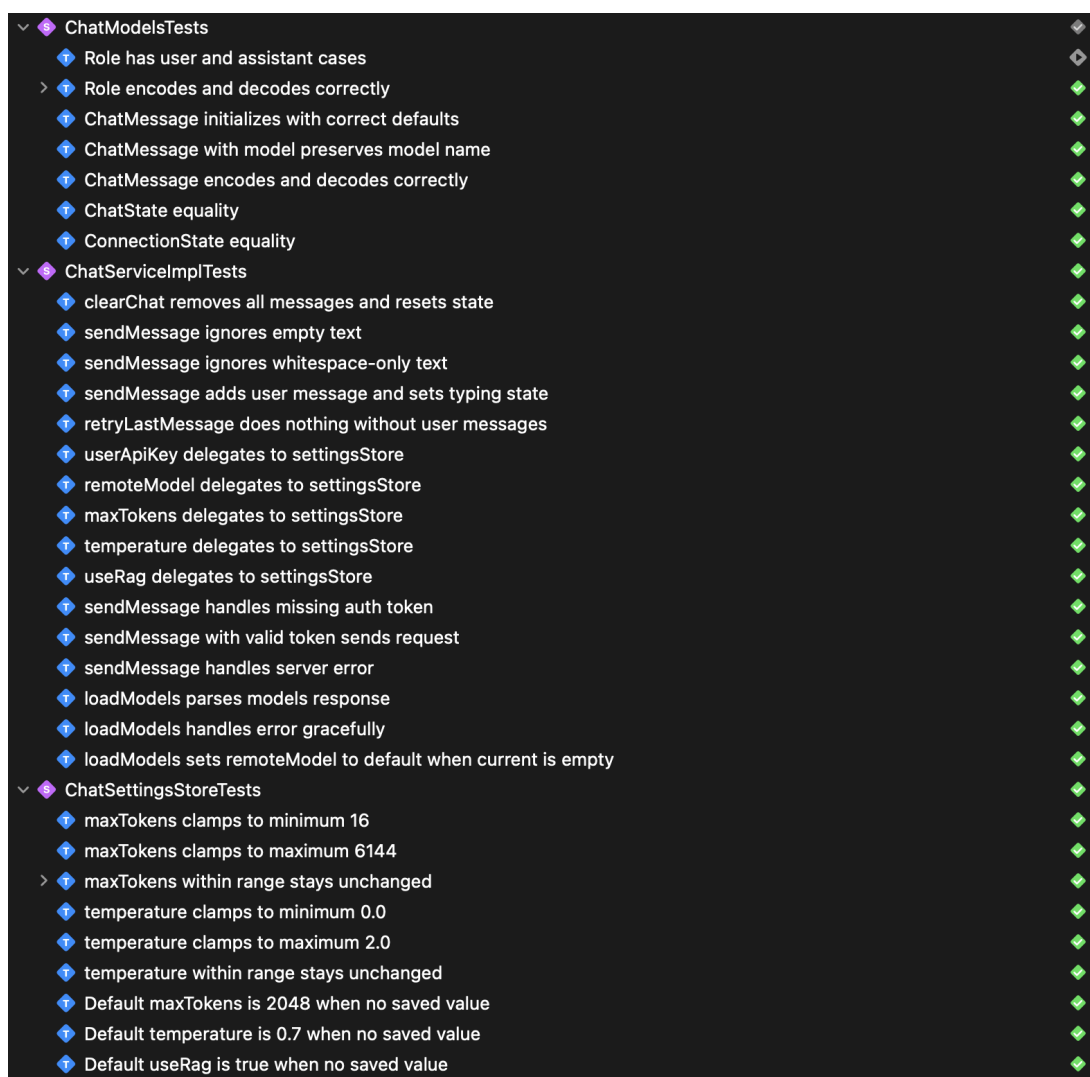


Рис.3.4. iOS Unit-тесты модуля чата

Также было написано множество тестов для навигации приложения, механизмов кеширования пользовательских данных и ViewModel-ей более мелких экранов. Всего удалось покрыть unit-тестами 29,4% - это 2779 из 9452 исполняемых строк. Относительно низкий процент объясняется архитектурными особенностями iOS-приложения - большая часть кода относится к слою пользовательского интерфейса, который содержит преимущественно декларативное описание интерфейса и ограниченное количество бизнес-логики, чтобы повысить процент покрытия будут написаны UI-тесты для важных экранов. Если же смотреть на покрытие бизнес-логики, то это будет 71,01% (1369 / 1928) (рисунок 3.5).

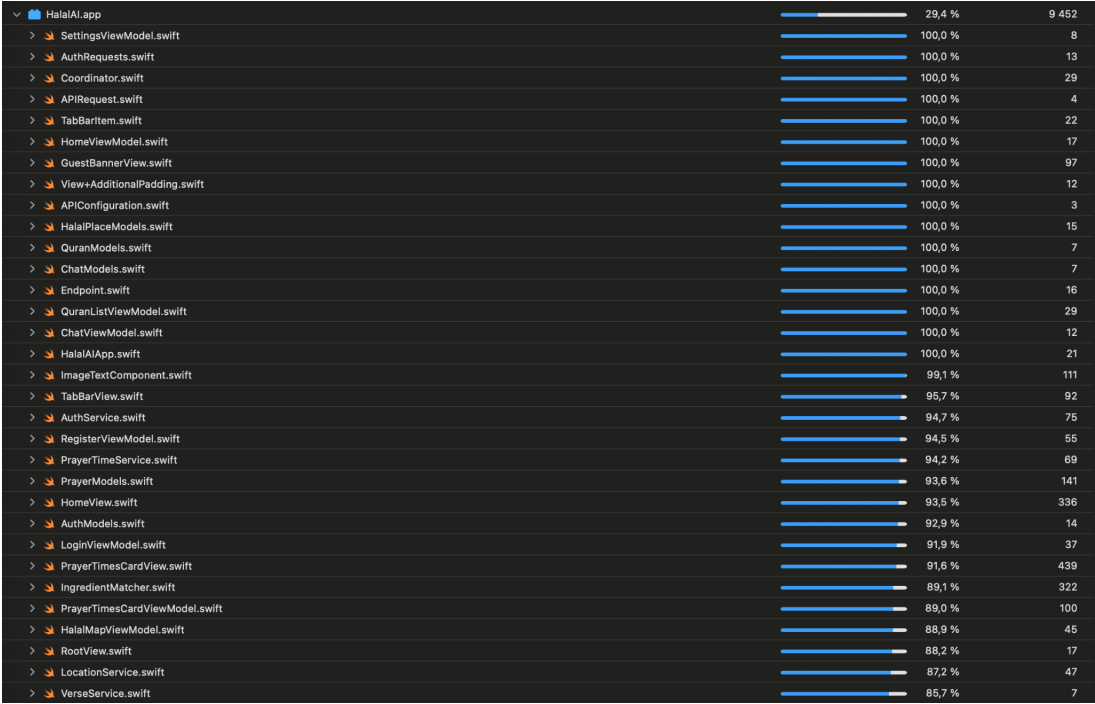


Рис.3.5. iOS Unit-тесты покрытие

3.1.2 UI-тестирование

UI-тестирование выполнялось с помощью фреймворка XCUITest и было направлено на проверку основных пользовательских сценариев приложения. Всего было написано 28 UI-тестов, охватывающих ключевые элементы навигации и взаимодействия пользователя с системой.

Основная часть тестов была сосредоточена на проверке экранов авторизации и регистрации, навигации между табами в приложении, корректности отображения основных экранов, доступности функциональности для гостевого режима и ограничений доступа к отдельным функциям приложения (рисунок 3.6).



Рис.3.6. iOS UI-тесты

После написания UI-тестов общее покрытие проекта iOS тестами увеличилось до 57.6% (рисунок 3.7).



Рис.3.7. Покрытие кода после написания UI-тестов

3.2 Тестирование backend-сервиса

Для backend были написаны автоматизированные тесты, покрывающие основные уровни архитектуры: контроллеры, сервисы, механизмы безопасности, обработку исключений и интеграцию с внешним LLM-сервисом.

Тестирование выполнялось с использованием фреймворков Spring Boot Test, JUnit 5, Mockito и MockMvc. Для изоляции тестируемых компонентов применялись mock-объекты, это позволило проверять бизнес-логику независимо от реальных внешних зависимостей, базы данных и сетевых запросов.

Были реализованы следующие категории тестов: unit-тесты сервисного слоя, тесты REST-контроллеров, тесты JWT-аутентификации и компонентов безопасности, тесты глобального обработчика исключений, интеграционные smoke-тесты Spring Boot-приложения, тесты загрузки и обработки CSV-данных, тесты интеграции с LLM-сервисом.

Также были написаны тесты для сервиса генерации ответов от LLM-Service. Были написаны тесты успешной генерации ответа, обработки параметров генерации и различных ошибочных сценариев.

Для подсистемы аутентификации были протестированы сценарии регистрации, входа пользователя, обновления JWT-токена, обработки неверных учетных данных, блокировки отключенных пользователей и защиты от использования некорректных токенов.

Тестирование REST-контроллеров выполнялось с использованием MockMvc и включало проверку: корректности HTTP-статусов, структуры JSON-ответов, обработки ошибок валидации, корректного преобразования исключений в HTTP-ответы.

Также были написаны тесты для загрузки аятов Корана из CSV-файлов, проверки корректности парсинга данных и пропуска некорректных строк. Всего backend-часть проекта содержит 15 файлов автоматизированных тестов, покрывающих основные критические сценарии работы системы, включая безопасность, бизнес-логику и интеграцию с внешними сервисами.

Для оценки качества тестирования проекта использовался инструмент JaCoCo, позволяющий анализировать степень покрытия кода автоматизированными тестами. По результатам тестирования были получены следующие показатели покрытия (рисунок 3.8):

- А. покрытие строк кода - 100% в анализируемом наборе классов.
- В. покрытие JVM-инструкций - 99%.
- С. покрытие ветвлений - 88%.

HalalAIBackend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.halalai.backend.controller		100 %		100 %	0	12	0	35	0	11	0	3
com.halalai.backend.service		98 %		91 %	7	46	0	161	2	17	0	5
com.halalai.backend.security		100 %		79 %	5	28	0	109	0	16	0	2
com.halalai.backend.config		100 %	n/a	n/a	0	11	0	37	0	11	0	2
com.halalai.backend.exception		100 %	n/a	n/a	0	12	0	36	0	12	0	1
com.halalai.backend.model		100 %	n/a	n/a	0	30	0	57	0	30	0	2
com.halalai.backend.dto		100 %	n/a	n/a	0	11	0	11	0	11	0	9
Total	11 of 1 823	99 %	10 of 84	88 %	12	150	0	446	2	108	0	24

Рис.3.8. Отчет JaCoCo о покрытии backend-сервиса

3.3 Тестирование LLM-сервиса

Тестирование LLM-сервиса проводилось с использованием фреймворка pytest. Основной целью тестирования является проверка RAG-системы, API-слоя, обработки ошибок, а также взаимодействия с внешними LLM-провайдерами. Были написаны Unit и интеграционные тесты. Unit тесты проверяли компоненты системы в изоляции от внешних зависимостей. Для этого использовались mock-объекты SentenceTransformer, SimpleRAG, OpenRouterClient и других сервисов. Интеграционные тесты проверяли работу HTTP-API FastAPI-приложения и взаимодействие компонентов между собой. Для ускорения и стабильности тестирования реальные embedding-модели и внешние LLM-запросы также заменялись mock-объектами.

Для VectorStore были написаны тесты, проверяющие работу поиска документов, корректность вычисления косинусной близости, обработку пустого хранилища, корректную работу параметра top_k, а также добавление новых документов и объединение embedding-векторов.

Для SimpleRAG тестировался поиск релевантных документов, обработка пустых запросов, ранжирование найденных результатов и корректное взаимодействие с embedding-моделью.

В сервисе обработки чатов ChatService были протестированы: извлечение пользовательского сообщения из истории диалога, формирование prompt, обработка отсутствующих сообщений, работа RAG, обработка ошибок OpenRouter API, обработка пустого ответа модели, сценарии работы без API-ключа, отключение и включение RAG-поиска.

Для OpenRouterClient были реализованы тесты успешного взаимодействия с API, обработки HTTP-ошибок, проверки корректности JSON-структуры ответа, формирования HTTP-заголовков и корректного использования системного prompt.

Для интеграционного тестирования API использовался FastAPI TestClient, это позволило не запускать отдельно сервер. Проверялись HTTP-коды ответа (200, 400, 422, 503), структура JSON-ответов и корректность обработки ошибок (рисунок 3.9).

```
tests/integration/test_hala_rag_api.py::test_root PASSED [ 1%]
tests/integration/test_hala_rag_api.py::test_llm_health PASSED [ 3%]
tests/integration/test_hala_rag_api.py::test_llm_info PASSED [ 5%]
tests/integration/test_hala_rag_api.py::test_llm_chat_empty_messages PASSED [ 7%]
tests/integration/test_hala_rag_api.py::test_llm_chat_missing_messages_key PASSED [ 8%]
tests/integration/test_hala_rag_api.py::test_llm_chat_service_not_initialized_returns_503 PASSED [ 10%]
tests/integration/test_hala_rag_api.py::test_llm_chat_happy_path PASSED [ 12%]
tests/integration/test_main_entrypoint.py::test_main_py_invokes_unicorn_when_run_as_script PASSED [ 14%]
tests/integration/test_main_lifespan.py::test_lifespan_raises_when_quran_file_missing PASSED [ 15%]
tests/integration/test_main_lifespan.py::test_lifespan_propagates_when_json_invalid PASSED [ 17%]
tests/unit/test_chat_service.py::test_extract_user_message_last_user_wins PASSED [ 19%]
tests/unit/test_chat_service.py::test_extract_user_message_empty PASSED [ 21%]
tests/unit/test_chat_service.py::test_search_sources_empty_query PASSED [ 22%]
tests/unit/test_chat_service.py::test_format_sources_empty PASSED [ 24%]
tests/unit/test_chat_service.py::test_format_sources_joins PASSED [ 26%]
tests/unit/test_chat_service.py::test_handle_error_none_message PASSED [ 28%]
tests/unit/test_chat_service.py::test_handle_error_429 PASSED [ 29%]
tests/unit/test_chat_service.py::test_handle_error_401 PASSED [ 31%]
tests/unit/test_chat_service.py::test_handle_error_generic PASSED [ 33%]
tests/unit/test_chat_service.py::test_generate_response_no_api_key PASSED [ 35%]
tests/unit/test_chat_service.py::test_generate_response_openrouter_failure PASSED [ 36%]
tests/unit/test_chat_service.py::test_process_chat_missing_user_message PASSED [ 38%]
tests/unit/test_chat_service.py::test_process_chat_success_with_rag PASSED [ 40%]
tests/unit/test_chat_service.py::test_process_chat_rag_disabled_skips_search PASSED [ 42%]
tests/unit/test_chat_service.py::test_process_chat_empty_llm_reply_triggers_handle_error PASSED [ 43%]
tests/unit/test_chat_service.py::test_build_prompt_with_and_without_sources PASSED [ 45%]
tests/unit/test_dependencies.py::test_set_rag_clears_chat_service PASSED [ 47%]
tests/unit/test_dependencies.py::test_set_llm_client_clears_chat_service PASSED [ 49%]
tests/unit/test_dependencies.py::test_get_llm_client_without_key_returns_none PASSED [ 50%]
tests/unit/test_dependencies.py::test_get_llm_client_with_env_creates_openrouter PASSED [ 52%]
tests/unit/test_dependencies.py::test_get_chat_service_requires_rag PASSED [ 54%]
tests/unit/test_dependencies.py::test_get_chat_service_builds_singleton PASSED [ 56%]
```

Рис.3.9. Отчет JaCoCo о покрытии backend-сервис

По результатам выполнения тестов было достигнуто 100% покрытие строк кода для основных модулей LLM-сервиса. Всего было покрыто 381 исполняемое выражение. Часть интерфейсных классов была исключена из расчета покрытия, поскольку они содержат только абстрактные объявления и не включают исполняемую бизнес-логику (рисунок 3.10).

Coverage report: 100%

Files Functions Classes

coverage.py v7.14.0, created at 2026-05-13 21:24 +0300

File ▲	statements	missing	excluded	coverage
src/halal_rag/__init__.py	0	0	0	100%
src/halal_rag/api/__init__.py	0	0	0	100%
src/halal_rag/api/dependencies.py	57	0	0	100%
src/halal_rag/api/dto/__init__.py	6	0	0	100%
src/halal_rag/api/dto/api_info_response.py	6	0	0	100%
src/halal_rag/api/dto/chat_request.py	9	0	0	100%
src/halal_rag/api/dto/chat_response.py	6	0	0	100%
src/halal_rag/api/dto/health_response.py	5	0	0	100%
src/halal_rag/api/dto/root_response.py	4	0	0	100%
src/halal_rag/api/interfaces.py	5	0	2	100%
src/halal_rag/api/main.py	57	0	0	100%
src/halal_rag/api/services.py	79	0	0	100%
src/halal_rag/llm/__init__.py	0	0	0	100%
src/halal_rag/llm/interfaces.py	5	0	2	100%
src/halal_rag/llm/open_router.py	42	0	0	100%
src/halal_rag/rag/__init__.py	0	0	0	100%
src/halal_rag/rag/embeddings.py	38	0	0	100%
src/halal_rag/rag/interfaces.py	16	0	12	100%
src/halal_rag/rag/retriever.py	16	0	0	100%
src/halal_rag/rag/vector_store.py	30	0	0	100%
Total	381	0	16	100%

Рис.3.10. Отчет pytest о покрытии LLM-service

Также, хочется отметить что проверка качества системы осуществлялась не только посредством классического тестирования программного обеспечения, но и в рамках экспериментального исследования, проведенного на этапе разработки LLM-сервиса. В разделе 3.2.2 выполнялась проверка сформулированных гипотез, связанных с влиянием RAG-подсистемы, embedding-моделей и их дообучения на качество генерации ответов. Для этого были проведены эксперименты с различными конфигурациями системы и выполнена оценка качества retrieval-подсистемы и итоговых ответов языковой модели по набору критериев.

3.4 Проверка соответствия требованиям

3.4.1 Проверка качества распознавания состава продукта

Для проверки использовалось ручное тестирование. Для этого использовались продуктовые упаковки и было проведено сканирование состава через приложение. Результаты приведены в таблице 3.1.

Таблица 3.1

Результаты распознавания состава продуктов

№.	Продукт	Состав на упаковке	Распознано системой	% совпадения
1	Мармелад	Патока, сахар, вода, желатин, регулятор кислотности (лимонная кислота), концентрированный яблочный сок, масло пальмовое, сироп карамельный, ароматизаторы натуральные, глазирователь, красители (антоцианы, экстракт паприки, куркумин).	сахар, патока, желатин, антоцианы, ароматизаторы, лимонная кислота, пальмовое масло, вода.	72%

Продолжение табл. 3.1

No.	Продукт	Состав на упаковке	Распознано системой	% совпадения
2	Бекон	Грудореберная часть свиная, ширицовочный рассол (вода питьевая, комплексная пищевая добавка для мясной продукции с йодом (соль пищевая); фиксатор окраски нитрит натрия; йодат калия; агент антислеживающий E538), соль пищевая выварочная йодированная (соль пищевая выварочная, йодат калия, агент антислеживающий E538), комплексная пищевая добавка (регулятор кислотности E4351, стабилизатор E4501), комплексная пищевая добавка (консервант E2622(1)), регуляторы кислотности (E327, E331(1)(II)), пищевая добавка (загуститель E407A), белок говяжий животный, комплексная пищевая добавка (ароматизатор «говядина» (вещества вкусоароматические), усилитель вкуса и аромата E621, сухая патока глюкозы, декстроза, соль, лактоза), жидкий ароматизатор (масло чеснока, натуральный ароматизатор чеснока), пищевая добавка (агент желеобразующий E850), пищевая добавка (антислеживающий E300), пищевая добавка (загуститель E415).	глютен, патока, лактоза, (часть) свиная, декстроза, нитрит натрия, нитрит калия, вода, соль	38%

Продолжение табл. 3.1

№.	Продукт	Состав на упаковке	Распознано системой	% совпадения
3	Хлеб	вода, натуральная мука ржаная хлебопекарная обдирная, мука пшеничная хлебопекарная первого сорта, соль, дрожжи хлебопекарные	пшеничная мука, вода, мука, соль	100%

В ходе тестирования была проведена проверка работы системы распознавания состава продуктов на основе реальных упаковок и их сканирования через приложение (рисунок 3.11).

Полученные значения показывают, что система в целом корректно определяет ключевые ингредиенты продуктов, однако уровень совпадения варьируется в зависимости от сложности состава. Наилучший результат был достигнут для простого состава (хлеб), где совпадение составило 100%, что свидетельствует о высокой точности распознавания базовых ингредиентов. Для более сложных продуктов (мармелад и бекон) наблюдается снижение процента совпадения. Это связано с наличием комплексных пищевых добавок, вложенных составов и технологических компонентов (Е-добавок), не всегда явно интерпретируемых системой. Следует отметить, что система распознавания ориентирована не на полное восстановление состава продукта, а на выделение ключевых ингредиентов с точки зрения классификации по категориям халяль, харам и сомнительных компонентов.

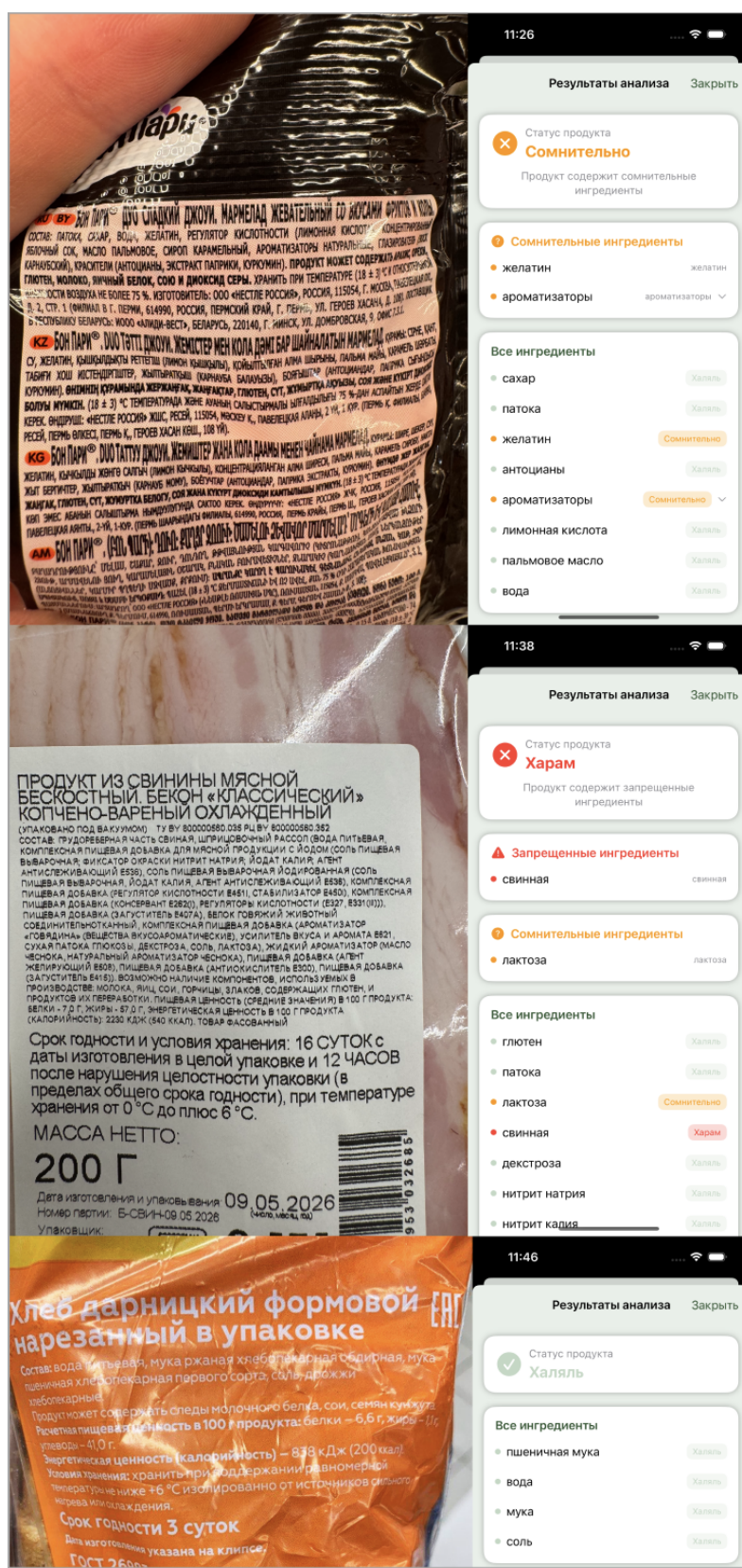


Рис.3.11. Сканирование реальных составов через приложение

3.4.2 Проверка корректности классификации ингредиентов

Для проверки корректности классификации ингредиентов были написаны Unit тесты. Объектом тестирования служил компонент IngredientMatcher. Результаты представлены в таблице 3.2.

Таблица 3.2

Результаты классификации ингредиентов

№.	Наименование теста	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	Классификация халяль-ингредиента по Е-коду	E100	Статус: Халяль	Статус: Халяль	Пройден
2	Классификация харам-ингредиента по Е-коду	E120	Статус: Харам	Статус: Харам	Пройден
3	Классификация мушбукх-ингредиента по Е-коду	E471	Статус: Мушбукх	Статус: Мушбукх	Пройден
4	Классификация харам-ингредиента по названию на русском	желатин свиной	Статус: Харам	Статус: Харам	Пройден
5	Классификация харам-ингредиента по названию на английском	pork gelatin	Статус: Харам	Статус: Харам	Пройден
6	Классификация мушбукх-ингредиента по названию	натуральный ароматизатор	Статус: Мушбукх	Статус: Мушбукх	Пройден
7	Приоритет харам над халяль	E100, E120	Итоговый статус: Харам	Итоговый статус: Харам	Пройден
8	Приоритет харам над мушбукх	E120, E471	Итоговый статус: Харам	Итоговый статус: Харам	Пройден

Продолжение табл. 3.2

№.	Наименование теста	Входные данные	Ожидаемый результат	Фактический результат	Статус
9	Приоритет мушбук над халяль при отсутствии харам	E100, E471	Итоговый статус: Мушбук	Итоговый статус: Мушбук	Пройден
10	Полная цепочка приоритетов: харам > мушбук > халяль	E100, E471, E120	Итоговый статус: Харам, обнаружено 3 ингредиента	Итоговый статус: Харам, обнаружено 3 ингредиента	Пройден
11	Ингредиент, отсутствующий в базе данных	E999	Статус: Неизвестно, список пуст	Статус: Неизвестно, список пуст	Пройден

Система корректно классифицирует ингредиенты по всем статусам как по E-коду, так и по текстовому названию. Соблюдается иерархия приоритетов - харам > мушбук > халяль > неизвестно. Все тесты пройдены успешно (рисунок 3.12).

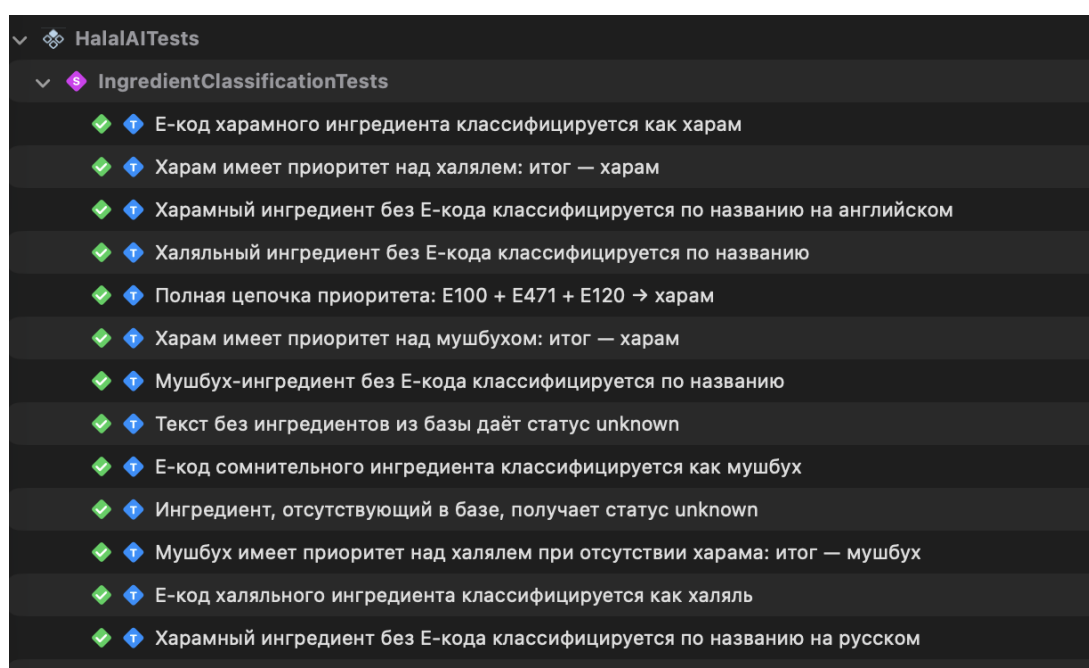


Рис.3.12. Результаты прохождения тестов для IngredientMatcher

3.4.3 Проверка реализации других модулей приложения

В приложении успешно реализован чат с AI-ассистентом, отвечающим на вопросы пользователей на основе текстов Корана (рисунок 3.13). В приложении

также реализован модуль чтения Корана, обеспечивающий доступ ко всем сурам и аятам, настройку размера текста и возможность продолжения чтения с последнего сохраненного места (рисунок 3.14). Также была реализована гибкая система настройки уведомлений о времени намаза с учетом выбранного пользователем мазхаба и метода расчета времени молитв (рисунок 3.15). Дополнительно в приложение была внедрена система локализации, обеспечивающая поддержку нескольких языков интерфейса и адаптацию контента в соответствии с языковыми предпочтениями пользователя (рисунки 3.16, 3.17).

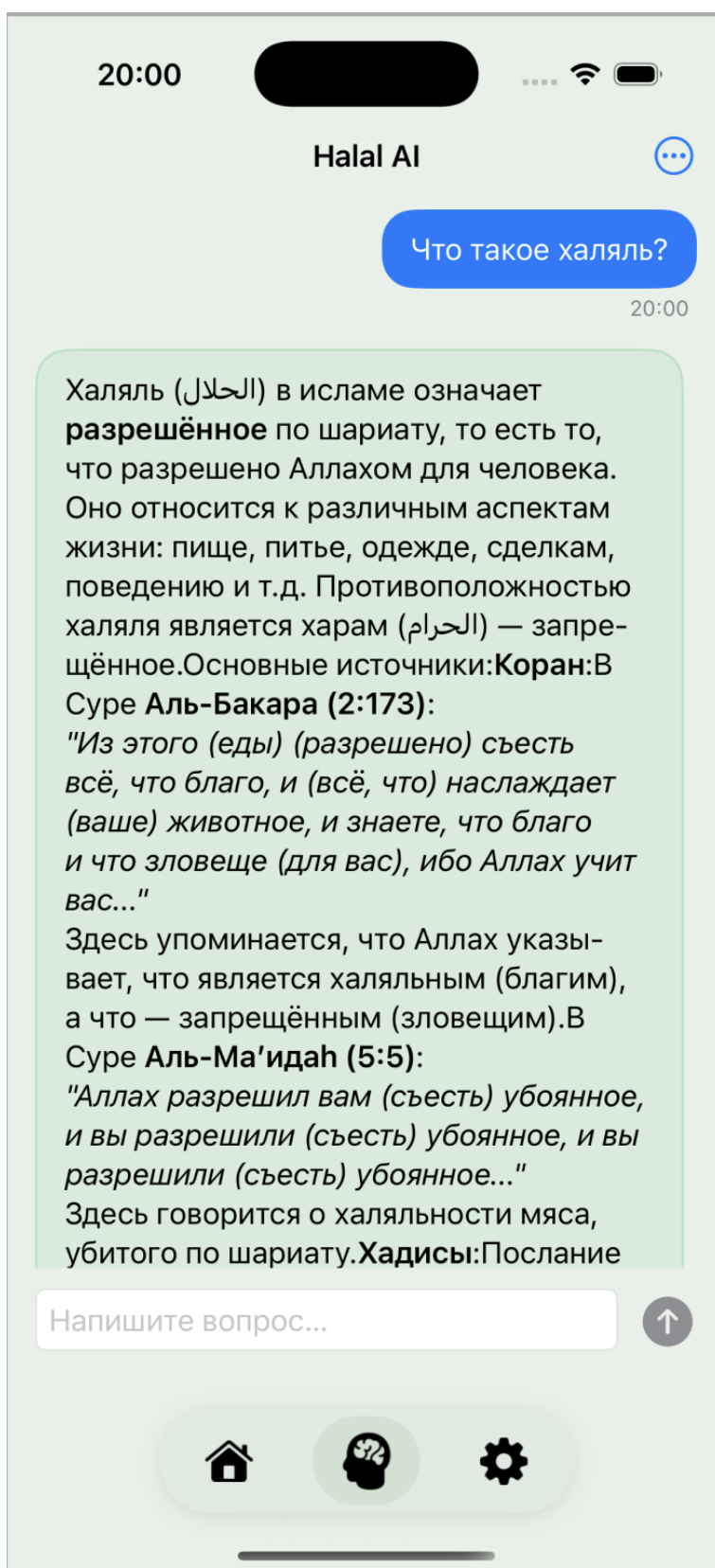


Рис.3.13. Реализованный экран с AI-ассистентом

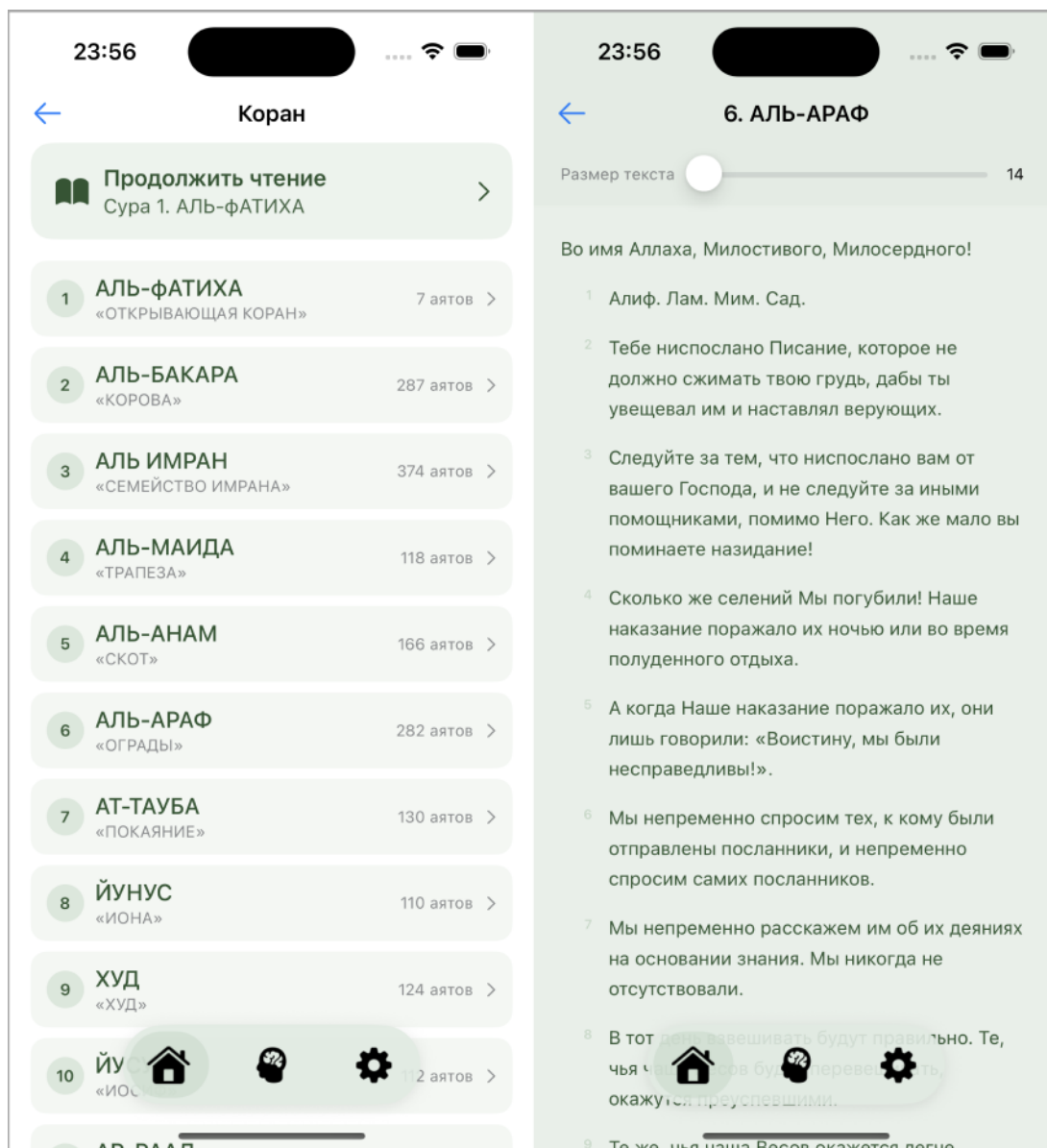


Рис.3.14. Реализованный экран чтения Корана

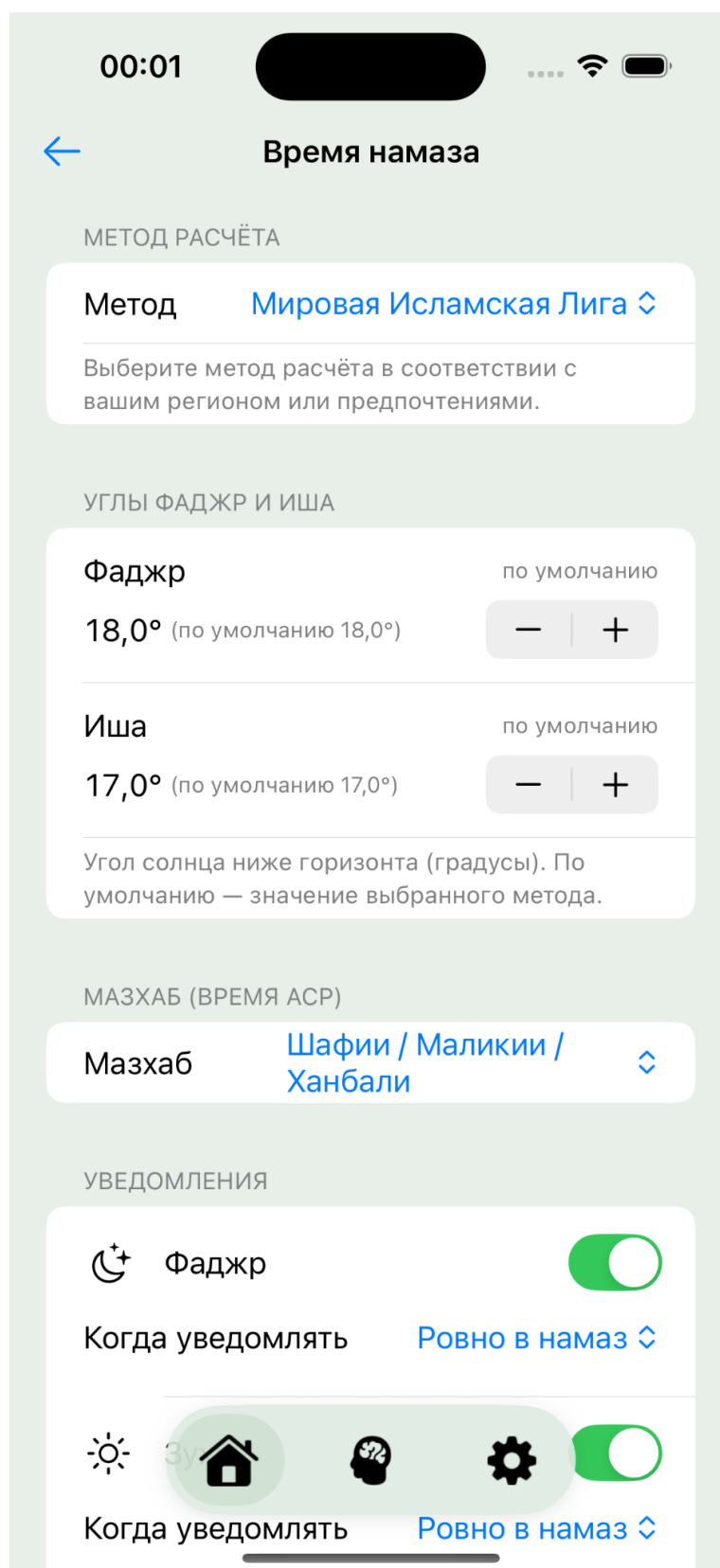


Рис.3.15. Реализованный экран настройки уведомлений о времени молитвы

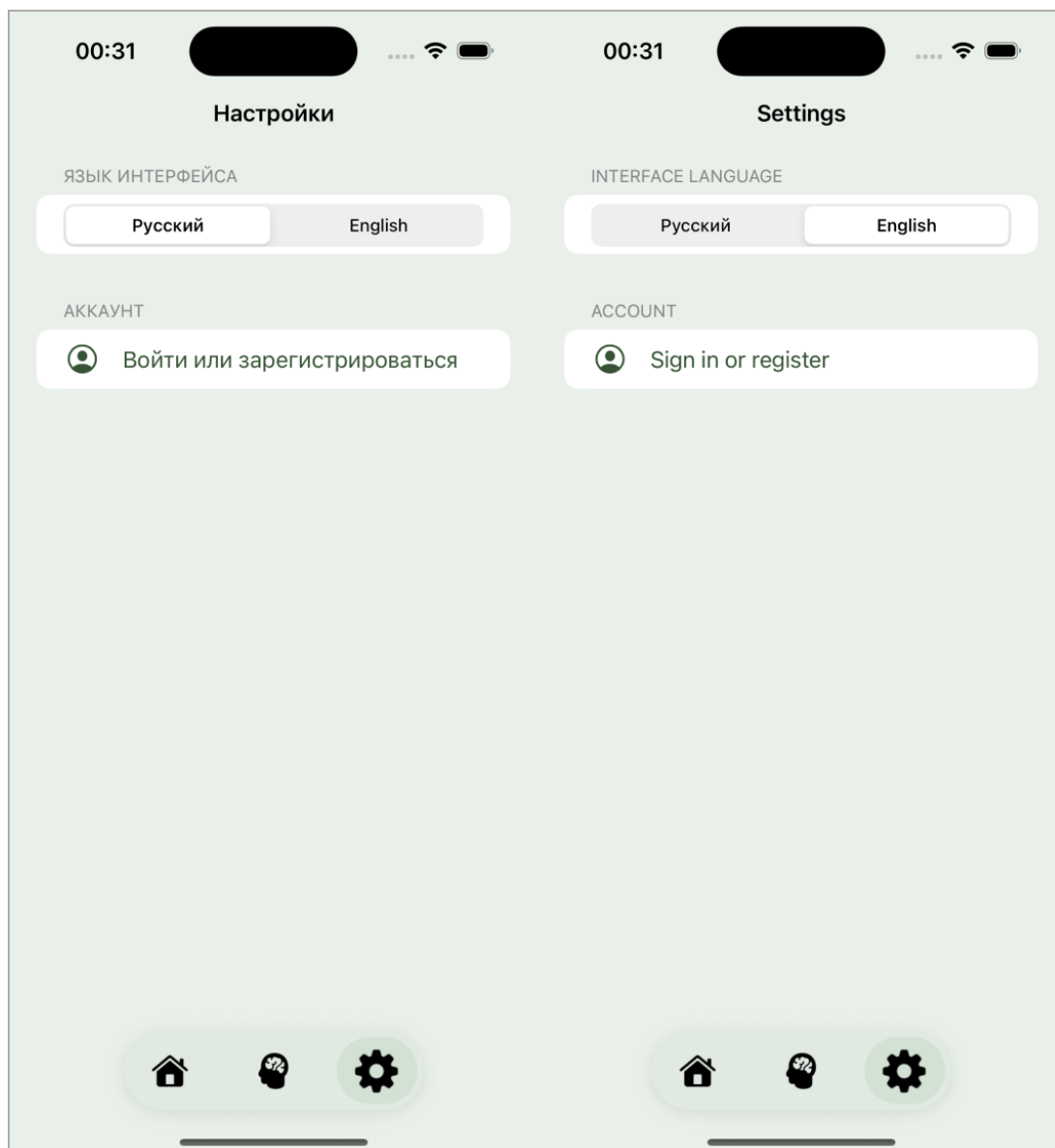


Рис.3.16. Реализованный экран настройки выбора языка



Рис.3.17. Главный экран на русском и английском языке

3.4.4 Проверка производительности системы

Проверка производительности проводилась с целью подтверждения соответствия времени отклика компонентов установленным требованиям. Тестирование охватывало три основных компонента системы: backend-приложение, LLM-сервис и мобильное iOS-приложение.

В рамках работы использовался подход SLA-тестирования (Service Level Agreement), при котором измеряется время выполнения конкретной операции и сравнивается с заранее установленным пороговым значением. В отличие от полноценного нагрузочного тестирования, данный подход позволяет проверить производительность приложения без необходимости разворачивания продуктивной инфраструктуры и внешних сервисов.


```

def test_health_endpoint_responds_within_sla(client):
    start = time.perf_counter()
    r = client.get("/llm/health")
    elapsed_ms = (time.perf_counter() - start) * 1000

    assert r.status_code == 200
    assert elapsed_ms < HEALTH_SLA_MS, (
        f"health превысил SLA: {elapsed_ms:.1f} мс > {HEALTH_SLA_MS} мс"
    )

def test_info_endpoint_responds_within_sla(client):
    start = time.perf_counter()
    r = client.get("/llm/info")
    elapsed_ms = (time.perf_counter() - start) * 1000

    assert r.status_code == 200
    assert elapsed_ms < INFO_SLA_MS, (
        f"info превысил SLA: {elapsed_ms:.1f} мс > {INFO_SLA_MS} мс"
    )

def test_chat_endpoint_responds_within_sla(client):
    start = time.perf_counter()
    r = client.post(
        "/llm/chat",
        json={
            "messages": [{"role": "user", "content": "Вопрос о халяле"}],
            "api_key": "test-key",
        },
    )
    elapsed_ms = (time.perf_counter() - start) * 1000

    assert r.status_code == 200
    assert elapsed_ms < CHAT_SLA_MS, (
        f"chat превысил SLA: {elapsed_ms:.1f} мс > {CHAT_SLA_MS} мс"
    )

```

Рис.3.19. Тесты производительности LLM-сервиса

В iOS производительность проверялась с использованием Swift Testing и ContinuousClock.measure. Тестировалось время выполнения операций авторизации, регистрации, обновления токена, загрузки списка моделей, а также скорость локальных синхронных операций, связанных с обновлением состояния интерфейса. Особое внимание уделялось тому, чтобы операции изменения состояния чата не блокировали основной UI-поток (рисунок 3.20).

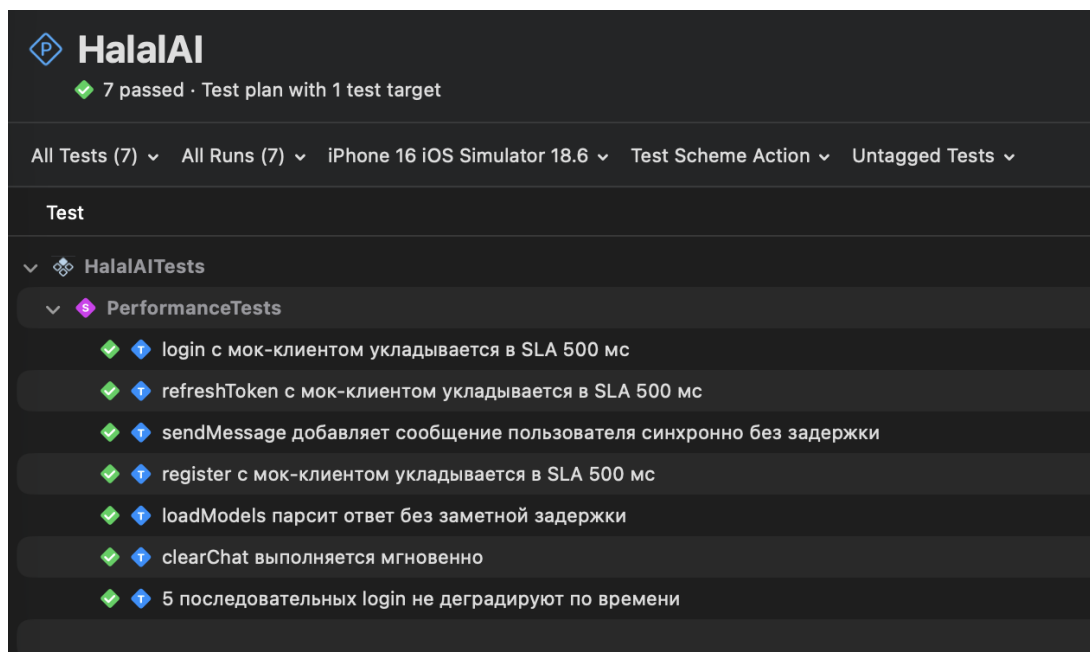


Рис.3.20. Пройденные тесты производительности iOS

Всего для проверки производительности было реализовано 20 автоматизированных тестов:

- A. 7 тестов для backend.
- B. 6 тестов для LLM-сервиса.
- C. 7 тестов для iOS-приложения.

Результаты тестирования показали, что все реализованные операции укладываются в установленные SLA-пороговые значения. Это подтверждает, что архитектура системы и выбранные технологии обеспечивают достаточную производительность для использования в мобильном приложении.

3.4.5 Проверка надежности системы

Проверка надежности проводилась с целью оценки устойчивости приложения к ошибкам, сбоям внешних сервисов и некорректным входным данным. Основная задача тестирования заключалась в подтверждении того, что система сохраняет работоспособность и корректно обрабатывает исключительные ситуации без аварийного завершения работы.

В ходе тестирования проверялись следующие свойства надежности:

- A. корректная деградация при отказе внешних сервисов.
- B. независимость компонентов системы.
- C. восстановление после временных сбоев.
- D. устойчивость к повторным ошибочным запросам.

Е. отсутствие критических сбоев приложения при некорректных данных.

На стороне backend были реализованы тесты с использованием JUnit 5 и MockMvc. Проверялись сценарии недоступности LLM-сервиса, возврата ошибок со стороны внешних зависимостей, повторных неудачных запросов и восстановления после временного отказа. Например, при выбросе исключения со стороны LLM-сервиса backend должен был возвращать структурированный JSON-ответ с кодом ошибки, а не необработанный 500 Internal Server Error. Также проверялось, что отказ чат-сервиса не влияет на доступность эндпоинтов аутентификации (рисунок 3.21).

```

41 class ReliabilityTest {
42     void chat_when_llm_unavailable_then_chat_service_returns_exception() throws Exception {
43     }
44 }
45
46 // --- Авторизация работает независимо от состояния LLM
47
48 @test
49 void auth_service_remains_usable_when_chat_service_fails() throws Exception {
50     when(llmService.generateCompletion(any(), any(), any(), any(), any()))
51         .thenReturn(new RuntimeException("LLM не отвечает"));
52     when(authService.login(any()))
53         .thenReturn(new Authentication("test", "password", 1L, "mb.com"));
54
55     // chat user
56     mockMvc.perform(post("/api/chat")
57         .contentType(MediaType.APPLICATION_JSON)
58         .content(objectMapper.writeValueAsString(new ChatRequest(
59             List.of(Map.of("role", "user", "password", "q1")),
60             null, null, null, null)))
61         .andExpect(status().isOk()));
62
63     // auth user password
64     mockMvc.perform(post("/api/auth/login")
65         .contentType(MediaType.APPLICATION_JSON)
66         .content(objectMapper.writeValueAsString(
67             new LoginRequest("mb.com", "password123"))))
68         .andExpect(status().isOk())
69         .andExpect(jsonPath("$.token").value("test"));
70 }
71
72 // --- Параллельность и восстановление
73
74 @test
75 void login_fails_if_requests_are_failed() throws Exception {
76     when(authService.login(any()))
77         .thenReturn(new Authentication("test", "password", 1L, "mb.com"));
78
79     for (int i = 0; i < 10; i++) {
80         mockMvc.perform(post("/api/auth/login")
81             .contentType(MediaType.APPLICATION_JSON)
82             .content(objectMapper.writeValueAsString(
83                 new LoginRequest("mb.com", "password123"))))
84             .andExpect(status().isOk());
85     }
86 }
87
88 @test
89 void chat_recovers_after_transient_failure() throws Exception {
90     when(llmService.generateCompletion(any(), any(), any(), any(), any()))
91         .thenReturn(new RuntimeException("временная ошибка"));
92     when(authService.login(any()))
93         .thenReturn(new Authentication("test", "password", 1L, "mb.com"));
94
95     mockMvc.perform(post("/api/chat")
96         .contentType(MediaType.APPLICATION_JSON)
97         .content(objectMapper.writeValueAsString(new ChatRequest(
98             List.of(Map.of("role", "user", "password", "q1")),
99             null, null, null, null)))
100         .andExpect(status().isOk()));

```

```

[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Ti
[INFO] Results:
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ halal-ai-backend
[INFO] Loading execution data file /Users/user/Documents/ha
[INFO] Analyzed bundle 'HalalAIBackend' with 24 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.422 s
[INFO] Finished at: 2026-05-17T10:45:04+03:00
[INFO]

```

Рис.3.21. Тесты надежности backend

Для LLM-сервиса тестирование проводилось с использованием pytest и FastAPI TestClient. Проверялись сценарии неинициализированного сервиса, ошибок со стороны OpenRouter, пустых сообщений, последовательных запросов и восстановления после временных сбоев. Важным требованием являлось отсутствие необработанных исключений и корректное возвращение HTTP-статусов (рисунки 3.22, 3.23).


```

def test_chat_service_not_initialized_returns_503(client, monkeypatch):
    """Если ChatService не инициализирован – 503, не 500 и не краш."""
    monkeypatch.setattr("hahal_rag.api.dependencies.get_chat_service", lambda: None)

    r = client.post(
        "/llm/chat",
        json={"messages": [{"role": "user", "content": "Q"}], "api_key": "k"},
    )
    assert r.status_code == 503
    assert "not initialized" in r.json().get("detail", "").lower()

def test_chat_llm_error_returns_reply_not_500(client, monkeypatch):
    """При сбое OpenRouter сервис возвращает 200 с сообщением об ошибке, не 500."""
    monkeypatch.setattr(
        "hahal_rag.llm.open_router.OpenRouterClient.generate",
        AsyncMock(side_effect=RuntimeError("upstream timeout")),
    )

    r = client.post(
        "/llm/chat",
        json={
            "messages": [{"role": "user", "content": "Свинина дозволена?"}],
            "api_key": "test-key",
        },
    )
    assert r.status_code == 200
    data = r.json()
    assert "reply" in data
    assert data["reply"], "reply не должен быть пустым при ошибке LLM"

def test_empty_messages_returns_400_not_500(client):
    """Пустой список messages → 400 Bad Request, не 500."""
    r = client.post("/llm/chat", json={"messages": []})
    assert r.status_code == 400

```

Рис.3.22. Тесты надежности LLM-сервиса

```

cd /Users/user/Documents/hahalaINew/HalaIAI/HalaIAI-backend/LLM-service
/opt/homebrew/bin/python3.9 -m pytest tests/integration/test_reliability.py -v --override-ini="addopts="
===== test session starts =====
platform darwin -- Python 3.9.23, pytest-8.4.2, pluggy-1.6.0 -- /opt/homebrew/opt/python@3.9/bin/python3.9
cachedir: .pytest_cache
rootdir: /Users/user/Documents/hahalaINew/HalaIAI/HalaIAI-backend/LLM-service
configfile: pyproject.toml
plugins: anyio-4.11.0, asyncio-1.2.0
asyncio: mode=auto, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 11 items

tests/integration/test_reliability.py::test_health_always_returns_ok PASSED
tests/integration/test_reliability.py::test_health_shows_rag_ready_when_initialized PASSED
tests/integration/test_reliability.py::test_health_remains_available_after_multiple_chat_requests PASSED
tests/integration/test_reliability.py::test_chat_service_not_initialized_returns_503 PASSED
tests/integration/test_reliability.py::test_chat_llm_error_returns_reply_not_500 PASSED
tests/integration/test_reliability.py::test_empty_messages_returns_400_not_500 PASSED
tests/integration/test_reliability.py::test_sequential_chat_requests_all_succeed PASSED
tests/integration/test_reliability.py::test_service_recovers_after_llm_failure PASSED
tests/integration/test_reliability.py::test_missing_messages_field_returns_422_not_500 PASSED
tests/integration/test_reliability.py::test_non_json_body_returns_422_not_500 PASSED
tests/integration/test_reliability.py::test_repeated_health_checks_all_succeed PASSED
===== 11 passed in 0.07s =====

```

Рис.3.23. Пройденные тесты надежности LLM-сервиса

В мобильном приложении iOS проверялась устойчивость клиентской логики к сетевым ошибкам и сбоям API. Тесты подтвердили, что:

- А. приложение не завершается аварийно при отсутствии сети.

- В. состояние `isLoading` всегда корректно сбрасывается после ошибки.
- С. повторные ошибки не приводят к деградации состояния `ViewModel`.
- Д. после успешного запроса приложение корректно восстанавливается после предыдущего сбоя.
- Е. очистка состояния чата и выход из аккаунта корректно восстанавливают исходное состояние приложения.

Особое внимание уделялось сценариям, связанным с пользовательским интерфейсом. Для мобильного приложения критично, чтобы даже при ошибках серверной части интерфейс не переходил в "зависшее" состояние (рисунок 3.24).

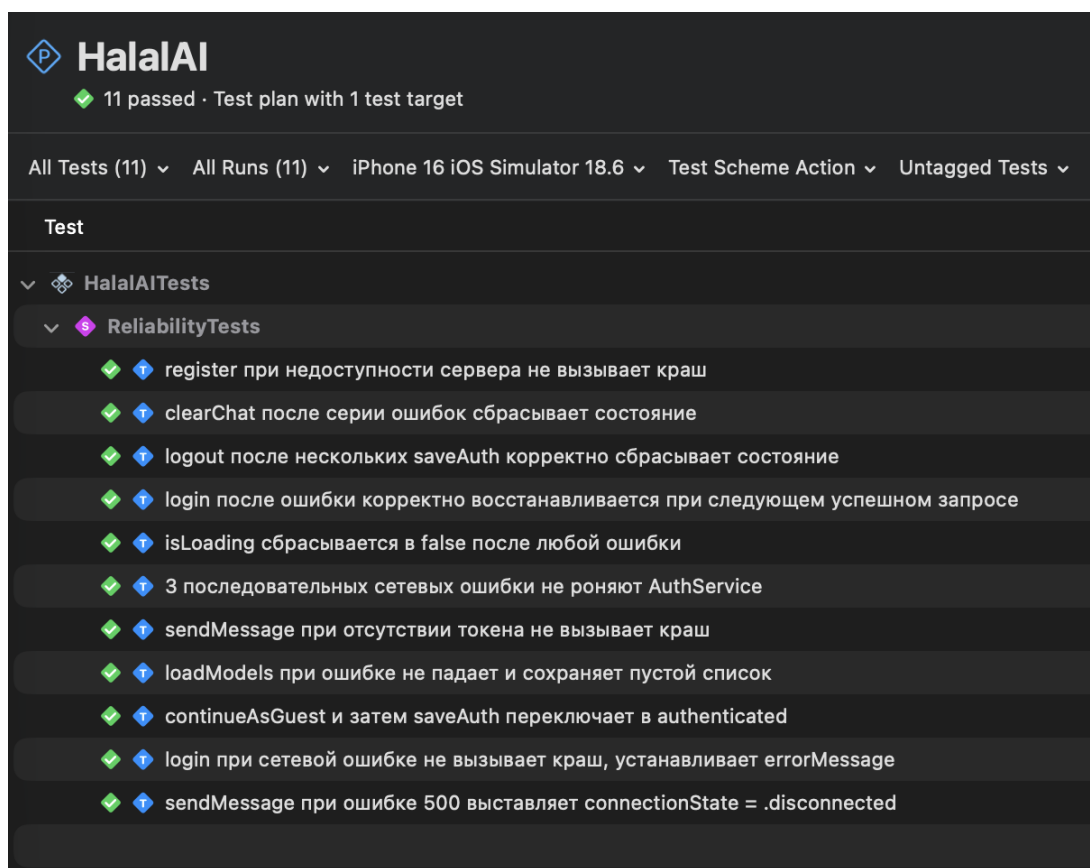


Рис.3.24. Пройденные тесты надежности iOS

Всего для проверки надежности было реализовано 29 автоматизированных тестов:

- А. 9 тестов для backend.
- В. 10 тестов для LLM-сервиса.
- С. 10 тестов для iOS-приложения.

Результаты тестирования показали, что система корректно обрабатывает сбои, не завершается аварийно и способна восстанавливаться после временных

ошибок. Это подтверждает устойчивость реализованной архитектуры и корректную обработку исключительных ситуаций на всех уровнях системы.

3.4.6 Проверка безопасности

Проверка безопасности системы проводилась с целью подтверждения корректной реализации механизмов аутентификации, контроля доступа и обработки потенциально опасных входных данных.

Тестирование безопасности охватывало три направления:

- А. контроль доступа к защищенным ресурсам.
- В. корректную работу механизма JWT-аутентификации.
- С. устойчивость системы к некорректным и потенциально вредоносным данным.

Для backend-приложения были реализованы тесты на основе JUnit 5, MockMvc и Spring Security. В отличие от большинства тестов проекта, данные тесты выполнялись с использованием реальной цепочки фильтров Spring Security. Это позволило проверить фактическую работу механизмов авторизации и JWT-фильтрации.

В ходе тестирования проверялись следующие сценарии:

- А. доступ к защищённым эндпоинтам без JWT-токена.
- В. использование некорректного JWT.
- С. использование просроченного токена.
- Д. использование токена, подписанного неправильным ключом.
- Е. попытки передачи SQL-инъекций в заголовке Authorization.
- Ф. доступ к публичным эндпоинтам без авторизации.
- Г. корректная работа CORS-заголовков.

Результаты тестирования безопасности backend приведены на рисунках 3.25 и 3.26.

```

2026-05-17T10:49:36.821+03:00 INFO 19240 --- [HalalAIBackend]
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time
[INFO] Results:
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.12:report (report) @ halal-ai-backend ---
[INFO] Loading execution data file /Users/user/Documents/haha
[INFO] Analyzed bundle 'HalalAIBackend' with 24 classes
[INFO] BUILD SUCCESS
[INFO] Total time: 3.625 s
[INFO] Finished at: 2026-05-17T10:49:37+03:00

```

Рис.3.25. Пройденные тесты безопасности backend

```

@Test
void chat_withGarbledAuthorizationHeader_isRejected() throws Exception {
    var result = mockMvc.perform(post("/api/chat")
        .contentType(MediaType.APPLICATION_JSON)
        .header("Authorization", "' OR '1'='1'")
        .content(objectMapper.writeValueAsString(
            Map.of("messages",
                List.of(Map.of("role", "user", "content", "Q")))))
        .andReturn());

    int status = result.getResponse().getStatus();
    assertTrue(status == 401 || status == 403,
        "Мусорный заголовок Authorization должен быть отвергнут, получен: " + status);
}

```

Рис.3.26. Тесты на попытку передачи SQL-инъекций в заголовке Authorization

Отдельно проверялось, что только корректно подписанный JWT предоставляет доступ к защищенным ресурсам системы.

Для LLM-сервиса были реализованы тесты безопасности с использованием pytest. Проверялась устойчивость сервиса к различным типам вредоносных входных данных:

- SQL-инъекциям.
- XSS-строкам.
- shell-инъекциям.
- чрезмерно большим payload.
- некорректным Unicode-символам.
- null-значениям.
- отсутствующим обязательным полям.

Главным критерием являлось отсутствие внутренних ошибок сервера (500 Internal Server Error) даже при передаче потенциально опасных данных. Сервис

должен был либо успешно обработать запрос, либо вернуть ошибку валидации (400, 422, 413) (рисунки 3.27, 3.28).

```
def test_sql_injection_in_message_content_handled_safely(client):
    """SQL-инъекция в content не должна вызывать 500."""
    r = client.post(
        "/llm/chat",
        json={
            "messages": [{"role": "user", "content": "'; DROP TABLE users; --"}],
            "api_key": "test-key",
        },
    )
    assert r.status_code == 200, f"SQL-инъекция вызвала ошибку: {r.status_code}"
    assert "reply" in r.json()

def test_xss_in_message_content_handled_safely(client):
    """XSS-попытка в content не должна вызывать ошибку сервера."""
    r = client.post(
        "/llm/chat",
        json={
            "messages": [
                {"role": "user", "content": "<script>alert('xss')</script>"}
            ],
            "api_key": "test-key",
        },
    )
    assert r.status_code == 200
    assert "reply" in r.json()

def test_shell_injection_in_message_content_handled_safely(client):
    """Shell-инъекция в content не должна вызывать 500."""
    r = client.post(
        "/llm/chat",
        json={
            "messages": [{"role": "user", "content": "$(rm -rf /) && echo pwned"}],
            "api_key": "test-key",
        },
    )
    assert r.status_code == 200
    assert "reply" in r.json()
```

Рис.3.27. Тесты безопасности LLM-сервиса

```

/Users/user/Documents/hahalAINew/HalalAI/HalalAI-backend/LLM-service
/opt/homebrew/bin/python3.9 -m pytest tests/integration/test_reliability.py -v --override-ini="addopts="
===== test session start =====
platform darwin -- Python 3.9.23, pytest-8.4.2, pluggy-1.6.0 -- /opt/homebrew/opt/python@3.9/bin/python3.9
cachedir: .pytest_cache
rootdir: /Users/user/Documents/hahalAINew/HalalAI/HalalAI-backend/LLM-service
configfile: pyproject.toml
plugins: anyio-4.11.0, asyncio-1.2.0
asyncio: mode=auto, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 11 items

/integration/test_reliability.py::test_health_always_returns_ok PASSED
/integration/test_reliability.py::test_health_shows_rag_ready_when_initialized PASSED
/integration/test_reliability.py::test_health_remains_available_after_multiple_chat_requests PASSED
/integration/test_reliability.py::test_chat_service_not_initialized_returns_503 PASSED
/integration/test_reliability.py::test_chat_llm_error_returns_reply_not_500 PASSED
/integration/test_reliability.py::test_empty_messages_returns_400_not_500 PASSED
/integration/test_reliability.py::test_sequential_chat_requests_all_succeed PASSED
/integration/test_reliability.py::test_service_recovers_after_llm_failure PASSED
/integration/test_reliability.py::test_missing_messages_field_returns_422_not_500 PASSED
/integration/test_reliability.py::test_non_json_body_returns_422_not_500 PASSED
/integration/test_reliability.py::test_repeated_health_checks_all_succeed PASSED

===== 11 passed in 0.07s =====

```

Рис.3.28. Пройденные тесты безопасности LLM-сервиса

В iOS-приложении проверялись механизмы хранения и удаления токенов аутентификации, а также защита клиентской логики от некорректных пользовательских данных. Проверялось:

- А. отсутствие токена до авторизации;
- В. корректное сохранение JWT после входа;
- С. полное удаление токена и пользовательских данных после logout;
- Д. невозможность отправки API-запросов без токена;
- Е. защита от некорректных email-адресов и слишком длинных строк;
- Ф. устойчивость к SQL-инъекциям и XSS-строкам в полях ввода;
- Г. ограничение диапазонов параметров генерации (maxTokens, temperature).

Результаты тестирования безопасности iOS показаны на рисунке 3.29.

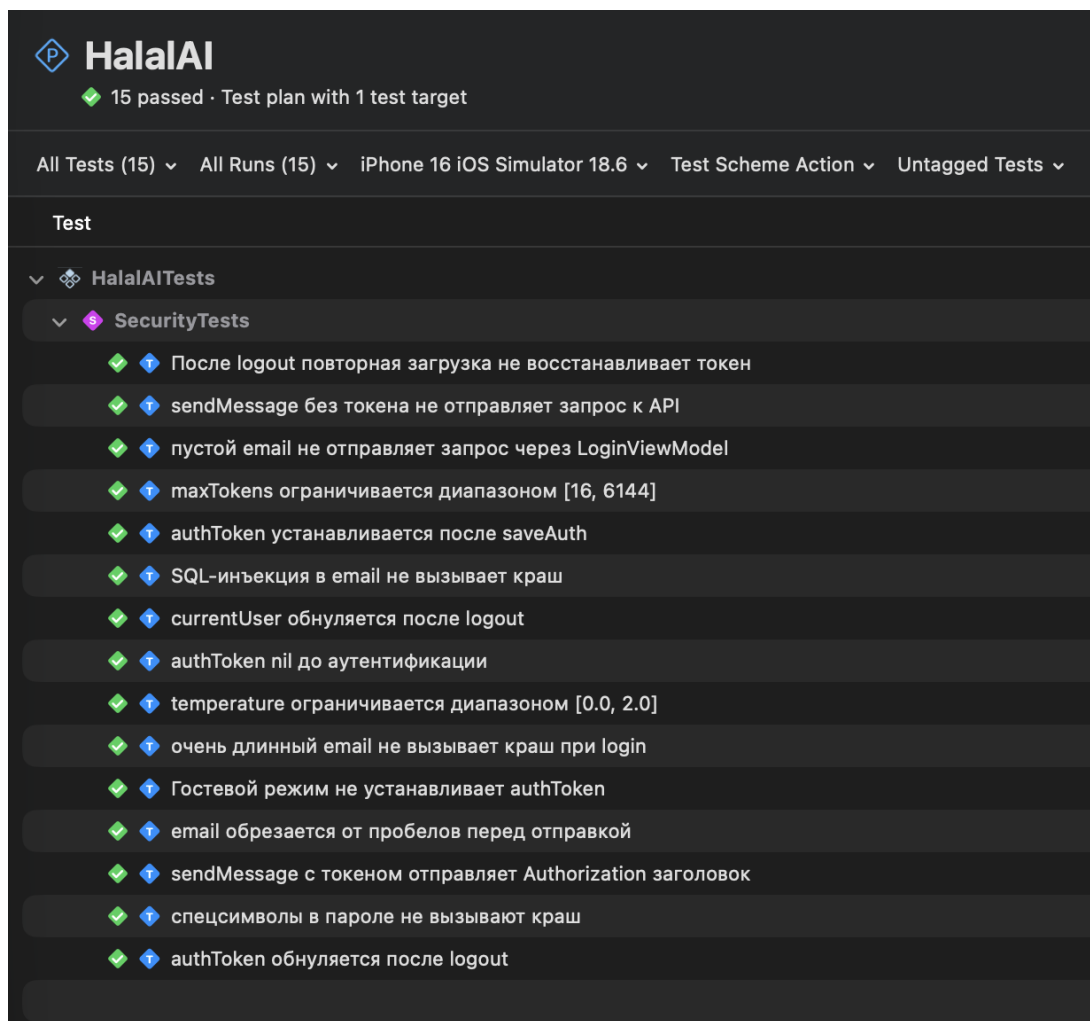


Рис.3.29. Пройденные тесты безопасности iOS

Всего для проверки безопасности было реализовано 34 автоматизированных теста:

- А. 9 тестов для backend.
- В. 11 тестов для LLM-сервиса.
- С. 14 тестов для iOS-приложения.

Результаты тестирования подтвердили корректную реализацию механизмов аутентификации и контроля доступа, а также устойчивость системы к некорректным и потенциально опасным входным данным. Дополнительно было установлено, что backend-приложение корректно использует цепочку фильтров Spring Security и действительно блокирует несанкционированный доступ к защищенным ресурсам.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы была разработана интеллектуальная система HalalAI, объединяющая мобильное приложение, backend и LLM-сервис для генерации ответов, семантического поиска и работы с релевантными исламскими источниками на основе технологий RAG.

В процессе разработки был реализован backend на Spring Boot, обеспечивающий REST API, аутентификацию пользователей, работу с JWT-токенами и взаимодействие с LLM-сервисом. Также было разработано iOS-приложение на SwiftUI, включающее систему авторизации, AI-чат, экран чтения Корана, модуль анализа состава продуктов и систему уведомлений о времени намаза.

Особое внимание было уделено разработке и исследованию retrieval-подсистемы. В рамках экспериментального исследования были проверены гипотезы о влиянии использования RAG и дообучения embedding-моделей на качество ответов системы. Проведенные эксперименты показали, что использование retrieval-механизма действительно повышает обоснованность и релевантность ответов, а применение дообученных embedding-моделей улучшает качество семантического поиска.

Для обеспечения надежности проекта были написаны тесты для всех ключевых компонентов системы. Backend покрыт Unit и интеграционными тестами с использованием JUnit, Mockito и MockMvc. LLM-сервис протестирован с помощью pytest, было достигнуто полное покрытие строк тестируемого кода. Клиентское приложение покрыто Unit и UI-тестами, проверяющими основные пользовательские сценарии и бизнес-логику системы.

В ходе выполнения работы были решены следующие задачи:

- A. Проведен анализ предметной области и существующих решений для мусульманской аудитории.
- B. Сформулированы функциональные и нефункциональные требования к разрабатываемой системе.
- C. Разработана архитектура взаимодействия мобильного клиента, backend-сервиса и LLM-модуля.
- D. Спроектирована структура базы данных и retrieval-подсистема для семантического поиска.
- E. Разработан REST API для взаимодействия компонентов системы.
- F. Реализован LLM-сервис с использованием RAG-подхода.

Г. Разработано iOS-приложение, удовлетворяющее поставленным требованиям.

Н. Проведена интеграция компонентов системы и их тестирование.

Таким образом, цель выпускной квалификационной работы была достигнута. Разработанная система продемонстрировала работоспособность выбранной архитектуры и эффективность применения retrieval-подсистемы и современных языковых моделей для решения задач, связанных с исламским информационным контекстом и анализом халяльности продуктов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Key attributes of mobile apps for halal tourism: an analysis of online reviews using Leximancer / ResearchGate. — URL: https://www.researchgate.net/publication/395269599_Key_attributes_of_mobile_apps_for_halal_tourism_an_analysis_of_online_reviews_using_Leximancer.
2. The future of the global Muslim population / Pew Research Center. — URL: <https://www.pewresearch.org/religion/2011/01/27/the-future-of-the-global-muslim-population/>.
3. Halal Food Market / Research Nester. — URL: <https://www.researchnester.com/ru/reports/halal-food-market/6076>.
4. Mobile-first marketing in app advertising / Halal Matchmaking API. — URL: <https://api.halalmatchmaking.org/advertiser/blog/mobile-first-marketing-in-app-advertising>.
5. App developers cater to Islamic consumer / Gulf News. — URL: <https://gulfnews.com/technology/media/app-developers-cater-to-islamic-consumer-1.1865595>.
6. Muzz / Muzz.com. — URL: <https://muzz.com/>.
7. К 2023 году рынок халяль-продуктов и мусульманских услуг достигнет 3 млрд долларов / Produkt.by. — URL: <https://produkt.by/news/novosti-mira/k-2023-godu-rynok-khalyal-produktov-i-musulmanskikh-uslug-dostignet-3>.
8. Artificial Intelligence in the Halal Industry: Bibliometric Analysis and Future Research Directions / Indonesian Journal of Halal Industry. — URL: <https://journal.uin.ac.id/IJHI/article/view/39184/18195>.
9. Survey, Analysis and Issues of Islamic Mobile Applications. — URL: <https://d1wqtxts1xzle7.cloudfront.net/69684260/pdf-libre.pdf>.
10. Цифровизация религии: ислам в условиях информационного общества / КиберЛенинка. — URL: <https://cyberleninka.ru/article/n/tsifrovizatsiya-religii-islam>.
11. Специфика цифровизации ислама и IT-халяль / КиберЛенинка. — URL: <https://cyberleninka.ru/article/n/spetsifika-tsifrovizatsii-islama-i-it-halyal>.
12. Artificial Intelligence-Based Quranic Chatbots and Digital Religious Tools / Journal of Contemporary Humanitarian Research. — URL: <https://publish.kne-publishing.com/index.php/JCHR/article/download/14792/13916>.
13. HALALCheck: A Multi-Faceted Approach for Intelligent Halal Packaged Food Recognition and Analysis / ResearchGate. — URL: <https://www.researchgate.net/>

publication/378348937_HALALCheck_A_Multi-Faceted_Approach_for_Intelligent_Halal_Packaged_Food_Recognition_and_Analysis.

14. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus, [et al.]. — URL: <https://arxiv.org/abs/2005.11401>.

15. Dense Passage Retrieval for Open-Domain Question Answering / V. Karpukhin, B. Oguz, S. Min, [et al.] // Proceedings of EMNLP 2020. — 2020. — URL: <https://aclanthology.org/2020.emnlp-main.550/>.

16. RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering / Y. Qu, Y. Cai, H. Lin, [et al.]. — URL: <https://arxiv.org/abs/2010.08191>.

Код LLM-Service

src/halal_rag/__init__.py

```
"""HalalAI RAG Service"""
```

src/halal_rag/api/__init__.py

```
"""API module"""
```

src/halal_rag/api/dto/__init__.py

```
from .chat_request import ChatRequest
from .chat_response import ChatResponse
from .health_response import HealthResponse
from .api_info_response import ApiInfoResponse
from .root_response import RootResponse

__all__ = ["ChatRequest", "ChatResponse", "HealthResponse", "ApiInfoResponse", "
           RootResponse"]
```

src/halal_rag/api/dto/api_info_response.py

```
from pydantic import BaseModel

class ApiInfoResponse(BaseModel):
    name: str
    version: str
    description: str
    endpoints: dict[str, str]
```

src/halal_rag/api/dto/root_response.py

```
from pydantic import BaseModel

class RootResponse(BaseModel):
    message: str
    docs: str
```

src/halal_rag/api/dto/health_response.py

```
from pydantic import BaseModel

class HealthResponse(BaseModel):
    status: str
```

```
rag_ready: str
llm_ready: str
```

src/halal_rag/api/dto/chat_request.py

```
from typing import Optional
from pydantic import BaseModel

class ChatRequest(BaseModel):
    """Request model for /llm/chat endpoint"""
    messages: list[dict[str, str]]
    max_tokens: int = 256
    api_key: Optional[str] = None
    remote_model: str = "qwen/qwen3.6-plus:free"
    temperature: float = 0.7
    use_rag: bool = True
```

src/halal_rag/api/dto/chat_response.py

```
from typing import Optional
from pydantic import BaseModel

class ChatResponse(BaseModel):
    """Response model for /llm/chat endpoint"""
    reply: str
    used_remote: bool = False
    remote_error: Optional[str] = None
```

src/halal_rag/api/interfaces.py

```
"""Service interfaces for API layer"""

from abc import ABC, abstractmethod
from .dto import ChatRequest, ChatResponse

class IChatService(ABC):
    """Interface for chat service"""

    @abstractmethod
    async def process_chat(self, request: ChatRequest) -> ChatResponse:
        """Process chat request end-to-end"""
        pass
```

src/halal_rag/api/services.py

```
"""Business logic services"""

import logging
from typing import Optional

from halal_rag.llm.interfaces import ILLMClient
from halal_rag.rag.interfaces import IRAGPipeline
```

```

from .interfaces import IChatService
from .dto import ChatRequest, ChatResponse
from halal_rag.llm.open_router import OpenRouterClient

logger = logging.getLogger(__name__)

class ChatService(IChatService):
    """Service for handling chat requests"""

    def __init__(self, rag: Optional[IRAGPipeline], llm_client: Optional[
        ILLMClient]):
        self.rag = rag
        self.llm_client = llm_client
        # Create one reusable OpenRouter client
        self.openrouter_client = OpenRouterClient()

    def extract_user_message(self, messages: list[dict[str, str]]) -> str:
        """Extract last user message from conversation history"""
        for msg in reversed(messages):
            if msg.get("role") == "user":
                return msg.get("content", "").strip()
        return ""

    def search_sources(self, query: str, top_k: int = 3) -> list[dict]:
        """Search for relevant Quranic verses"""
        if not self.rag or not query:
            return []
        return self.rag.search(query, top_k=top_k)

    def format_sources(self, sources: list[dict]) -> str:
        """Format sources for LLM prompt"""
        if not sources:
            return "No sources found"
        return "\n\n".join([f"Cypa {r['sura']}: {r['verse']}\n{r['text']}" for r
            in sources])

    async def generate_response(
        self, query: str, sources: str, api_key: Optional[str], model: str,
        max_tokens: int, temperature: float = 0.7
    ) -> tuple[str, bool, Optional[str]]:
        """Generate response using LLM"""
        if not api_key:
            return "", False, "API key is required"

        try:
            reply = await self.openrouter_client.generate(
                query=query,
                sources=sources,
                api_key=api_key,
                model=model,
                max_tokens=max_tokens,
                temperature=temperature
            )
            return reply, True, None

        except Exception as e:
            error_msg = str(e)
            print(f" LLM generation failed: {error_msg}")
            return "", False, error_msg

    def handle_error(self, error: Optional[str]) -> str:

```

```

        """Generate user-friendly error message"""
        if not error:
            return "Извините, удаленная модель недоступна. Пожалуйста, проверьте ваш API ключ и попробуйте снова."

        if "429" in error or "Too Many Requests" in error:
            return "OpenRouter API вернул ошибку 429: слишком много запросов. Это может быть из-за лимита free модели или превышения rate limit. Пожалуйста, попробуйте позже."
        elif "401" in error or "Unauthorized" in error or "authentication" in error.lower():
            return "Ошибка аутентификации OpenRouter: ваш API ключ недействителен или истек. Проверьте настройки."
        else:
            return f"Ошибка при обращении к OpenRouter: {error}"

def build_prompt(self, query: str, sources: str) -> tuple[str, str]:
    """Build system prompt and user prompt"""
    system_prompt = """
        # Ты - HalalAI, опытный специалист по исламу.
        1. Задача - точно отвечать на вопросы, **основываясь на Коране и Хадисах**.
        2. Давать **четкие**, уважительные ответы, основанные на исламском учении.
        3. Всегда приводите конкретные номера сур и аятов.
        4. Отвечай на **русском** языке.
        """

    # Only include sources section if sources are provided
    if sources.strip():
        user_prompt = f"""
        # Вопрос : {query}
        Соответствующие аяты Корана:
        {sources}
        """
    else:
        user_prompt = f"""
        # Вопрос : {query}
        """

    return system_prompt, user_prompt

async def process_chat(self, request: ChatRequest) -> ChatResponse:
    """Process chat request end-to-end"""
    print("\n" + "="*80)
    print(f" PROCESSING NEW REQUEST (RAG={request.use_rag})")
    print("="*80)

    # 1. Extract message
    query = self.extract_user_message(request.messages)
    if not query:
        return ChatResponse(
            reply="No user message found",
            used_remote=False,
            remote_error="Invalid request"
        )

    print(f" Chat query: {query}\n model={request.remote_model}, use_rag={request.use_rag}")

    # 2. Search sources (only if RAG enabled)
    sources = []

```

```

sources_text = ""
if request.use_rag:
    sources = self.search_sources(query)
    sources_text = self.format_sources(sources)
    # Log retrieved sources
    print(f"\n RAG RETRIEVED {len(sources)} SOURCES:")
    for i, source in enumerate(sources, 1):
        score = source.get('score', 'N/A')
        sura = source.get('sura', 'N/A')
        verse = source.get('verse', 'N/A')
        text = source.get('text', '')[:80]
        print(f" {i}. [Score: {score:.3f}] Cypa {sura}:{verse} - {text
}...")

# 3. Build prompt
system_prompt, user_prompt = self.build_prompt(query, sources_text)
full_prompt = f"[SYSTEM]\n{system_prompt}\n\n[USER]\n{user_prompt}"

# 4. Generate response via LLM
reply, used_remote, error = await self.generate_response(
    query=query,
    sources=sources_text,
    api_key=request.api_key,
    model=request.remote_model,
    max_tokens=request.max_tokens,
    temperature=request.temperature
)

# 5. Handle errors if needed
if not reply:
    reply = self.handle_error(error)

print("="*80)
print(f" REQUEST COMPLETED")
print("="*80 + "\n")

return ChatResponse(
    reply=reply,
    used_remote=used_remote,
    remote_error=error
)

```

src/halal_rag/api/dependencies.py

```

"""Dependency Injection for FastAPI"""

import logging
import os
from typing import Optional

from halal_rag.llm.interfaces import ILLMClient
from halal_rag.llm.open_router import OpenRouterClient
from halal_rag.rag.interfaces import IRAGPipeline
from halal_rag.rag.retriever import SimpleRAG
from .interfaces import IChatService

logger = logging.getLogger(__name__)

class DependencyContainer:
    """Manages dependencies for the application"""

```

```

_rag: Optional[IRAGPipeline] = None
_llm_client: Optional[ILLMClient] = None
_chat_service: Optional[IChatService] = None

@classmethod
def set_rag(cls, rag: IRAGPipeline) -> None:
    """Set RAG instance (called during app startup)"""
    cls._rag = rag

@classmethod
def set_llm_client(cls, client: ILLMClient) -> None:
    """Set LLM client instance"""
    cls._llm_client = client

@classmethod
def get_rag(cls) -> Optional[IRAGPipeline]:
    """Get RAG instance"""
    return cls._rag

@classmethod
def get_llm_client(cls, api_key: Optional[str] = None) -> Optional[ILLMClient]:
    """Get or create LLM client (lazy initialization)"""
    if cls._llm_client is None:
        try:
            key = api_key or os.getenv("OPEN_ROUTER_KEY")
            if not key:
                raise ValueError("OPEN_ROUTER_KEY must be provided or set as environment variable")
            cls._llm_client = OpenRouterClient(api_key=key)
            print(" OpenRouter client initialized")
        except Exception as e:
            print(f" Failed to initialize OpenRouter client: {e}")
            cls._llm_client = None
    return cls._llm_client

@classmethod
def get_chat_service(cls) -> Optional[IChatService]:
    """Get or create ChatService singleton"""
    if cls._chat_service is None:
        rag = cls.get_rag()
        llm_client = cls.get_llm_client()
        if rag:
            from .services import ChatService
            cls._chat_service = ChatService(rag=rag, llm_client=llm_client)
            print(" ChatService initialized")
    return cls._chat_service

# Module-level convenience functions for backward compatibility
def set_rag(rag: IRAGPipeline) -> None:
    """Set RAG instance (called during app startup)"""
    DependencyContainer.set_rag(rag)
    # Reset chat service when RAG changes
    DependencyContainer._chat_service = None

def set_llm_client(client: ILLMClient) -> None:
    """Set LLM client instance"""
    DependencyContainer.set_llm_client(client)
    # Reset chat service when LLM client changes

```

```

        DependencyContainer._chat_service = None

def get_rag() -> Optional[IRAGPipeline]:
    """Get RAG instance"""
    return DependencyContainer.get_rag()

def get_llm_client(api_key: Optional[str] = None) -> Optional[ILLMClient]:
    """Get or create LLM client (lazy initialization)"""
    return DependencyContainer.get_llm_client(api_key=api_key)

def get_chat_service() -> Optional[IChatService]:
    """Get ChatService singleton"""
    return DependencyContainer.get_chat_service()

```

src/halal_rag/api/main.py

```

"""FastAPI application for HalalAI RAG Service"""

import logging
import json
from contextlib import asynccontextmanager
from pathlib import Path
from fastapi import FastAPI, HTTPException
from halal_rag.rag.retriever import SimpleRAG
from halal_rag.api import dependencies
from halal_rag.api.dto import ChatRequest, ChatResponse, HealthResponse,
    ApiInfoResponse, RootResponse

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Application startup and shutdown"""
    print(" Starting HalalAI RAG API...")

    # Startup: Initialize RAG
    try:
        print(" Loading RAG system...")
        data_file = Path(__file__).parent.parent.parent.parent / "data" / "quran_ru.jsonl"

        if not data_file.exists():
            raise FileNotFoundError(f"Quran data not found at {data_file}")

        docs = []
        with open(data_file, 'r', encoding='utf-8') as f:
            for line in f:
                line = line.strip()
                if line:
                    docs.append(json.loads(line))

        print(f" Loaded {len(docs)} Quranic verses")
        rag = SimpleRAG(documents=docs, model_type="paraphrase", use_finetuned=True)
        dependencies.set_rag(rag)
        print(" RAG system ready")
    except Exception as e:
        logger.error(f"Error during startup: {e}")

```



```

    except Exception as e:
        print(f" Failed to initialize RAG: {e}")
        raise

    print(" Application startup complete")
    yield

    # Shutdown
    print(" Shutting down RAG system...")

app = FastAPI(
    title="HalalAI RAG API",
    description="Semantic search and QA system for Quranic questions",
    version="1.0.0",
    lifespan=lifespan
)

@app.get("/llm/health", response_model=HealthResponse, tags=["Health"])
async def health_check() -> HealthResponse:
    """Check service availability"""
    rag = dependencies.get_rag()
    llm_client = dependencies.get_llm_client()

    return HealthResponse(
        status="ok",
        rag_ready="ready" if rag else "initializing",
        llm_ready="ready" if llm_client else "not initialized"
    )

@app.post("/llm/chat", response_model=ChatResponse, tags=["Chat"])
async def chat(request: ChatRequest):
    """Main chat endpoint for Q&A"""
    if not request.messages:
        raise HTTPException(status_code=400, detail="Messages cannot be empty")

    service = dependencies.get_chat_service()
    if not service:
        raise HTTPException(status_code=503, detail="Chat service not initialized")

    return await service.process_chat(request)

@app.get("/llm/info", response_model=ApiInfoResponse, tags=["Docs"])
async def api_info() -> ApiInfoResponse:
    """Get API information"""
    return ApiInfoResponse(
        name="HalalAI RAG API",
        version="1.0.0",
        description="LLM service with RAG integration for Quranic questions",
        endpoints={
            "health": "/llm/health",
            "chat": "/llm/chat (POST)",
            "info": "/llm/info",
            "docs": "/docs"
        }
    )

```

```

@app.get("/", response_model=RootResponse, tags=["Docs"], include_in_schema=False)
)
async def root() -> RootResponse:
    """Redirect to API info"""
    return RootResponse(
        message="See /llm/info for API documentation",
        docs="/docs"
    )

if __name__ == "__main__":
    import uvicorn

    uvicorn.run(
        app,
        host="localhost",
        port=8001,
        log_level="info"
    )

```

src/halal_rag/rag/__init__.py

```

"""RAG module"""

```

src/halal_rag/rag/interfaces.py

```

from abc import ABC, abstractmethod
from typing import Any
import torch

class IEmbeddingEncoder(ABC):
    """Interface for embedding dto"""

    @abstractmethod
    def encode(self, texts: list[str]) -> torch.Tensor:
        """Encode multiple texts to embeddings"""
        pass

    @abstractmethod
    def encode_single(self, text: str) -> torch.Tensor:
        """Encode single text to embedding"""
        pass

class IVectorSearcher(ABC):
    """Interface for vector search and storage"""

    @abstractmethod
    def add_documents(self, documents: list[dict[str, Any]], embeddings: torch.
Tensor) -> None:
        """Add documents and their embeddings to the store"""
        pass

    @abstractmethod
    def search(self, query_embedding: torch.Tensor, top_k: int = 3) -> list[dict[
str, Any]]:
        """Search for similar documents"""

```

```

        pass

class IRAGPipeline(ABC):
    """Interface for RAG pipeline"""

    @abstractmethod
    def search(self, query: str, top_k: int = 3) -> list[dict[str, Any]]:
        """Search for relevant documents based on query"""
        pass

```

src/halal_rag/rag/embeddings.py

```

from pathlib import Path

import torch
from sentence_transformers import SentenceTransformer

from .interfaces import IEmbeddingEncoder

class EmbeddingModel(IEmbeddingEncoder):

    # Маппинг типов моделей на полные имена
    MODEL_MAPPING = {
        "paraphrase": "sentence-transformers/paraphrase-multilingual-mpnet-base-v2",
        "sbert": "ai-forever/sbert_large_nlu_ru",
    }

    def __init__(
        self,
        model_type: str = "paraphrase",
        use_finetuned: bool = False,
    ):
        model_name = self.MODEL_MAPPING.get(model_type, self.MODEL_MAPPING["paraphrase"])

        if use_finetuned:
            if "sbert" in model_type.lower():
                finetuned_dir = "sbert-quranic-embeddings"
            else:
                finetuned_dir = "quranic-embeddings"

            finetuned_path = (
                Path(__file__).parent.parent.parent.parent
                / "models"
                / finetuned_dir
            )
            if finetuned_path.exists():
                print(f"Loading fine-tuned model from {finetuned_path}")
                self.model = SentenceTransformer(str(finetuned_path), device="cpu")

                print("Fine-tuned model loaded")
            else:
                print(
                    f"Fine-tuned model not found at {finetuned_path}, "
                    f"falling back to {model_name}"
                )
                try:

```

```

        self.model = SentenceTransformer(
            model_name, device="cpu", local_files_only=True
        )
    except Exception:
        print(f"Downloading model: {model_name}")
        self.model = SentenceTransformer(model_name, device="cpu")
else:
    try:
        self.model = SentenceTransformer(
            model_name, device="cpu", local_files_only=True
        )
        print(f"Loaded cached model: {model_name}")
    except Exception:
        print(f"Downloading model: {model_name}")
        self.model = SentenceTransformer(model_name, device="cpu")
        print(f"Model downloaded successfully")

print("Getting embedding dimension...")
self.embedding_dim = self.model.get_sentence_embedding_dimension()
print(f"Embedding dimension: {self.embedding_dim}")

def encode(self, texts: list[str]) -> torch.Tensor:
    embeddings = self.model.encode(
        texts,
        convert_to_tensor=True,
        normalize_embeddings=True,
        show_progress_bar=False,
    )
    return embeddings.cpu()

def encode_single(self, text: str) -> torch.Tensor:
    return self.encode([text])[0]

```

src/halal_rag/rag/vector_store.py

```

from __future__ import annotations
from typing import Any, Optional
import torch
from torch import nn

from .interfaces import IVectorSearcher

class VectorStore(IVectorSearcher):

    def __init__(self):
        self.documents: list[dict[str, Any]] = []
        self.embeddings: Optional[torch.Tensor] = None

    def add_documents(self, documents: list[dict[str, Any]], embeddings: torch.Tensor):
        self.documents.extend(documents)

        if self.embeddings is None:
            self.embeddings = embeddings
        else:
            self.embeddings = torch.cat([self.embeddings, embeddings], dim=0)

    def search(self, query_embedding: torch.Tensor, top_k: int = 3) -> list[dict[str, Any]]:
        if self.embeddings is None or not self.documents:

```

```

        return []

    if query_embedding.dim() == 1:
        query_embedding = query_embedding.unsqueeze(0)

    query = nn.functional.normalize(query_embedding, p=2, dim=1)
    doc_embeddings = nn.functional.normalize(self.embeddings, p=2, dim=1)

    scores = torch.matmul(doc_embeddings, query.T).squeeze(1)

    k = min(top_k, len(self.documents))
    top_scores, indices = torch.topk(scores, k=k)

    results = []
    for score, idx in zip(top_scores.tolist(), indices.tolist()):
        doc = self.documents[idx].copy()
        doc['score'] = float(score)
        results.append(doc)

    return results

```

src/halal_rag/rag/retriever.py

```

from typing import Any

from .embeddings import EmbeddingModel
from .vector_store import VectorStore
from .interfaces import IRAGPipeline, IEmbeddingEncoder, IVectorSearcher

class SimpleRAG(IRAGPipeline):
    def __init__(
        self,
        documents: list[dict[str, Any]],
        model_type: str = "paraphrase", # "paraphrase" или "sbert"
        use_finetuned: bool = False,
    ):

        self.embeddings: IEmbeddingEncoder = EmbeddingModel(model_type=model_type, use_finetuned=use_finetuned)
        self.store: IVectorSearcher = VectorStore()

        texts = [doc['text'] for doc in documents]
        embeddings = self.embeddings.encode(texts)

        self.store.add_documents(documents, embeddings)

    def search(self, query: str, top_k: int = 3) -> list[dict[str, Any]]:
        if not query or not query.strip():
            return []

        query_embedding = self.embeddings.encode_single(query)

        return self.store.search(query_embedding, top_k=top_k)

```

src/halal_rag/llm/__init__.py

```

"""LLM module"""

```

src/halal_rag/llm/interfaces.py

```
from __future__ import annotations
from abc import ABC, abstractmethod

class ILLMClient(ABC):
    """Interface for LLM clients"""

    @abstractmethod
    async def generate(
        self,
        query: str,
        sources: str,
        system_prompt: str | None = None,
        max_tokens: int = 1000,
        temperature: float = 0.7,
        model: str | None = None
    ) -> str:
        """Generate response using LLM"""
        pass
```

src/halal_rag/llm/open_router.py

```
import httpx
import logging
from typing import Optional
from .interfaces import ILLMClient

logger = logging.getLogger(__name__)

class OpenRouterClient(ILLMClient):

    def __init__(
        self,
        model: str = "openrouter/auto",
        base_url: str = "https://openrouter.ai/api/v1"
    ):
        self.model = model
        self.base_url = base_url
        self.client = httpx.AsyncClient(base_url=self.base_url)

    async def generate(
        self,
        query: str,
        sources: str,
        api_key: str,
        system_prompt: Optional[str] = None,
        max_tokens: int = 1000,
        temperature: float = 0.7,
        model: Optional[str] = None
    ) -> str:
        if not system_prompt:
            system_prompt = """
            # Ты - HalalAI, опытный специалист по исламу.
            1. Задача - точно отвечать на вопросы, **основываясь на Коране и Хадисах**.
```

```

2. Давать **четкие**, уважительные ответы, основанные на исламском учении.
3. Всегда приводите конкретные номера сур и аятов.
4. Отвечай на **русском** языке.
"""

prompt = f"""
# Вопрос : {query}
"""

if sources:
    prompt += f"""
        Соответствующие аяты Корана:
        {sources}
        """

try:
    effective_model = model if model else self.model
    print(f" Используем llm-модель: {effective_model}")

    print(f"=== SYSTEM PROMPT ===\n{system_prompt}")
    print(f"=== USER PROMPT ===\n{prompt}")

    response = await self.client.post(
        "/chat/completions",
        json={
            "model": effective_model,
            "messages": [
                {
                    "role": "system",
                    "content": system_prompt
                },
                {
                    "role": "user",
                    "content": prompt
                }
            ],
            "temperature": temperature,
            "max_tokens": max_tokens,
        },
        headers={
            "Authorization": f"Bearer {api_key}",
            "HTTP-Referer": "https://halalai.app",
            "X-Title": "HalalAI"
        }
    )

    response.raise_for_status()
    data = response.json()

    answer = data["choices"][0]["message"]["content"]
    print(
        f" OpenRouter response: "
        f"{len(answer)} chars, "
        f"tokens: {data['usage']['total_tokens']}"
    )
    print(f"=== LLM RESPONSE ===\n{answer}")

    return answer

except httpx.HTTPError as e:
    print(f" OpenRouter API error: {e}")
    raise

```

```
        except KeyError as e:
            print(f" Unexpected OpenRouter response format: {e}")
            raise ValueError(f"Invalid OpenRouter response: {e}")

    async def close(self):
        try:
            await self.client.aclose()
        except Exception as e:
            pass # Ignore close errors
```


Код Backend

src/main/java/com/halalai/backend/HalalAiBackendApplication.java

```
package com.halalai.backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class HalalAiBackendApplication {

    public static void main(String[] args) {
        SpringApplication.run(HalalAiBackendApplication.class, args);
    }

}
```

src/main/java/com/halalai/backend/config/DataLoader.java

```
package com.halalai.backend.config;

import com.halalai.backend.model.Verse;
import com.halalai.backend.repository.VerseRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.CommandLineRunner;
import org.springframework.core.io.ClassPathResource;
import org.springframework.stereotype.Component;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;

@Component
public class DataLoader implements CommandLineRunner {

    private static final Logger logger = LoggerFactory.getLogger(DataLoader.class);
    private final VerseRepository verseRepository;

    public DataLoader(VerseRepository verseRepository) {
        this.verseRepository = verseRepository;
    }

    @Override
    public void run(String... args) throws Exception {
        // Проверяем, есть ли уже данные в базе
        if (verseRepository.count() > 0) {
            logger.info("База данных аятов уже содержит {} записей. Пропускаем загрузку.", verseRepository.count());
            return;
        }

        logger.info("Начинаем загрузку аятов из CSV файла...");
    }
}
```

```

    try {
        ClassPathResource resource = new ClassPathResource("sury.csv");
        int loadedCount = 0;
        int skippedCount = 0;

        try (BufferedReader reader = new BufferedReader(
            new InputStreamReader(resource.getInputStream(),
StandardCharsets.UTF_8))) {

            // Пропускаем заголовок
            String header = reader.readLine();
            if (header == null || !header.startsWith("sura_index")) {
                logger.warn("Неверный формат CSV файла. Ожидается заголовок с
'sura_index'");
                return;
            }

            String line;
            while ((line = reader.readLine()) != null) {
                if (line.trim().isEmpty()) {
                    continue;
                }

                try {
                    Verse verse = parseCsvLine(line);
                    if (verse != null) {
                        verseRepository.save(verse);
                        loadedCount++;

                        if (loadedCount % 1000 == 0) {
                            logger.info("Загружено {} аятов...", loadedCount)
;

                                }
                            } else {
                                skippedCount++;
                            }
                        } catch (Exception e) {
                            logger.warn("Ошибка при парсинге строки: {}. Пропускаем.
Ошибка: {}", line.substring(0, Math.min(50, line.length())), e.getMessage());
                            skippedCount++;
                        }
                    }

                }

                logger.info("Загрузка завершена. Загружено: {}, Пропущено: {}, Всего
в базе: {}",
                    loadedCount, skippedCount, verseRepository.count());
            } catch (Exception e) {
                logger.error("Ошибка при загрузке данных из CSV: {}", e.getMessage(),
e);
            }
        }

        private Verse parseCsvLine(String line) {
            // Простой парсинг CSV с учетом кавычек
            // Формат: sura_index,sura_title,sura_subtitle,verse_number,text
            try {
                String[] parts = parseCsvFields(line);

                if (parts.length < 5) {
                    return null;

```

```

    }

    Integer suraIndex = parseInteger(parts[0]);
    String suraTitle = parts[1].trim();
    String suraSubtitle = parts[2].trim();
    Integer verseNumber = parseInteger(parts[3]);
    String text = parts[4].trim();

    if (suraIndex == null || suraTitle.isEmpty() || text.isEmpty()) {
        return null;
    }

    return new Verse(suraIndex, suraTitle, suraSubtitle, verseNumber,
text);
    } catch (Exception e) {
        logger.debug("Ошибка парсинга строки: {}", e.getMessage());
        return null;
    }
}

private String[] parseCsvFields(String line) {
    // Простой парсер CSV, который учитывает кавычки
    java.util.List<String> fields = new java.util.ArrayList<>();
    StringBuilder currentField = new StringBuilder();
    boolean inQuotes = false;

    for (int i = 0; i < line.length(); i++) {
        char c = line.charAt(i);

        if (c == '"') {
            inQuotes = !inQuotes;
        } else if (c == ',' && !inQuotes) {
            fields.add(currentField.toString());
            currentField = new StringBuilder();
        } else {
            currentField.append(c);
        }
    }
    fields.add(currentField.toString());

    return fields.toArray(new String[0]);
}

private Integer parseInteger(String value) {
    if (value == null || value.trim().isEmpty()) {
        return null;
    }
    try {
        return Integer.parseInt(value.trim());
    } catch (NumberFormatException e) {
        return null;
    }
}
}

```

src/main/java/com/halalai/backend/config/RestTemplateConfig.java

```

package com.halalai.backend.config;

import org.springframework.boot.web.client.RestTemplateBuilder;
import org.springframework.context.annotation.Bean;

```

```

import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.web.client.RestTemplate;

@Configuration
public class RestTemplateConfig {

    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        SimpleClientHttpRequestFactory factory = new
        SimpleClientHttpRequestFactory();
        factory.setConnectTimeout(10000);
        factory.setReadTimeout(180000);

        return builder
            .requestFactory(() -> factory)
            .build();
    }
}

```

src/main/java/com/halalai/backend/config/SecurityConfig.java

```

package com.halalai.backend.config;

import com.halalai.backend.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.ProviderManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.method.configuration.
    EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.
    EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.
    AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.
    UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.Arrays;
import java.util.List;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;
    private final UserDetailsService userDetailsService;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthenticationFilter,
        UserDetailsService userDetailsService) {

```

```

        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
        this.userDetailsService = userDetailsService;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }

    @Bean
    public AuthenticationManager authenticationManager() {
        DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
        authProvider.setUserDetailsService(userDetailsService);
        authProvider.setPasswordEncoder(passwordEncoder());
        return new ProviderManager(authProvider);
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
    Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
        )
            .sessionManagement(session -> session.sessionCreationPolicy(
                SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/models").permitAll()
                .requestMatchers("/api/verse-of-the-day").permitAll()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthenticationFilter,
                UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(List.of("*"));
        configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "
DELETE", "OPTIONS"));
        configuration.setAllowedHeaders(List.of("*"));
        configuration.setExposedHeaders(List.of("Authorization"));
        configuration.setAllowCredentials(false);
        configuration.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource source = new
        UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}

```

src/main/java/com/halalai/backend/controller/AuthController.java

```

package com.halalai.backend.controller;

import com.halalai.backend.dto.AuthResponse;

```

```

import com.halalai.backend.dto.LoginRequest;
import com.halalai.backend.dto.RefreshTokenRequest;
import com.halalai.backend.dto.RegisterRequest;
import com.halalai.backend.service.IAuthService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
public class AuthController {
    private final IAuthService authService;

    public AuthController(IAuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/register")
    public ResponseEntity<AuthResponse> register(@Valid @RequestBody
    RegisterRequest request) {
        AuthResponse response = authService.register(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest
    request) {
        AuthResponse response = authService.login(request);
        return ResponseEntity.ok(response);
    }

    @PostMapping("/refresh")
    public ResponseEntity<AuthResponse> refreshToken(@Valid @RequestBody
    RefreshTokenRequest request) {
        AuthResponse response = authService.refreshToken(request.token());
        return ResponseEntity.ok(response);
    }
}

```

src/main/java/com/halalai/backend/controller/ChatController.java

```

package com.halalai.backend.controller;

import java.util.Map;

import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.halalai.backend.dto.ChatRequest;
import com.halalai.backend.dto.ChatResponse;
import com.halalai.backend.service.IConfigService;
import com.halalai.backend.service.ILLMService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@RestController

```

```

@RequestMapping("/api")
public class ChatController {

    private static final Logger logger = LoggerFactory.getLogger(ChatController.class);

    private final ILLMService llmService;
    private final IConfigService configService;

    public ChatController(ILLMService llmService, IConfigService configService) {
        this.llmService = llmService;
        this.configService = configService;
    }

    @PostMapping("/chat")
    public ChatResponse createChatCompletion(@RequestBody ChatRequest request) {
        logger.info("POST /api/chat - сообщений: {}, maxTokens: {}, temperature: {}, useRag: {}",
            request.messages() != null ? request.messages().size() : 0,
            request.maxTokens(),
            request.temperature(),
            request.useRag());

        return llmService.generateCompletion(
            request.messages(),
            request.apiKey(),
            request.remoteModel(),
            request.maxTokens(),
            request.temperature(),
            request.useRag());
    }

    @GetMapping(value = "/models", produces = MediaType.APPLICATION_JSON_VALUE + ";charset=UTF-8")
    public Map<String, Object> listModels() {
        logger.info("GET /api/models");
        return configService.getAvailableModels();
    }
}

```

src/main/java/com/halalai/backend/controller/VerseController.java

```

package com.halalai.backend.controller;

import com.halalai.backend.dto.VerseResponse;
import com.halalai.backend.service.IVerseService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api")
public class VerseController {

    private static final Logger logger = LoggerFactory.getLogger(VerseController.class);
    private final IVerseService verseService;

```

```

    public VerseController(IVerseService verseService) {
        this.verseService = verseService;
    }

    @GetMapping("/verse-of-the-day")
    public ResponseEntity<VerseResponse> getVerseOfTheDay() {
        logger.info("GET /api/verse-of-the-day");
        VerseResponse response = verseService.getVerseOfTheDay();
        return ResponseEntity.ok(response);
    }
}

```

src/main/java/com/halalai/backend/dto/AuthResponse.java

```

package com.halalai.backend.dto;

public record AuthResponse(
    String token,
    String type,
    Long userId,
    String email
) {
    public static AuthResponse of(String token, Long userId, String email) {
        return new AuthResponse(token, "Bearer", userId, email);
    }
}

```

src/main/java/com/halalai/backend/dto/ChatRequest.java

```

package com.halalai.backend.dto;

import java.util.List;
import java.util.Map;

import com.fasterxml.jackson.annotation.JsonProperty;

public record ChatRequest(
    List<Map<String, String>> messages,
    @JsonProperty("api_key") String apiKey,
    @JsonProperty("remote_model") String remoteModel,
    @JsonProperty("max_tokens") Integer maxTokens,
    @JsonProperty("temperature") Double temperature,
    @JsonProperty("use_rag") Boolean useRag
) {
}

```

src/main/java/com/halalai/backend/dto/ChatResponse.java

```

package com.halalai.backend.dto;

import com.fasterxml.jackson.annotation.JsonProperty;

public record ChatResponse(
    String reply,
    @JsonProperty("used_remote") Boolean usedRemote,
    @JsonProperty("remote_error") String remoteError
) {
}

```



```
}

```

src/main/java/com/halalai/backend/dto/ConfigResponse.java

```
package com.halalai.backend.dto;

public record ConfigResponse(
    String systemPrompt
) {
}

```

src/main/java/com/halalai/backend/dto/ErrorResponse.java

```
package com.halalai.backend.dto;

import java.time.LocalDateTime;

public record ErrorResponse(
    String message,
    LocalDateTime timestamp,
    String path
) {
    public static ErrorResponse of(String message, String path) {
        return new ErrorResponse(message, LocalDateTime.now(), path);
    }
}

```

src/main/java/com/halalai/backend/dto/LoginRequest.java

```
package com.halalai.backend.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

public record LoginRequest(
    @NotBlank(message = "Email не может быть пустым")
    @Email(message = "Email должен быть валидным")
    String email,

    @NotBlank(message = "Пароль не может быть пустым")
    String password
) {
}

```

src/main/java/com/halalai/backend/dto/RefreshTokenRequest.java

```
package com.halalai.backend.dto;

import jakarta.validation.constraints.NotBlank;

public record RefreshTokenRequest(
    @NotBlank(message = "Токен не может быть пустым")
    String token
) {
}

```

src/main/java/com/halalai/backend/dto/RegisterRequest.java

```
package com.halalai.backend.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record RegisterRequest(
    @NotBlank(message = "Email не может быть пустым")
    @Email(message = "Email должен быть валидным")
    String email,

    @NotBlank(message = "Пароль не может быть пустым")
    @Size(min = 8, message = "Пароль должен содержать минимум 8 символов")
    String password
) {
}
```

src/main/java/com/halalai/backend/dto/VerseResponse.java

```
package com.halalai.backend.dto;

public record VerseResponse(
    Long id,
    Integer suraIndex,
    String suraTitle,
    String suraSubtitle,
    Integer verseNumber,
    String text
) {
}
```

src/main/java/com/halalai/backend/exception/GlobalExceptionHandler.java

```
package com.halalai.backend.exception;

import com.halalai.backend.dto.ErrorResponse;
import jakarta.validation.ConstraintViolation;
import jakarta.validation.ConstraintViolationException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;

import java.util.HashMap;
import java.util.Map;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {
```

```

private static final Logger logger = LoggerFactory.getLogger(
GlobalExceptionHandler.class);

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, Object>> handleValidationExceptions(
    MethodArgumentNotValidException ex, WebRequest request) {
    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult().getAllErrors().forEach((error) -> {
        String fieldName = ((FieldError) error).getField();
        String errorMessage = error.getDefaultMessage();
        errors.put(fieldName, errorMessage);
    });

    return getMapResponseEntity(request, errors);
}

@ExceptionHandler(ConstraintViolationException.class)
public ResponseEntity<Map<String, Object>> handleConstraintViolationException(
    ConstraintViolationException ex, WebRequest request) {
    Map<String, String> errors = ex.getConstraintViolations().stream()
        .collect(Collectors.toMap(
            violation -> violation.getPropertyPath().toString(),
            ConstraintViolation::getMessage
        ));

    return getMapResponseEntity(request, errors);
}

private ResponseEntity<Map<String, Object>> getMapResponseEntity(WebRequest
request, Map<String, String> errors) {
    Map<String, Object> response = new HashMap<>();
    response.put("message", "Ошибка валидации");
    response.put("errors", errors);
    response.put("timestamp", java.time.LocalDateTime.now());
    response.put("path", request.getDescription(false).replace("uri=", ""));

    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(response);
}

@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<ErrorResponse> handleIllegalArgumentException(
    IllegalArgumentException ex, WebRequest request) {
    return ResponseEntity.status(HttpStatus.BAD_REQUEST)
        .body(ErrorResponse.of(ex.getMessage(), request.getDescription(
false).replace("uri=", "")));
}

@ExceptionHandler(BadCredentialsException.class)
public ResponseEntity<ErrorResponse> handleBadCredentialsException(
    BadCredentialsException ex, WebRequest request) {
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
        .body(ErrorResponse.of("Неверное имя пользователя или пароль",
request.getDescription(false).replace("uri=", "")));
}

@ExceptionHandler(org.springframework.security.core.userdetails.
UsernameNotFoundException.class)
public ResponseEntity<ErrorResponse> handleUsernameNotFoundException(
    org.springframework.security.core.userdetails.
UsernameNotFoundException ex, WebRequest request) {
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED)

```

```

        .body(ErrorResponse.of("Пользователь не найден",
            request.getDescription(false).replace("uri=", "")));
    }

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<ErrorResponse> handleRuntimeException(
        RuntimeException ex, WebRequest request) {
        logger.error("RuntimeException: {}", ex.getMessage(), ex);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(ErrorResponse.of(ex.getMessage(),
                request.getDescription(false).replace("uri=", "")));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGlobalException(
        Exception ex, WebRequest request) {
        logger.error("Необработанное исключение: {}", ex.getMessage(), ex);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(ErrorResponse.of("Внутренняя ошибка сервера",
                request.getDescription(false).replace("uri=", "")));
    }
}

```

src/main/java/com/halalai/backend/model/User.java

```

package com.halalai.backend.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import java.time.LocalDateTime;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Email
    @Column(unique = true, nullable = false)
    private String email;

    @NotBlank
    @Size(min = 8)
    @Column(nullable = false)
    private String password;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @Column(name = "enabled", nullable = false)
    private Boolean enabled = true;

    @PrePersist

```

```
protected void onCreate() {
    createdAt = LocalDateTime.now();
    updatedAt = LocalDateTime.now();
}

@PreUpdate
protected void onUpdate() {
    updatedAt = LocalDateTime.now();
}

public User() {
}

public User(String email, String password) {
    this.email = email;
    this.password = password;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}

public LocalDateTime getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDateTime createdAt) {
    this.createdAt = createdAt;
}

public LocalDateTime getUpdatedAt() {
    return updatedAt;
}

public void setUpdatedAt(LocalDateTime updatedAt) {
    this.updatedAt = updatedAt;
}

public Boolean getEnabled() {
    return enabled;
}
```

```

    public void setEnabled(Boolean enabled) {
        this.enabled = enabled;
    }
}

```

src/main/java/com/halalai/backend/model/Verse.java

```

package com.halalai.backend.model;

import jakarta.persistence.*;

@Entity
@Table(name = "verses")
public class Verse {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "sura_index", nullable = false)
    private Integer suraIndex;

    @Column(name = "sura_title", nullable = false, length = 200)
    private String suraTitle;

    @Column(name = "sura_subtitle", length = 200)
    private String suraSubtitle;

    @Column(name = "verse_number")
    private Integer verseNumber;

    @Column(name = "text", nullable = false, columnDefinition = "TEXT")
    private String text;

    public Verse() {
    }

    public Verse(Integer suraIndex, String suraTitle, String suraSubtitle,
Integer verseNumber, String text) {
        this.suraIndex = suraIndex;
        this.suraTitle = suraTitle;
        this.suraSubtitle = suraSubtitle;
        this.verseNumber = verseNumber;
        this.text = text;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public Integer getSuraIndex() {
        return suraIndex;
    }

    public void setSuraIndex(Integer suraIndex) {
        this.suraIndex = suraIndex;
    }
}

```

```

    public String getSuraTitle() {
        return suraTitle;
    }

    public void setSuraTitle(String suraTitle) {
        this.suraTitle = suraTitle;
    }

    public String getSuraSubtitle() {
        return suraSubtitle;
    }

    public void setSuraSubtitle(String suraSubtitle) {
        this.suraSubtitle = suraSubtitle;
    }

    public Integer getVerseNumber() {
        return verseNumber;
    }

    public void setVerseNumber(Integer verseNumber) {
        this.verseNumber = verseNumber;
    }

    public String getText() {
        return text;
    }

    public void setText(String text) {
        this.text = text;
    }
}

```

src/main/java/com/halalai/backend/repository/UserRepository.java

```

package com.halalai.backend.repository;

import com.halalai.backend.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}

```

src/main/java/com/halalai/backend/repository/VerseRepository.java

```

package com.halalai.backend.repository;

import com.halalai.backend.model.Verse;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

```

```

@Repository
public interface VerseRepository extends JpaRepository<Verse, Long> {
    long count();
    Page<Verse> findAllByIdAsc(Pageable pageable);
}

```

src/main/java/com/halalai/backend/security/JwtAuthenticationFilter.java

```

package com.halalai.backend.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.lang.NonNull;
import org.springframework.security.authentication.
    UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.authentication.
    WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.io.IOException;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private static final Logger logger = LoggerFactory.getLogger(
        JwtAuthenticationFilter.class);

    private final JwtTokenProvider tokenProvider;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtTokenProvider tokenProvider,
        UserDetailsService userDetailsService) {
        this.tokenProvider = tokenProvider;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain
    ) throws ServletException, IOException {
        String jwt = getJwtFromRequest(request);

        if (jwt != null) {
            try {
                String email = tokenProvider.getUsernameFromToken(jwt);

                if (email != null && SecurityContextHolder.getContext().
                    getAuthentication() == null) {
                    UserDetails userDetails = userDetailsService.
                        loadUserByUsername(email);

```



```

        if (tokenProvider.validateToken(jwt, userDetails)) {
            UsernamePasswordAuthenticationToken authentication = new
UsernamePasswordAuthenticationToken(
                userDetails, null, userDetails.getAuthorities());
            authentication.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(
authentication);
            logger.debug("Аутентификация установлена для пользователя
: {}", email);
        } else {
            logger.warn("JWT токен не прошел валидацию для пользовате
ля: {}", email);
        }
    } catch (Exception e) {
        logger.error("Ошибка при валидации JWT токена: {}", e.getMessage
(), e);
    }
}

filterChain.doFilter(request, response);
}

private String getJwtFromRequest(HttpServletRequest request) {
    String bearerToken = request.getHeader("Authorization");

    if (bearerToken != null) {
        bearerToken = bearerToken.trim();
        if (bearerToken.startsWith("Bearer ") || bearerToken.startsWith("
bearer ")) {
            String token = bearerToken.substring(7).trim();
            if (!token.isEmpty()) {
                return token;
            }
        }
    }
    return null;
}
}

```

src/main/java/com/halalai/backend/security/JwtTokenProvider.java

```

package com.halalai.backend.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;

```

```

import java.util.function.Function;

@Component
public class JwtTokenProvider {

    private static final Logger logger = LoggerFactory.getLogger(JwtTokenProvider
.class);

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private Long expiration;

    private SecretKey getSigningKey() {
        byte[] keyBytes = secret.getBytes(StandardCharsets.UTF_8);
        return Keys.hmacShaKeyFor(keyBytes);
    }

    /** subject JWT -- email пользователя (Spring UserDetails#getUsername() тоже
email). */
    public String generateToken(String email, Long userId) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("userId", userId);
        return createToken(claims, email);
    }

    private String createToken(Map<String, Object> claims, String subject) {
        Date now = new Date();
        Date expiryDate = new Date(now.getTime() + expiration);

        return Jwts.builder()
            .claims(claims)
            .subject(subject)
            .issuedAt(now)
            .expiration(expiryDate)
            .signWith(getSigningKey())
            .compact();
    }

    public String getUsernameFromToken(String token) {
        return getClaimFromToken(token, Claims::getSubject);
    }

    /**
     * Извлекает subject (email) из токена, даже если он истек (для refresh)
     */
    public String getUsernameFromExpiredToken(String token) {
        try {
            Claims claims = Jwts.parser()
                .verifyWith(getSigningKey())
                .build()
                .parseSignedClaims(token)
                .getPayload();
            return claims.getSubject();
        } catch (io.jsonwebtoken.ExpiredJwtException e) {
            return e.getClaims().getSubject();
        } catch (Exception e) {
            logger.error("Ошибка при извлечении email из истекшего токена: {}", e
.getMessage(), e);
            throw e;
        }
    }
}

```

```

}

/**
 * Извлекает userId из токена, даже если он истек (для refresh)
 */
public Long getUserIdFromExpiredToken(String token) {
    try {
        Claims claims = Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
        Object userId = claims.get("userId");
        if (userId instanceof Number) {
            return ((Number) userId).longValue();
        }
        return null;
    } catch (io.jsonwebtoken.ExpiredJwtException e) {
        // Для истекших токенов все равно извлекаем userId
        Object userId = e.getClaims().get("userId");
        if (userId instanceof Number) {
            return ((Number) userId).longValue();
        }
        return null;
    } catch (Exception e) {
        logger.error("Ошибка при извлечении userId из истекшего токена: " + e
            .getMessage(), e);
        return null;
    }
}

public <T> T getClaimFromToken(String token, Function<Claims, T>
claimsResolver) {
    final Claims claims = getAllClaimsFromToken(token);
    return claimsResolver.apply(claims);
}

private Claims getAllClaimsFromToken(String token) {
    try {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    } catch (io.jsonwebtoken.ExpiredJwtException e) {
        logger.debug("JWT токен истек");
        throw e;
    } catch (io.jsonwebtoken.security.SignatureException e) {
        logger.error("Неверная подпись JWT токена", e);
        throw e;
    } catch (io.jsonwebtoken.MalformedJwtException e) {
        logger.error("Некорректный формат JWT токена", e);
        throw e;
    } catch (Exception e) {
        logger.error("Ошибка при парсинге JWT токена", e);
        throw e;
    }
}

public Boolean isTokenExpired(String token) {
    try {
        Claims claims = getAllClaimsFromToken(token);

```

```

        Date expiration = claims.getExpiration();
        return expiration.before(new Date());
    } catch (io.jsonwebtoken.ExpiredJwtException e) {
        // Если токен истек, возвращаем true
        return true;
    } catch (Exception e) {
        logger.debug("Ошибка при проверке истечения токена: {}", e.getMessage());
        return true;
    }
}

public Boolean validateToken(String token, UserDetails userDetails) {
    try {
        final String username = getUsernameFromToken(token);
        boolean isExpired = isTokenExpired(token);
        boolean usernameMatches = username.equals(userDetails.getUsername());

        return usernameMatches && !isExpired;
    } catch (Exception e) {
        logger.debug("Ошибка при валидации токена: {}", e.getMessage());
        return false;
    }
}
}

```

src/main/java/com/halalai/backend/service/AuthService.java

```

package com.halalai.backend.service;

import com.halalai.backend.dto.AuthResponse;
import com.halalai.backend.dto.LoginRequest;
import com.halalai.backend.dto.RegisterRequest;
import com.halalai.backend.model.User;
import com.halalai.backend.repository.UserRepository;
import com.halalai.backend.security.JwtTokenProvider;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.
    UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class AuthService implements IAuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtTokenProvider tokenProvider;
    private final AuthenticationManager authenticationManager;

    public AuthService(
        UserRepository userRepository,
        PasswordEncoder passwordEncoder,
        JwtTokenProvider tokenProvider,
        AuthenticationManager authenticationManager
    ) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.tokenProvider = tokenProvider;
        this.authenticationManager = authenticationManager;
    }
}

```

```

}

@Transactional
public AuthResponse register(RegisterRequest request) {
    if (userRepository.existsByEmail(request.email())) {
        throw new IllegalArgumentException("Пользователь с таким email уже су
ществует");
    }

    User user = new User();
    user.setEmail(request.email());
    user.setPassword(passwordEncoder.encode(request.password()));
    user.setEnabled(true);

    user = userRepository.save(user);

    String token = tokenProvider.generateToken(user.getEmail(), user.getId())
;

    return AuthResponse.of(token, user.getId(), user.getEmail());
}

public AuthResponse login(LoginRequest request) {
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            request.email(),
            request.password()
        )
    );

    User user = userRepository.findByEmail(request.email())
        .orElseThrow(() -> new IllegalArgumentException("Пользователь не
найден"));

    if (!user.isEnabled()) {
        throw new IllegalArgumentException("Аккаунт пользователя отключен");
    }
    String token = tokenProvider.generateToken(user.getEmail(), user.getId())
;

    return AuthResponse.of(token, user.getId(), user.getEmail());
}

public AuthResponse refreshToken(String oldToken) {
    String email = tokenProvider.getUsernameFromExpiredToken(oldToken);
    Long userId = tokenProvider.getUserIdFromExpiredToken(oldToken);

    if (email == null) {
        throw new IllegalArgumentException("Не удалось извлечь email из токен
a");
    }
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new IllegalArgumentException("Пользователь не
найден"));

    if (!user.isEnabled()) {
        throw new IllegalArgumentException("Аккаунт пользователя отключен");
    }
    if (userId != null && !userId.equals(user.getId())) {
        throw new IllegalArgumentException("Неверный токен");
    }
}

```

```

        String newToken = tokenProvider.generateToken(user.getEmail(), user.getId
        ());

        return AuthResponse.of(newToken, user.getId(), user.getEmail());
    }
}

```

src/main/java/com/halalai/backend/service/ConfigService.java

```

package com.halalai.backend.service;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@Service
public class ConfigService implements IConfigService {

    private final String defaultModel;
    private final List<String> allowedModels;

    public ConfigService(
        @Value("${llm.models.default}") String defaultModel,
        @Value("${llm.models.allowed}") String allowedModelsStr) {
        this.defaultModel = defaultModel;
        this.allowedModels = List.of(allowedModelsStr.split(","));
    }

    public Map<String, Object> getAvailableModels() {
        Map<String, Object> result = new HashMap<>();
        result.put("default_model", defaultModel);
        result.put("allowed_models", allowedModels);
        return result;
    }
}

```

src/main/java/com/halalai/backend/service/IAuthService.java

```

package com.halalai.backend.service;

import com.halalai.backend.dto.AuthResponse;
import com.halalai.backend.dto.LoginRequest;
import com.halalai.backend.dto.RegisterRequest;

public interface IAuthService {
    AuthResponse register(RegisterRequest request);
    AuthResponse login(LoginRequest request);
    AuthResponse refreshToken(String token);
}

```

src/main/java/com/halalai/backend/service/IConfigService.java

```

package com.halalai.backend.service;

import java.util.Map;

```

```
public interface IConfigService {
    Map<String, Object> getAvailableModels();
}
```

src/main/java/com/halalai/backend/service/ILLMSERVICE.java

```
package com.halalai.backend.service;

import com.halalai.backend.dto.ChatResponse;
import java.util.List;
import java.util.Map;

public interface ILLMSERVICE {
    ChatResponse generateCompletion(List<Map<String, String>> messages, String
    apiKey, String remoteModel, Integer maxTokens, Double temperature, Boolean
    useRag);
}
```

src/main/java/com/halalai/backend/service/IVerseService.java

```
package com.halalai.backend.service;

import com.halalai.backend.dto.VerseResponse;

public interface IVerseService {
    VerseResponse getVerseOfTheDay();
}
```

src/main/java/com/halalai/backend/service/LLMSERVICE.java

```
package com.halalai.backend.service;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpMethod;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
import org.springframework.web.client.RestClientException;
import org.springframework.web.server.ResponseStatusException;

import com.fasterxml.jackson.databind.JsonNode;
import com.halalai.backend.dto.ChatResponse;

@Service
public class LLMSERVICE implements ILLMSERVICE {
```

```

private static final Logger logger = LoggerFactory.getLogger(LLMService.class
);

private final RestTemplate restTemplate;
private final String llmServiceUrl;
private final int defaultMaxTokens;
private final String defaultModel;
private final List<String> allowedModels;

public LLMService(
    RestTemplate restTemplate,
    @Value("${llm.service.url}") String llmServiceUrl,
    @Value("${llm.service.max-tokens:256}") int maxTokens,
    @Value("${llm.models.default}") String defaultModel,
    @Value("${llm.models.allowed}") String allowedModelsStr) {
    this.restTemplate = restTemplate;
    this.llmServiceUrl = llmServiceUrl;
    this.defaultMaxTokens = maxTokens;
    this.defaultModel = defaultModel;
    this.allowedModels = List.of(allowedModelsStr.split(","));

    logger.info("Инициализация LLM Service... URL: {}", llmServiceUrl);
    logger.info("Доступные модели: default={}, count={}", defaultModel, this.
allowedModels.size());
    logger.info("Примечание: системный промпт управляется Python LLM сервисом
");
}

public ChatResponse generateCompletion(List<Map<String, String>>
clientMessages, String apiKey, String remoteModel, Integer maxTokensOverride,
Double temperature, Boolean useRag) {
    logger.debug("Запрос к LLM сервису");

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);
    headers.set("Accept-Charset", "UTF-8");

    List<Map<String, String>> messages;

    if (clientMessages != null && !clientMessages.isEmpty()) {
        messages = new ArrayList<>(clientMessages);
        logger.debug("Получена история от клиента: {} сообщений",
clientMessages.size());
    } else {
        throw new IllegalArgumentException("Не указаны ни messages, ни prompt
");
    }

    Map<String, Object> requestBody = new HashMap<>();
    requestBody.put("messages", messages);

    int effectiveMaxTokens = maxTokensOverride != null ? maxTokensOverride :
defaultMaxTokens;
    requestBody.put("max_tokens", effectiveMaxTokens);
    if (apiKey != null && !apiKey.isBlank()) {
        requestBody.put("api_key", apiKey.trim());
    }
    if (remoteModel != null && !remoteModel.isBlank()) {
        requestBody.put("remote_model", remoteModel.trim());
    }
    if (temperature != null) {
        requestBody.put("temperature", temperature);
    }
}

```



```

    }
    if (useRag != null) {
        requestBody.put("use_rag", useRag);
    }

    HttpEntity<Map<String, Object>> requestEntity = new HttpEntity<>(
requestBody, headers);
    String chatUrl = llmServiceUrl + "/llm/chat";

    try {
        logger.debug("Отправка запроса к LLM сервису: {}", chatUrl);
        logger.debug("Тело запроса: messages={}, max_tokens={}{}{}{}{}",
            messages.size(),
            effectiveMaxTokens,
            requestBody.containsKey("api_key") ? ", api_key=***" : "",
            requestBody.containsKey("remote_model") ? ", remote_model=" +
requestBody.get("remote_model") : "",
            requestBody.containsKey("temperature") ? ", temperature=" +
requestBody.get("temperature") : "",
            requestBody.containsKey("use_rag") ? ", use_rag=" +
requestBody.get("use_rag") : "");

        ResponseEntity<JsonNode> responseEntity = restTemplate.exchange(
            chatUrl,
            HttpMethod.POST,
            requestEntity,
            JsonNode.class
        );

        logger.debug("Получен ответ от LLM сервиса. Статус: {}",
responseEntity.getStatusCode());

        JsonNode responseBody = responseEntity.getBody();
        if (responseBody == null) {
            throw new RuntimeException("LLM сервис вернул пустой ответ");
        }
        String reply = responseBody.has("reply") ? responseBody.get("reply").
asText("") : "";
        Boolean usedRemote = responseBody.has("used_remote") ? responseBody.
get("used_remote").asBoolean(false) : Boolean.FALSE;
        String remoteError = null;
        if (responseBody.has("remote_error") && !responseBody.get("
remote_error").isNull()) {
            String error = responseBody.get("remote_error").asText("");
            remoteError = error.isEmpty() ? null : error;
        }

        if (reply.isEmpty()) {
            throw new RuntimeException("Ответ не содержит поле 'reply' или он
о пустое. Структура: " + responseBody);
        }

        logger.info("Ответ от LLM получен: длина={} символов, remote={},
            reply.length(), usedRemote);

        return new ChatResponse(reply, usedRemote, remoteError);

    } catch (org.springframework.web.client.HttpClientErrorException e) {
        logger.error("Ошибка HTTP при обращении к LLM сервису: статус={}, URL
={}, тело={}",
            e.getStatusCode(), chatUrl, e.getResponseBodyAsString());
    }

```

```

        if (e.getStatusCode().value() == 503) {
            throw new RuntimeException("LLM сервис не готов. Убедитесь, что
Python сервис запущен и модель загружена.", e);
        }

        throw new ResponseStatusException(e.getStatusCode(), "Ошибка LLM серв
иса: " + e.getResponseBodyAsString(), e);

    } catch (org.springframework.web.client.ResourceAccessException e) {
        logger.error("Ошибка подключения к LLM сервису: URL={}, сообщение
={},", chatUrl, e.getMessage());
        throw new RuntimeException("Не удалось подключиться к LLM сервису по
адресу " + chatUrl + ". " +
            "Убедитесь, что Python сервис запущен на " + llmServiceUrl, e
    );

    } catch (RestClientException e) {
        logger.error("Ошибка REST клиента при обращении к LLM сервису: URL
={}, сообщение={},", chatUrl, e.getMessage());
        throw new RuntimeException("Не удалось подключиться к LLM сервису по
адресу " + chatUrl + ". " +
            "Убедитесь, что Python сервис запущен на " + llmServiceUrl, e
    );

    } catch (Exception e) {
        logger.error("Неизвестная ошибка при генерации ответа: {}", e.
getMessage(), e);
        throw new RuntimeException("Ошибка при генерации ответа: " + e.
getMessage(), e);
    }
}
}

```

src/main/java/com/halalai/backend/service/UserDetailsServiceImpl.java

```

package com.halalai.backend.service;

import com.halalai.backend.model.User;
import com.halalai.backend.repository.UserRepository;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class UserDetailsServiceImpl implements UserDetailsService {

    private final UserRepository userRepository;

    public UserDetailsServiceImpl(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    @Transactional
    public UserDetails loadUserByUsername(String email) throws
UsernameNotFoundException {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException("Пользователь не
найден: " + email));
    }
}

```

```

        return org.springframework.security.core.userdetails.User.builder()
            .username(user.getEmail())
            .password(user.getPassword())
            .disabled(!user.getEnabled())
            .accountExpired(false)
            .accountLocked(false)
            .credentialsExpired(false)
            .authorities("ROLE_USER")
            .build();
    }
}

```

src/main/java/com/halalai/backend/service/VerseService.java

```

package com.halalai.backend.service;

import com.halalai.backend.dto.VerseResponse;
import com.halalai.backend.model.Verse;
import com.halalai.backend.repository.VerseRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.data.domain.PageRequest;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

@Service
public class VerseService implements IVerseService {

    private static final Logger logger = LoggerFactory.getLogger(VerseService.class);
    private final VerseRepository verseRepository;

    public VerseService(VerseRepository verseRepository) {
        this.verseRepository = verseRepository;
    }

    public VerseResponse getVerseOfTheDay() {
        long totalVerses = verseRepository.count();

        if (totalVerses == 0) {
            logger.warn("База данных аятов пуста");
            throw new RuntimeException("База данных аятов пуста. Пожалуйста, загрузите данные.");
        }

        // Используем день года (1-365/366) для выбора аята
        // Это гарантирует, что один и тот же аят будет возвращаться в течение дня
        LocalDate today = LocalDate.now();
        LocalDate startOfYear = LocalDate.of(today.getYear(), 1, 1);
        long dayOfYear = ChronoUnit.DAYS.between(startOfYear, today) + 1;

        // Используем день года как индекс для выбора аята
        int verseIndex = (int) ((dayOfYear - 1) % totalVerses);

        // Используем пагинацию для эффективного получения одного аята
        Verse verse = verseRepository.findAllByIdAsc(
            PageRequest.of(verseIndex, 1)

```

```
        ).getContent().get(0);

        logger.info("Возвращен аят дня: сура {}, аят {}", verse.getSuraIndex(),
            verse.getVerseNumber());

        return new VerseResponse(
            verse.getId(),
            verse.getSuraIndex(),
            verse.getSuraTitle(),
            verse.getSuraSubtitle(),
            verse.getVerseNumber(),
            verse.getText()
        );
    }
}
```

Код iOS-приложения

HalalAI/App/HalalAIApp.swift

```
//
//  HalalAIApp.swift
//  HalalAI
//
//  Created by Мирсаит Сабирзянов on 05.10.2025.
//

import SwiftUI

@main
struct HalalAIApp: App {
    var body: some Scene {
        WindowGroup {
            ZStack {
                Color.greenBackground.ignoresSafeArea()
                screenFactory.makeRootView()
                    .preferredColorScheme(.light)
            }
        }
    }
}
```

HalalAI/App/RootView.swift

```
//
//  RouterView.swift
//  HalalAI
//
//  Created by Мирсаит Сабирзянов on 05.10.2025.
//

import SwiftUI

struct RootView: View {
    var authManager: AuthManager

    var body: some View {
        Group {
            if authManager.isAuthenticated || authManager.isGuest {
                RouterView()
            } else {
                AuthFlowView()
            }
        }
    }
}
```

HalalAI/App/ScreenFactory.swift

```

//
//  ScreenFactory.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 07.01.2026.
//

import Foundation
import SwiftUI

protocol ScreenFactory {
    func makeLoginView(path: Binding<[AuthCoordinator]>) -> LoginView
    func makeRegisterView(path: Binding<[AuthCoordinator]>) -> RegisterView
    func makeRootView() -> RootView
    func makeSettingsView() -> SettingsView
    func makeChatView() -> ChatView
    func makeScannerView() -> ScannerView
    func makeHomeView() -> HomeView
    func makeQuranListView() -> QuranListView
    func makeSuraReaderView(suraIndex: Int) -> SuraReaderView
    func makePrayerNotificationSettingsView() -> PrayerNotificationSettingsView
}

@MainActor
let screenFactory = ScreenFactoryImpl()

@MainActor
final class ScreenFactoryImpl {
    fileprivate init() {}
    fileprivate let dc = DependencyContainer()

    func makeLoginView(path: Binding<[AuthCoordinator]>) -> LoginView {
        return LoginView(authManager: dc.authManager, authService: dc.authService) {
            path.wrappedValue = [.register]
        }
    }

    func makeRegisterView(path: Binding<[AuthCoordinator]>) -> RegisterView {
        return RegisterView(authManager: dc.authManager, authService: dc.authService) {
            path.wrappedValue = []
        }
    }

    func makeRootView() -> RootView {
        RootView(authManager: dc.authManager)
    }

    func makeSettingsView() -> SettingsView {
        return SettingsView(chatService: dc.chatService, authManager: dc.authManager)
    }

    func makeChatView() -> ChatView {
        return ChatView(chatService: dc.chatService, authManager: dc.authManager)
    }

    func makeScannerView() -> ScannerView {
        return ScannerView(ingredientService: dc.ingredientService, authManager: dc.authManager)
    }
}

```

```

func makeHomeView() -> HomeView {
    return HomeView(
        verseService: dc.verseService,
        locationService: dc.locationService,
        prayerTimeService: dc.prayerTimeService,
        settingsStore: dc.prayerSettingsStore,
        authManager: dc.authManager
    )
}

func makePrayerNotificationSettingsView() -> PrayerNotificationSettingsView {
    return PrayerNotificationSettingsView(
        settingsStore: dc.prayerSettingsStore,
        notificationService: dc.prayerNotificationService,
        locationService: dc.locationService
    )
}

func makeQuranListView() -> QuranListView {
    return QuranListView(quranStorage: dc.quranStorage)
}

func makeSuraReaderView(suraIndex: Int) -> SuraReaderView {
    return SuraReaderView(suraIndex: suraIndex, quranStorage: dc.quranStorage)
}

func makeHalalMapView() -> HalalMapView {
    return HalalMapView(placesService: dc.halalPlacesService, locationService: dc.locationService)
}
}

@MainActor
final class DependencyContainer {
    fileprivate var authManager: AuthManager
    fileprivate var authService: AuthService
    fileprivate var chatSettingsStore: ChatSettingsStore
    fileprivate var chatService: ChatService
    fileprivate var ingredientService: IngredientService
    fileprivate var verseService: VerseService
    fileprivate var quranStorage: QuranStorageService
    fileprivate var locationService: LocationService
    fileprivate var prayerTimeService: PrayerTimeService
    fileprivate var prayerSettingsStore: PrayerSettingsStore
    fileprivate var prayerNotificationService: PrayerNotificationService
    fileprivate var halalPlacesService: HalalPlacesService

    init() {
        self.authManager = AuthManagerImpl()
        self.authService = AuthServiceImpl()
        self.chatSettingsStore = ChatSettingsStore()
        self.chatService = ChatServiceImpl(authManager: authManager, settingsStore: chatSettingsStore)
        self.ingredientService = IngredientServiceImpl()
        self.verseService = VerseServiceImpl()
        self.quranStorage = QuranStorageServiceImpl()
        self.locationService = LocationServiceImpl()
        self.prayerTimeService = PrayerTimeServiceImpl()
        self.prayerSettingsStore = PrayerSettingsStore()
    }
}

```

```

        self.prayerNotificationService = PrayerNotificationServiceImpl(
            prayerTimeService: prayerTimeService)
        self.halalPlacesService = HalalPlacesServiceImpl()
    }
}

```

HalalAI/Core/Components/ErrorMessageView.swift

```

//
//  ErrorMessageView.swift
//  HalalAI
//
//  Extracted from ChatView.swift
//

import SwiftUI

struct ErrorMessageView: View {
    let message: String
    let onRetry: () -> Void

    var body: some View {
        HStack {
            VStack(alignment: .leading, spacing: 8) {
                HStack {
                    Image(systemName: "exclamationmark.triangle.fill")
                        .foregroundColor(.red)
                    Text("Ошибка соединения")
                        .font(.headline)
                        .foregroundColor(.red)
                }

                Text(message)
                    .font(.body)
                    .foregroundColor(.secondary)
                    .textSelection(.enabled)

                HStack {
                    Button(action: onRetry) {
                        Text("Повторить отправку")
                            .font(.subheadline)
                            .foregroundColor(.blue)
                            .padding(.horizontal, 16)
                            .padding(.vertical, 8)
                            .background {
                                RoundedRectangle(cornerRadius: 16)
                                    .fill(Color.blue.opacity(0.1))
                            }
                    }

                    Button("Копировать", systemImage: "doc.on.doc") {
                        UIPasteboard.general.string = message
                    }
                    .font(.subheadline)
                    .foregroundColor(.secondary)
                }
            }
            .padding()
            .background {
                RoundedRectangle(cornerRadius: 12)
                    .fill(Color.red.opacity(0.1))
            }
        }
    }
}

```



```

        .overlay {
            RoundedRectangle(cornerRadius: 12)
                .stroke(Color.red.opacity(0.3), lineWidth: 1)
        }

        Spacer()
    }
    .transition(.asymmetric(
        insertion: .scale.combined(with: .opacity),
        removal: .opacity
    ))
}
}

```

HalalAI/Core/Components/ErrorView.swift

```

//
//  ErrorView.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 14.02.2026.
//

import SwiftUI

struct ErrorView: View {
    var message: String
    var body: some View {
        VStack(spacing: 12) {
            Image(systemName: "exclamationmark.triangle.fill")
                .font(.largeTitle)
                .foregroundColor(.orange)
            Text(message)
                .multilineTextAlignment(.center)
                .foregroundColor(.secondary)
                .padding(.horizontal)
        }
        .frame(maxWidth: .infinity, maxHeight: .infinity)
    }
}

#Preview {
    ErrorView(message: "Ошибка")
}

```

HalalAI/Core/Components/GuestAuthPromptView.swift

```

//
//  GuestAuthPromptView.swift
//  HalalAI
//

import SwiftUI

struct GuestAuthPromptView: View {
    let featureName: String
    let authManager: AuthManager
}

```

```

var body: some View {
    ZStack {
        Color.greenBackground
            .ignoresSafeArea()

        VStack(spacing: 24) {
            Image(systemName: "lock.fill")
                .font(.largeTitle)
                .foregroundColor(.darkGreen)

            VStack(spacing: 8) {
                Text("Нужна авторизация")
                    .font(.title2)
                    .bold()
                    .foregroundColor(.darkGreen)

                Text("Войдите в аккаунт, чтобы использовать \((featureName)")
                    .font(.subheadline)
                    .foregroundColor(.secondary)
                    .multilineTextAlignment(.center)
            }

            Button(action: {
                authManager.logout()
            }) {
                Text("Войти")
                    .fontWeight(.semibold)
                    .frame(maxWidth: .infinity)
                    .frame(height: 50)
                    .background(Color.darkGreen)
                    .foregroundColor(.white)
                    .clipShape(.rect(cornerRadius: 12))
            }
            .padding(.horizontal, 32)
        }
        .padding()
    }
}

```

HalalAI/Core/Components/GuestBannerView.swift

```

//
// GuestBannerView.swift
// HalalAI
//

import SwiftUI

struct GuestBannerView: View {
    let onLogin: () -> Void

    var body: some View {
        Button(action: onLogin) {
            HStack(spacing: 12) {
                Image(systemName: "person.crop.circle.badge.plus")
                    .font(.title2)
                    .foregroundColor(.darkGreen)

                VStack(alignment: .leading, spacing: 2) {
                    Text("Войдите в аккаунт")

```

```

        .font(.subheadline)
        .fontWeight(.semibold)
        .foregroundColor(.darkGreen)
Text("Чтобы использовать чат и сканер")
        .font(.caption)
        .foregroundColor(.darkGreen)
    }

    Spacer()

    Image(systemName: "chevron.right")
        .font(.caption)
        .fontWeight(.semibold)
        .foregroundColor(.greenForeground)
    }
    .padding(.horizontal, 16)
    .padding(.vertical, 14)
    .background(.greenForeground)
    .clipShape(RoundedRectangle(cornerRadius: 16))
    .overlay(
        RoundedRectangle(cornerRadius: 16)
            .stroke(Color.greenForeground.opacity(0.3), lineWidth: 1)
    )
    }
}
}

```

HalalAI/Core/Components/ImageTextComponent.swift

```

//
// ImageTextComponent.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 18.11.2025.
//

import SwiftUI

struct ImageTextComponent: View {
    let componentSize: ImageTextComponentSize
    let image: ImageResource
    let title: String
    let description: String
    var locked: Bool = false

    private var imageSize: (CGFloat, CGFloat) {
        switch componentSize {
            case .small:
                return (UIScreen.main.bounds.width * 0.3, UIScreen.main.bounds.width
* 0.3)
            case .medium:
                return (UIScreen.main.bounds.width * 0.4, UIScreen.main.bounds.width
* 0.4)
            case .large:
                return (UIScreen.main.bounds.width * 0.88, UIScreen.main.bounds.width
* 0.4)
        }
    }

    var body: some View {
        VStack(alignment: .leading) {

```

```

        Image(image)
            .resizable()
            .frame(width: imageSize.0, height: imageSize.1)
            .padding([.top])
        VStack(alignment: .leading) {
            Text(title)
                .font(.system(size: 18, weight: .bold))
                .lineLimit(2)
                .fixedSize(horizontal: false, vertical: true)
            Text(description)
                .font(.system(size: 14, weight: .regular))
                .lineLimit(2)
                .fixedSize(horizontal: false, vertical: true)
            Spacer()
        }
        .foregroundColor(.darkGreen)
        .frame(height: 60)
    }
    .frame(width: imageSize.0)
    .padding(.horizontal, 13)
    .background {
        RoundedRectangle(cornerRadius: 20)
            .fill(Color.greenForeground)
    }
    .opacity(locked ? 0.6 : 1)
    .overlay(alignment: .topTrailing) {
        if locked {
            HStack(spacing: 4) {
                Image(systemName: "lock.fill")
                    .font(.system(size: 11, weight: .semibold))
                Text("Үйжөн аккаунт")
                    .font(.system(size: 11, weight: .semibold))
            }
            .foregroundColor(.darkGreen)
            .padding(.horizontal, 8)
            .padding(.vertical, 4)
            .background(.white)
            .clipShape(Capsule())
            .padding(10)
        }
    }
}

enum ImageTextComponentSize {
    case small
    case medium
    case large
}

#Preview {
    ScrollView(.vertical) {
        VStack {
            HStack {
                ImageTextComponent(
                    componentSize: .small,
                    image: .quran,
                    title: "Title",
                    description: "
DescriptionDescriptionDescriptionDescriptionDescription"
                )
                ImageTextComponent(

```

```

        componentSize: .small,
        image: .quran,
        title: "Title",
        description: "Description"
    )
}
HStack {
    ImageTextComponent(
        componentSize: .medium,
        image: .quran,
        title: "Title",
        description: "
DescriptionDescriptionDescriptionDescriptionDescription"
    )
    ImageTextComponent(
        componentSize: .medium,
        image: .quran,
        title: "Title",
        description: "Description"
    )
}
ImageTextComponent(
    componentSize: .large,
    image: .mosque,
    title: "Title",
    description: "
DescriptionDescriptionDescriptionDescriptionDescription"
)
}
}
}
}

```

HalalAI/Core/Extensions/HapticFeedback+Modes.swift

```

//
//  HapticFeedback+Modes.swift
//  HalalAI
//
//  Created by Мирсаит Сабирзянов on 25.10.2025.
//

import SwiftUI

// TODO: Рассмотреть миграцию на .sensoryFeedback() модификатор SwiftUI
struct HapticFeedback {
    static func light() {
        let impactFeedback = UIImpactFeedbackGenerator(style: .light)
        impactFeedback.impactOccurred()
    }

    static func medium() {
        let impactFeedback = UIImpactFeedbackGenerator(style: .medium)
        impactFeedback.impactOccurred()
    }

    static func heavy() {
        let impactFeedback = UIImpactFeedbackGenerator(style: .heavy)
        impactFeedback.impactOccurred()
    }
}

```

```

static func success() {
    let notificationFeedback = UINotificationFeedbackGenerator()
    notificationFeedback.notificationOccurred(.success)
}

static func error() {
    let notificationFeedback = UINotificationFeedbackGenerator()
    notificationFeedback.notificationOccurred(.error)
}
}

```

HalalAI/Core/Extensions/View+AdditionalPadding.swift

```

//
//  View+AdditionalPadding.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 14.02.2026.
//

import SwiftUI

struct AdditionalPaddingModifier: ViewModifier {
    let shouldShow: Bool
    let height: Int
    func body(content: Content) -> some View {
        content
            .safeAreaInset(edge: .bottom, spacing: 0) {
                Color.clear.frame(height: CGFloat(shouldShow ? height : .zero))
            }
    }
}

extension View {
    func additionalPaddingIfNeeded(_ shouldShow: Bool, _ height: Int) -> some View {
        self.modifier(AdditionalPaddingModifier(shouldShow: shouldShow, height: height))
    }
}

```

HalalAI/Core/Extensions/View+HideKeyboard.swift

```

//
//  View+hideKeyboard.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 27.12.2025.
//

import SwiftUI

extension View {
    func hideKeyboard() {
        UIApplication.shared.sendAction(#selector(UIResponder.resignFirstResponder), to: nil, from: nil, for: nil)
    }
}

```

HalalAI/Core/Location/LocationService.swift

```

//
//  LocationService.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 26.02.2026.
//

import Foundation
import CoreLocation
import Observation

@MainActor
protocol LocationService: AnyObject {
    var currentLocation: CLLocation? { get }
    var authorizationStatus: CLAuthorizationStatus { get }
    func requestLocation()
}

@Observable
@MainActor
final class LocationServiceImpl: NSObject, LocationService {
    var currentLocation: CLLocation?
    var authorizationStatus: CLAuthorizationStatus = .notDetermined

    private let manager = CLLocationManager()

    override init() {
        super.init()
        manager.delegate = self
        manager.desiredAccuracy = kCLLocationAccuracyKilometer
        authorizationStatus = manager.authorizationStatus
    }

    func requestLocation() {
        switch manager.authorizationStatus {
        case .notDetermined:
            manager.requestWhenInUseAuthorization()
        case .authorizedWhenInUse, .authorizedAlways:
            manager.requestLocation()
        default:
            break
        }
    }
}

// MARK: - CLLocationManagerDelegate

extension LocationServiceImpl: CLLocationManagerDelegate {
    nonisolated func locationManagerDidChangeAuthorization(_ manager:
        CLLocationManager) {
        Task { @MainActor in
            self.authorizationStatus = manager.authorizationStatus
            if manager.authorizationStatus == .authorizedWhenInUse ||
                manager.authorizationStatus == .authorizedAlways {
                manager.requestLocation()
            }
        }
    }
}

```

```

        nonisolated func locationManager(_ manager: CLLocationManager,
        didUpdateLocations locations: [CLLocation]) {
            guard let location = locations.last else { return }
            Task { @MainActor in
                self.currentLocation = location
            }
        }

        nonisolated func locationManager(_ manager: CLLocationManager,
        didFailWithError error: Error) {
            print("LocationService error: \(error.localizedDescription)")
        }
    }
}

```

HalalAI/Core/Network/APIConfiguration.swift

```

//
//  APIConfiguration.swift
//  HalalAI
//
//  Единая точка конфигурации URL бекенда.
//

import Foundation

enum APIConfiguration {
    static var backendURL: String {
        #if DEBUG
            return "http://localhost:8080"
        #else
            // TODO: Заменить на production URL
            return "https://your-production-url.com"
        #endif
    }
}

```

HalalAI/Core/Network/APIRequest.swift

```

//
//  APIRequest.swift
//  HalalAI
//
//  Протокол типизированного API-запроса с ассоциированным типом ответа.
//

import Foundation

protocol APIRequest: Sendable {
    associatedtype Response: Decodable & Sendable

    var endpoint: Endpoint { get }
    var body: (any Encodable & Sendable)? { get }
    var token: String? { get }
    var timeout: TimeInterval { get }
    var expectedStatus: Int { get }
}

// MARK: - Дефолтные значения

```



```

extension APIRequest {
    var body: (any Encodable & Sendable)? { nil }
    var token: String? { nil }
    var timeout: TimeInterval { 30 }
    var expectedStatus: Int { 200 }
}

```

HalalAI/Core/Network/Endpoint.swift

```

//
// Endpoint.swift
// HalalAI
//
// Типизированный enum со всеми API-маршрутами приложения.
//

import Foundation

enum Endpoint {
    // Auth
    case login
    case register
    case refreshToken

    // Chat
    case chat
    case models

    // Verse
    case verseOfTheDay

    var path: String {
        switch self {
            case .login:           "/api/auth/login"
            case .register:        "/api/auth/register"
            case .refreshToken:    "/api/auth/refresh"
            case .chat:            "/api/chat"
            case .models:          "/api/models"
            case .verseOfTheDay:   "/api/verse-of-the-day"
        }
    }

    var method: HTTPMethod {
        switch self {
            case .login, .register, .refreshToken, .chat: .post
            case .models, .verseOfTheDay:                  .get
        }
    }
}

```

HalalAI/Core/Network/NetworkClient.swift

```

//
// NetworkClient.swift
// HalalAI
//
// Sendable обертка над URLSession. Принимает типизированные APIRequest.
//

```

```

import Foundation

struct NetworkClient: Sendable {

    private let session: URLSession = .shared
    private var baseURL: String { APIConfiguration.backendURL }

    // MARK: - Типизированный запрос

    /// Выполняет APIRequest и декодирует ответ в 'R.Response'.
    func send<R: APIRequest>(_ request: R) async throws(NetworkError) -> R.
        Response {
        let (data, response) = try await sendRaw(request)

        guard response.statusCode == request.expectedStatus else {
            throw .serverError(statusCode: response.statusCode, data: data)
        }

        do {
            return try JSONDecoder().decode(R.Response.self, from: data)
        } catch {
            throw .decodingError(error)
        }
    }

    // MARK: - Сырой запрос

    /// Выполняет APIRequest и возвращает сырые данные + HTTPURLResponse.
    /// Используется, когда нужен ручной разбор ответа (например ChatService).
    func sendRaw<R: APIRequest>(_ request: R) async throws(NetworkError) -> (Data
        , HTTPURLResponse) {
        guard let url = URL(string: "\(baseURL)\(request.endpoint.path)") else {
            throw .invalidURL
        }

        var urlRequest = URLRequest(url: url)
        urlRequest.httpMethod = request.endpoint.method.rawValue
        urlRequest.timeoutInterval = request.timeout
        urlRequest.setValue("application/json; charset=utf-8", forHTTPHeaderField
            : "Content-Type")
        urlRequest.setValue("application/json; charset=utf-8", forHTTPHeaderField
            : "Accept")

        if let token = request.token {
            urlRequest.setValue("Bearer \(token)", forHTTPHeaderField: "
                Authorization")
        }

        if let body = request.body {
            do {
                urlRequest.httpBody = try JSONEncoder().encode(body)
            } catch {
                throw .unknown(error)
            }
        }

        let data: Data
        let response: URLResponse
        do {
            (data, response) = try await session.data(for: urlRequest)
        } catch {
            throw .unknown(error)
        }
    }
}

```

```

    }

    guard let httpResponse = response as? HTTPURLResponse else {
        throw .invalidResponse
    }

    return (data, httpResponse)
}

// MARK: - HTTPMethod

enum HTTPMethod: String, Sendable {
    case get = "GET"
    case post = "POST"
    case put = "PUT"
    case delete = "DELETE"
}

```

HalalAI/Core/Network/NetworkError.swift

```

//
// NetworkError.swift
// HalalAI
//
// Единый тип ошибки сетевого слоя.
//

import Foundation

enum NetworkError: Error {
    /// Невалидный URL
    case invalidURL
    /// Ответ не является HTTPURLResponse
    case invalidResponse
    /// Статус-код не совпал с ожидаемым; data содержит тело ответа для доп. парсинга
    case serverError(statusCode: Int, data: Data)
    /// Ошибка декодирования JSON
    case decodingError(Error)
    /// Ошибка URLSession (таймаут, нет сети и т.д.)
    case unknown(Error)
}

```

HalalAI/Features/Auth/Models/AuthModels.swift

```

//
// AuthModels.swift
// HalalAI
//

import Foundation

// MARK: - Auth Models

struct RegisterRequest: Codable {
    let email: String
    let password: String
}

```

```

}

struct LoginRequest: Codable {
    let email: String
    let password: String
}

struct RefreshTokenRequest: Codable {
    let token: String
}

struct AuthResponse: Codable {
    let token: String
    let type: String
    let userId: Int64
    let email: String
}

struct ErrorResponse: Codable {
    let message: String
    let timestamp: String?
    let path: String?
}

// MARK: - Auth State

enum AuthState: Equatable {
    case authenticated
    case unauthenticated
    case loading
    case guest
}

enum AuthError: LocalizedError {
    case invalidCredentials
    case userAlreadyExists
    case networkError(String)
    case validationError(String)
    case unknown(String)

    var errorDescription: String? {
        switch self {
            case .invalidCredentials:
                return "Неверный email или пароль"
            case .userAlreadyExists:
                return "Пользователь с таким email уже существует"
            case .networkError(let message):
                return "Ошибка сети: \(message)"
            case .validationError(let message):
                return "Ошибка валидации: \(message)"
            case .unknown(let message):
                return message
        }
    }
}

```

HalalAI/Features/Auth/Service/AuthManager.swift

```

//
//  AuthManager.swift
//  HalalAI

```

```
//

import SwiftUI

@MainActor
protocol AuthManager {
    var authState: AuthState { get set }
    var currentUser: AuthResponse? { get set }
    var errorMessage: String? { get set }
    var isAuthenticated: Bool { get }
    var isGuest: Bool { get }
    var authToken: String? { get }
    func saveAuth(_ response: AuthResponse)
    func logout()
    func continueAsGuest()
    func refreshToken() async throws
}

@MainActor
@Observable
final class AuthManagerImpl: AuthManager {
    var authState: AuthState = .unauthenticated
    var currentUser: AuthResponse?
    var errorMessage: String?

    // TODO: Перенести хранение токена в Keychain для безопасности
    private let tokenKey = "HalalAI.authToken"
    private let userKey = "HalalAI.currentUser"
    private let guestKey = "HalalAI.isGuest"

    init() {
        loadStoredAuth()
    }

    // MARK: - Public Methods

    var isAuthenticated: Bool {
        return authState == .authenticated && currentUser != nil
    }

    var isGuest: Bool {
        return authState == .guest
    }

    var authToken: String? {
        return UserDefaults.standard.string(forKey: tokenKey)
    }

    func saveAuth(_ response: AuthResponse) {
        currentUser = response
        UserDefaults.standard.set(response.token, forKey: tokenKey)
        UserDefaults.standard.removeObject(forKey: guestKey)

        if let userData = try? JSONEncoder().encode(response) {
            UserDefaults.standard.set(userData, forKey: userKey)
        }

        authState = .authenticated
        errorMessage = nil
    }
}
```

```

func continueAsGuest() {
    authState = .guest
    UserDefaults.standard.set(true, forKey: guestKey)
}

func logout() {
    currentUser = nil
    cleanUserDefaults()
    authState = .unauthenticated
    errorMessage = nil
}

func refreshToken() async throws {
    guard let oldToken = authToken, !oldToken.isEmpty else {
        throw NSError(domain: "AuthManager", code: 401, userInfo: [
            NSLocalizedDescriptionKey: "Токен не найден"])
    }

    // Создаем временный AuthService для refresh
    let authService = AuthServiceImpl()
    let newAuthResponse = try await authService.refreshToken(oldToken)

    // Сохраняем новый токен
    saveAuth(newAuthResponse)
}

// MARK: - Private Methods

private func loadStoredAuth() {
    guard let token = UserDefaults.standard.string(forKey: tokenKey),
        !token.isEmpty,
        let userData = UserDefaults.standard.data(forKey: userKey),
        let user = try? JSONDecoder().decode(AuthResponse.self, from:
userData) else {
        authState = UserDefaults.standard.bool(forKey: guestKey) ? .guest : .
unauthenticated
        return
    }

    currentUser = user
    authState = .authenticated
}

private func cleanUserDefaults() {
    UserDefaults.standard.removeObject(forKey: tokenKey)
    UserDefaults.standard.removeObject(forKey: userKey)
    UserDefaults.standard.removeObject(forKey: guestKey)
}
}

```

HalalAI/Features/Auth/Service/AuthRequests.swift

```

//
//  AuthRequests.swift
//  HalalAI
//
//  Типизированные API-запросы для аутентификации.
//

import Foundation

```

```

struct LoginAPIRequest: APIRequest {
    typealias Response = AuthResponse

    let endpoint = Endpoint.login
    let body: (any Encodable & Sendable)?

    init(body: LoginRequest) {
        self.body = body
    }
}

struct RegisterAPIRequest: APIRequest {
    typealias Response = AuthResponse

    let endpoint = Endpoint.register
    let body: (any Encodable & Sendable)?
    let expectedStatus = 201

    init(body: RegisterRequest) {
        self.body = body
    }
}

struct RefreshTokenAPIRequest: APIRequest {
    typealias Response = AuthResponse

    let endpoint = Endpoint.refreshToken
    let body: (any Encodable & Sendable)?

    init(body: RefreshTokenRequest) {
        self.body = body
    }
}

```

HalalAI/Features/Auth/Service/AuthService.swift

```

//
//  AuthService.swift
//  HalalAI
//

import Foundation

@MainActor
protocol AuthService {
    var isLoading: Bool { get set }
    var errorMessage: String? { get set }
    func register(email: String, password: String) async throws -> AuthResponse
    func login(email: String, password: String) async throws -> AuthResponse
    func refreshToken(_ oldToken: String) async throws -> AuthResponse
}

@MainActor
@Observable
final class AuthServiceImpl: AuthService {
    var isLoading = false
    var errorMessage: String?

    private let networkClient: NetworkClient

```

```

init(networkClient: NetworkClient = NetworkClient()) {
    self.networkClient = networkClient
}

// MARK: - Public Methods

func register(email: String, password: String) async throws -> AuthResponse {
    let body = RegisterRequest(
        email: email.trimmingCharacters(in: .whitespacesAndNewlines),
        password: password
    )
    return try await performRequest(RegisterAPIRequest(body: body))
}

func login(email: String, password: String) async throws -> AuthResponse {
    let body = LoginRequest(
        email: email.trimmingCharacters(in: .whitespacesAndNewlines),
        password: password
    )
    return try await performRequest(LoginAPIRequest(body: body))
}

func refreshToken(_ oldToken: String) async throws -> AuthResponse {
    let body = RefreshTokenRequest(token: oldToken)
    return try await performRequest(RefreshTokenAPIRequest(body: body))
}

// MARK: - Private

private func performRequest<R: APIRequest>(
    _ request: R
) async throws -> AuthResponse where R.Response == AuthResponse {
    isLoading = true
    errorMessage = nil
    defer { isLoading = false }

    do {
        return try await networkClient.send(request)
    } catch {
        let authError = mapToAuthError(error)
        errorMessage = authError.errorDescription
        throw authError
    }
}

/// Мappings NetworkError AuthError с сохранением бизнес-логики статус-кодов
private func mapToAuthError(_ error: NetworkError) -> AuthError {
    switch error {
    case .invalidURL:
        return .networkError("Неверный URL бекенда")

    case .invalidResponse:
        return .networkError("Неверный ответ от сервера")

    case .serverError(let statusCode, let data):
        switch statusCode {
        case 400:
            if let errorResponse = try? JSONDecoder().decode(ErrorResponse.self, from: data) {
                if errorResponse.message.contains("уже существует") {
                    return .userAlreadyExists
                }
            }
        }
    }
}

```



```

        return .validationError(errorResponse.message)
    }
    return .validationError("Ошибка валидации данных")

    case 401:
        return .invalidCredentials

    default:
        if let errorResponse = try? JSONDecoder().decode(ErrorResponse.self, from: data) {
            return .unknown(errorResponse.message)
        }
        return .networkError("Ошибка сервера (\(statusCode))")
    }

    case .decodingError:
        return .unknown("Неверный формат ответа от сервера")

    case .unknown(let underlying):
        return .networkError(underlying.localizedDescription)
    }
}
}

```

HalalAI/Features/Auth/UI/AuthFlowView.swift

```

//
//  AuthFlowView.swift
//  HalalAI
//

import SwiftUI

struct AuthFlowView: View {
    @State private var path: [AuthCoordinator] = []

    var body: some View {
        NavigationStack(path: $path) {
            screenFactory.makeLoginView(path: $path)
                .navigationDestination(for: AuthCoordinator.self) { step in
                    switch step {
                    case .login:
                        screenFactory.makeLoginView(path: $path)
                    case .register:
                        screenFactory.makeRegisterView(path: $path)
                            .navigationBarBackButtonHidden()
                    }
                }
        }
    }
}

```

HalalAI/Features/Auth/UI/AuthTextField.swift

```

//
//  AuthTextField.swift
//  HalalAI
//

```

```

import SwiftUI

struct AuthTextField: View {
    let icon: String
    let placeholder: String
    @Binding var text: String
    var isSecure: Bool = false
    @State private var showText: Bool = false

    var body: some View {
        HStack(spacing: 12) {
            Image(systemName: icon)
                .foregroundColor(.darkGreen)
                .frame(width: 20)

            Group {
                if isSecure && !showText {
                    SecureField(placeholder, text: $text)
                } else {
                    TextField(placeholder, text: $text)
                        .textInputAutocapitalization(.never)
                        .disableAutocorrection(true)
                }
            }
                .foregroundColor(.primary)

            if isSecure {
                Button(showText ? "Скрыть пароль" : "Показать пароль",
                    systemImage: showText ? "eye.slash.fill" : "eye.fill",
                    action: { showText.toggle() })
                    .labelStyle(.iconOnly)
                    .foregroundColor(.secondary)
            }
        }
        .padding(.horizontal, 16)
        .padding(.vertical, 15)
        .background(Color.white)
        .clipShape(.rect(cornerRadius: 14))
        .shadow(color: .black.opacity(0.07), radius: 8, x: 0, y: 2)
    }
}

```

HalalAI/Features/Auth/UI/Login/LoginView.swift

```

//
//  LoginView.swift
//  HalalAI
//

import SwiftUI

struct LoginView: View {
    @State private var viewModel: ViewModel
    private let onShowRegister: (() -> Void)?

    init(
        authManager: AuthManager,
        authService: AuthService,
        onShowRegister: (() -> Void)? = nil
    ) {
        self.viewModel = ViewModel(authManager, authService, onShowRegister)
    }
}

```

```

) {
    _viewModel = State(initialValue: ViewModel(authManager: authManager,
authService: authService))
    self.onShowRegister = onShowRegister
}

var body: some View {
    @Bindable var vm = viewModel
    ZStack(alignment: .bottom) {
        Color.darkGreen.ignoresSafeArea()

        VStack(spacing: 0) {
            // Header
            VStack(spacing: 12) {
                Image(systemName: "moon.stars.fill")
                    .font(.largeTitle)
                    .foregroundColor(.white)

                Text("Halal AI")
                    .font(.largeTitle)
                    .bold()
                    .foregroundColor(.white)

                Text("Войдите в свой аккаунт")
                    .font(.body)
                    .foregroundColor(.white.opacity(0.8))
            }
            .frame(maxWidth: .infinity)
            .padding(.top, 70)
            .padding(.bottom, 40)

            // White card
            ZStack(alignment: .top) {
                // Background extends to bottom edge
                UnevenRoundedRectangle(
                    topLeadingRadius: 32,
                    bottomLeadingRadius: 0,
                    bottomTrailingRadius: 0,
                    topTrailingRadius: 32
                )
                .fill(Color(.systemBackground))
                .ignoresSafeArea(edges: .bottom)

                // ScrollView moves with keyboard
                ScrollView {
                    VStack(spacing: 20) {
                        VStack(spacing: 14) {
                            AuthTextField(
                                icon: "envelope.fill",
                                placeholder: "Email",
                                text: $vm.email
                            )

                            AuthTextField(
                                icon: "lock.fill",
                                placeholder: "Пароль",
                                text: $vm.password,
                                isSecure: true
                            )
                        }

                        // Login button

```

```

        Button(action: {
            Task { await vm.login() }
        }) {
            HStack {
                if vm.authService.isLoading {
                    ProgressView()
                        .progressViewStyle(
CircularProgressViewStyle(tint: .white))
                } else {
                    Text("Войти")
                        .fontWeight(.semibold)
                }
            }
            .frame(maxWidth: .infinity)
            .frame(height: 52)
            .background(vm.isDisable ? Color.greenForeground
: Color.darkGreen)
            .foregroundColor(.white)
            .clipShape(.rect(cornerRadius: 14))
        }
        .disabled(vm.isDisable)
        .opacity(vm.isDisable ? 0.6 : 1.0)

// Register link
HStack(spacing: 4) {
    Text("Нет аккаунта?")
        .foregroundColor(.secondary)
    Button(action: { onShowRegister?() }) {
        Text("Зарегистрироваться")
            .fontWeight(.semibold)
            .foregroundColor(.darkGreen)
    }
}

// Guest login
Button(action: {
    vm.authManager.continueAsGuest()
}) {
    Text("Продолжить без аккаунта")
        .font(.subheadline)
        .foregroundColor(.secondary)
}

}
.padding(.horizontal, 28)
.padding(.top, 36)
.padding(.bottom, 40)
}
.scrollDismissesKeyboard(.interactively)
}
}
.alert("Ошибка", isPresented: $vm.showError) {
    Button("OK", role: .cancel) { }
} message: {
    Text(vm.errorMessage)
}
}
}

```

```

//
// LoginViewModel.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 29.12.2025.
//

import Foundation

extension LoginView {
    @MainActor
    @Observable
    final class ViewModel {
        var authManager: AuthManager
        var authService: AuthService

        init(authManager: AuthManager, authService: AuthService) {
            self.authManager = authManager
            self.authService = authService
        }

        var email: String = ""
        var password: String = ""
        var showPassword: Bool = false
        var showError: Bool = false
        var errorMessage: String = ""
        var isDisable: Bool {
            authService.isLoading || email.isEmpty || password.isEmpty
        }

        func login() async {
            guard !email.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty,
                  !password.isEmpty else {
                errorMessage = "Заполните все поля"
                showError = true
                return
            }

            do {
                let response = try await authService.login(
                    email: email,
                    password: password
                )
                authManager.saveAuth(response)
            } catch let error as AuthError {
                errorMessage = error.errorDescription ?? "Неизвестная ошибка"
                showError = true
            } catch {
                errorMessage = "Произошла ошибка при входе"
                showError = true
            }
        }
    }
}

```

HalalAI/Features/Auth/UI/Register/RegisterView.swift

```

//
// RegisterView.swift
// HalalAI
//

```



```

        text: $vm.email
    )

    VStack(alignment: .leading, spacing: 6) {
        AuthTextField(
            icon: "lock.fill",
            placeholder: "Пароль",
            text: $vm.password,
            isSecure: true
        )
        if !vm.password.isEmpty && vm.password.count
< 8 {
            Text("Пароль должен содержать минимум 8 с
имболов")

            .font(.caption)
            .foregroundColor(.red)
            .padding(.horizontal, 4)
        }
    }

    VStack(alignment: .leading, spacing: 6) {
        AuthTextField(
            icon: "lock.fill",
            placeholder: "Подтвердите пароль",
            text: $vm.confirmPassword,
            isSecure: true
        )
        if !vm.confirmPassword.isEmpty && vm.password
!= vm.confirmPassword {
            Text("Пароли не совпадают")
            .font(.caption)
            .foregroundColor(.red)
            .padding(.horizontal, 4)
        }
    }
}

// Register button
Button(action: {
    Task { await vm.register() }
}) {
    HStack {
        if vm.authService.isLoading {
            ProgressView()
                .progressViewStyle(
CircularProgressViewStyle(tint: .white))
        } else {
            Text("Зарегистрироваться")
                .fontWeight(.semibold)
        }
    }
    .frame(maxWidth: .infinity)
    .frame(height: 52)
    .background(vm.isFormValid ? Color.darkGreen :
Color.greenForeground)
    .foregroundColor(.white)
    .clipShape(.rect(cornerRadius: 14))
}
.disabled(!vm.isFormValid || vm.authService.isLoading)
)
.opacity((!vm.isFormValid || vm.authService.isLoading)
? 0.6 : 1.0)

```

```

        // Login link
        HStack(spacing: 4) {
            Text("Уже есть аккаунт?")
                .foregroundColor(.secondary)
            Button(action: { onShowLogin?() }) {
                Text("Войти")
                    .fontWeight(.semibold)
                    .foregroundColor(.darkGreen)
            }
        }

        Button(action: {
            vm.authManager.continueAsGuest()
        }) {
            Text("Продолжить без регистрации")
                .font(.subheadline)
                .foregroundColor(.secondary)
        }

        .padding(.horizontal, 28)
        .padding(.top, 36)
        .padding(.bottom, 40)
    }
    .scrollDismissesKeyboard(.interactively)
}

}

.alert("Ошибка", isPresented: $vm.showError) {
    Button("OK", role: .cancel) { }
} message: {
    Text(vm.errorMessage)
}

}

}

```

HalalAI/Features/Auth/UI/Register/RegisterViewModel.swift

```

//
// RegisterViewModel.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 29.12.2025.
//

import Foundation

extension RegisterView {
    @MainActor
    @Observable
    final class ViewModel {
        var email: String = ""
        var password: String = ""
        var confirmPassword: String = ""
        var showPassword: Bool = false
        var showConfirmPassword: Bool = false
        var showError: Bool = false
        var errorMessage: String = ""

        var authManager: AuthManager
        var authService: AuthService
    }
}

```



```

init(authManager: AuthManager, authService: AuthService) {
    self.authManager = authManager
    self.authService = authService
}

var isValidForm: Bool {
    !email.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty &&
    password.count >= 8 &&
    password == confirmPassword &&
    isValidEmail(email)
}

func isValidEmail(_ email: String) -> Bool {
    let emailRegex = "[A-Z0-9a-z._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,64}"
    let emailPredicate = NSPredicate(format:"SELF MATCHES %@", emailRegex)

    return emailPredicate.evaluate(with: email)
}

func register() async {
    guard !email.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty,
          password.count >= 8,
          password == confirmPassword else {
        errorMessage = "Проверьте правильность заполнения всех полей"
        showError = true
        return
    }

    guard isValidEmail(email) else {
        errorMessage = "Введите корректный email адрес"
        showError = true
        return
    }

    do {
        let response = try await authService.register(
            email: email,
            password: password
        )
        authManager.saveAuth(response)
    } catch let error as AuthError {
        errorMessage = error.errorDescription ?? "Неизвестная ошибка"
        showError = true
    } catch {
        errorMessage = "Произошла ошибка при регистрации"
        showError = true
    }
}
}

```

HalalAI/Features/Chat/Models/ChatModels.swift

```

//
// ChatModels.swift
// HalalAI
//
// Created by Мирсайт Сабирзянов on 25.10.2025.
//

```

```

import Foundation

// MARK: - Chat Models

enum Role: String, CaseIterable, Codable {
    case user
    case assistant
}

struct ChatMessage: Identifiable, Codable {
    let id: UUID
    let role: Role
    let text: String
    let date: Date
    let model: String?

    init(role: Role, text: String, model: String? = nil) {
        self.id = UUID()
        self.role = role
        self.text = text
        self.date = Date()
        self.model = model
    }
}

// MARK: - Chat States

enum ChatState: Equatable {
    case idle
    case typing
    case error(String)
}

// MARK: - Connection State

enum ConnectionState: Equatable {
    case connected
    case disconnected
    case connecting
}

```

HalalAI/Features/Chat/Service/ChatService.swift

```

//
//  ChatService.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 25.10.2025.
//

import Foundation

@MainActor
protocol ChatService {
    var messages: [ChatMessage] { get set }
    var chatState: ChatState { get set }
    var connectionState: ConnectionState { get set }
    var userApiKey: String { get set }
    var remoteModel: String { get set }
    var maxTokens: Int { get set }
    var temperature: Double { get set }
}

```

```

    var useRag: Bool { get set }
    var availableModels: [String] { get set }
    var defaultRemoteModel: String { get set }
    func loadModels() async
    func sendMessage(_ text: String)
    func retryLastMessage()
    func clearChat()
}

@MainActor
@Observable
final class ChatServiceImpl: ChatService {
    var messages: [ChatMessage] = []
    var chatState: ChatState = .idle
    var connectionState: ConnectionState = .connected

    var userApiKey: String {
        get { settingsStore.userApiKey }
        set { settingsStore.userApiKey = newValue }
    }
    var remoteModel: String {
        get { settingsStore.remoteModel }
        set { settingsStore.remoteModel = newValue }
    }
    var maxTokens: Int {
        get { settingsStore.maxTokens }
        set { settingsStore.maxTokens = newValue }
    }
    var temperature: Double {
        get { settingsStore.temperature }
        set { settingsStore.temperature = newValue }
    }
    var useRag: Bool {
        get { settingsStore.useRag }
        set { settingsStore.useRag = newValue }
    }

    var availableModels: [String] = []
    var defaultRemoteModel: String = ""

    // MARK: - Private

    private var isSending = false
    private var lastSendAt: Date?
    private let settingsStore: ChatSettingsStore
    private let authManager: AuthManager
    private let networkClient: NetworkClient

    init(authManager: AuthManager, settingsStore: ChatSettingsStore,
networkClient: NetworkClient = NetworkClient()) {
        self.authManager = authManager
        self.settingsStore = settingsStore
        self.networkClient = networkClient
    }

    // MARK: - Public Methods

    func loadModels() async {
        do {
            let (data, response) = try await networkClient.sendRaw(
ModelsAPIRequest())

```

```

        guard response.statusCode == 200 else {
            print("    Ошибка загрузки моделей: \((String(data: data, encoding:
.utf8) ?? "nil")")
            return
        }

        let modelsResponse = try JSONDecoder().decode(modelsResponse.self,
from: data)

        let defaultModel = modelsResponse.defaultModel ?? ""
        var allowed = modelsResponse.allowedModels ?? []
        if allowed.isEmpty, !defaultModel.isEmpty {
            allowed = [defaultModel]
        }

        let currentModel = remoteModel.trimmingCharacters(in: .
whitespacesAndNewlines)
        self.defaultRemoteModel = defaultModel
        self.availableModels = allowed
        if !allowed.isEmpty, currentModel.isEmpty, !defaultModel.isEmpty {
            self.remoteModel = defaultModel
        }
        print("    Модели загружены: default=\(defaultModel), count=\(allowed.
count)")
    } catch {
        print("    Ошибка при загрузке моделей: \((error)")
    }
}

func sendMessage(_ text: String) {
    guard !text.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty else
{ return }
    if isSending, let last = lastSendAt, Date.now.timeIntervalSince(last) >
15 {
        isSending = false
        print("    sendMessage: предыдущий запрос завис >15с, сбрасываем
isSending")
    }
    guard !isSending else {
        print("    sendMessage: блокируем отправку, предыдущий запрос еще обра
батывается")
        return
    }

    let userMessage = ChatMessage(role: .user, text: text)
    messages.append(userMessage)

    chatState = .typing
    connectionState = .connecting
    isSending = true
    lastSendAt = Date.now

    Task {
        await sendRequestToBackend(userMessage: userMessage, isRetry: false)
    }
}

func retryLastMessage() {
    guard !isSending else { return }

    guard let lastUserIndex = messages.lastIndex(where: { $0.role == .user })
else { return }

```

```

let lastUserMessage = messages[lastUserIndex]
messages.removeSubrange((lastUserIndex + 1)..

```

```

        let errorMessage = String(data: data, encoding: .utf8) ?? "Ошибка
сервера"
        await handleError("Ошибка сервера \(response.statusCode): \(
errorMessage)")
        return
    }

    let chatResponse = try JSONDecoder().decode(ChatResponse.self, from:
data)

    let aiMessage = ChatMessage(role: .assistant, text: chatResponse.
reply)
    messages.append(aiMessage)

    chatState = .idle
    connectionState = .connected
    isSending = false
    lastSendAt = nil
} catch {
    await handleError("Ошибка сети: \(error)")
}
}

private func handleError(_ message: String) async {
    let errorMessage = ChatMessage(role: .assistant, text: "У нас что-то слом
алось, попробуйте позже или повторите попытку.")
    messages.append(errorMessage)

    chatState = .error(message)
    connectionState = .disconnected
    isSending = false
    lastSendAt = nil
}
}

// MARK: - Private API

private struct ModelsAPIRequest: APIRequest {
    typealias Response = ModelsResponse

    let endpoint = Endpoint.models
    let timeout: TimeInterval = 15
}

private struct ChatAPIRequest: APIRequest {
    typealias Response = ChatResponse

    let endpoint = Endpoint.chat
    let body: (any Encodable & Sendable)?
    let token: String?
    let timeout: TimeInterval = 300

    init(body: ChatRequest, token: String) {
        self.body = body
        self.token = token
    }
}

// MARK: - Private DTO

private struct ChatMessageDTO: Codable, Sendable {

```

```

        let role: String
        let content: String
    }

    private struct ChatRequest: Codable, Sendable {
        let messages: [ChatMessageDTO]
        let maxTokens: Int
        let temperature: Double
        let useRag: Bool
        let apiKey: String
        let remoteModel: String

        enum CodingKeys: String, CodingKey {
            case messages
            case maxTokens = "max_tokens"
            case temperature
            case useRag = "use_rag"
            case apiKey = "api_key"
            case remoteModel = "remote_model"
        }
    }

    private struct ChatResponse: Codable, Sendable {
        let reply: String
    }

    private struct ModelsResponse: Codable, Sendable {
        let defaultModel: String?
        let allowedModels: [String]?

        enum CodingKeys: String, CodingKey {
            case defaultModel = "default_model"
            case allowedModels = "allowed_models"
        }
    }
}

```

HalalAI/Features/Chat/Service/ChatSettingsStore.swift

```

//
//  ChatSettingsStore.swift
//  HalalAI
//
//  Персистенция настроек чата (API key, модель, токены, temperature, RAG).
//

import Foundation

@MainActor
@Observable
final class ChatSettingsStore {
    var userApiKey: String {
        didSet { UserDefaults.standard.set(userApiKey, forKey: Keys.apiKey) }
    }
    var remoteModel: String {
        didSet { UserDefaults.standard.set(remoteModel, forKey: Keys.remoteModel) }
    }
    var maxTokens: Int {
        didSet {
            let clamped = max(16, min(maxTokens, 6144))
            if clamped != maxTokens {

```

```

        maxTokens = clamped
        return
    }
    UserDefaults.standard.set(maxTokens, forKey: Keys.maxTokens)
}
}
var temperature: Double {
    didSet {
        let clamped = max(0.0, min(temperature, 2.0))
        if clamped != temperature {
            temperature = clamped
            return
        }
        UserDefaults.standard.set(temperature, forKey: Keys.temperature)
    }
}
var useRag: Bool {
    didSet { UserDefaults.standard.set(useRag, forKey: Keys.useRag) }
}

init() {
    self.userApiKey = UserDefaults.standard.string(forKey: Keys.apiKey) ?? ""
    self.remoteModel = UserDefaults.standard.string(forKey: Keys.remoteModel)
    ?? ""
    let savedMax = UserDefaults.standard.integer(forKey: Keys.maxTokens)
    self.maxTokens = savedMax == 0 ? 2048 : savedMax
    let savedTemp = UserDefaults.standard.double(forKey: Keys.temperature)
    self.temperature = savedTemp == 0 ? 0.7 : savedTemp
    self.useRag = UserDefaults.standard.object(forKey: Keys.useRag) as? Bool
    ?? true
}

private enum Keys {
    static let apiKey = "HalalAI.userApiKey"
    static let remoteModel = "HalalAI.remoteModel"
    static let maxTokens = "HalalAI.maxTokens"
    static let temperature = "HalalAI.temperature"
    static let useRag = "HalalAI.useRag"
}
}

```

HalalAI/Features/Chat/UI/ChatUtils.swift

```

//
//  ChatUtils.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 25.10.2025.
//

import Foundation
import SwiftUI

// MARK: - Chat Utilities

struct ChatUtils {

    // MARK: - Text Processing

    static func formatMessageTime(_ date: Date) -> String {
        date.formatted(date: .omitted, time: .shortened)
    }
}

```



```

    }

    static func formatDate(_ date: Date) -> String {
        date.formatted(date: .abbreviated, time: .omitted)
    }

    static func isToday(_ date: Date) -> Bool {
        Calendar.current.isDateInToday(date)
    }

    static func isYesterday(_ date: Date) -> Bool {
        Calendar.current.isDateInYesterday(date)
    }

    // MARK: - Message Validation

    static func isValidMessage(_ text: String) -> Bool {
        let trimmed = text.trimmingCharacters(in: .whitespacesAndNewlines)
        return !trimmed.isEmpty && trimmed.count <= 2000
    }

    static func sanitizeMessage(_ text: String) -> String {
        return text.trimmingCharacters(in: .whitespacesAndNewlines)
    }

    // MARK: - UI Helpers

    static func getMessageBubbleColor(for role: Role) -> Color {
        switch role {
        case .user:
            return .blue
        case .assistant:
            return .green.opacity(0.1)
        }
    }

    static func getMessageTextColor(for role: Role) -> Color {
        switch role {
        case .user:
            return .white
        case .assistant:
            return .primary
        }
    }
}

// MARK: - Chat Constants

struct ChatConstants {
    static let maxMessageLength = 2000
    static let maxMessagesInHistory = 100
    static let typingIndicatorDelay: TimeInterval = 0.5
    static let messageAnimationDuration: TimeInterval = 0.3
    static let scrollAnimationDuration: TimeInterval = 0.3
}

// MARK: - Chat Colors

struct ChatColors {
    static let userBubble = Color.blue
    static let assistantBubble = Color.green.opacity(0.1)
    static let assistantBubbleBorder = Color.green.opacity(0.3)
}

```

```

static let errorBubble = Color.red.opacity(0.1)
static let errorBubbleBorder = Color.red.opacity(0.3)
static let inputBackground = Color(.systemGray6)
static let sendButton = Color.blue
static let microphoneButton = Color.blue
static let quickQuestionButton = Color.blue.opacity(0.1)
static let quickQuestionBorder = Color.blue.opacity(0.3)
}

```

HalalAI/Features/Chat/UI/ChatView.swift

```

//
//  ChatView.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 12.10.2025.
//

import SwiftUI

struct ChatView: View {
    @State private var viewModel: ViewModel
    @Environment(Coordinator.self) var coordinator

    init(chatService: ChatService, authManager: AuthManager) {
        _viewModel = State(initialValue: ViewModel(chatService: chatService,
authManager: authManager))
    }

    var body: some View {
        @Bindable var vm = viewModel
        VStack(spacing: 0) {
            ZStack {
                if vm.chatService.messages.isEmpty {
                    EmptyChatView(chatService: vm.chatService)
                } else {
                    ScrollViewReader { proxy in
                        ScrollView {
                            LazyVStack(spacing: 16) {
                                ForEach(vm.chatService.messages) { message in
                                    MessageBubble(message: message)
                                        .id(message.id)
                                        .transition(.asymmetric(
                                            insertion: .scale.combined(with: .
opacity),
                                            removal: .opacity
                                        ))
                                }

                                // Индикатор печати
                                if vm.chatService.chatState == .typing {
                                    TypingIndicator()
                                        .id("typing")
                                }

                                // Сообщение об ошибке
                                if case .error(let errorMessage) = vm.chatService
.chatState {
                                    ErrorMessageView(
                                        message: errorMessage,
                                        onRetry: {

```

```

                                vm.chatService.retryLastMessage()
                            }
                        )
                    .id("error")
                }
            }
        .padding(.horizontal, 16)
        .padding(.vertical, 8)
    }
}

}

}

InputBar(
    messageText: $vm.messageText,
    onSend: vm.sendMessage,
    onMicrophoneTap: {
        // TODO: Реализовать голосовой ввод, пока кнопка скрыта
        print("Микрофон нажат")
    }
)

}

.navigationTitle("Halal AI")
.toolbar(vm.authManager.isGuest ? .hidden : .visible, for: .navigationBar
)

.navigationBarTitleDisplayMode(.inline)
.toolbar {
    ToolbarItem(placement: .topBarTrailing) {
        if !vm.authManager.isGuest {
            Menu {
                Button(action: {
                    coordinator.currentSelectedTab = .settings
                }) {
                    Label("Настройки модели", systemImage: "gearshape")
                }

                Button(action: {
                    vm.chatService.clearChat()
                }) {
                    Label("Очистить чат", systemImage: "trash")
                }
            } label: {
                Label("Опции", systemImage: "ellipsis.circle")
                    .labelStyle(.iconOnly)
            }
        }
    }
}

.background {
    Color.greenBackground.ignoresSafeArea()
}

.overlay {
    if vm.authManager.isGuest {
        GuestAuthPromptView(featureName: "ИИ-чат", authManager: vm.
authManager)
    }
}

}

}

```

```
//
// ChatViewModel.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 07.01.2026.
//

import SwiftUI

extension ChatView {
    @Observable
    @MainActor
    final class ViewModel {
        var messageText = ""
        let chatService: ChatService
        let authManager: AuthManager

        init(chatService: ChatService, authManager: AuthManager) {
            self.chatService = chatService
            self.authManager = authManager
        }

        func sendMessage() {
            let trimmedText = messageText.trimmingCharacters(in: .
whitespacesAndNewlines)
            guard !trimmedText.isEmpty else { return }

            chatService.sendMessage(trimmedText)
            messageText = ""
        }
    }
}
```

HalalAI/Features/Chat/UI/EmptyChatView.swift

```
//
// EmptyChatView.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 25.10.2025.
//

import SwiftUI

struct EmptyChatView: View {
    @State private var isAnimating = false
    var chatService: ChatService

    var body: some View {
        VStack(spacing: 24) {
            Spacer()
            VStack(spacing: 16) {
                Text("Ассаламу алейкум ")
                    .font(.title2)
                    .fontWeight(.semibold)
                    .foregroundColor(.primary)

                Text("Я -- HalalAI, ваш халяль-помощник.")
                    .font(.headline)
            }
        }
    }
}
```

```

        .foregroundColor(.secondary)
        .multilineTextAlignment(.center)
    }
    Spacer()

    VStack(alignment: .leading, spacing: 12) {
        ScrollView(.horizontal) {
            HStack {
                Spacer()
                QuickQuestionButton(text: "Что такое халяль?", onTap: {
                    chatService.sendMessage("Что такое халяль?") })
                QuickQuestionButton(text: "Халяль мясо", onTap: {
                    chatService.sendMessage("Расскажи о халяль мясо") })
                QuickQuestionButton(text: "Молочные продукты", onTap: {
                    chatService.sendMessage("Какие молочные продукты халяль?") })
                QuickQuestionButton(text: "Популярные бренды", onTap: {
                    chatService.sendMessage("Какие популярные халяль бренды?") })
                Spacer()
            }
        }
        .scrollIndicators(.hidden)
    }
    .onAppear {
        isAnimating = true
    }
}

struct QuickQuestionButton: View {
    let text: String
    let onTap: () -> Void
    @State private var isPressed = false

    var body: some View {
        Button(action: {
            HapticFeedback.light()
            onTap()
        }) {
            Text(text)
                .font(.caption)
                .foregroundColor(.darkGreen)
                .padding(.horizontal, 12)
                .padding(.vertical, 8)
                .background {
                    RoundedRectangle(cornerRadius: 16)
                        .fill(Color.greenForeground)
                }
        }
        .scaleEffect(isPressed ? 0.95 : 1.0)
        .onLongPressGesture(minimumDuration: 0) {
            // Действие при долгом нажатии
        } onPressingChanged: { pressing in
            isPressed = pressing
        }
    }
}

```

HalalAI/Features/Chat/UI/InputBar.swift

```
//
```

```

// InputBar.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 25.10.2025.
//

import SwiftUI

struct InputBar: View {
    @Binding var messageText: String
    let onSend: () -> Void
    let onMicrophoneTap: () -> Void
    @State private var textHeight: CGFloat = 40

    var body: some View {
        HStack(alignment: .bottom, spacing: 12) {
            // Кнопка микрофона, пока ее отключем
            if false {
                Button(action: {
                    HapticFeedback.medium()
                    onMicrophoneTap()
                }) {
                    Image(systemName: "mic.fill")
                        .font(.title3)
                        .foregroundColor(.blue)
                        .frame(width: 40, height: 40)
                        .background {
                            Circle()
                                .fill(Color.blue.opacity(0.1))
                        }
                }
            }

            TextField("Напишите вопрос...", text: $messageText)
                .textFieldStyle(.roundedBorder)
                .disableAutocorrection(true)

            // Кнопка отправки
            Button(action: {
                HapticFeedback.light()
                onSend()
            }) {
                Image(systemName: "arrow.up.circle.fill")
                    .font(.title)
                    .foregroundColor(messageText.trimmingCharacters(in: .
whitespacesAndNewlines).isEmpty ? .gray : .blue)
            }
                .disabled(messageText.trimmingCharacters(in: .whitespacesAndNewlines)
.isEmpty)
                .padding(.horizontal, 16)
                .padding(.vertical, 12)
        }
    }
}

```

HalalAI/Features/Chat/UI/MessageBubble.swift

```

//
// MessageBubble.swift
// HalalAI
//

```

```
// Created by Мирсайт Сабирзянов on 25.10.2025.
//

import SwiftUI

struct MessageBubble: View {
    let message: ChatMessage
    @State private var isPressed = false

    var body: some View {
        HStack {
            if message.role == .user {
                Spacer()
            }

            VStack(alignment: message.role == .user ? .trailing : .leading,
spacing: 4) {
                // Для сообщений ассистента используем markdown, для пользователя
                - обычный текст
                if message.role == .assistant {
                    messageTextView
                        .padding(.horizontal, 16)
                        .padding(.vertical, 12)
                        .background {
                            RoundedRectangle(cornerRadius: 18)
                                .fill(Color.green.opacity(0.1))
                                .overlay {
                                    RoundedRectangle(cornerRadius: 18)
                                        .stroke(Color.green.opacity(0.3),
lineWidth: 1)
                                }
                        }
                }
                .overlay(alignment: .topTrailing) {
                    if let model = message.model, !model.isEmpty {
                        Text(model)
                            .font(.caption)
                            .foregroundColor(.secondary)
                            .padding(.horizontal, 8)
                            .padding(.vertical, 4)
                            .background(
                                RoundedRectangle(cornerRadius: 10)
                                    .fill(Color.secondary.opacity(0.1))
                            )
                            .padding([.top, .trailing], 6)
                    }
                }
                .contextMenu {
                    Button(action: {
                        UIPasteboard.general.string = message.text
                    }) {
                        Label("Копировать", systemImage: "doc.on.doc")
                    }
                }
            } else {
                Text(message.text)
                    .font(.body)
                    .foregroundColor(.white)
                    .padding(.horizontal, 16)
                    .padding(.vertical, 12)
                    .background {
                        RoundedRectangle(cornerRadius: 18)
                            .fill(Color.blue)
                    }
            }
        }
    }
}
```

```

        }
        .contextMenu {
            Button(action: {
                UIPasteboard.general.string = message.text
            }) {
                Label("Копировать", systemImage: "doc.on.doc")
            }
        }
    }

    Text(formatTime(message.date))
        .font(.caption)
        .foregroundStyle(.secondary)
        .padding(.horizontal, 4)
    }

    if message.role == .assistant {
        Spacer()
    }
}
.scaleEffect(isPressed ? 0.95 : 1.0)
.animation(.easeInOut(duration: 0.1), value: isPressed)
.onLongPressGesture(minimumDuration: 0.1) {
    // Дополнительные действия при долгом нажатии
} onPressingChanged: { pressing in
    isPressed = pressing
}
}

private func formatTime(_ date: Date) -> String {
    date.formatted(date: .omitted, time: .shortened)
}

// MARK: - Markdown Support

private var messageTextView: some View {
    Text(parseMarkdown(message.text))
        .font(.body)
        .foregroundStyle(.primary)
        .textSelection(.enabled)
}

private func parseMarkdown(_ text: String) -> AttributedString {
    do {
        let options = AttributedString.MarkdownParsingOptions(
            interpretedSyntax: .full,
            failurePolicy: .returnPartiallyParsedIfPossible
        )
        let attributedString = try AttributedString(markdown: text, options:
options)
        return attributedString
    } catch {
        return AttributedString(text)
    }
}

}

struct TypingIndicator: View {
    @State private var animationOffset: CGFloat = 0

    var body: some View {
        HStack {

```



```

        HStack(spacing: 4) {
            ForEach(0..<3) { index in
                Circle()
                    .fill(Color.green.opacity(0.6))
                    .frame(width: 8, height: 8)
                    .offset(y: animationOffset)
                    .animation(
                        .easeInOut(duration: 0.6)
                        .repeatForever()
                        .delay(Double(index) * 0.2),
                        value: animationOffset
                    )
            }
        }
        .padding(.horizontal, 16)
        .padding(.vertical, 12)
        .background {
            RoundedRectangle(cornerRadius: 18)
                .fill(Color.green.opacity(0.1))
                .overlay {
                    RoundedRectangle(cornerRadius: 18)
                        .stroke(Color.green.opacity(0.3), lineWidth: 1)
                }
        }
        .Spacer()
    }
    .onAppear {
        animationOffset = -4
    }
}

#Preview {
    VStack(spacing: 16) {
        MessageBubble(message: ChatMessage(role: .user, text: "Привет! Расскажи о халяль продуктах"))
        MessageBubble(message: ChatMessage(role: .assistant, text: "Ассаламу алей кум! \n\nЯ -- HalalAI, ваш халяль-помощник. Готов ответить на ваши вопросы о халяль продуктах, брендах и исламских принципах питания."))
        TypingIndicator()
    }
    .padding()
}

```

HalalAI/Features/HalalMap/Models/HalalPlaceModels.swift

```

//
//  HalalPlaceModels.swift
//  HalalAI
//

import Foundation
import CoreLocation
import MapKit

struct HalalPlace: Identifiable, Hashable {
    let id: String
    let name: String
    let address: String
    let coordinate: CLLocationCoordinate2D
}

```

```

let phoneNumber: String?
let url: URL?
let mapItem: MKMapItem

static func == (lhs: HalalPlace, rhs: HalalPlace) -> Bool {
    lhs.id == rhs.id
}

func hash(into hasher: inout Hasher) {
    hasher.combine(id)
}

/// Distance from a given location, formatted as a readable string.
func formattedDistance(from location: CLLocation) -> String {
    let placeLocation = CLLocation(latitude: coordinate.latitude, longitude:
coordinate.longitude)
    let distance = location.distance(from: placeLocation)
    if distance < 1000 {
        return distance.formatted(.number.precision(.fractionLength(0))) + "
m"
    } else {
        return (distance / 1000).formatted(.number.precision(.fractionLength
(1))) + " км"
    }
}
}

```

HalalAI/Features/HalalMap/Service/HalalPlacesService.swift

```

//
//  HalalPlacesService.swift
//  HalalAI
//

import Foundation
import MapKit

protocol HalalPlacesService {
    func searchNearby(coordinate: CLLocationCoordinate2D) async throws -> [
        HalalPlace]
}

final class HalalPlacesServiceImpl: HalalPlacesService {

    func searchNearby(coordinate: CLLocationCoordinate2D) async throws -> [
        HalalPlace] {
        let request = MKLocalSearch.Request()
        request.naturalLanguageQuery = "халляль"
        request.resultTypes = .pointOfInterest
        request.region = MKCoordinateRegion(
            center: coordinate,
            latitudinalMeters: 5000,
            longitudinalMeters: 5000
        )

        let search = MKLocalSearch(request: request)
        let response = try await search.start()

        return response.mapItems.compactMap { item in
            guard let name = item.name else { return nil }
            let placemark = item.placemark

```

```

        let address = [
            placemark.thoroughfare,
            placemark.subThoroughfare,
            placemark.locality
        ]

        .compactMap { $0 }
        .joined(separator: ", ")

        return HalalPlace(
            id: UUID().uuidString,
            name: name,
            address: address.isEmpty ? "Адрес не указан" : address,
            coordinate: placemark.coordinate,
            phoneNumber: item.phoneNumber,
            url: item.url,
            mapItem: item
        )
    }
}

```

HalalAI/Features/HalalMap/UI/HalalMapView.swift

```

//
//  HalalMapView.swift
//  HalalAI
//

import SwiftUI
import MapKit

struct HalalMapView: View {
    @Environment(Coordinator.self) var coordinator
    @State private var viewModel: ViewModel

    init(placesService: HalalPlacesService, locationService: LocationService) {
        _viewModel = State(initialValue: ViewModel(placesService: placesService,
            locationService: locationService))
    }

    var body: some View {
        ZStack {
            Map(position: Binding(
                get: { viewModel.cameraPosition },
                set: { viewModel.cameraPosition = $0 }
            )) {
                UserAnnotation()

                ForEach(viewModel.places) { place in
                    Marker(place.name, coordinate: place.coordinate)
                        .tint(.green)
                }
            }
            .mapControls {
                MapUserLocationButton()
                MapCompass()
            }

            if viewModel.isLoading {
                ProgressView("Поиск халяль мест...")
                    .padding()
            }
        }
    }
}

```

```

        .background(.ultraThinMaterial, in: .rect(cornerRadius: 12))
    }

    if let errorMessage = viewModel.errorMessage, viewModel.places.
isEmpty {
        VStack {
            Spacer()
            Text(errorMessage)
                .font(.subheadline)
                .foregroundColor(.secondary)
                .padding()
                .background(.ultraThinMaterial, in: .rect(cornerRadius:
12))
                .padding()
            Spacer()
        }
    }
}
.navigationTitle("Ҳаляль места")
.navigationBarTitleDisplayMode(.inline)
.toolbar {
    ToolbarItem(placement: .topBarLeading) {
        Button("Назад", systemImage: "chevron.left") {
            coordinator.dismiss()
        }
        .tint(.darkGreen)
    }
}
.sheet(item: Binding(
    get: { viewModel.selectedPlace },
    set: { viewModel.selectedPlace = $0 }
)) { place in
    PlaceDetailSheet(place: place, viewModel: viewModel)
        .presentationDetents([.fraction(0.3)])
}
.task {
    await viewModel.loadPlaces()
}
.onMapCameraChange { context in
    // Allow camera to move freely after initial load
}
}
}

// MARK: - Place Detail Sheet

private struct PlaceDetailSheet: View {
    let place: HalalPlace
    let viewModel: HalalMapView.ViewModel

    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            Text(place.name)
                .font(.title3)
                .bold()

            Label(place.address, systemImage: "mappin.and.ellipse")
                .font(.subheadline)
                .foregroundColor(.secondary)

            if let distance = viewModel.distanceText(for: place) {
                Label(distance, systemImage: "location")
            }
        }
    }
}

```

```

        .font(.subheadline)
        .foregroundColor(.secondary)
    }

    if let phone = place.phoneNumber {
        Label(phone, systemImage: "phone")
        .font(.subheadline)
        .foregroundColor(.secondary)
    }

    Button("Построить маршрут", systemImage: "arrow.triangle.turn.up.
right.diamond") {
        viewModel.openInAppleMaps(place: place)
    }
    .buttonStyle(.borderedProminent)
    .tint(.green)
}
.frame(maxWidth: .infinity, alignment: .leading)
.padding()
}
}

```

HalalAI/Features/HalalMap/UI/HalalMapViewModel.swift

```

//
//  HalalMapViewModel.swift
//  HalalAI
//

import Foundation
import MapKit
import SwiftUI

extension HalalMapView {
    @MainActor
    @Observable
    final class ViewModel {
        var places: [HalalPlace] = []
        var selectedPlace: HalalPlace?
        var isLoading = false
        var errorMessage: String?
        var cameraPosition: MapCameraPosition = .userLocation(fallback: .
automatic)

        private let placesService: HalalPlacesService
        private let locationService: LocationService

        init(placesService: HalalPlacesService, locationService: LocationService)
        {
            self.placesService = placesService
            self.locationService = locationService
        }

        func loadPlaces() async {
            guard !isLoading else { return }

            locationService.requestLocation()

            // Wait briefly for location if not yet available
            if locationService.currentLocation == nil {
                try? await Task.sleep(for: .seconds(1))
            }
        }
    }
}

```

```

    }

    guard let location = locationService.currentLocation else {
        errorMessage = "Не удалось определить местоположение"
        return
    }

    isLoading = true
    errorMessage = nil

    do {
        places = try await placesService.searchNearby(coordinate:
location.coordinate)
        if places.isEmpty {
            errorMessage = "Халяль заведения не найдены поблизости"
        }
    } catch {
        errorMessage = "Ошибка поиска: \(error.localizedDescription)"
    }

    isLoading = false
}

func openInAppleMaps(place: HalalPlace) {
    place.mapItem.openInMaps(launchOptions: [
        MKLaunchOptionsDirectionsModeKey:
MKLaunchOptionsDirectionsModeDriving
    ])
}

func distanceText(for place: HalalPlace) -> String? {
    guard let location = locationService.currentLocation else { return
nil }
    return place.formattedDistance(from: location)
}
}
}

```

HalalAI/Features/Home/Models/Verse.swift

```

//
// Verse.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 09.01.2026.
//

import Foundation

struct Verse: Codable, Equatable, Hashable {
    let id: Int
    let suraIndex: Int
    let suraTitle: String
    let suraSubtitle: String
    let verseNumber: Int
    let text: String
}

```

HalalAI/Features/Home/Service/VerseService.swift

```

//
// VerseService.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 09.01.2026.
//

import Foundation

@MainActor
protocol VerseService {
    var verseOfTheDay: Verse? { get set }
    func fetchVerseOfTheDay() async throws
}

@Observable
@MainActor
final class VerseServiceImpl: VerseService {
    var verseOfTheDay: Verse?

    private let networkClient: NetworkClient

    init(networkClient: NetworkClient = NetworkClient()) {
        self.networkClient = networkClient
    }

    func fetchVerseOfTheDay() async throws {
        verseOfTheDay = try await networkClient.send(VerseOfTheDayAPIRequest())
    }
}

// MARK: - API

struct VerseOfTheDayAPIRequest: APIRequest {
    typealias Response = Verse

    let endpoint = Endpoint.verseOfTheDay
}

```

HalalAI/Features/Home/UI/HomeView.swift

```

//
// HomeView.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 18.11.2025.
//

import SwiftUI

struct HomeView: View {
    @Environment(Coordinator.self) var coordinator
    @State private var viewModel: ViewModel

    init(
        verseService: VerseService,
        locationService: LocationService,
        prayerTimeService: PrayerTimeService,
        settingsStore: PrayerSettingsStore,
        authManager: AuthManager
    ) {
        self.viewModel = ViewModel(
            verseService: verseService,
            locationService: locationService,
            prayerTimeService: prayerTimeService,
            settingsStore: settingsStore,
            authManager: authManager
        )
    }
}

```

```

    ) {
        let prayerCardVM = PrayerTimesCardView.ViewModel(
            locationService: locationService,
            prayerTimeService: prayerTimeService,
            settingsStore: settingsStore
        )
        _viewModel = State(
            initialValue: ViewModel(
                verseService: verseService,
                prayerCardViewModel: prayerCardVM,
                authManager: authManager
            )
        )
    }

var body: some View {
    VStack {
        ScrollView(showsIndicators: false) {
            VerseView(verseService: viewModel.verseService)

            if viewModel.authManager.isGuest {
                GuestBannerView {
                    viewModel.authManager.logout()
                }
            }

            PrayerTimesCardView(viewModel: viewModel.prayerCardViewModel)

            Button {
                guard !viewModel.authManager.isGuest else { return }
                coordinator.nextStep(step: .home(.scanner))
            } label: {
                ImageTextComponent(
                    componentSize: .large,
                    image: .scan,
                    title: "Сканировать состав",
                    description: "Проверь ингредиенты на халяльность",
                    locked: viewModel.authManager.isGuest
                )
            }
            .buttonStyle(.plain)

            HStack(spacing: 8) {
                Button {
                    guard !viewModel.authManager.isGuest else { return }
                    coordinator.selectTab(item: .chat)
                } label: {
                    ImageTextComponent(
                        componentSize: .medium,
                        image: .quran,
                        title: "Чат с AI",
                        description: "Задай вопрос",
                        locked: viewModel.authManager.isGuest
                    )
                }
                .buttonStyle(.plain)

                Button {
                    coordinator.nextStep(step: .home(.halalMap))
                } label: {
                    ImageTextComponent(
                        componentSize: .medium,

```



```

        image: .map,
        title: "Найти заведение",
        description: "Халяль места рядом"
    )
}
.buttonStyle(.plain)
}

Button {
    withAnimation {
        coordinator.nextStep(step: .home(.quran))
    }
} label: {
    ImageTextComponent(
        componentSize: .large,
        image: .mosque,
        title: "Изучай Ислам",
        description: "Суры и Аяты из Корана"
    )
}
.buttonStyle(.plain)
}
.scrollIndicators(.hidden)
}
.padding(.horizontal, 24)
.background {
    Color.greenBackground.ignoresSafeArea()
}
}
}
}

```

HalalAI/Features/Home/UI/HomeViewModel.swift

```

//
// HomeViewModel.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 14.02.2026.
//

import Foundation

extension HomeView {
    @Observable
    @MainActor
    final class ViewModel {
        var verseService: VerseService
        var prayerCardViewModel: PrayerTimesCardView.ViewModel
        var authManager: AuthManager

        init(
            verseService: VerseService,
            prayerCardViewModel: PrayerTimesCardView.ViewModel,
            authManager: AuthManager
        ) {
            self.verseService = verseService
            self.prayerCardViewModel = prayerCardViewModel
            self.authManager = authManager
        }

        deinit {

```

```

        print("deinit HomeViewModel")
    }
}

```

HalalAI/Features/Home/UI/VerseView.swift

```

//
// VerseView.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 09.01.2026.
//

import SwiftUI

struct VerseView: View {
    @State var verseService: VerseService

    var body: some View {
        VStack {
            if let verse = verseService.verseOfTheDay {
                VStack(alignment: .leading, spacing: 12) {
                    // Информация о суре
                    VStack(alignment: .leading, spacing: 4) {
                        Text("Аят дня")
                            .font(.headline)
                        HStack {
                            Text(verse.suraTitle)
                            Text("Сурa \("\(verse.suraIndex)\): \("\(verse.verseNumber)")")
                        }
                            .font(.caption)
                    }
                    .foregroundColor(.darkGreen)

                    Divider()
                        .padding(.vertical, 4)

                    // Текст аята
                    Text(verse.text)
                        .font(.subheadline)
                        .foregroundColor(.black)
                        .padding(.top, 4)
                }
                .padding(16)
                .background(.greenForeground)
                .clipShape(.rect(cornerRadius: 12))
                .shadow(color: Color.black.opacity(0.06), radius: 8, x: 0, y: 2)
            }
        }
        .task {
            do {
                try await verseService.fetchVerseOfTheDay()
            } catch {
                print("Ошибка при получении аята-дня: \(error)")
            }
        }
    }
}

```

HalalAI/Features/Prayer/Models/PrayerModels.swift

```

//
// PrayerModels.swift
// HalalAI
//
// Created by Мирсали Сабирзянов on 26.02.2026.
//

import Foundation

// MARK: - Prayer

enum Prayer: String, CaseIterable, Codable {
    case fajr
    case sunrise
    case dhuhr
    case asr
    case maghrib
    case isha

    var localizedName: String {
        switch self {
            case .fajr:
                return "Фаджр"
            case .sunrise:
                return "Шурук"
            case .dhuhr:
                return "Зухр"
            case .asr:
                return "Аср"
            case .maghrib:
                return "Магриб"
            case .isha:
                return "Иша"
        }
    }

    var systemImage: String {
        switch self {
            case .fajr:
                return "moon.stars"
            case .sunrise:
                return "sunrise"
            case .dhuhr:
                return "sun.max"
            case .asr:
                return "sun.min"
            case .maghrib:
                return "sunset"
            case .isha:
                return "moon"
        }
    }

    static var notifiablePrayers: [Prayer] {
        [.fajr, .dhuhr, .asr, .maghrib, .isha]
    }
}

// MARK: - DailyPrayerTimes

```

```

struct DailyPrayerTimes {
    let date: Date
    let fajr: Date
    let sunrise: Date
    let dhuhr: Date
    let asr: Date
    let maghrib: Date
    let isha: Date

    func time(for prayer: Prayer) -> Date {
        switch prayer {
        case .fajr:
            return fajr
        case .sunrise:
            return sunrise
        case .dhuhr:
            return dhuhr
        case .asr:
            return asr
        case .maghrib:
            return maghrib
        case .isha:
            return isha
        }
    }

    var allPrayers: [(Prayer, Date)] {
        Prayer.allCases.map { ($0, time(for: $0)) }
    }
}

// MARK: - Calculation Method

enum PrayerCalculationMethod: String, CaseIterable, Codable {
    case russia = "russia"
    case tatarstan = "tatarstan"
    case muslimWorldLeague = "mwl"
    case isna = "isna"
    case egypt = "egypt"
    case makkah = "makkah"
    case karachi = "karachi"
    case tehran = "tehran"

    var localizedName: String {
        switch self {
        case .russia:
            return "ДУМ России"
        case .tatarstan:
            return "ДУМ Республики Татарстан"
        case .muslimWorldLeague:
            return "Мировая Исламская Лига"
        case .isna:
            return "ISNA (Северная Америка)"
        case .egypt:
            return "Египетский"
        case .makkah:
            return "Умм аль-Кура (Мекка)"
        case .karachi:
            return "Карачи"
        case .tehran:
            return "Тегеран"
        }
    }
}

```

```

    }
}

var fajrAngle: Double {
    switch self {
    case .muslimWorldLeague:
        return 18.0
    case .isna:
        return 15.0
    case .egypt:
        return 19.5
    case .makkah:
        return 18.5
    case .karachi:
        return 18.0
    case .tehran:
        return 17.7
    case .russia:
        return 16.0
    case .tatarstan:
        return 18.0
    }
}

var ishaAngle: Double {
    switch self {
    case .muslimWorldLeague:
        return 17.0
    case .isna:
        return 15.0
    case .egypt:
        return 17.5
    case .makkah:
        return 90.0 / 60 // ~1.5
    case .karachi:
        return 18.0
    case .tehran:
        return 14.0
    case .russia:
        return 15.0
    case .tatarstan:
        return 15.0
    }
}

}

// MARK: - Madhab

enum Madhab: String, CaseIterable, Codable {
    case shafi = "shafi"
    case hanafi = "hanafi"

    var localizedName: String {
        switch self {
        case .shafi:
            return "Шафии / Маликии / Ханбали"
        case .hanafi:
            return "Ханафи"
        }
    }
}

var asrShadowFactor: Double {

```

```

        switch self {
        case .shafi:
            return 1.0
        case .hanafi:
            return 2.0
        }
    }
}

// MARK: - Notification Settings

struct PrayerNotificationSetting: Codable, Equatable {
    var isEnabled: Bool
    var offsetMinutes: Int

    static let `default` = PrayerNotificationSetting(isEnabled: false,
    offsetMinutes: 0)
}

// MARK: - Prayer Settings

struct PrayerSettings: Codable {
    var calculationMethod: PrayerCalculationMethod
    var madhab: Madhab
    var customFajrAngle: Double?
    var customIshaAngle: Double?
    var notifications: [String: PrayerNotificationSetting]

    static let `default` = PrayerSettings(
        calculationMethod: .muslimWorldLeague,
        madhab: .shafi,
        customFajrAngle: nil,
        customIshaAngle: nil,
        notifications: Prayer.notifiablePrayers.reduce(into: [:]) { dict, prayer
in
            dict[prayer.rawValue] = .default
        }
    )

    func notificationSetting(for prayer: Prayer) -> PrayerNotificationSetting {
        notifications[prayer.rawValue] ?? .default
    }

    mutating func setNotification(_ setting: PrayerNotificationSetting, for
prayer: Prayer) {
        notifications[prayer.rawValue] = setting
    }
}

```

HalalAI/Features/Prayer/Service/PrayerNotificationService.swift

```

//
// PrayerNotificationService.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 26.02.2026.
//

import Foundation
import UserNotifications
import CoreLocation

```

```

protocol PrayerNotificationService: AnyObject {
    func requestAuthorization() async -> Bool
    func scheduleNotifications(settings: PrayerSettings, location: CLLocation)
    async
    func cancelAllPrayerNotifications() async
    func sendTestNotification() async
}

final class PrayerNotificationServiceImpl: PrayerNotificationService {

    private let prayerTimeService: PrayerTimeService
    private let center = UNUserNotificationCenter.current()

    init(prayerTimeService: PrayerTimeService) {
        self.prayerTimeService = prayerTimeService
    }

    // MARK: - Authorization

    func requestAuthorization() async -> Bool {
        do {
            return try await center.requestAuthorization(options: [.alert, .sound
, .badge])
        } catch {
            print("Notification auth error: \(error)")
            return false
        }
    }

    // MARK: - Scheduling

    /// Schedules prayer notifications for the next 7 days (max 35 notifications)
    .
    func scheduleNotifications(settings: PrayerSettings, location: CLLocation)
    async {
        await cancelAllPrayerNotifications()

        let calendar = Calendar.current
        let today = Date()

        for dayOffset in 0..<7 {
            guard let targetDate = calendar.date(byAdding: .day, value: dayOffset
, to: today) else { continue }

            guard let times = prayerTimeService.calculateTimes(
                for: targetDate,
                location: location,
                settings: settings
            ) else { continue }

            for prayer in Prayer.notifiablePrayers {
                let notifSetting = settings.notificationSetting(for: prayer)
                guard notifSetting.isEnabled else { continue }

                let prayerDate = times.time(for: prayer)
                let fireDate = prayerDate.addingTimeInterval(-Double(notifSetting
.offsetMinutes) * 60)

                guard fireDate > Date() else { continue }

                let identifier = "prayer_\(prayer.rawValue)_\((dayOffset)"

```

```

        await schedule(
            identifier: identifier,
            prayer: prayer,
            offsetMinutes: notifSetting.offsetMinutes,
            fireDate: fireDate
        )
    }
}

func cancelAllPrayerNotifications() async {
    let pending = await center.pendingNotificationRequests()
    let prayerIDs = pending
        .map(\.identifier)
        .filter { $0.hasPrefix("prayer_") }
    center.removePendingNotificationRequests(withIdentifiers: prayerIDs)
}

// MARK: - Test

func sendTestNotification() async {
    let content = UNMutableNotificationContent()
    content.title = "Тест уведомления"
    content.body = "Уведомления о намазе работают!"
    content.sound = .default

    let trigger = UNTimeIntervalNotificationTrigger(timeInterval: 5, repeats:
false)
    let request = UNNotificationRequest(identifier: "prayer_test", content:
content, trigger: trigger)

    do {
        try await center.add(request)
    } catch {
        print("Test notification error: \(error)")
    }
}

// MARK: - Private

private func schedule(
    identifier: String,
    prayer: Prayer,
    offsetMinutes: Int,
    fireDate: Date
) async {
    let content = UNMutableNotificationContent()

    if offsetMinutes == 0 {
        content.title = "Время намаза: \(prayer.localizedName)"
        content.body = "Наступило время \(prayer.localizedName)"
    } else {
        content.title = "\(prayer.localizedName) через \(offsetMinutes) мин"
        content.body = "Подготовьтесь к намазу \(prayer.localizedName)"
    }
    content.sound = .default

    let components = Calendar.current.dateComponents(
        [.year, .month, .day, .hour, .minute],
        from: fireDate
    )
}

```



```

        let trigger = UNCalendarNotificationTrigger(dateMatching: components,
repeats: false)
        let request = UNNotificationRequest(identifier: identifier, content:
content, trigger: trigger)

        do {
            try await center.add(request)
        } catch {
            print("Schedule notification error \(identifier): \(error)")
        }
    }
}

```

HalalAI/Features/Prayer/Service/PrayerSettingsStore.swift

```

//
//  PrayerSettingsStore.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 26.02.2026.
//

import Foundation
import Observation

private let kPrayerSettingsKey = "HalalAI.prayerSettings"

@Observable
@MainActor
final class PrayerSettingsStore {
    var settings: PrayerSettings {
        didSet { persist() }
    }

    init() {
        if let data = UserDefaults.standard.data(forKey: kPrayerSettingsKey),
            let decoded = try? JSONDecoder().decode(PrayerSettings.self, from:
data) {
            settings = decoded
        } else {
            settings = .default
        }
    }

    private func persist() {
        if let data = try? JSONEncoder().encode(settings) {
            UserDefaults.standard.set(data, forKey: kPrayerSettingsKey)
        }
    }
}

```

HalalAI/Features/Prayer/Service/PrayerTimeService.swift

```

//
//  PrayerTimeService.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 26.02.2026.
//

```

```

import Foundation
import CoreLocation
import Adhan

protocol PrayerTimeService: AnyObject {
    func calculateTimes(
        for date: Date,
        location: CLLocation,
        settings: PrayerSettings
    ) -> DailyPrayerTimes?

    func nextPrayer(from times: DailyPrayerTimes) -> (Prayer, Date)?
}

final class PrayerTimeServiceImpl: PrayerTimeService {

    func calculateTimes(
        for date: Date,
        location: CLLocation,
        settings: PrayerSettings
    ) -> DailyPrayerTimes? {
        let coords = Adhan.Coordinates(
            latitude: location.coordinate.latitude,
            longitude: location.coordinate.longitude
        )

        let dc = Calendar(identifier: .gregorian)
            .dateComponents([.year, .month, .day], from: date)

        var params = settings.calculationMethod.adhanParams
        params.madhab = settings.madhab.adhanMadhab

        if let fajr = settings.customFajrAngle { params.fajrAngle = fajr }
        if let isha = settings.customIshaAngle { params.ishaAngle = isha }

        guard let times = Adhan.PrayerTimes(
            coordinates: coords,
            date: dc,
            calculationParameters: params
        ) else { return nil }

        let fmt = DateFormatter()
        fmt.dateFormat = "HH:mm"
        print("[PrayerTime] params: \(params)")

        print("[PrayerTime] Fajr=\(fmt.string(from: times.fajr)) Sunrise=\(fmt.
            string(from: times.sunrise)) Dhuhr=\(fmt.string(from: times.dhuhr)) Asr=\(fmt.
            string(from: times.asr)) Maghrib=\(fmt.string(from: times.maghrib)) Isha=\(fmt.
            string(from: times.isha))")

        return DailyPrayerTimes(
            date: date,
            fajr: times.fajr,
            sunrise: times.sunrise,
            dhuhr: times.dhuhr,
            asr: times.asr,
            maghrib: times.maghrib,
            isha: times.isha
        )
    }
}

```

```

    func nextPrayer(from times: DailyPrayerTimes) -> (Prayer, Date)? {
        let now = Date()
        return times.allPrayers.first { $0.1 > now }
    }
}

private extension PrayerCalculationMethod {
    var adhanParams: Adhan.CalculationParameters {
        switch self {
            case .muslimWorldLeague:
                return Adhan.CalculationMethod.muslimWorldLeague.params
            case .isna:
                return Adhan.CalculationMethod.northAmerica.params
            case .egypt:
                return Adhan.CalculationMethod.egyptian.params
            case .makkah:
                return Adhan.CalculationMethod.ummAlQura.params
            case .karachi:
                return Adhan.CalculationMethod.karachi.params
            case .tehran:
                return Adhan.CalculationMethod.tehran.params
            case .russia, .tatarstan:
                var params = Adhan.CalculationMethod.other.params
                params.fajrAngle = self.fajrAngle
                params.ishaAngle = self.ishaAngle
                return params
        }
    }
}

private extension Madhab {
    var adhanMadhab: Adhan.Madhab {
        switch self {
            case .shafi: return .shafi
            case .hanafi: return .hanafi
        }
    }
}

```

HalalAI/Features/Prayer/UI/PrayerNotificationSettingsView.swift

```

//
// PrayerNotificationSettingsView.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 26.02.2026.
//

import SwiftUI
import UserNotifications

struct PrayerNotificationSettingsView: View {
    @State private var viewModel: ViewModel
    @Environment(Coordinator.self) var coordinator

    init(
        settingsStore: PrayerSettingsStore,
        notificationService: PrayerNotificationService,
        locationService: LocationService
    ) {
        _viewModel = State(

```

```

        initialValue: ViewModel(
            settingsStore: settingsStore,
            notificationService: notificationService,
            locationService: locationService
        )
    )
}

var body: some View {
    @Bindable var vm = viewModel
    Form {
        calculationSection
        anglesSection
        madhabSection
        notificationSection
        testSection
    }
    .scrollContentBackground(.hidden)
    .background(Color.greenBackground.ignoresSafeArea())
    .navigationTitle("Время намаза")
    .navigationBarTitleDisplayMode(.inline)
    .toolbar {
        ToolbarItem(placement: .topBarLeading) {
            Button("Назад", systemImage: "arrow.left") {
                coordinator.dismiss()
            }
            .labelStyle(.iconOnly)
        }
    }
    .task {
        await vm.onAppear()
    }
}

// MARK: - Sections

private var calculationSection: some View {
    Section(header: Text("Метод расчёта")) {
        Picker("Метод", selection: $viewModel.settings.calculationMethod) {
            ForEach(PrayerCalculationMethod.allCases, id: \.self) { method in
                Text(method.localizedName).tag(method)
            }
        }
        .pickerStyle(.menu)
        .onChange(of: viewModel.settings.calculationMethod) { _, _ in
            viewModel.onSettingsChanged()
        }

        Text("Выберите метод расчёта в соответствии с вашим регионом или пред-
почтениями.")
            .font(.footnote)
            .foregroundColor(.secondary)
    }
}

private var anglesSection: some View {
    Section(header: Text("Углы Фаджр и Иша")) {
        AngleStepper(
            label: "Фаджр",
            defaultAngle: viewModel.settings.calculationMethod.fajrAngle,
            value: Binding(

```

```

        get: { viewModel.settings.customFajrAngle ?? viewModel.
settings.calculationMethod.fajrAngle },
        set: { viewModel.settings.customFajrAngle = $0; viewModel.
onSettingsChanged() }
    ),
    isCustom: viewModel.settings.customFajrAngle != nil,
    onReset: { viewModel.settings.customFajrAngle = nil; viewModel.
onSettingsChanged() }
)
AngleStepper(
    label: "Иша",
    defaultAngle: viewModel.settings.calculationMethod.ishaAngle,
    value: Binding(
        get: { viewModel.settings.customIshaAngle ?? viewModel.
settings.calculationMethod.ishaAngle },
        set: { viewModel.settings.customIshaAngle = $0; viewModel.
onSettingsChanged() }
    ),
    isCustom: viewModel.settings.customIshaAngle != nil,
    onReset: { viewModel.settings.customIshaAngle = nil; viewModel.
onSettingsChanged() }
)
Text("Угол солнца ниже горизонта (градусы). По умолчанию -- значение
выбранного метода.")
    .font(.footnote)
    .foregroundColor(.secondary)
}
}

private var madhabSection: some View {
    Section(header: Text("Мазхаб (время Аср)")) {
        Picker("Мазхаб", selection: $viewModel.settings.madhab) {
            ForEach(Madhab.allCases, id: \.self) { madhab in
                Text(madhab.localizedName).tag(madhab)
            }
        }
        .pickerStyle(.menu)
        .onChange(of: viewModel.settings.madhab) { _, _ in
            viewModel.onSettingsChanged()
        }
    }
}

private var notificationSection: some View {
    Section(header: Text("Уведомления")) {
        if viewModel.notificationsAuthorized == false {
            HStack {
                Image(systemName: "bell.slash")
                    .foregroundColor(.orange)
                VStack(alignment: .leading, spacing: 2) {
                    Text("Уведомления отключены")
                        .fontWeight(.medium)
                    Text("Разрешите в Настройках HalalAI Уведомления")
                        .font(.caption)
                        .foregroundColor(.secondary)
                }
            }
        }
        Button("Открыть настройки") {
            if let url = URL(string: UIApplication.openSettingsURLString)
{
                UIApplication.shared.open(url)
            }
        }
    }
}

```

```

        }
        .foregroundColor(Color.greenForeground)
    }

    ForEach(Prayer.notifiablePrayers, id: \.self) { prayer in
        PrayerNotificationRow(
            prayer: prayer,
            setting: viewModel.bindingSetting(for: prayer),
            onChanged: viewModel.onSettingsChanged
        )
    }
}

private var testSection: some View {
    Section {
        Button {
            Task { await viewModel.sendTestNotification() }
        } label: {
            HStack {
                Spacer()
                Label("Тестовое уведомление (через 5 сек)", systemImage: "
bell.badge.waveform")
                Spacer()
            }
        }
        .foregroundColor(Color.greenForeground)
        .disabled(viewModel.notificationsAuthorized == false)

        Text("Отправит уведомление через 5 секунд для проверки работы.")
        .font(.footnote)
        .foregroundColor(.secondary)
    }
}

// MARK: - Angle Stepper

private struct AngleStepper: View {
    let label: String
    let defaultAngle: Double
    @Binding var value: Double
    let isCustom: Bool
    let onReset: () -> Void

    var body: some View {
        VStack(alignment: .leading, spacing: 6) {
            HStack {
                Text(label)
                Spacer()
                if isCustom {
                    Button("Сбросить", action: onReset)
                        .font(.caption)
                        .foregroundColor(Color.greenForeground)
                } else {
                    Text("по умолчанию")
                        .font(.caption)
                        .foregroundColor(.secondary)
                }
            }
            HStack {
                Stepper(

```

```

        value: $value,
        in: 10.0...25.0,
        step: 0.5
    ) {
        HStack(spacing: 4) {
            Text("\(value.formatted(.number.precision(.fractionLength
(1)))) ")
                .monospacedDigit()
                .fontWeight(isCustom ? .semibold : .regular)
                .foregroundStyle(isCustom ? Color.greenForeground : .
primary)
                if !isCustom {
                    Text("(по умолчанию \(defaultAngle.formatted(.number.
precision(.fractionLength(1)))) ")
                        .font(.caption)
                        .foregroundStyle(.secondary)
                }
            }
        }
    }
    .padding(.vertical, 2)
}

// MARK: - Prayer Notification Row

private struct PrayerNotificationRow: View {
    let prayer: Prayer
    @Binding var setting: PrayerNotificationSetting
    let onChanged: () -> Void

    private let offsetOptions: [(label: String, minutes: Int)] = [
        ("Ровно в намаз", 0),
        ("За 5 минут", 5),
        ("За 10 минут", 10),
        ("За 15 минут", 15),
        ("За 30 минут", 30),
    ]

    var body: some View {
        VStack(alignment: .leading, spacing: 8) {
            Toggle(isOn: $setting.isEnabled) {
                Label(prayer.localizedName, systemImage: prayer.systemImage)
                    .foregroundStyle(.primary)
            }
            .toggleStyle(SwitchToggleStyle(tint: .green))
            .onChange(of: setting.isEnabled) { _, _ in onChanged() }

            if setting.isEnabled {
                Picker("Когда уведомлять", selection: $setting.offsetMinutes) {
                    ForEach(offsetOptions, id: \.minutes) { option in
                        Text(option.label).tag(option.minutes)
                    }
                }
                .pickerStyle(.menu)
                .onChange(of: setting.offsetMinutes) { _, _ in onChanged() }
            }
        }
        .padding(.vertical, 4)
    }
}

```

```

// MARK: - ViewModel

extension PrayerNotificationSettingsView {
    @Observable
    @MainActor
    final class ViewModel {
        var settings: PrayerSettings
        var notificationsAuthorized: Bool? = nil

        private let settingsStore: PrayerSettingsStore
        private let notificationService: PrayerNotificationService
        private let locationService: LocationService

        init(
            settingsStore: PrayerSettingsStore,
            notificationService: PrayerNotificationService,
            locationService: LocationService
        ) {
            self.settingsStore = settingsStore
            self.notificationService = notificationService
            self.locationService = locationService
            self.settings = settingsStore.settings
        }

        func onAppear() async {
            await checkNotificationStatus()
            if notificationsAuthorized == false {
                notificationsAuthorized = await notificationService.
requestAuthorization()
            }
        }

        func onSettingsChanged() {
            settingsStore.settings = settings
            Task {
                guard let location = locationService.currentLocation else {
return }
                await notificationService.scheduleNotifications(settings:
settings, location: location)
            }
        }

        func sendTestNotification() async {
            await notificationService.sendTestNotification()
        }

        func bindingSetting(for prayer: Prayer) -> Binding<
PrayerNotificationSetting> {
            Binding(
                get: { self.settings.notificationSetting(for: prayer) },
                set: { self.settings.setNotification($0, for: prayer) }
            )
        }

        private func checkNotificationStatus() async {
            let settings = await UNUserNotificationCenter.current().
notificationSettings()
            switch settings.authorizationStatus {
            case .authorized, .provisional, .ephemeral:
                notificationsAuthorized = true
            case .denied:

```



```

        .font(.body.weight(.semibold))
        .frame(width: 36, height: 32)
    }
    .disabled(!viewModel.canShiftToPreviousDay)
    .opacity(viewModel.canShiftToPreviousDay ? 1 : 0.35)

    Spacer()

    Text(viewModel.displayedDayTitle)
        .font(.subheadline.weight(.medium))

    Spacer()

    Button {
        viewModel.shiftDisplayedDay(by: 1)
    } label: {
        Image(systemName: "chevron.right")
            .font(.body.weight(.semibold))
            .frame(width: 36, height: 32)
    }
    .disabled(!viewModel.canShiftToNextDay)
    .opacity(viewModel.canShiftToNextDay ? 1 : 0.35)
}
.foregroundColor(.darkGreen)
.padding(.horizontal)
.padding(.vertical, 8)
}
default:
    EmptyView()
}
}

// MARK: - Header

@ViewBuilder
private var cardHeader: some View {
    HStack(alignment: .top) {
        VStack(alignment: .leading, spacing: 4) {
            Text("Время намаза")
                .font(.headline)
                .foregroundColor(.darkGreen)

            switch viewModel.locationService.authorizationStatus {
            case .notDetermined:
                Button {
                    viewModel.locationService.requestLocation()
                } label: {
                    Label("Разрешить геолокацию", systemImage: "location")
                        .font(.subheadline)
                        .foregroundColor(.darkGreen)
                }
            case .denied, .restricted:
                Text("Нет доступа к геолокации")
                    .font(.subheadline)
                    .foregroundColor(.darkGreen)
            case .authorizedWhenInUse, .authorizedAlways:
                if let (prayer, time) = viewModel.nextPrayer {
                    VStack(alignment: .leading, spacing: 2) {
                        Text("Следующий: \(prayer.localizedName)")
                            .font(.subheadline)
                            .foregroundColor(.secondary)
                    }
                    HStack(spacing: 6) {

```

```

        Text(time, format: .dateTime.hour().minute())
            .font(.title2.bold())
            .foregroundColor(.darkGreen)
        TimelineView(.periodic(from: .now, by: 60)) { _
in
            Text(countdown(to: time))
                .font(.caption)
                .foregroundColor(.darkGreen)
                .padding(.horizontal, 8)
                .padding(.vertical, 3)
                .background(Capsule().fill(Color.
secondary.opacity(0.15)))
        }
    }
}
} else if viewModel.displayedTimes == nil {
    Text("Определяем местоположение...")
        .font(.subheadline)
        .foregroundColor(.darkGreen)
} else {
    Text("Все намазы прошли")
        .font(.subheadline)
        .foregroundColor(.darkGreen)
}
@unknown default:
    EmptyView()
}
}

Spacer()

Button("Настройки уведомлений", systemImage: "bell.badge") {
    coordinator.nextStep(step: .home(.prayerSettings))
}
    .font(.title3)
    .foregroundColor(.darkGreen)
    .labelStyle(.iconOnly)
    .padding(8)
    .background(Circle().fill(Color.greenForeground.opacity(0.12)))
}
    .padding()
}

// MARK: - Prayer List

@ViewBuilder
private func prayerList(times: DailyPrayerTimes) -> some View {
    VStack(spacing: 0) {
        ForEach(Prayer.allCases, id: \.self) { prayer in
            PrayerRowView(
                prayer: prayer,
                time: times.time(for: prayer),
                isNext: viewModel.isNextPrayerRow(prayer: prayer, time: times
.time(for: prayer))
            )
            if prayer != Prayer.allCases.last {
                Divider().padding(.horizontal)
            }
        }
    }
    .padding(.vertical, 4)
}
}

```

```

// MARK: - Countdown

private func countdown(to date: Date) -> String {
    let diff = Int(date.timeIntervalSinceNow)
    guard diff > 0 else { return "" }
    let h = diff / 3600
    let m = (diff % 3600) / 60
    if h > 0 {
        return "через \ (h)ч \ (m)м"
    } else {
        return "через \ (m)м"
    }
}

// MARK: - Prayer Row

private struct PrayerRowView: View {
    let prayer: Prayer
    let time: Date
    let isNext: Bool

    var body: some View {
        HStack {
            Image(systemName: prayer.systemImage)
                .frame(width: 24)
                .foregroundColor(isNext ? .darkGreen : .secondary)

            Text(prayer.localizedName)
                .foregroundColor(isNext ? .darkGreen : .secondary)
                .fontWeight(isNext ? .semibold : .regular)

            Spacer()

            Text(time, format: .dateTime.hour().minute())
                .foregroundColor(isNext ? .darkGreen : .secondary)
                .fontWeight(isNext ? .semibold : .regular)
        }
        .padding(.horizontal)
        .padding(.vertical, 10)
        .background(isNext ? Color.greenForeground.opacity(0.08) : Color.clear)
    }
}

```

HalalAI/Features/Prayer/UI/PrayerTimesCardViewModel.swift

```

//
// PrayerTimesCardViewModel.swift
// HalalAI
//
// Created by Мирсаит Сабирзянов on 01.04.2026.
//

import Foundation

extension PrayerTimesCardView {
    @Observable
    @MainActor
    final class ViewModel {
        let locationService: LocationService
    }
}

```

```

private let prayerTimeService: PrayerTimeService
private let settingsStore: PrayerSettingsStore

private let calendar = Calendar(identifier: .gregorian)
private var cachedStartOfToday: Date?
/// Если 'nil', используется умный режим: сегодня, пока есть ожидающий на
маз; иначе завтра.
private var preferredDayOffset: Int?

private(set) var todayTimes: DailyPrayerTimes?
var displayedTimes: DailyPrayerTimes?
var nextPrayer: (Prayer, Date)?

private let minDayOffset = -7
private let maxDayOffset = 30

var effectiveDayOffset: Int {
    guard let today = todayTimes else { return 0 }
    let smart = prayerTimeService.nextPrayer(from: today) != nil ? 0 : 1
    return preferredDayOffset ?? smart
}

var displayedDayTitle: String {
    let offset = effectiveDayOffset
    let start = calendar.startOfDay(for: Date.now)
    guard let day = calendar.date(byAdding: .day, value: offset, to:
start) else { return "" }
    switch offset {
    case 0:
        return "Сегодня"
    case 1:
        return "Завтра"
    case -1:
        return "Вчера"
    default:
        return day.formatted(.dateTime.day().month(.wide).weekday(.wide).
locale(Locale(identifier: "ru_RU"))).capitalized
    }
}

var canShiftToPreviousDay: Bool { effectiveDayOffset > minDayOffset }
var canShiftToNextDay: Bool { effectiveDayOffset < maxDayOffset }

init(
    locationService: LocationService,
    prayerTimeService: PrayerTimeService,
    settingsStore: PrayerSettingsStore
) {
    self.locationService = locationService
    self.prayerTimeService = prayerTimeService
    self.settingsStore = settingsStore
}

func refresh() {
    locationService.requestLocation()
    recalculate()
}

func shiftDisplayedDay(by delta: Int) {
    if todayTimes == nil {
        recalculate()
    }
}

```

```

        guard let today = todayTimes else { return }
        let smart = prayerTimeService.nextPrayer(from: today) != nil ? 0 : 1
        let current = preferredDayOffset ?? smart
        preferredDayOffset = min(maxDayOffset, max(minDayOffset, current +
delta))
        recalculate()
    }

    func isNextPrayerRow(prayer: Prayer, time: Date) -> Bool {
        guard let (p, when) = nextPrayer else { return false }
        guard p == prayer else { return false }
        return calendar.isDate(when, equalTo: time, toGranularity: .minute)
    }

    func recalculate() {
        guard let loc = locationService.currentLocation else { return }
        let settings = settingsStore.settings
        let startOfToday = calendar.startOfDay(for: Date.now)
        if cachedStartOfToday != startOfToday {
            cachedStartOfToday = startOfToday
            preferredDayOffset = nil
        }

        todayTimes = prayerTimeService.calculateTimes(
            for: startOfToday,
            location: loc,
            settings: settings
        )

        guard let today = todayTimes else {
            displayedTimes = nil
            nextPrayer = nil
            return
        }

        if let next = prayerTimeService.nextPrayer(from: today) {
            nextPrayer = next
        } else if let tomorrowStart = calendar.date(byAdding: .day, value: 1,
to: startOfToday),
            let tomorrowTimes = prayerTimeService.calculateTimes(
                for: tomorrowStart,
                location: loc,
                settings: settings
            ) {
            nextPrayer = prayerTimeService.nextPrayer(from: tomorrowTimes) ??
(.fajr, tomorrowTimes.fajr)
        } else {
            nextPrayer = nil
        }

        let smartOffset = prayerTimeService.nextPrayer(from: today) != nil ?
0 : 1
        let offset = preferredDayOffset ?? smartOffset
        let clamped = min(maxDayOffset, max(minDayOffset, offset))
        guard let displayStart = calendar.date(byAdding: .day, value: clamped
, to: startOfToday) else {
            displayedTimes = nil
            return
        }
        displayedTimes = prayerTimeService.calculateTimes(
            for: displayStart,
            location: loc,

```

```

        settings: settings
    )
}
}
}

```

HalalAI/Features/Quran/Models/QuranModels.swift

```

//
//  QuranModels.swift
//  HalalAI
//

import Foundation

struct QuranVerse: Identifiable, Equatable {
    let id: String
    /// Номер аята (может быть пустым для Бисмилля)
    let verseNumber: Int?
    let text: String

    init(suraIndex: Int, verseNumber: Int?, text: String) {
        self.id = "\(suraIndex)-\(verseNumber ?? 0)"
        self.verseNumber = verseNumber
        self.text = text
    }
}

struct Sura: Identifiable, Equatable {
    let id: Int
    let index: Int
    let title: String
    let subtitle: String
    let verses: [QuranVerse]

    var displayTitle: String { "\(index). \(title)" }
}

```

HalalAI/Features/Quran/Service/QuranStorageService.swift

```

//
//  QuranStorageService.swift
//  HalalAI
//

import Foundation

@MainActor
protocol QuranStorageService {
    var suras: [Sura] { get }
    var lastReadSuraIndex: Int? { get }
    var lastReadVerseNumber: Int? { get }
    func loadQuranFromBundle() throws
    func saveProgress(suraIndex: Int, verseNumber: Int)
    func clearProgress()
}

private enum UserDefaultsKeys {
    static let lastReadSuraIndex = "quran.lastReadSuraIndex"
}

```

```

        static let lastReadVerseNumber = "quran.lastReadVerseNumber"
    }

    @Observable
    @MainActor
    final class QuranStorageServiceImpl: QuranStorageService {
        private(set) var suras: [Sura] = []
        private var surasLoaded = false

        var lastReadSuraIndex: Int? {
            guard UserDefaults.standard.object(forKey: UserDefaultsKeys.
lastReadSuraIndex) != nil else { return nil }
            let v = UserDefaults.standard.integer(forKey: UserDefaultsKeys.
lastReadSuraIndex)
            return v > 0 ? v : nil
        }
        var lastReadVerseNumber: Int? {
            guard UserDefaults.standard.object(forKey: UserDefaultsKeys.
lastReadVerseNumber) != nil else { return nil }
            let v = UserDefaults.standard.integer(forKey: UserDefaultsKeys.
lastReadVerseNumber)
            return v > 0 ? v : nil
        }

        func loadQuranFromBundle() throws {
            if surasLoaded { return }
            guard let url = Bundle.main.url(forResource: "sury", withExtension: "csv
") else {
                throw QuranError.fileNotFound
            }
            let data = try Data(contentsOf: url)
            guard let content = String(data: data, encoding: .utf8) else {
                throw QuranError.encodingError
            }
            suras = try Self.parseCSV(content)
            surasLoaded = true
        }

        func saveProgress(suraIndex: Int, verseNumber: Int) {
            UserDefaults.standard.set(suraIndex, forKey: UserDefaultsKeys.
lastReadSuraIndex)
            UserDefaults.standard.set(verseNumber, forKey: UserDefaultsKeys.
lastReadVerseNumber)
        }

        func clearProgress() {
            UserDefaults.standard.removeObject(forKey: UserDefaultsKeys.
lastReadSuraIndex)
            UserDefaults.standard.removeObject(forKey: UserDefaultsKeys.
lastReadVerseNumber)
        }

        // MARK: - CSV parsing

        private static func parseCSV(_ content: String) throws -> [Sura] {
            let lines = content.components(separatedBy: .newlines)
            guard !lines.isEmpty else { throw QuranError.emptyFile }
            let columnCount = 5
            var currentSuraIndex: Int?
            var currentTitle = ""
            var currentSubtitle = ""
            var currentVerses: [QuranVerse] = []

```



```

var allSuras: [Sura] = []

for (i, line) in lines.enumerated() {
    if i == 0 && line.hasPrefix("sura_index") { continue }
    let fields = parseCSVLine(line)
    guard fields.count >= columnCount else { continue }
    guard let suraIndex = Int(fields[0]) else { continue }
    let title = fields[1].trimmingCharacters(in: .whitespaces)
    let subtitle = fields[2].trimmingCharacters(in: .whitespaces)
    let verseNumRaw = fields[3].trimmingCharacters(in: .whitespaces)
    let verseNumber = Int(verseNumRaw)
    var text = fields[4].trimmingCharacters(in: .whitespaces)
    if text.hasPrefix("\") { text.removeFirst() }
    if text.hasSuffix("\") { text.removeLast() }

    if currentSuraIndex != suraIndex {
        if let idx = currentSuraIndex, !currentVerses.isEmpty {
            allSuras.append(Sura(
                id: idx,
                index: idx,
                title: currentTitle,
                subtitle: currentSubtitle,
                verses: currentVerses
            ))
        }
        currentSuraIndex = suraIndex
        currentTitle = title
        currentSubtitle = subtitle
        currentVerses = []
    }
    currentVerses.append(QuranVerse(suraIndex: suraIndex, verseNumber:
verseNumber, text: text))
}
if let idx = currentSuraIndex, !currentVerses.isEmpty {
    allSuras.append(Sura(
        id: idx,
        index: idx,
        title: currentTitle,
        subtitle: currentSubtitle,
        verses: currentVerses
    ))
}
return allSuras
}

/// Парсит одну строку CSV с учетом кавычек (поля в кавычках могут содержать
запятые).
private static func parseCSVLine(_ line: String) -> [String] {
    var result: [String] = []
    var current = ""
    var inQuotes = false
    for ch in line {
        if ch == "\"" {
            inQuotes.toggle()
            continue
        }
        if !inQuotes && ch == "," {
            result.append(current)
            current = ""
            continue
        }
        current.append(ch)
    }
}

```

```

        }
        result.append(current)
        return result
    }
}

enum QuranError: LocalizedError {
    case fileNotFound
    case encodingError
    case emptyFile

    var errorDescription: String? {
        switch self {
            case .fileNotFound: return "Файл Корана не найден в приложении."
            case .encodingError: return "Ошибка кодировки файла."
            case .emptyFile: return "Файл Корана пуст."
        }
    }
}

```

HalalAI/Features/Quran/UI/QuranListView.swift

```

//
//  QuranListView.swift
//  HalalAI
//

import SwiftUI

struct QuranListView: View {
    @Environment(Coordinator.self) var coordinator
    @State private var viewModel: ViewModel

    init(quranStorage: QuranStorageService) {
        _viewModel = State(initialValue: ViewModel(quranStorage: quranStorage))
    }

    var body: some View {
        Group {
            if let msg = viewModel.errorMessage {
                ErrorView(message: msg)
            } else if viewModel.isLoading {
                ProgressView("Загрузка Корана...")
                    .frame(maxWidth: .infinity, maxHeight: .infinity)
            } else {
                listContent
            }
        }
        .navigationTitle("Коран")
        .navigationBarTitleDisplayMode(.inline)
        .toolbar {
            ToolbarItem(placement: .topBarLeading) {
                Button("Назад", systemImage: "arrow.left") {
                    coordinator.dismiss()
                }
                .labelStyle(.iconOnly)
            }
        }
        .onAppear { viewModel.loadQuran() }
    }
}

```

```

private var listContent: some View {
    ScrollView {
        VStack(alignment: .leading, spacing: 16) {
            if viewModel.quranStorage.lastReadSuraIndex != nil, viewModel.
quranStorage.lastReadVerseNumber != nil {
                continueReadingButton
            }
            LazyVStack(alignment: .leading, spacing: 8) {
                ForEach(viewModel.suras) { sura in
                    suraRow(sura)
                }
            }
        }
        .padding(.horizontal, 16)
        .padding(.bottom, 24)
    }
}

private var continueReadingButton: some View {
    Button {
        if let idx = viewModel.quranStorage.lastReadSuraIndex,
viewModel.suras.contains(where: { $0.index == idx }) {
            coordinator.nextStep(step: .home(.sura(suraIndex: idx)))
        }
    } label: {
        HStack {
            Image(systemName: "book.fill")
                .font(.title2)
            VStack(alignment: .leading, spacing: 2) {
                Text("Продолжить чтение")
                    .font(.headline)
                if let s = viewModel.quranStorage.lastReadSuraIndex,
let name = viewModel.suras.first(where: { $0.index == s })
?.displayTitle {
                    Text("Cypa \"(name)\")")
                        .font(.subheadline)
                        .opacity(0.9)
                }
            }
            Spacer()
            Image(systemName: "chevron.right")
        }
        .padding()
        .background(RoundedRectangle(cornerRadius: 16).fill(Color.
greenForeground.opacity(0.3)))
        .foregroundColor(.darkGreen)
    }
    .buttonStyle(.plain)
}

private func suraRow(_ sura: Sura) -> some View {
    Button {
        coordinator.nextStep(step: .home(.sura(suraIndex: sura.index)))
    } label: {
        HStack {
            Text("\(sura.index)")
                .font(.caption)
                .fontWeight(.semibold)
                .frame(width: 28, height: 28)
                .background(Circle().fill(Color.greenForeground.opacity(0.5))
)
                .foregroundColor(.darkGreen)
        }
    }
}

```

```

        VStack(alignment: .leading, spacing: 2) {
            Text(sura.title)
                .font(.body)
                .fontWeight(.medium)
                .foregroundColor(.darkGreen)
            Text(sura.subtitle)
                .font(.caption)
                .foregroundColor(.secondary)
        }
        Spacer()
        Text("\(sura.verses.count) аятов")
            .font(.caption)
            .foregroundColor(.secondary)
        Image(systemName: "chevron.right")
            .font(.caption)
            .foregroundColor(.secondary)
    }
    .padding(.vertical, 10)
    .padding(.horizontal, 12)
    .background(RoundedRectangle(cornerRadius: 12).fill(Color.
greenForeground.opacity(0.2)))
    }
    .buttonStyle(.plain)
}
}

```

HalalAI/Features/Quran/UI/QuranListViewModel.swift

```

//
//  QuranListViewModel.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 14.02.2026.
//

import Foundation

extension QuranListView {
    @Observable
    @MainActor
    final class ViewModel {
        var quranStorage: QuranStorageService
        var suras: [Sura] = []
        var isLoading = true
        var errorMessage: String?

        init(quranStorage: QuranStorageService) {
            self.quranStorage = quranStorage
        }

        func loadQuran() {
            isLoading = true
            errorMessage = nil
            Task {
                do {
                    try quranStorage.loadQuranFromBundle()
                    suras = quranStorage.suras
                    isLoading = false
                } catch {
                    errorMessage = error.localizedDescription
                    isLoading = false
                }
            }
        }
    }
}

```

```

        }
    }
}
}
}

```

HalalAI/Features/Quran/UI/SuraReaderView.swift

```

//
//  SuraReaderView.swift
//  HalalAI
//

import SwiftUI

struct SuraReaderView: View {
    @Environment(Coordinator.self) var coordinator
    @State private var viewModel: ViewModel

    init(suraIndex: Int, quranStorage: QuranStorageService) {
        _viewModel = State(initialValue: ViewModel(suraIndex: suraIndex,
            quranStorage: quranStorage))
    }

    var body: some View {
        Group {
            if let s = viewModel.sura {
                readerContent(sura: s)
            } else {
                ProgressView("3арпызка...")
                    .frame(maxWidth: .infinity, maxHeight: .infinity)
            }
        }
        .toolbar {
            ToolbarItem(placement: .topBarLeading) {
                Button("Ҳазар", systemImage: "arrow.left") {
                    coordinator.dismiss()
                }
                .labelStyle(.iconOnly)
            }
        }
        .navigationTitle(viewModel.sura?.displayTitle ?? "Cypa")
        .navigationBarTitleDisplayMode(.inline)
        .background(Color.greenBackground.ignoresSafeArea())
        .onAppear { viewModel.loadSura() }
        .onDisappear { viewModel.saveProgressIfNeeded() }
    }

    private func readerContent(sura: Sura) -> some View {
        VStack(spacing: 0) {
            fontControls
            ScrollView {
                LazyVStack(alignment: .leading, spacing: 20) {
                    ForEach(Array(sura.verses.enumerated()), id: \.element.id) {
                        index, verse in
                            verseRow(verse: verse, sura: sura)
                                .id(verse.id)
                                .onAppear { viewModel.trackVisibleVerse(suraIndex:
                                    sura.index, verseNumber: verse.verseNumber ?? index + 1) }
                    }
                }
            }
        }
    }
}

```

```

        }
    }
    .padding(.horizontal, 20)
    .padding(.vertical, 24)
}
.scrollIndicators(.hidden)
}

private var fontControls: some View {
    HStack {
        Text("Пазмep  peкcтpa")
            .font(.caption)
            .foregroundColor(.secondary)
        Slider(value: $viewModel.fontSize, in: 14...28, step: 1)
            .tint(Color.greenForeground)
        Text("\(Int(viewModel.fontSize))")
            .font(.caption)
            .frame(width: 24)
    }
    .padding(.horizontal, 20)
    .padding(.vertical, 8)
    .background(Color.greenForeground.opacity(0.15))
}

private func verseRow(verse: QuranVerse, sura: Sura) -> some View {
    HStack(alignment: .top, spacing: 12) {
        if let num = verse.verseNumber {
            Text("\(num)")
                .font(.system(size: viewModel.fontSize * 0.7, weight: .medium))
        }
        .foregroundColor(.greenForeground)
        .frame(width: 24, alignment: .trailing)
    }
    Text(verse.text)
        .font(.system(size: viewModel.fontSize))
        .foregroundColor(.darkGreen)
        .lineSpacing(6)
        .fixedSize(horizontal: false, vertical: true)
    }
}
}

```

HalalAI/Features/Quran/UI/SuraReaderViewModel.swift

```

//
//  SuraReaderViewModel.swift
//  HalalAI
//
//  Created by Мирсаит Сабирзянов on 14.02.2026.
//

import SwiftUI

extension SuraReaderView {
    @Observable
    @MainActor
    final class ViewModel {
        var suraIndex: Int
        var quranStorage: QuranStorageService
        var sura: Sura?
    }
}

```

```

var fontSize: CGFloat = 18
var lastTrackedSura: Int?
var lastTrackedVerse: Int?

init(suraIndex: Int, quranStorage: QuranStorageService) {
    self.suraIndex = suraIndex
    self.quranStorage = quranStorage
}

func trackVisibleVerse(suraIndex: Int, verseNumber: Int) {
    lastTrackedSura = suraIndex
    lastTrackedVerse = verseNumber
}

func saveProgressIfNeeded() {
    if let s = lastTrackedSura, let v = lastTrackedVerse {
        quranStorage.saveProgress(suraIndex: s, verseNumber: v)
    }
}

func loadSura() {
    if !quranStorage.suras.isEmpty {
        sura = quranStorage.suras.first { $0.index == suraIndex }
        return
    }
    Task {
        try? quranStorage.loadQuranFromBundle()
        sura = quranStorage.suras.first { $0.index == suraIndex }
    }
}
}

```

HalalAI/Features/Scanner/Models/IngredientModels.swift

```

//
// IngredientModels.swift
// HalalAI
//
// Created by Мирсаит Сабирзянов on 29.12.2025.
//

import Foundation
import SwiftUI

enum IngredientStatus: String, Codable {
    case halal = "halal"
    case haram = "haram"
    case mushbooh = "mushbooh"
    case unknown = "unknown"

    var displayName: String {
        switch self {
            case .halal:
                return "Халяль"
            case .haram:
                return "Харам"
            case .mushbooh:
                return "Сомнительно"
            case .unknown:
                return "Неизвестно"
        }
    }
}

```

```

    }
}

var color: Color {
    switch self {
    case .halal:
        return Color.greenForeground
    case .haram:
        return .red
    case .mushbooh:
        return .orange
    case .unknown:
        return .gray
    }
}
}

struct Ingredient: Codable, Identifiable {
    let id: UUID
    let eCode: String?
    let status: IngredientStatus
    let nameRu: String
    let nameEn: String
    let note: String?

    enum CodingKeys: String, CodingKey {
        case eCode = "e_code"
        case status
        case nameRu = "name_ru"
        case nameEn = "name_en"
        case note
    }

    init(id: UUID = UUID(), eCode: String?, status: IngredientStatus, nameRu:
String, nameEn: String, note: String? = nil) {
        self.id = id
        self.eCode = eCode
        self.status = status
        self.nameRu = nameRu
        self.nameEn = nameEn
        self.note = note
    }

    init(from decoder: Decoder) throws {
        let container = try decoder.container(keyedBy: CodingKeys.self)
        self.id = UUID()
        self.eCode = try container.decodeIfPresent(String.self, forKey: .eCode)?.
isEmpty == false
            ? try container.decodeIfPresent(String.self, forKey: .eCode)
            : nil
        let statusString = try container.decode(String.self, forKey: .status)
        self.status = IngredientStatus(rawValue: statusString) ?? .unknown
        self.nameRu = try container.decode(String.self, forKey: .nameRu)
        self.nameEn = try container.decode(String.self, forKey: .nameEn)
        self.note = try container.decodeIfPresent(String.self, forKey: .note)?.
isEmpty == false
            ? try container.decodeIfPresent(String.self, forKey: .note)
            : nil
    }
}

struct ProductAnalysis {

```



```

    let ingredients: [DetectedIngredient]
    let overallStatus: IngredientStatus
    let haramIngredients: [DetectedIngredient]
    let mushboohIngredients: [DetectedIngredient]

    var isHalal: Bool {
        return overallStatus == .halal && haramIngredients.isEmpty &&
            mushboohIngredients.isEmpty
    }
}

struct DetectedIngredient: Identifiable {
    let id: UUID
    let name: String
    let matchedIngredient: Ingredient?
    let status: IngredientStatus

    init(name: String, matchedIngredient: Ingredient? = nil) {
        self.id = UUID()
        self.name = name
        self.matchedIngredient = matchedIngredient
        self.status = matchedIngredient?.status ?? .unknown
    }
}

```

HalalAI/Features/Scanner/Service/IngredientCSVParser.swift

```

//
// IngredientCSVParser.swift
// HalalAI
//
// Extracted from IngredientService.swift
//

import Foundation

/// Отвечает за парсинг CSV-файла с ингредиентами.
struct IngredientCSVParser {

    func parseFromBundle() throws -> [Ingredient] {
        guard let url = Bundle.main.url(forResource: "ingr", withExtension: "csv")
        else {
            throw IngredientServiceError.fileNotFound
        }

        let data = try Data(contentsOf: url)
        let csvString = String(data: data, encoding: .utf8) ?? ""

        let lines = csvString.components(separatedBy: .newlines)
            .filter { !$0.isEmpty }

        guard lines.count > 1 else {
            throw IngredientServiceError.invalidFormat
        }

        let dataLines = Array(lines.dropFirst())
        var parsedIngredients: [Ingredient] = []

        for line in dataLines {
            let components = parseCSVLine(line)
            guard components.count >= 4 else { continue }

```

```

        let eCode = components[0].isEmpty ? nil : components[0]
        let statusString = components[1]
        let nameRu = components[2].trimmingCharacters(in: .
whitespacesAndNewlines)
        let nameEn = components[3].trimmingCharacters(in: .
whitespacesAndNewlines)
        let note = components[4].trimmingCharacters(in: .
whitespacesAndNewlines)

        guard let status = IngredientStatus(rawValue: statusString) else {
continue }

        let ingredient = Ingredient(
            eCode: eCode,
            status: status,
            nameRu: nameRu,
            nameEn: nameEn,
            note: note.isEmpty == false ? note : nil
        )
        parsedIngredients.append(ingredient)
    }

    return parsedIngredients
}

// MARK: - Private

private func parseCSVLine(_ line: String) -> [String] {
    var result: [String] = []
    var currentField = ""
    var insideQuotes = false

    for char in line {
        if char == "\"" {
            insideQuotes.toggle()
        } else if char == "," && !insideQuotes {
            result.append(currentField.trimmingCharacters(in: .whitespaces))
            currentField = ""
        } else {
            currentField.append(char)
        }
    }
    result.append(currentField.trimmingCharacters(in: .whitespaces))

    return result
}
}

```

HalalAI/Features/Scanner/Service/IngredientMatcher.swift

```

//
// IngredientMatcher.swift
// HalalAI
//
// Extracted from IngredientService.swift
//

import Foundation

/// Алгоритм сопоставления текста с базой ингредиентов:

```

```

/// извлечение E-кодов, скоринг совпадений, фильтрация лучших результатов.
struct IngredientMatcher {

    typealias CandidateMatch = (ingredient: Ingredient, name: String, score: Double)

    // MARK: - Public

    /// Анализирует текст и возвращает найденные ингредиенты с общим статусом.
    func analyze(text: String, ingredients: [Ingredient]) -> ProductAnalysis {
        var detectedIngredients: [DetectedIngredient] = []
        var foundECodes: Set<String> = []

        // 1. Извлекаем E-коды из текста
        let ecodes = extractECodes(from: text)

        // 2. Проверяем каждый E-код
        for ecode in ecodes {
            if let ingredient = ingredients.first(where: { $0.eCode?.uppercased() == ecode.uppercased() }) {
                let detected = DetectedIngredient(name: ecode, matchedIngredient: ingredient)
                detectedIngredients.append(detected)
                foundECodes.insert(ecode.uppercased())
            }
        }

        // 3. Проверяем по названиям ингредиентов (гибкий поиск по словам)
        let textLower = text.lowercased()
        var candidateMatches: [CandidateMatch] = []

        for ingredient in ingredients {
            if let eCode = ingredient.eCode, foundECodes.contains(eCode.uppercased()) {
                continue
            }

            let nameRuLower = ingredient.nameRu.lowercased().trimmingCharacters(in: .whitespacesAndNewlines)
            if !nameRuLower.isEmpty,
                let score = matchesIngredientWithScore(name: nameRuLower, in: textLower) {
                candidateMatches.append((ingredient: ingredient, name: ingredient.nameRu, score: score))
            }

            let nameEnLower = ingredient.nameEn.lowercased().trimmingCharacters(in: .whitespacesAndNewlines)
            if !nameEnLower.isEmpty,
                let score = matchesIngredientWithScore(name: nameEnLower, in: textLower) {
                candidateMatches.append((ingredient: ingredient, name: ingredient.nameEn, score: score))
            }
        }

        // 4. Фильтруем и выбираем лучшие совпадения
        let filteredMatches = filterBestMatches(candidates: candidateMatches, text: textLower)

        var foundIngredientKeys: Set<String> = []
        for match in filteredMatches {

```

```

        let key = "\\(match.ingredient.eCode ?? "")_\\(match.ingredient.id)"
        if !foundIngredientKeys.contains(key) {
            foundIngredientKeys.insert(key)
            let detected = DetectedIngredient(name: match.name,
matchedIngredient: match.ingredient)
            detectedIngredients.append(detected)
        }
    }

    let haramIngredients = detectedIngredients.filter { $0.status == .haram }
    let mushboohIngredients = detectedIngredients.filter { $0.status == .
mushbooh }

    let overallStatus: IngredientStatus
    if !haramIngredients.isEmpty {
        overallStatus = .haram
    } else if !mushboohIngredients.isEmpty {
        overallStatus = .mushbooh
    } else if !detectedIngredients.isEmpty && detectedIngredients.allSatisfy
({ $0.status == .halal }) {
        overallStatus = .halal
    } else {
        overallStatus = .unknown
    }

    return ProductAnalysis(
        ingredients: detectedIngredients,
        overallStatus: overallStatus,
        haramIngredients: haramIngredients,
        mushboohIngredients: mushboohIngredients
    )
}

// MARK: - E-Code Extraction

private func extractECodes(from text: String) -> [String] {
    var ecodes: [String] = []

    let words = text.components(separatedBy: CharacterSet.alphanumerics.
inverted)
        .filter { !$0.isEmpty }

    for word in words {
        let wordUpper = word.uppercased()
        if (wordUpper.hasPrefix("E") || wordUpper.hasPrefix("e")) &&
wordUpper.count > 1 {
            let remaining = String(wordUpper.dropFirst())

            if !remaining.isEmpty {
                let lastChar = remaining.last!
                let validEndChars = CharacterSet(charactersIn: "0123456789
abcdefi")

                if validEndChars.contains(lastChar.unicodeScalars.first!) {
                    let validChars = CharacterSet.alphanumerics
                    if remaining.rangeOfCharacter(from: validChars.inverted)
== nil {
                        ecodes.append(wordUpper)
                    }
                }
            }
        }
    }
}

```

```

    }

    return Array(Set(ecodes))
}

// MARK: - Score Matching

/// Возвращает оценку совпадения (0.0-1.0) или nil, если не найдено.
private func matchesIngredientWithScore(name: String, in text: String) ->
Double? {
    let nameLower = name.lowercased().trimmingCharacters(in: .
whitespacesAndNewlines)
    let textLower = text.lowercased()

    // 1. Если название очень короткое (< 5 символов), только точное совпаден
ие
    if nameLower.count < 5 {
        if hasWordBoundaryMatch(nameLower, in: textLower) {
            return 0.7
        }
        return nil
    }

    // 2. Точное совпадение с границами слов -- самый высокий score
    if hasWordBoundaryMatch(nameLower, in: textLower) {
        return 1.0
    }

    // 3. Поиск по словам
    let allIngredientWords = extractWords(from: name, minLength: 1)
    let significantIngredientWords = extractWords(from: name, minLength: 4)

    guard !significantIngredientWords.isEmpty else {
        return nil
    }

    var matchedSignificantWords = 0
    for word in significantIngredientWords {
        if hasWordBoundaryMatch(word, in: textLower) {
            matchedSignificantWords += 1
        }
    }

    let shortWords = allIngredientWords.filter { $0.count >= 3 && $0.count <
4 }
    var matchedShortWords = 0
    for word in shortWords {
        if hasWordBoundaryMatch(word, in: textLower) {
            matchedShortWords += 1
        }
    }

    if !shortWords.isEmpty && matchedShortWords < shortWords.count {
        return nil
    }

    let matchRatio = Double(matchedSignificantWords) / Double(
significantIngredientWords.count)

    let minRequiredRatio: Double
    if significantIngredientWords.count == 1 {
        minRequiredRatio = 1.0
    }

```

```

    } else if significantIngredientWords.count == 2 {
        minRequiredRatio = 1.0
    } else if significantIngredientWords.count == 3 {
        minRequiredRatio = 0.67
    } else {
        minRequiredRatio = 0.75
    }

    if matchRatio >= minRequiredRatio {
        let score = matchRatio * 0.85
        return score
    }

    return nil
}

// MARK: - Best Match Filtering

/// Фильтрует кандидатов, оставляя только лучшие совпадения.
private func filterBestMatches(candidates: [CandidateMatch], text: String) ->
[CandidateMatch] {
    if candidates.isEmpty { return [] }

    let filteredByScore = candidates.filter { $0.score >= 0.7 }

    let sorted = filteredByScore.sorted { first, second in
        if first.score != second.score {
            return first.score > second.score
        }
        return first.name.count < second.name.count
    }

    var result: [CandidateMatch] = []
    var usedPositions: Set<Int> = []
    let textLower = text.lowercased()

    for (index, candidate) in sorted.enumerated() {
        if usedPositions.contains(index) { continue }

        let candidateNameLower = candidate.name.lowercased().
trimmingCharacters(in: .whitespacesAndNewlines)
        let hasExactMatch = hasWordBoundaryMatch(candidateNameLower, in:
textLower)
        var isSubset = false

        for (otherIndex, other) in sorted.enumerated() {
            if index == otherIndex || usedPositions.contains(otherIndex) {
continue }

            let otherNameLower = other.name.lowercased().trimmingCharacters(
in: .whitespacesAndNewlines)

            if otherNameLower.contains(candidateNameLower) &&
candidateNameLower.count < otherNameLower.count {

                let candidateWords = Set(extractWords(from:
candidateNameLower, minLength: 4))
                let otherWords = Set(extractWords(from: otherNameLower,
minLength: 4))
                let additionalWords = otherWords.subtracting(candidateWords)

```

```

        let candidateIsStartOfPhrase = hasFollowingWordsMatch(
candidateNameLower, in: textLower)
        let hasExactMatchForOther = hasWordBoundaryMatch(
otherNameLower, in: textLower)
        let allAdditionalWordsInText = additionalWords.isEmpty ||
additionalWords.allSatisfy {
            hasWordBoundaryMatch($0, in: textLower)
        }

        if hasExactMatch && !hasExactMatchForOther { continue }
        if hasExactMatch && !allAdditionalWordsInText { continue }
        if candidateIsStartOfPhrase && !hasExactMatchForOther {
continue }

        if hasExactMatchForOther && !hasExactMatch {
            isSubset = true
            break
        }

        if hasExactMatchForOther && allAdditionalWordsInText {
            isSubset = true
            break
        }

        if hasExactMatch && hasExactMatchForOther {
            if other.score > candidate.score ||
                (other.score == candidate.score && otherNameLower.
count < candidateNameLower.count) {
                isSubset = true
                break
            }
        }

        if !hasExactMatch && !hasExactMatchForOther &&
allAdditionalWordsInText &&
other.score > candidate.score + 0.1 {
            let lengthDiff = otherNameLower.count -
candidateNameLower.count
            if lengthDiff > 2 {
                isSubset = true
                break
            }
        }

        if !allAdditionalWordsInText { continue }
    }

    // Обратная проверка: этот ингредиент содержит другой
    if candidateNameLower.contains(otherNameLower) &&
otherNameLower.count < candidateNameLower.count {
        let hasExactMatchForOther = hasWordBoundaryMatch(
otherNameLower, in: textLower)
        if hasExactMatchForOther && !hasExactMatch {
            usedPositions.insert(index)
            break
        }
    }
}

if !isSubset {
    result.append(candidate)
    usedPositions.insert(index)
}

```

```

        }
    }

    return result
}

// MARK: - Helpers

private func hasWordBoundaryMatch(_ word: String, in text: String) -> Bool {
    let pattern = "\\b\\(NSRegularExpression.escapedPattern(for: word))\\b"
    guard let regex = try? NSRegularExpression(pattern: pattern, options: [.
caseInsensitive]) else {
        return false
    }
    let range = NSRange(location: 0, length: text.utf16.count)
    return regex.firstMatch(in: text, options: [], range: range) != nil
}

private func hasFollowingWordsMatch(_ word: String, in text: String) -> Bool
{
    let pattern = "\\b\\(NSRegularExpression.escapedPattern(for: word))\\s+\\w
+"
    guard let regex = try? NSRegularExpression(pattern: pattern, options: [.
caseInsensitive]) else {
        return false
    }
    let range = NSRange(location: 0, length: text.utf16.count)
    return regex.firstMatch(in: text, options: [], range: range) != nil
}

private func extractWords(from string: String, minLength: Int) -> [String] {
    string.components(separatedBy: CharacterSet.alphanumerics.inverted)
        .map { $0.lowercased().trimmingCharacters(in: .whitespacesAndNewlines
) }
        .filter { !$0.isEmpty && $0.count >= minLength }
}
}

```

HalalAI/Features/Scanner/Service/IngredientService.swift

```

//
//  IngredientService.swift
//  HalalAI
//
//  Created by Мирсаят Сабырзянов on 29.12.2025.
//

import Foundation

protocol IngredientService {
    func loadIngredients() async throws -> [Ingredient]
    func analyzeText(_ text: String) async -> ProductAnalysis
}

@MainActor
final class IngredientServiceImpl: IngredientService {
    private var ingredients: [Ingredient] = []
    private var ingredientsLoaded = false

    private let parser = IngredientCSVParser()
    private let matcher = IngredientMatcher()
}

```



```

func loadIngredients() async throws -> [Ingredient] {
    if ingredientsLoaded {
        return ingredients
    }

    ingredients = try parser.parseFromBundle()
    ingredientsLoaded = true
    return ingredients
}

func analyzeText(_ text: String) async -> ProductAnalysis {
    _ = try? await loadIngredients()
    return matcher.analyze(text: text, ingredients: ingredients)
}
}

enum IngredientServiceError: LocalizedError {
    case fileNotFound
    case invalidFormat
    case loadingFailed

    var errorDescription: String? {
        switch self {
            case .fileNotFound:
                return "Файл с ингредиентами не найден"
            case .invalidFormat:
                return "Неверный формат файла"
            case .loadingFailed:
                return "Ошибка загрузки данных"
        }
    }
}
}

```

HalalAI/Features/Scanner/UI/CameraView.swift

```

//
// CameraView.swift
// HalalAI
//
// Extracted from ScannerView.swift
//

import SwiftUI
import UIKit

enum ImageSource {
    case camera
    case photoLibrary
}

struct CameraView: UIViewControllerRepresentable {
    let sourceType: ImageSource
    let onImageCaptured: ([UIImage]) -> Void
    @Environment(\.dismiss) var dismiss

    func makeUIViewController(context: Context) -> UIImagePickerController {
        let picker = UIImagePickerController()
        picker.delegate = context.coordinator
        picker.sourceType = sourceType == .camera ? .camera : .photoLibrary
        if sourceType == .camera {

```

```

        picker.cameraCaptureMode = .photo
    }
    picker.allowsEditing = false
    return picker
}

func updateUIViewController(_ viewController: UIImagePickerController,
context: Context) {}

func makeCoordinator() -> Coordinator {
    Coordinator(onImageCaptured: onImageCaptured)
}

class Coordinator: NSObject, UIImagePickerControllerDelegate,
UINavigationControllerDelegate {
    let onImageCaptured: ([UIImage]) -> Void

    init(onImageCaptured: @escaping ([UIImage]) -> Void) {
        self.onImageCaptured = onImageCaptured
    }

    func imagePickerController(_ picker: UIImagePickerController,
didFinishPickingMediaWithInfo info: [UIImagePickerController.InfoKey: Any]) {
        if let image = info[.originalImage] as? UIImage {
            picker.dismiss(animated: true) {
                self.onImageCaptured([image])
            }
        } else {
            picker.dismiss(animated: true)
        }
    }

    func imagePickerControllerDidCancel(_ picker: UIImagePickerController) {
        picker.dismiss(animated: true)
    }
}
}

```

HalalAI/Features/Scanner/UI/IngredientResultsView.swift

```

//
//  IngredientResultsView.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 29.12.2025.
//

import SwiftUI

struct IngredientResultsView: View {
    let analysis: ProductAnalysis?
    @Environment(\.dismiss) private var dismiss

    var body: some View {
        NavigationStack {
            ZStack {
                Color.greenBackground.ignoresSafeArea()

                if let analysis = analysis {
                    ScrollView {
                        VStack(spacing: 20) {

```

```

        // Общий статус
        overallStatusCard(analysis)

        // Харам ингредиенты
        if !analysis.haramIngredients.isEmpty {
            haramIngredientsCard(analysis.haramIngredients)
        }

        // Сомнительные ингредиенты
        if !analysis.mushboohIngredients.isEmpty {
            mushboohIngredientsCard(analysis.
mushboohIngredients)
        }

        // Все ингредиенты
        allIngredientsCard(analysis.ingredients)
    }
    .padding()
}
} else {
    Text("Ошибка анализа")
        .foregroundColor(.gray)
}
}
.navigationTitle("Результаты анализа")
.navigationBarTitleDisplayMode(.inline)
.toolbar {
    ToolbarItem(placement: .topBarTrailing) {
        Button("Закрыть") {
            dismiss()
        }
        .foregroundColor(.darkGreen)
    }
}
}
}

private func overallStatusCard(_ analysis: ProductAnalysis) -> some View {
    VStack(spacing: 12) {
        HStack {
            Image(systemName: analysis.isHalal ? "checkmark.circle.fill" : "
xmark.circle.fill")
                .font(.largeTitle)
                .foregroundColor(analysis.overallStatus.color)

            VStack(alignment: .leading, spacing: 4) {
                Text("Статус продукта")
                    .font(.subheadline)
                    .foregroundColor(.gray)
                Text(analysis.overallStatus.displayName)
                    .font(.title2)
                    .bold()
                    .foregroundColor(analysis.overallStatus.color)
            }

            Spacer()
        }

        if !analysis.isHalal {
            Text(analysis.overallStatus == .haram
                ? "Продукт содержит запрещенные ингредиенты"
                : "Продукт содержит сомнительные ингредиенты")
        }
    }
}

```

```

        .font(.subheadline)
        .foregroundColor(.gray)
        .multilineTextAlignment(.center)
    }
}
.padding()
.background(Color.white)
.clipShape(.rect(cornerRadius: 16))
.shadow(radius: 4)
}

private func haramIngredientsCard(_ ingredients: [DetectedIngredient]) ->
some View {
    VStack(alignment: .leading, spacing: 12) {
        HStack {
            Image(systemName: "exclamationmark.triangle.fill")
                .foregroundColor(.red)
            Text("Запрещенные ингредиенты")
                .font(.headline)
                .foregroundColor(.red)
        }

        ForEach(ingredients) { ingredient in
            IngredientRowView(ingredient: ingredient)
        }
    }
    .padding()
    .background(Color.white)
    .clipShape(.rect(cornerRadius: 16))
    .shadow(radius: 4)
}

private func mushboohIngredientsCard(_ ingredients: [DetectedIngredient]) ->
some View {
    VStack(alignment: .leading, spacing: 12) {
        HStack {
            Image(systemName: "questionmark.circle.fill")
                .foregroundColor(.orange)
            Text("Сомнительные ингредиенты")
                .font(.headline)
                .foregroundColor(.orange)
        }

        ForEach(ingredients) { ingredient in
            IngredientRowView(ingredient: ingredient)
        }
    }
    .padding()
    .background(Color.white)
    .clipShape(.rect(cornerRadius: 16))
    .shadow(radius: 4)
}

private func allIngredientsCard(_ ingredients: [DetectedIngredient]) -> some
View {
    VStack(alignment: .leading, spacing: 12) {
        Text("Все ингредиенты")
            .font(.headline)
            .foregroundColor(.darkGreen)

        if ingredients.isEmpty {
            Text("В тексте не найдено совпадений с базой ингредиентов.")
        }
    }
}

```

```

        .font(.subheadline)
        .foregroundColor(.gray)
    } else {
        ForEach(ingredients) { ingredient in
            IngredientRowView(ingredient: ingredient, showStatus: true)
        }
    }
    .padding()
    .background(Color.white)
    .clipShape(.rect(cornerRadius: 16))
    .shadow(radius: 4)
}

}

// MARK: - Ingredient Row View

struct IngredientRowView: View {
    let ingredient: DetectedIngredient
    var showStatus: Bool = false
    @State private var isExpanded = false

    var body: some View {
        VStack(alignment: .leading, spacing: 8) {
            HStack {
                Circle()
                    .fill(ingredient.status.color)
                    .frame(width: 8, height: 8)

                Text(ingredient.name)
                    .font(.body)

                Spacer()

                if showStatus {
                    Text(ingredient.status.displayName)
                        .font(.caption)
                        .foregroundColor(ingredient.status.color)
                        .padding(.horizontal, 8)
                        .padding(.vertical, 4)
                        .background(ingredient.status.color.opacity(0.2))
                        .clipShape(.rect(cornerRadius: 8))
                } else if let matched = ingredient.matchedIngredient {
                    Text(matched.nameRu)
                        .font(.caption)
                        .foregroundColor(.gray)
                }
            }

            // Кнопка раскрытия, если есть note
            if ingredient.matchedIngredient?.note != nil {
                Button(isExpanded ? "Свернуть" : "Развернуть", systemImage:
isExpanded ? "chevron.up" : "chevron.down") {
                    withAnimation {
                        isExpanded.toggle()
                    }
                }
                .font(.caption)
                .foregroundColor(.gray)
                .labelStyle(.iconOnly)
            }
        }
    }
}

```

```

        // Раскрывающееся поле с note
        if isExpanded, let note = ingredient.matchedIngredient?.note, !note.isEmpty {
            VStack(alignment: .leading, spacing: 4) {
                Divider()
                Text(note)
                    .font(.caption)
                    .foregroundColor(.gray)
                    .padding(.leading, 16)
            }
            .transition(.opacity.combined(with: .move(edge: .top)))
        }
        .padding(.vertical, 4)
    }
}

```

HalalAI/Features/Scanner/UI/ScannerView.swift

```

//
// IngredientScannerView.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 29.12.2025.
//

import SwiftUI

struct ScannerView: View {
    @State private var viewModel: ViewModel
    @Environment(\.dismiss) private var dismiss

    init(ingredientService: IngredientService, authManager: AuthManager) {
        _viewModel = State(initialValue: ViewModel(ingredientService: ingredientService, authManager: authManager))
    }

    var body: some View {
        @Bindable var vm = viewModel
        ZStack {
            Color.greenBackground.ignoresSafeArea()

            VStack(spacing: 0) {
                // Заголовок
                HStack {
                    Button("Назад", systemImage: "chevron.left") {
                        dismiss()
                    }
                    .font(.title2)
                    .labelStyle(.iconOnly)
                    Spacer()
                    Text("Сканирование состава")
                        .font(.headline)
                    Spacer()
                    Button("Ввод вручную", systemImage: vm.showManualInput ? "camera" : "keyboard") {
                        vm.showManualInput.toggle()
                    }
                    .font(.title2)
                    .labelStyle(.iconOnly)

```

```

        }
        .animation(.easeInOut, value: vm.showManualInput)
        .foregroundColor(.darkGreen)
        .padding()

        if vm.showManualInput {
            manualInputView
        } else {
            scannerView
        }
    }
}

.sheet(isPresented: $vm.showCamera) {
    CameraView(sourceType: vm.cameraSource) { images in
        vm.processImages(images)
    }
}

.sheet(isPresented: $vm.showResults) {
    IngredientResultsView(analysis: vm.analysis)
}

.overlay {
    if vm.authManager.isGuest {
        GuestAuthPromptView(featureName: "сканирование продуктов",
authManager: vm.authManager)
    }
}

.overlay {
    if vm.isLoading {
        ZStack {
            Color.black.opacity(0.3)
                .ignoresSafeArea()
            VStack(spacing: 20) {
                ProgressView()
                    .scaleEffect(1.5)
                    .foregroundColor(.white)
                Text("Обработка изображения...")
                    .foregroundColor(.black)
                    .font(.headline)
            }
            .padding(30)
            .background(.darkGreen.opacity(0.9))
            .clipShape(.rect(cornerRadius: 20))
        }
    }
}

private var scannerView: some View {
    VStack(spacing: 20) {
        Spacer()

        Image(systemName: "camera.fill")
            .font(.system(size: 70))
            .foregroundColor(.darkGreen)

        Text("Сфотографируйте или выберите в галерее состав продукта")
            .font(.title2)
            .multilineTextAlignment(.center)
            .foregroundColor(.darkGreen)
            .padding(.horizontal)

        VStack(spacing: 15) {

```

```

        Button(action: viewModel.openCamera) {
            HStack {
                Image(systemName: "camera.fill")
                Text("Сфотографировать")
            }
            .font(.headline)
            .foregroundColor(.white)
            .frame(maxWidth: .infinity)
            .padding()
            .background(Color.darkGreen)
            .clipShape(.rect(cornerRadius: 12))
        }

        Button(action: viewModel.openPhotoLibrary) {
            HStack {
                Image(systemName: "photo.on.rectangle")
                Text("Выбрать из галереи")
            }
            .font(.headline)
            .foregroundColor(.darkGreen)
            .frame(maxWidth: .infinity)
            .padding()
            .background(Color.white)
            .overlay(
                RoundedRectangle(cornerRadius: 12)
                    .stroke(Color.darkGreen, lineWidth: 2)
            )
            .clipShape(.rect(cornerRadius: 12))
        }
    }
    .padding(.horizontal, 40)
    .padding(.top, 20)

    Spacer()
}

private var manualInputView: some View {
    VStack(spacing: 20) {
        Spacer()

        Image(systemName: "keyboard.fill")
            .font(.system(size: 70))
            .foregroundColor(.darkGreen)

        Text("Введите состав продукта")
            .font(.title2)
            .multilineTextAlignment(.center)
            .foregroundColor(.darkGreen)
            .padding(.horizontal)

        TextEditor(text: $viewModel.manualInput)
            .frame(height: 200)
            .padding()
            .background(Color.white)
            .clipShape(.rect(cornerRadius: 12))
            .padding(.horizontal)

        Button(action: {
            viewModel.processManualInput()
        }) {
            HStack {

```



```

        Image(systemName: "checkmark.circle.fill")
        Text("Проверить")
    }
    .font(.headline)
    .foregroundColor(.white)
    .frame(maxWidth: .infinity)
    .padding()
    .background(Color.darkGreen)
    .clipShape(.rect(cornerRadius: 12))
}
.padding(.horizontal)
.disabled(viewModel.manualInput.trimmingCharacters(in: .
whitespacesAndNewlines).isEmpty)

    Spacer()
}
.padding()
}
}

```

HalalAI/Features/Scanner/UI/ScannerViewModel.swift

```

//
// IngredientScannerViewModel.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 05.01.2026.
//

import SwiftUI
import Vision

extension ScannerView {
    @MainActor
    @Observable
    final class ViewModel {
        var showCamera = false
        var showManualInput = false
        var showResults = false
        var manualInput = ""
        var analysis: ProductAnalysis?
        var isLoading = false
        var cameraSource: ImageSource = .camera

        private let ingredientService: IngredientService
        let authManager: AuthManager

        init(ingredientService: IngredientService, authManager: AuthManager) {
            self.ingredientService = ingredientService
            self.authManager = authManager
            Task {
                try? await ingredientService.loadIngredients()
            }
        }

        func openCamera() {
            cameraSource = .camera
            showCamera = true
        }
    }
}

```

```

func openPhotoLibrary() {
    cameraSource = .photoLibrary
    showCamera = true
}

func processImages(_ images: [UIImage]) {
    guard !images.isEmpty else { return }

    isLoading = true

    Task {
        // Обрабатываем все изображения и объединяем текст
        var allText: [String] = []

        for image in images {
            let text = await recognizeText(from: image)
            if !text.isEmpty {
                allText.append(text)
            }
        }

        let combinedText = allText.joined(separator: " ")

        await MainActor.run {
            if !combinedText.isEmpty {
                processText(combinedText)
            } else {
                print("Не удалось распознать текст на изображении")
            }
            isLoading = false
        }
    }
}

func processManualInput() {
    let text = manualInput
    processText(text)
}

private func processText(_ text: String) {
    print("Распознанный текст: \(text)")

    Task {
        let analysis = await ingredientService.analyzeText(text)
        await MainActor.run {
            print("Найдено ингредиентов: \(analysis.ingredients.count)")
            print("Харам ингредиентов: \(analysis.haramIngredients.count)")
        }

        self.analysis = analysis
        self.showResults = true
    }
}

")

private func recognizeText(from image: UIImage) async -> String {
    guard let cgImage = image.cgImage else {
        print("Ошибка: не удалось получить CGImage")
        return ""
    }

    return await withCheckedContinuation { continuation in
        let request = VNRecognizeTextRequest { request, error in

```



```

@Binding var useCustomModel: Bool
var availableModels: [String]
var defaultRemoteModel: String
var onRefreshModels: () -> Void

var body: some View {
    Section(header: Text("Удаленная модель")) {
        modelPickerContent
        maxTokensContent
        temperatureContent
        ragToggleContent
        refreshButton
    }
}

// MARK: - Model Picker

@ViewBuilder
private var modelPickerContent: some View {
    if !availableModels.isEmpty {
        Picker("Выберите модель", selection: $remoteModel) {
            ForEach(availableModels, id: \.self) { model in
                Text(model).tag(model)
            }
        }
        .pickerStyle(.menu)
        .font(.system(.body, design: .monospaced))

        Text("Доступно \(availableModels.count) моделей. По умолчанию: \(
defaultRemoteModel)")
            .font(.footnote)
            .foregroundColor(.secondary)

        Toggle("Использовать custom модель", isOn: $useCustomModel)
            .toggleStyle(SwitchToggleStyle(tint: .green))

        if useCustomModel {
            TextField(
                "Введите имя модели (например, meta-llama/llama-3.3-70b-
instruct:free)",
                text: $remoteModel
            )
            .textInputAutocapitalization(.never)
            .disableAutocorrection(true)
            .font(.system(.body, design: .monospaced))

            Text("Укажите имя модели в формате провайдера/модель.")
                .font(.footnote)
                .foregroundColor(.secondary)
        }
    } else {
        TextField(
            "Введите имя модели (например, meta-llama/llama-3.3-70b-instruct:
free)",
            text: $remoteModel
        )
        .textInputAutocapitalization(.never)
        .disableAutocorrection(true)
        .font(.system(.body, design: .monospaced))

        Text("Введите имя модели вручную. Список пока не загружен.")
            .font(.footnote)
    }
}

```

```

        .foregroundColor(.secondary)
    }
}

// MARK: - Max Tokens

private var maxTokensContent: some View {
    VStack(alignment: .leading, spacing: 8) {
        HStack {
            Text("max_tokens")
            Spacer()
            Text("\(Int(maxTokensSlider))")
                .foregroundColor(.secondary)
                .font(.system(.body, design: .monospaced))
        }

        Slider(
            value: $maxTokensSlider,
            in: 16...6144,
            step: 16,
            label: { Text("max_tokens") },
            minimumValueLabel: {
                Text("16")
                    .font(.caption)
                    .foregroundColor(.secondary)
            },
            maximumValueLabel: {
                Text("6144")
                    .font(.caption)
                    .foregroundColor(.secondary)
            }
        )

        Text("Лимит токенов для генерации. Сервер принимает до 6144.")
            .font(.footnote)
            .foregroundColor(.secondary)
    }
}

// MARK: - Temperature

private var temperatureContent: some View {
    VStack(alignment: .leading, spacing: 8) {
        HStack {
            Text("temperature")
            Spacer()
            Text(temperatureSlider, format: .number.precision(.fractionLength
(2)))
                .foregroundColor(.secondary)
                .font(.system(.body, design: .monospaced))
        }

        Slider(
            value: $temperatureSlider,
            in: 0.0...2.0,
            step: 0.1,
            label: { Text("temperature") },
            minimumValueLabel: {
                Text("0.0")
                    .font(.caption)
                    .foregroundColor(.secondary)
            },
        ),
    }
}

```

```

        maximumValueLabel: {
            Text("2.0")
                .font(.caption)
                .foregroundColor(.secondary)
        }
    )

    Text("Контролирует случайность ответов. 0 = детерминированно, 2.0 = м
аксимально случайно.")
        .font(.footnote)
        .foregroundColor(.secondary)
    }
}

// MARK: - RAG Toggle

private var ragToggleContent: some View {
    Group {
        Toggle("Использовать RAG (семантический поиск)", isOn: $useRag)
            .toggleStyle(SwitchToggleStyle(tint: .green))

        Text("RAG извлекает релевантные аяты из Корана для контекста. Выключи
те для ответов без контекста.")
            .font(.footnote)
            .foregroundColor(.secondary)
    }
}

// MARK: - Refresh

private var refreshButton: some View {
    Button("Обновить список моделей", action: onRefreshModels)
        .font(.footnote)
}
}

```

HalalAI/Features/Settings/UI/SettingsView.swift

```

//
//  SettingsView.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 12.10.2025.
//

import SwiftUI

struct SettingsView: View {
    @State private var viewModel: ViewModel

    init(chatService: ChatService, authManager: AuthManager) {
        _viewModel = State(initialValue: ViewModel(chatService: chatService,
authManager: authManager))
    }

    var body: some View {
        @Bindable var vm = viewModel
        VStack {
            Form {
                if vm.authManager.isGuest {
                    guestLoginSection

```

```

    } else {
        accountSection
        apiKeySection
        RemoteModelSettingsSection(
            remoteModel: Binding(
                get: { vm.chatService.remoteModel },
                set: { vm.chatService.remoteModel = $0 }
            ),
            useRag: Binding(
                get: { vm.chatService.useRag },
                set: { vm.chatService.useRag = $0 }
            ),
            maxTokensSlider: $vm.maxTokensSlider,
            temperatureSlider: $vm.temperatureSlider,
            useCustomModel: $vm.useCustomModel,
            availableModels: vm.chatService.availableModels,
            defaultRemoteModel: vm.chatService.defaultRemoteModel,
            onRefreshModels: {
                Task { await vm.chatService.loadModels() }
            }
        )
        dataSourcesSection
        logoutSection
    }
}

.scrollContentBackground(.hidden)
.navigationTitle("Настройки")
.navigationBarTitleDisplayMode(.inline)
.background(Color.greenBackground.ignoresSafeArea())
}

.background(Color.greenBackground.ignoresSafeArea())
.onAppear {
    vm.maxTokensSlider = Double(vm.chatService.maxTokens)
    vm.temperatureSlider = vm.chatService.temperature
    Task {
        await vm.chatService.loadModels()
    }
}

.onChange(of: vm.maxTokensSlider) { _, newValue in
    vm.chatService.maxTokens = Int(newValue)
}

.onChange(of: vm.temperatureSlider) { _, newValue in
    vm.chatService.temperature = newValue
}
}

// MARK: - Subviews

private var accountSection: some View {
    Section(header: Text("Аккаунт")) {
        if let user = viewModel.authManager.currentUser {
            HStack {
                Text("Email")
                Spacer()
                Text(user.email)
                .foregroundColor(.secondary)
            }
        } else {
            Text("Не авторизован")
            .foregroundColor(.secondary)
        }
    }
}

```

```

}

private var apiKeySection: some View {
    @Bindable var vm = viewModel
    return Section(header: Text("API ключ провайдера")) {
        Group {
            if vm.isApiKeyVisible {
                TextField("Введите ключ", text: Binding(
                    get: { vm.chatService.userApiKey },
                    set: { vm.chatService.userApiKey = $0 }
                ))
            } else {
                SecureField("Введите ключ", text: Binding(
                    get: { vm.chatService.userApiKey },
                    set: { vm.chatService.userApiKey = $0 }
                ))
            }
        }
        .textInputAutocapitalization(.never)
        .disableAutocorrection(true)
        .font(.system(.body, design: .monospaced))

        Toggle("Показать ключ", isOn: $vm.isApiKeyVisible)
            .toggleStyle(SwitchToggleStyle(tint: .green))

        if !vm.chatService.userApiKey.isEmpty {
            Button(role: .destructive) {
                vm.chatService.userApiKey = ""
            } label: {
                Label("Очистить ключ", systemImage: "trash")
            }
        }

        Text("Если указать API ключ (например, OpenAI или совместимый), ответы  
будут генерироваться через удаленную модель. Без ключа применяется локальная  
модель HalalAI.")
            .font(.footnote)
            .foregroundColor(.secondary)
            .padding(.top, 4)
    }
}

private var dataSourcesSection: some View {
    Section(header: Text("Источники данных")) {
        Text("Система RAG формирует контекст из Перевода смыслов Священного  
Корана Э.Р. Кулиева.")
            .font(.footnote)
            .foregroundColor(.secondary)
            .padding(.vertical, 4)

        if let url = URL(string: "https://xn---8sbemuhsaeiwd9h5a9c.xn--p1ai/  
chitat-koran-na-russkom/elmir-kuliev/") {
            Link("Открыть источник", destination: url)
                .font(.footnote)
                .foregroundColor(.blue)
        }
    }
}

private var logoutSection: some View {
    Section {
        Button(role: .destructive) {

```



```

        viewModel.authManager.logout()
    } label: {
        HStack {
            Spacer()
            Label("Выйти из аккаунта", systemImage: "rectangle.portrait.and.arrow.right")
            Spacer()
        }
    }
}

private var guestLoginSection: some View {
    Section {
        Button {
            viewModel.authManager.logout()
        } label: {
            Label("Войти или зарегистрироваться", systemImage: "person.crop.circle")
                .frame(alignment: .leading)
        }
        .foregroundColor(.darkGreen)
    } header: {
        Text("Аккаунт")
    }
}
}
}

```

HalalAI/Features/Settings/UI/SettingsViewModel.swift

```

//
// SettingsViewModel.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 29.12.2025.
//

import Foundation

extension SettingsView {
    @MainActor
    @Observable
    final class ViewModel {
        var chatService: ChatService
        var authManager: AuthManager

        init(chatService: ChatService, authManager: AuthManager) {
            self.chatService = chatService
            self.authManager = authManager
        }

        var isApiKeyVisible = false
        var maxTokensSlider: Double = 2048
        var temperatureSlider: Double = 0.7
        var useCustomModel = false
    }
}

```

HalalAI/Navigation/AuthCoordinator.swift

```
//
// AuthCoordinator.swift
// HalalAI
//

import SwiftUI

enum AuthCoordinator: Hashable {
    case login
    case register
}
```

HalalAI/Navigation/ChatCoordinator.swift

```
//
// ChatCoordinator.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 12.10.2025.
//

import SwiftUI

enum ChatCoordinator {
    case chat

    @MainActor
    var view: some View {
        switch self {
            case .chat:
                screenFactory.makeChatView()
        }
    }
}
```

HalalAI/Navigation/Coordinator.swift

```
//
// CoordinatorService.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 12.10.2025.
//

import SwiftUI

enum Step: Hashable, Equatable {
    case chat(_ val: ChatCoordinator)
    case settings(_ val: SettingsCoordinator)
    case home(_ val: HomeCoordinator)
}

@MainActor
@Observable
final class Coordinator {
    var path: [Step] = []
    var currentSelectedTab: TabBarItem = .home
}
```

```

init() {}

func nextStep(step: Step) {
    path.append(step)
}

func dismiss() {
    if !path.isEmpty {
        path.removeLast()
    }
}

func toRoot() {
    path = []
}

func selectTab(item: TabBarItem) {
    if item == currentSelectedTab {
        path = []
        return
    }
    path = []
    currentSelectedTab = item
}

@ViewBuilder
func build(step: Step) -> some View {
    switch step {
    case .chat(let value): value.view
    case .settings(let value): value.view
    case .home(let value): value.view
    }
}
}

```

HalalAI/Navigation/HomeCoordinator.swift

```

//
//  HomeCoordinator.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 18.11.2025.
//

import SwiftUI

enum HomeCoordinator: Hashable {
    case home
    case scanner
    case quran
    case sura(suraIndex: Int)
    case prayerSettings
    case halalMap

    @MainActor
    @ViewBuilder
    var view: some View {
        switch self {
        case .home:
            screenFactory.makeHomeView()

```

```

        case .scanner:
            screenFactory.makeScannerView()
        case .quran:
            screenFactory.makeQuranListView()
        case .sura(let suraIndex):
            screenFactory.makeSuraReaderView(suraIndex: suraIndex)
        case .prayerSettings:
            screenFactory.makePrayerNotificationSettingsView()
        case .halalMap:
            screenFactory.makeHalalMapView()
    }
}
}

```

HalalAI/Navigation/RouterView.swift

```

//
// RouterView.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 25.10.2025.
//

import SwiftUI

struct RouterView: View {
    @State var coordinator = Coordinator()
    @State private var isKeyboardVisible = false
    private var tabBarHeight = 80

    var body: some View {
        ZStack(alignment: .bottom) {
            NavigationStack(path: $coordinator.path) {
                tabRootView
                    .additionalPaddingIfNeeded(shouldShowTabBar, tabBarHeight)
                    .navigationDestination(for: Step.self) { step in
                        coordinator.build(step: step)
                            .navigationBarBackButtonHidden()
                            .additionalPaddingIfNeeded(shouldShowTabBar,
tabBarHeight)
                    }
            }

            if shouldShowTabBar {
                TabBarView()
                    .edgesIgnoringSafeArea(.bottom)
                    .transition(.push(from: .bottom))
            }
        }
        .environment(coordinator)
        .ignoresSafeArea(.keyboard, edges: .bottom)
        .onReceive(NotificationCenter.default.publisher(for: UIResponder.
keyboardWillShowNotification)) { _ in
            withAnimation(.easeInOut(duration: 0.25)) {
                isKeyboardVisible = true
            }
        }
        .onReceive(NotificationCenter.default.publisher(for: UIResponder.
keyboardWillHideNotification)) { _ in
            withAnimation(.easeInOut(duration: 0.25)) {
                isKeyboardVisible = false
            }
        }
    }
}

```

```

        }
    }
}

@ViewBuilder
private var tabRootView: some View {
    switch coordinator.currentSelectedTab {
    case .home:
        coordinator.build(step: .home(.home))
    case .chat:
        coordinator.build(step: .chat(.chat))
    case .settings:
        coordinator.build(step: .settings(.settings))
    }
}

private var shouldShowTabBar: Bool {
    !isKeyboardVisible
}
}

```

HalalAI/Navigation/SettingsCoordinator.swift

```

//
// SettingsCoordinator.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 12.10.2025.
//

import SwiftUI

enum SettingsCoordinator {
    case settings

    @MainActor
    var view: some View {
        switch self {
        case .settings:
            screenFactory.makeSettingsView()
        }
    }
}

```

HalalAI/Navigation/TabBar/TabBarItem.swift

```

//
// TabBarItem.swift
// HalalAI
//
// Created by Мирсаят Сабирзянов on 12.10.2025.
//

import SwiftUI

struct TabBarItemModel {
    let indexInTab: Int
    let name: String
    let image: UIImage
}

```

```

enum TabBarItem {
    case home
    case chat
    case settings

    var model: TabBarItemModel {
        switch self {
            case .home:
                return TabBarItemModel(
                    indexInTab: 0,
                    name: "Homepage",
                    image: UIImage(systemName: "house.fill")!
                )
            case .chat:
                return TabBarItemModel(
                    indexInTab: 1,
                    name: "Chat",
                    image: UIImage(systemName: "brain.head.profile.fill")!
                )
            case .settings:
                return TabBarItemModel(
                    indexInTab: 2,
                    name: "Settings",
                    image: UIImage(systemName: "gearshape.fill")!
                )
        }
    }
}

```

HalalAI/Navigation/TabBar/TabBarView.swift

```

//
//  TabBarView.swift
//  HalalAI
//
//  Created by Мирсаят Сабирзянов on 12.10.2025.
//

import SwiftUI

struct TabBarView: View {
    @Environment(Coordinator.self) var coordinator
    var body: some View {
        HStack {
            ZStack {
                SelectedCapsule(selectedTab: coordinator.currentSelectedTab)
                HStack(spacing: 50) {
                    TabBarItem(tab: .home, coordinator: coordinator)
                        .id(0)
                    TabBarItem(tab: .chat, coordinator: coordinator)
                        .id(1)
                    TabBarItem(tab: .settings, coordinator: coordinator)
                        .id(2)
                }
                .frame(maxWidth: .infinity)
            }
        }
        .padding(.vertical, 5)
        .background(
            RoundedRectangle(cornerRadius: 100)

```

```

        .fill(Color.tabBar)
        .opacity(0.9)
        .shadow(color: Color.black.opacity(0.1), radius: 5, x: 0, y: 2)
    )
    .padding(.horizontal, 85)
}

struct SelectedCapsule: View {
    var selectedTab: TabBarItem
    private let positions: [CGFloat] = [-77.0, 0, 77.0]
    var body: some View {
        RoundedRectangle(cornerRadius: 100)
            .fill(Color.greenForeground.opacity(0.5))
            .frame(width: 60, height: 50)
            .animation(.bouncy(extraBounce: 0.017), value: selectedTab.model.
indexInTab)
            .offset(x: positions[selectedTab.model.indexInTab])
    }
}

struct TabBarItem: View {
    let tab: TabBarItem
    var coordinator: Coordinator
    var body: some View {
        Button {
            coordinator.selectTab(item: tab)
        } label: {
            Image(uiImage: tab.model.image)
                .resizable()
                .scaledToFit()
                .frame(width: 27, height: 27)
                .scaleEffect(coordinator.currentSelectedTab == tab ? 1.2 : 1.0)
                .animation(.easeInOut(duration: 0.2), value: coordinator.
currentSelectedTab)
        }
        .buttonStyle(.plain)
        .accessibilityLabel(tab.model.name)
    }
}

HalalAIUITests/LoginUITests.swift

//
//  LoginUITests.swift
//  HalalAIUITests
//
//  Created by Мирсаят Сабирзянов on 14.12.2025.
//

import XCTest

final class LoginUITests: XCTestCase {
    var app: XCUIApplication!

    override func setUp() {
        continueAfterFailure = false
        app = XCUIApplication()
        app.launchArguments = ["testing"]
        app.launch()
    }
}

```

```

override func tearDown() {
    app = nil
}

// MARK: - UI Elements Tests

func testWelcomeMessage() {
    XCTAssertTrue(app.staticTexts["Добро пожаловать"].exists)
}

func testLoginSubtitleExists() {
    XCTAssertTrue(app.staticTexts["Войдите в свой аккаунт"].exists)
}

func testEmailFieldExists() {
    let emailField = app.textFields["Email"]
    XCTAssertTrue(emailField.exists, "Поле для ввода email должно существовать")
}

func testPasswordFieldExists() {
    let passwordField = app.secureTextFields["Пароль"]
    XCTAssertTrue(passwordField.exists, "Поле для ввода пароля должно существовать")
}

func testLoginButtonExists() {
    let loginButton = app.buttons["Войти"]
    XCTAssertTrue(loginButton.exists, "Кнопка входа должна существовать")
}

func testRegisterButtonExists() {
    let registerButton = app.buttons["Зарегистрироваться"]
    XCTAssertTrue(registerButton.exists, "Кнопка регистрации должна существовать")
}

// MARK: - Validation Tests

func testLoginButtonExistsWhenFieldsEmpty() {
    let loginButton = app.buttons["Войти"]
    XCTAssertTrue(loginButton.exists, "Кнопка входа должна существовать даже при пустых полях")
}

func testCanFillEmailAndPasswordFields() {
    let emailField = app.textFields["Email"]
    let passwordField = app.secureTextFields["Пароль"]
    let loginButton = app.buttons["Войти"]

    emailField.tap()
    emailField.typeText("user@example.com")

    passwordField.tap()
    passwordField.typeText("password123")

    XCTAssertTrue(loginButton.exists, "Кнопка входа должна существовать после заполнения полей")
}

// MARK: - Input Tests

```



```

func testCanTypeInEmailField() {
    let emailField = app.textFields["Email"]
    emailField.tap()
    emailField.typeText("test@example.com")

    XCTAssertEqual(emailField.value as? String, "test@example.com", "Поле должно содержать введенный email")
}

func testCanTypeInPasswordField() {
    let passwordField = app.secureTextFields["Пароль"]
    XCTAssertTrue(passwordField.isHittable, "Поле пароля должно быть доступно для взаимодействия")

    passwordField.tap()
    passwordField.typeText("mypassword")

    XCTAssertTrue(passwordField.exists, "Поле пароля должно существовать после ввода текста")
}

// MARK: - Navigation Tests

func testNavigateToRegisterScreen() {
    let registerButton = app.buttons["Зарегистрироваться"]
    registerButton.tap()

    let registerTitle = app.staticTexts["Регистрация"]
    XCTAssertTrue(registerTitle.waitForExistence(timeout: 2), "Должен открыться экран регистрации")
}

func testNavigateBackFromRegisterToLogin() {
    let registerButton = app.buttons["Зарегистрироваться"]
    registerButton.tap()

    let registerTitle = app.staticTexts["Регистрация"]
    XCTAssertTrue(registerTitle.waitForExistence(timeout: 2))

    let loginButton = app.buttons["Войти"]
    if loginButton.exists {
        loginButton.tap()

        let welcomeText = app.staticTexts["Добро пожаловать"]
        XCTAssertTrue(welcomeText.waitForExistence(timeout: 2), "Должен вернуться экран входа")
    }
}

// MARK: - Accessibility Tests

func testLoginButtonHasAccessibilityLabel() {
    let loginButton = app.buttons["Войти"]
    XCTAssertTrue(loginButton.exists, "Кнопка входа должна быть доступна для accessibility")
}

func testFieldsAreAccessible() {
    let emailField = app.textFields["Email"]
    let passwordField = app.secureTextFields["Пароль"]

```

```

        XCTAssertTrue(emailField.isHittable, "Поле email должно быть доступно для
взаимодействия")
        XCTAssertTrue(passwordField.isHittable, "Поле пароля должно быть доступно
для взаимодействия")
    }
}

HalalAIUITests/RegisterUITests.swift

//
// RegisterUITests.swift
// HalalAIUITests
//

import XCTest

final class RegisterUITests: XCTestCase {
    var app: XCUIApplication!

    override func setUp() {
        continueAfterFailure = false
        app = XCUIApplication()
        app.launchArguments = ["testing"]
        app.launch()

        let registerButton = app.buttons["Зарегистрироваться"]
        if registerButton.waitForExistence(timeout: 2) {
            registerButton.tap()
        }
    }

    override func tearDown() {
        app = nil
    }

    // MARK: - UI Elements Tests

    func testRegisterTitleExists() {
        let registerTitle = app.staticTexts["Регистрация"]
        XCTAssertTrue(registerTitle.waitForExistence(timeout: 2), "Заголовок 'Рег
истрация' должен существовать")
    }

    func testRegisterSubtitleExists() {
        let subtitle = app.staticTexts["Создайте новый аккаунт"]
        XCTAssertTrue(subtitle.waitForExistence(timeout: 2), "Подзаголовок должен
существовать")
    }

    func testEmailFieldExists() {
        let emailField = app.textFields["Email"]
        XCTAssertTrue(emailField.waitForExistence(timeout: 2), "Поле для ввода
email должно существовать")
    }

    func testPasswordFieldExists() {
        let passwordField = app.secureTextFields["Пароль"]
        XCTAssertTrue(passwordField.waitForExistence(timeout: 2), "Поле для ввода
пароля должно существовать")
    }
}

```

```

func testConfirmPasswordFieldExists() {
    let confirmPasswordField = app.secureTextFields["Подтвердите пароль"]
    XCTAssertTrue(confirmPasswordField.waitForExistence(timeout: 2), "Поле для подтверждения пароля должно существовать")
}

func testRegisterButtonExists() {
    let registerButton = app.buttons["Зарегистрироваться"]
    XCTAssertTrue(registerButton.waitForExistence(timeout: 2), "Кнопка регистрации должна существовать")
}

func testLoginButtonExists() {
    let loginButton = app.buttons["Войти"]
    XCTAssertTrue(loginButton.waitForExistence(timeout: 2), "Кнопка входа должна существовать")
}

// MARK: - Validation Tests

func testRegisterButtonExistsWhenFieldsEmpty() {
    let registerButton = app.buttons["Зарегистрироваться"]
    XCTAssertTrue(registerButton.exists, "Кнопка регистрации должна существовать даже при пустых полях")
}

func testCanFillAllFields() {
    let emailField = app.textFields["Email"]
    let passwordField = app.secureTextFields["Пароль"]
    let confirmPasswordField = app.secureTextFields["Подтвердите пароль"]
    let registerButton = app.buttons["Зарегистрироваться"]

    emailField.tap()
    emailField.typeText("test@example.com")

    passwordField.tap()
    passwordField.typeText("password123")

    confirmPasswordField.tap()
    confirmPasswordField.typeText("password123")

    XCTAssertTrue(registerButton.exists, "Кнопка регистрации должна существовать после заполнения полей")
}

// MARK: - Input Tests

func testCanTypeInEmailField() {
    let emailField = app.textFields["Email"]
    emailField.tap()
    emailField.typeText("user@example.com")

    XCTAssertEqual(emailField.value as? String, "user@example.com", "Поле должно содержать введенный email")
}

func testCanTypeInPasswordField() {
    let passwordField = app.secureTextFields["Пароль"]
    XCTAssertTrue(passwordField.isHittable, "Поле пароля должно быть доступно для взаимодействия")

    passwordField.tap()

```

```

        passwordField.typeText("mypassword123")

        XCTAssertTrue(passwordField.exists, "Поле пароля должно существовать посл
е ввода текста")
    }

    func testCanTypeInConfirmPasswordField() {
        let confirmPasswordField = app.secureTextFields["Подтвердите пароль"]
        XCTAssertTrue(confirmPasswordField.isHittable, "Поле подтверждения пароля
должно быть доступно для взаимодействия")

        confirmPasswordField.tap()
        confirmPasswordField.typeText("mypassword123")

        XCTAssertTrue(confirmPasswordField.exists, "Поле подтверждения пароля дол
жно существовать после ввода текста")
    }

    func testCanTypeValidEmail() {
        let emailField = app.textFields["Email"]
        emailField.tap()
        emailField.typeText("valid.email@example.com")

        XCTAssertEqual(emailField.value as? String, "valid.email@example.com", "П
оле должно принимать валидный email адрес")
    }

    // MARK: - Password Validation Tests

    func testPasswordValidationMessageAppears() {
        let passwordField = app.secureTextFields["Пароль"]
        passwordField.tap()
        passwordField.typeText("short")

        app.swipeUp()

        let validationMessage = app.staticTexts["Пароль должен содержать минимум
8 символов"]
        if validationMessage.waitForExistence(timeout: 1) {
            XCTAssertTrue(validationMessage.exists, "Должно появиться сообщение о
валидации пароля")
        }
    }

    func testPasswordMismatchMessageAppears() {
        let passwordField = app.secureTextFields["Пароль"]
        let confirmPasswordField = app.secureTextFields["Подтвердите пароль"]

        passwordField.tap()
        passwordField.typeText("password123")

        confirmPasswordField.tap()
        confirmPasswordField.typeText("different")

        app.swipeUp()

        let mismatchMessage = app.staticTexts["Пароли не совпадают"]
        if mismatchMessage.waitForExistence(timeout: 1) {
            XCTAssertTrue(mismatchMessage.exists, "Должно появиться сообщение о н
есовпадении паролей")
        }
    }
}

```

```

// MARK: - Navigation Tests

func testNavigateToLoginScreen() {
    let loginButton = app.buttons["Войти"]
    loginButton.tap()

    let welcomeText = app.staticTexts["Добро пожаловать"]
    XCTAssertTrue(welcomeText.waitForExistence(timeout: 2), "Должен открыться экран входа")
}

func testNavigateBackFromLoginToRegister() {
    let loginButton = app.buttons["Войти"]
    loginButton.tap()

    let welcomeText = app.staticTexts["Добро пожаловать"]
    XCTAssertTrue(welcomeText.waitForExistence(timeout: 2))

    let registerButton = app.buttons["Зарегистрироваться"]
    if registerButton.exists {
        registerButton.tap()

        let registerTitle = app.staticTexts["Регистрация"]
        XCTAssertTrue(registerTitle.waitForExistence(timeout: 2), "Должен вернуться экран регистрации")
    }
}

// MARK: - Accessibility Tests

func testRegisterButtonHasAccessibilityLabel() {
    let registerButton = app.buttons["Зарегистрироваться"]
    XCTAssertTrue(registerButton.exists, "Кнопка регистрации должна быть доступна для accessibility")
}

func testAllFieldsAreAccessible() {
    let emailField = app.textFields["Email"]
    let passwordField = app.secureTextFields["Пароль"]
    let confirmPasswordField = app.secureTextFields["Подтвердите пароль"]

    XCTAssertTrue(emailField.isHittable, "Поле email должно быть доступно для взаимодействия")
    XCTAssertTrue(passwordField.isHittable, "Поле пароля должно быть доступно для взаимодействия")
    XCTAssertTrue(confirmPasswordField.isHittable, "Поле подтверждения пароля должно быть доступно для взаимодействия")
}

// MARK: - Form Filling Tests

func testCompleteFormFilling() {
    let emailField = app.textFields["Email"]
    let passwordField = app.secureTextFields["Пароль"]
    let confirmPasswordField = app.secureTextFields["Подтвердите пароль"]

    emailField.tap()
    emailField.typeText("testuser@example.com")

    passwordField.tap()
    passwordField.typeText("securepass123")
}

```

```
confirmPasswordField.tap()
confirmPasswordField.typeText("securepass123")

XCTAssertEqual(emailField.value as? String, "testuser@example.com", "
Email должен быть заполнен")
XCTAssertTrue(passwordField.exists, "Поле пароля должно существовать")
XCTAssertTrue(confirmPasswordField.exists, "Поле подтверждения пароля дол
жно существовать")
}

func testFormScrollable() {
    let emailField = app.textFields["Email"]
    emailField.tap()

    app.swipeUp()
    app.swipeDown()

    XCTAssertTrue(emailField.exists, "Форма должна быть прокручиваемой")
}
}
```